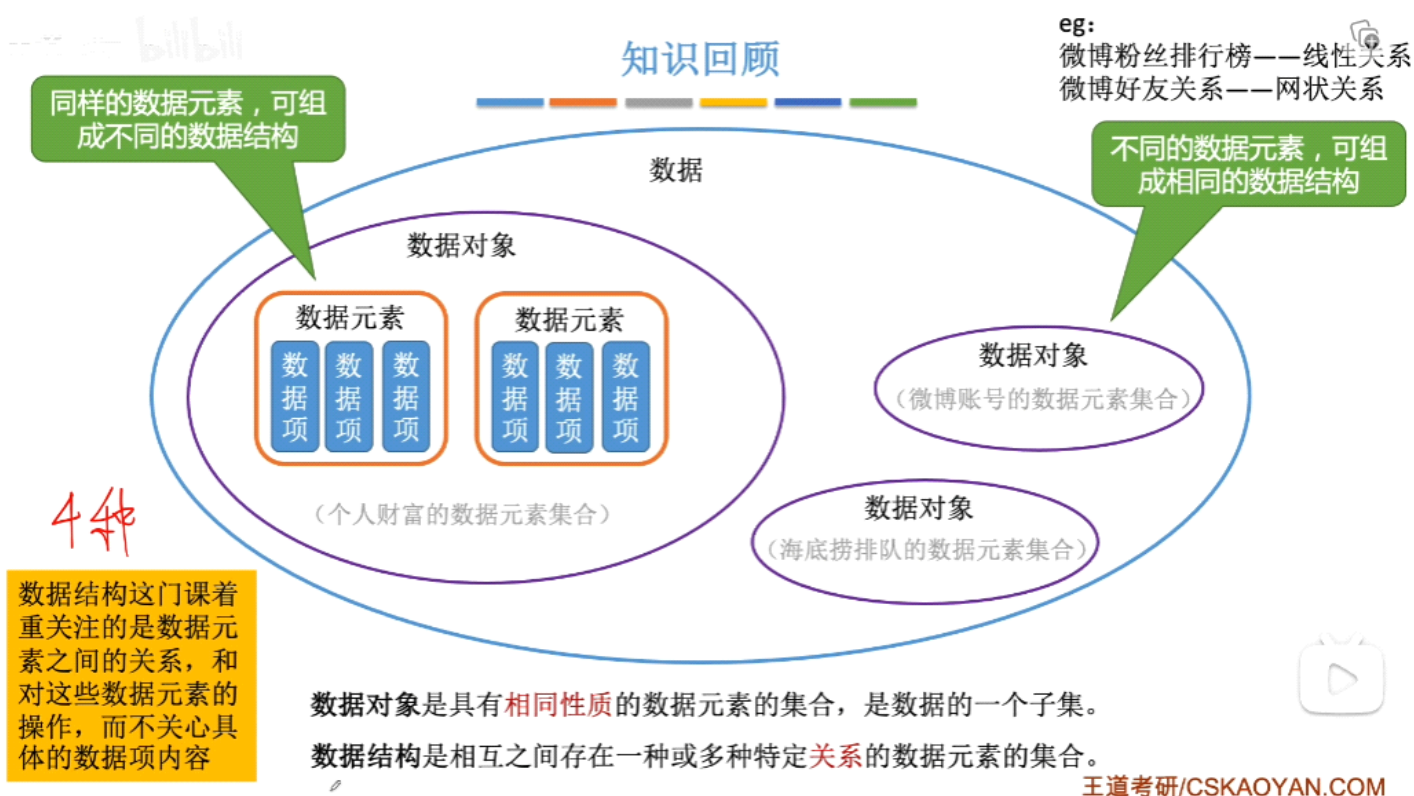


一、数据结构的基本概念：

- 1、数据：能被计算机识别，处理的符号的集合，可以看成是一个整体
- 2、数据元素：用于描述一个个体，由几个数据项组成
- 3、数据项：描述数据的最小单位，不可以再分割
- 4、数据对象：具有相同性质的数据元素的集合，是数据的子集
- 5、数据结构：描述数据元素间的关系

同一个数据对象里的数据元素，可以组成不同的数据结构，具体关系如下：



二、数据结构三要素

- 1、逻辑结构：数据元素之间的逻辑关系
 - 集合结构：无关系
 - 线性结构：一对一
 - 树形结构：一对多
 - 图形结构：多对多

- 2、存储结构（物理结构）：在计算机内部表示出数据元素之间的逻辑关系

顺序存储：物理地址和逻辑地址相同

链式存储：物理地址和逻辑地址不一定相同

索引存储：类似目录，每个数据有对应的地址表示

散列存储：哈希表

3、数据的运算：针对某种逻辑结构，结合实际需求，定义基本运算

增加

删除

修改

查找

排序

三、数据类型和抽象数据类型

数据类型分为

原子类型（他的值不可以再分，比如int float）

结构类型（值可以再分，比如数组结构体）

抽象数据类型 是对数据结构逻辑结构的描述，以及数据的运算在计算机内部的实现

四、程序=数据结构+算法

数据结构是如何用数据正确的描述现实世界并存入计算机

算法是对特定问题求解步骤的一种描述

五、算法的特性：

有穷性：有限个步骤

确定性：每条指令必须有确切的含义，相同输入只能得出相同输出，没有歧义

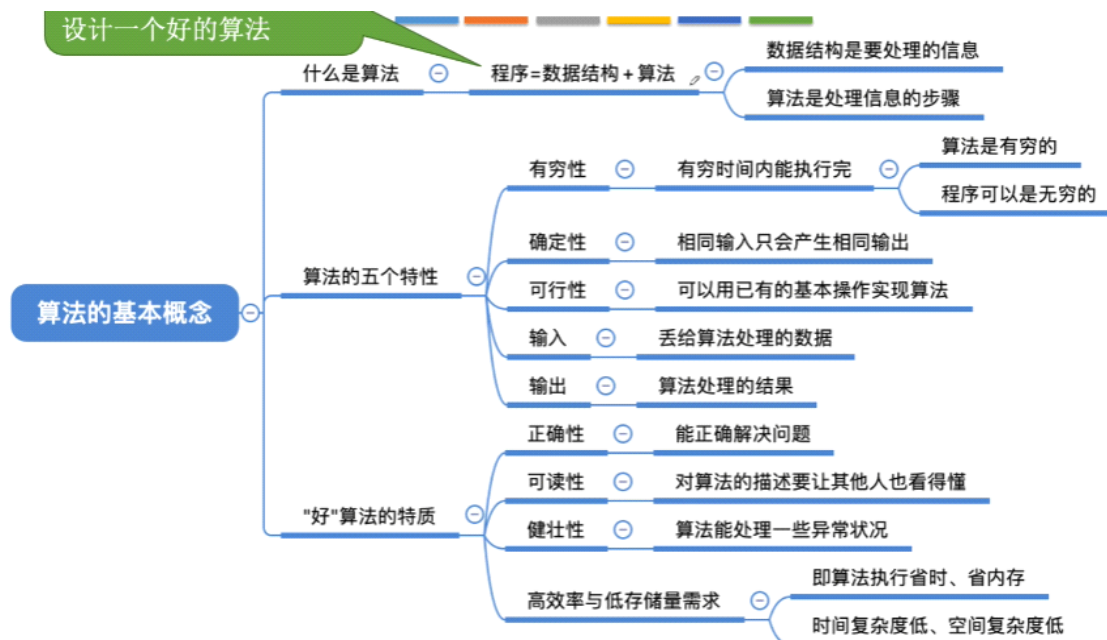
可行性：可以通过已经实现的基本运算执行有限次来实现

输入和输出：算法输入可以0个或多个，输出不可以0个，只能1个或多个

六、好算法的特性：

正确性：能解决问题





七、时间复杂度的计算（事前统计运行时间）

时间复杂度是指算法时间开销 T 与问题规模 n 的关系 $T=T(n)$

得出表达式后可以简化，找最大的项

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$



八、空间复杂度的计算（事前统计算法占用的空间）

一般都为 $S=S(1)$

可变一维数组为 $S=S(n)$

可变二维数组为 $S=S(n^2)$

知识回顾与重要考点



顺序表和链表

2024年2月3日 10:12

一、线性表的定义和基本操作

1、线性表的定义：

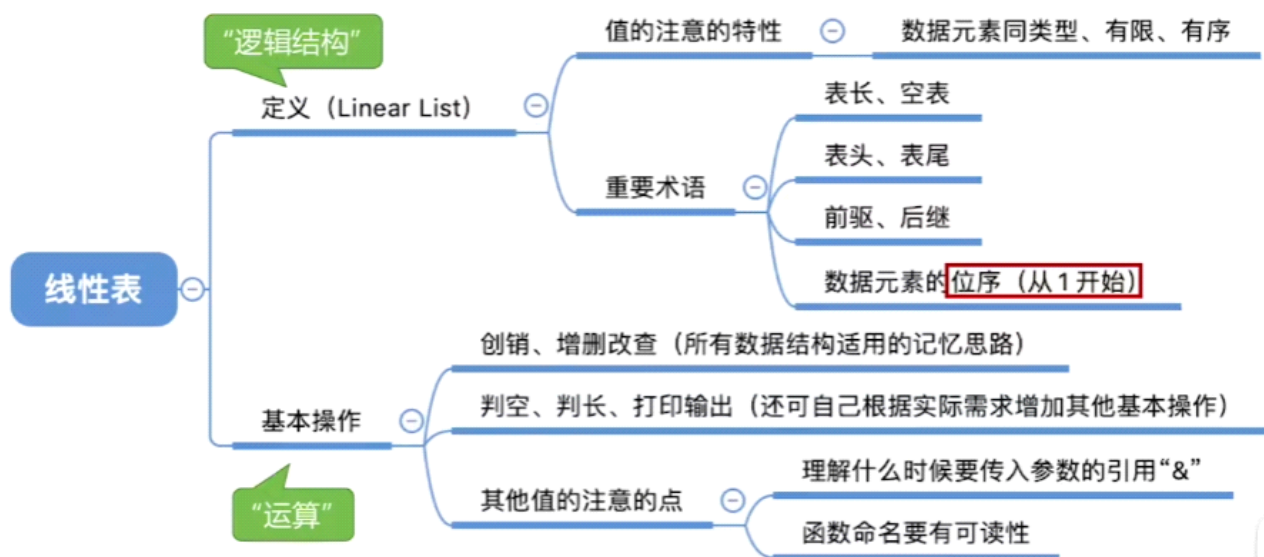
线性表是具有相同数据类型的 n 个数据元素的有限序列，其中 n 为表长， $n=0$ 时候线性表是空表。如果用 L 命名线性表，这一般表示为： $L=(a_1,a_2,a_3,a_4...a_i)$ 。其中 a_1 为表头元素， a_i 为表尾元素， i 为位序，从1开始，下标从0开始。除了第一个元素，每个元素有且仅有一个直接前驱。除了最后一个元素每个元素有且仅有一个直接后继。



2、线性表的基本操作：

- 1、初始化线性表 `InitList(&L)`
- 2、销毁一个线性表 `DestoryList(&L)`
- 3、插入操作 `ListInsert(&L,i,e)`
- 4、删除操作 `ListDelete(&L,i,&e)`
- 5、按值查找 `LocateElem(L,e)`
- 6、按位查找 `GetElem(L,i)`

知识回顾与重要考点



二、顺序表的定义

顺序表是用顺序存储的方式实现线性表。顺序存储是把逻辑上相邻的元素存储在物理位置上也相邻的存储单元

元中，元素之间的关系由存储单元的邻接关系来体现。

1、顺序表结构体定义：

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize]; //数据元素
    int length;             //表长
}Sqlist;
```

2、初始化一个顺序表：

```
void InitList(Sqlist &L){
    for(int i=0;i<MaxSize;i++) //给各个元素赋初始值
        L.data[i]=0;
    L.length=0;                //表长初始值为0
}
```

3、动态可增加长度的线性表实现：

```
#include<stdlib.h>
#include<stdio.h>
#define InitSize 10 //默认最大长度
typedef struct{
    int *data;       //动态分配数组的指针
    int MaxSize;     //顺序表的最大容量
    int length;      //顺序表的最大长度
}SeqList;

void InitList(SeqList &L){
    L.data=(int *)malloc(InitSize*sizeof(int)); //data指针指向新申请的一块内存区域
    L.length=0; //初始化当前长度
    L.MaxSize=InitSize; //初始化最大长度
}

void IncreaseSize(SeqList &L,int len){
    int *p=L.data; //p指针用于释放原来空间的内存
    L.data=(int *)malloc((L.MaxSize+len)*sizeof(int)); //扩大data指针指向的空间大小
    for(int i=0;i<L.length;i++)
        L.data[i]=p[i]; //将数据复制到新区域
    L.MaxSize=L.MaxSize+len; //最大长度增加len个
    free(p); //释放p指针指向的内存
}
```

缺点：花费一些时间

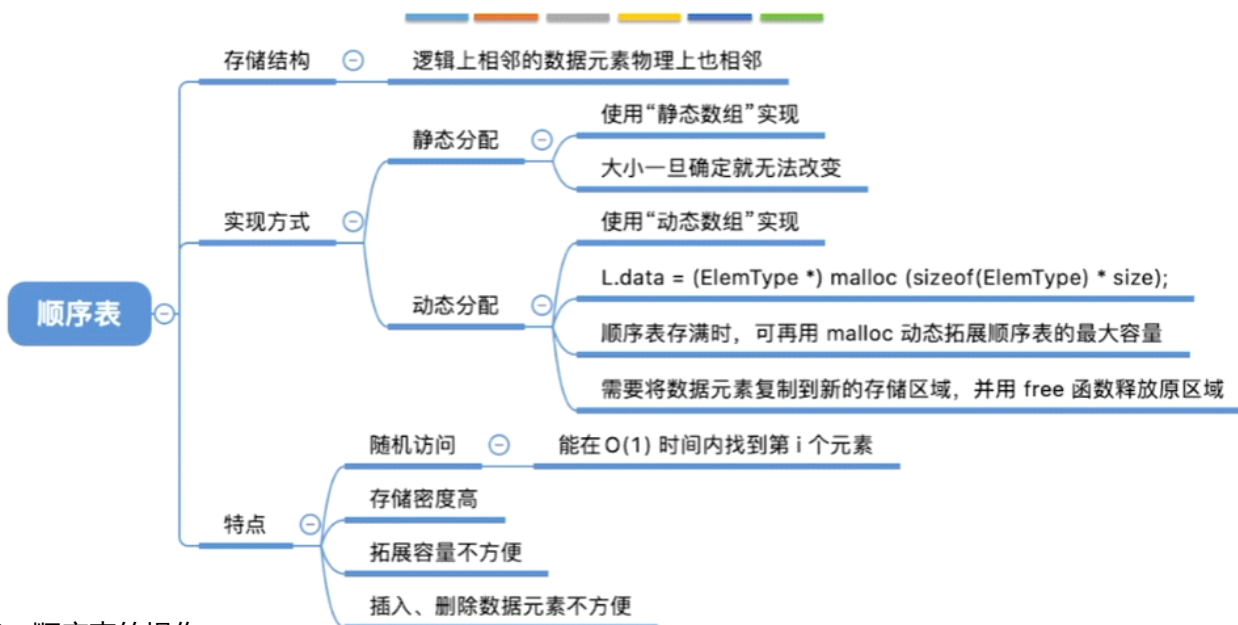
4、顺序表的特点：

随机访问：已知是第几个元素，访问的时间复杂度是 $O(1)$

存储密度高：每个节点只存储数据元素，节省空间

扩展容量不方便。

插入删除需要移动大量元素，不方便。



三、顺序表的操作

1、插入操作ListInsert(&L,i,e):

```
bool ListInsert(SqList &L,int i,int e){
    if(i<1||i>L.length+1) return false; //插入位置不合法
    if(L.length>=MaxSize) return false; //满了
    for(int j=L.length;j>=i;j--)           //插入前得腾出位置
        L.data[j]=L.data[j-1];           //从插入位置开始到最后一个元素位置，所有元素向后移动一位
    L.data[i-1]=e;                          //插入元素e在指定位置
    L.length++;                             //表长+1
    return true;
}
```

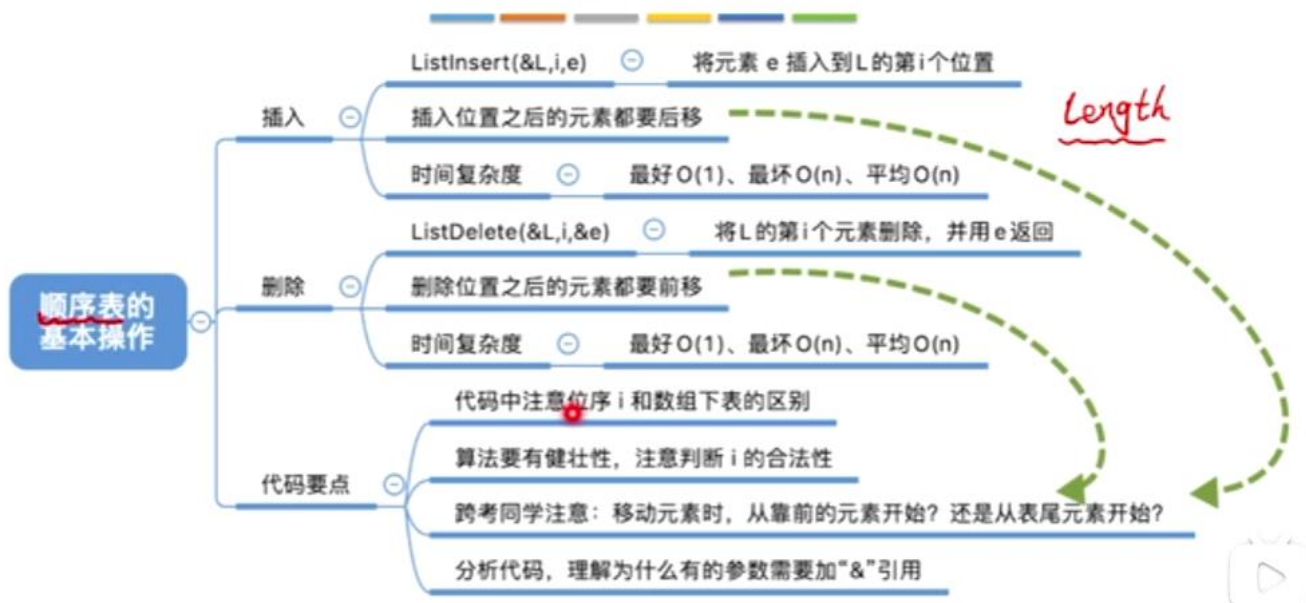
时间复杂度 $O(n)$

2、删除操作ListDelete(SqList &L,int i,int &e):

```
bool ListDelete(SqList &L,int i,int &e){
    if(i<0||i>L.length) return false; //删除位置不合法
    if(L.length==0) return false; //没有元素了
    e=L.data[i-1];                     //备份数据
    for(int j=i;j<L.length;j++)
        L.data[j-1]=L.data[j];         //从删除位置的后一位开始到最后一个位置，依次覆盖前驱元素
    L.length--;                         //表长-1
    return true;
}
```

时间复杂度： $O(n)$

知识回顾与重要考点

3、按位查找 `GetList(L,i)`:

```
ElemType GetElem(SqList L, int i){
    return L.data[i-1];
}
```

时间复杂度 $O(1)$

4、按值查找 `LocateElem(L,e)`:

```
int LocateElem(SqList L, ElemType e){
    for(int i=0; i<L.length; i++){
        if(L.data[i]==e) return i+1;
    }
    return 0;
}
```

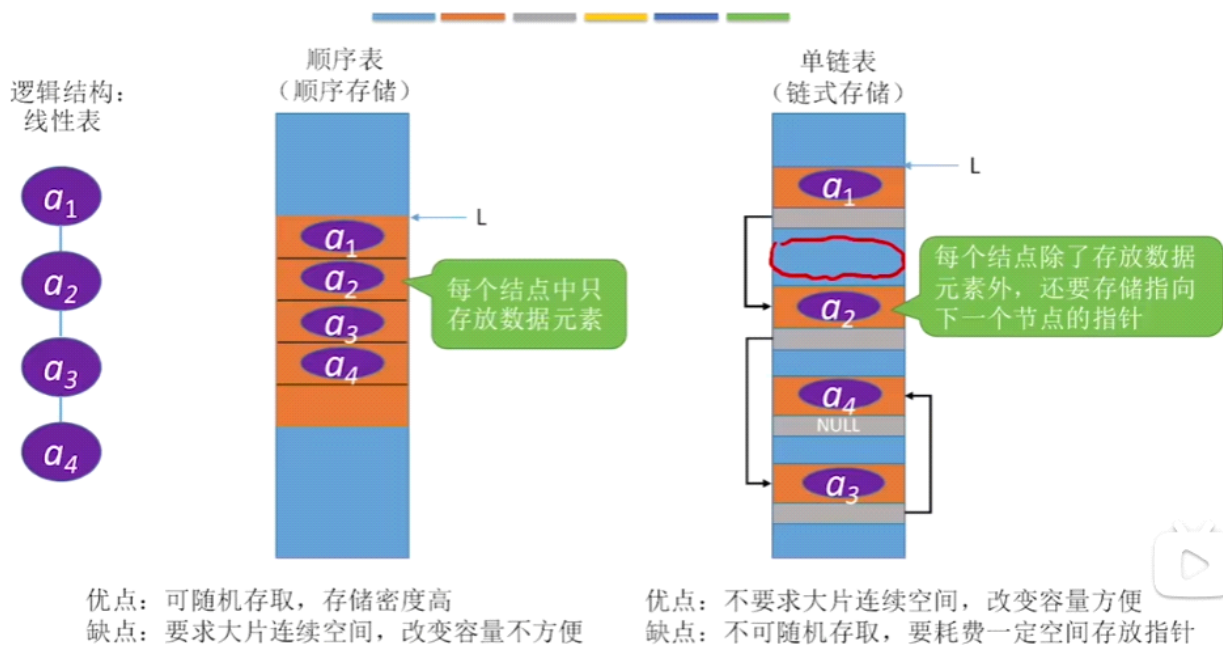
平均时间复杂度 $O(n)$



四、单链表定义。

1、定义：每个节点除了存放数据元素，还要存放指向下一个节点的指针

2、顺序表和单链表的区别：



3、定义单链表的结构体:

```
typedef struct LNode{
    ElemType data;    //数据域
    struct LNode *next; //指针域
}LNode,*LinkList;
//在形参表中如果是LinkList则是强调这是一个单链表, 是LNode *则是强调这是一个节点
```

4、初始化单链表(不带头结点):

```
bool InitList(LinkList &L){
    L=NULL;    //初始化为一个空表
    return true;
}
```

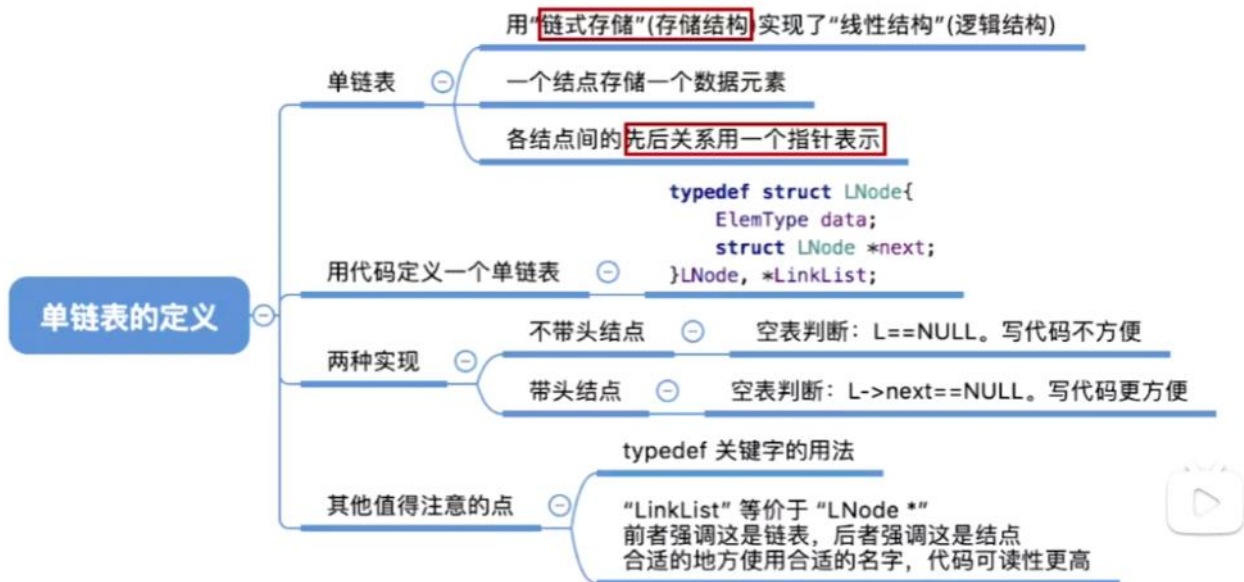
5、初始化单链表 (带头结点) :

```
bool InitList(LinkList &L){
    L=(LNode *)malloc(sizeof(LNode));    //加上一个头节点
    if(L==NULL) return false;    //内存不足, 分配失败
    L->next=NULL;    //初始化头结点后为空
    return true;
}
```

6、判断单链表是否为空 (不带头结点) :

```
bool Empty(LinkList L){
    if(L==NULL) return true;
    else return false;
}
```

用“链式存储”(存储结构)实现了“线性结构”(逻辑结构)



五、单链表的插入和删除

1、按照位序插入ListInsert(&L,i,e)(带头结点):

```
bool ListInsert(LinkList &L,int i,ElemType e){
    if(i<1) return false;           //插入位置不合法
    LNode *p=L;                     //p指向链表的头结点
    int j=0;                          //当前的p指向第0个节点(头节点)
    while(p!=NULL&& j<i-1)           //找到插入位置的前驱结点
    {
        p=p->next;                   //指针后移
        j++;
    }
    if(p==NULL) return false;        //i不合法
    LNode *s=(LNode *)malloc(sizeof(LNode)); //创建一个新节点, s指向他
    s->data=e;                        //新节点data域赋值
    s->next=p->next;                  //开始插入
    p->next=s;
    return true;
}
```

时间复杂度 $O(n)$

2、按照位序插入ListInsert(&L,i,e)(不带头结点):

//基本与上边相同, 但是需要考虑 $i=1$ 时候头结点更新的情况

```
bool ListInsert(LinkList &L,int i,ElemType e){
    if(i<1) return false;
    if(i==1){
        LNode *s=(LNode *)malloc(sizeof(LNode));
        s->data=e;
        s->next=L;                  //s接在头结点的前面
        L=s;                        //把s变成头结点
    }
}
```



```

return true; }
LNode *p=L; //p指向链表的头结点
int j=1; //当前的p已经指向了第1个节点
while(p!=NULL&& j<i-1) //p最终指向插入位置的前驱结点
{
    p=p->next; //指针后移
    j++;
}
if(p==NULL) return false; //没找到对应地址
LNode *s=(LNode *)malloc(sizeof(LNode)); //创建一个新节点，s指向他
s->data=e; //新节点data域赋值
s->next=p->next;
p->next=s;
return true;
}

```

平均时间复杂度 $O(n)$

3、指定节点的后插操作:

```

bool InsertNextNode(LNode *p,ElemType e){ //在p节点后插入
    if(p==NULL) return false;
    LNode *s=(LNode *)malloc(sizeof(LNode)); //开辟新内存空间
    if(s==NULL) return false; //内存分配失败
    s->data=e; //赋值新节点
    s->next=p->next; //后插
    p->next=s;
    return true;
}

```

平均时间复杂度 $O(1)$

4、指定节点的前插操作:

```

bool InsertPriorNode(LNode *p,ElemType e){
    if(p==NULL) return false;
    LNode *s=(LNode *)malloc(sizeof(LNode));
    if(s==NULL) return false;
    s->next=p->next; //后插
    p->next=s;
    s->data=p->data; //把p的数据移动到后边的s的data域
    p->data=e; //e给p的data域
}

bool InsertPriorNode(LNode *p,LNode *s){
    if(p==NULL||s==NULL) return false;
    s->next=p->next; //后插
    p->next=s;
    ElemType temp=p->data; //中间变量用于交换
}

```

```

        p->data=s->data;
        s->data=temp;
    return true;
}
    平均时间复杂度O(1)

```

5、按位序删除ListDelete(&L,i,&e) (带头结点) :

```

bool ListDelete(LinkList &L,int i,ElemType &e){
    if(i<1) return false;
    LNode *p=L;
    int j=0;
    while(p!=NULL&&j<i-1)
    {
        p=p->next;
        j++;
    }
    if(p==NULL) return false;
    if(p->next==NULL) return false;
    LNode *q=p->next;
    p->next=q->next;
    free(q);
    return true;
}
    平均时间复杂度O(n)

```

//p用于扫描满足条件的节点
 //p指向第0个节点
 //找到要删除节点的前驱节点
 //指针后移
 //没找到
 //前驱结点后没有节点了
 //q用于指向要删除节点，辅助删除

6、指定节点的删除DeleteNode(p)

```

bool DeleteNode(LNode*p){
    if(p==NULL) return false;
    LNode *q=p->next;
    p->data=p->next->data;
    p->next=q->next;
    free(q);
    return true;
}
    平均时间复杂度O(1)

```

//令q指向p的后继节点
 //和后继节点交换数据域
 //将q节点断开
 //释放q节点内存

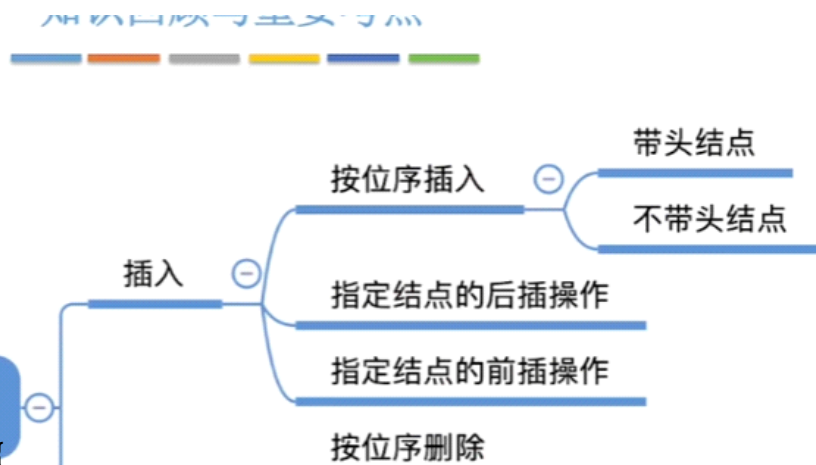
六、单链表的查找

1、按位查找GetElem (带头结点) :

```

LNode *GetElem(LinkList L,int i){

```



单链表的插入删除

```

LNode *GetElem(LinkList L, int i){
    if(i < 0) return NULL;
    LNode *p = L;
    int j = 0;
    while(p != NULL && j < i){
        p = p->next;
        j++;
    }
    return p;
}

```

删除

按位序删除

指定结点的删除

//p指向第i个节点的位置

- Tip:
1. 这些代码都要会写，都重要
 2. 打牢基础，慢慢加速
 3. 体会带头结点、不带头结点的代码区别
 4. 体会“封装”的好处

//返回节点地址

平均时间复杂度 $O(n)$

2、按值查找LocateElem:

```

LNode *LocateElem(LinkList L, ElemType e){
    LNode *p = L->next;
    while(p != NULL && p->data != e)
        p = p->next;
    return p;
}

```

平均时间复杂度 $O(n)$

3、求表的长度Length:

```

int Length(LinkList L){
    int len = 0;
    LNode *p = L;
    while(p->next != NULL) //从第一个元素开始遍历到最后一个元素
    { p = p->next; len++; }
    return len;
}

```

平均时间复杂度 $O(n)$

单链表的查找

按位查找

注意与“顺序表”对比

单链表不具备“随机访问”的特性，只能依次扫描

按值查找

求单链表长度

三种基本操作的时间复杂度都是 $O(n)$

Key

如何写循环扫描各个结点的代码逻辑

注意边界条件的处理

七、单链表的建立

1、尾插法建立

```

LinkedList list_TailInsert(LinkedList &L){
    char x;
    L=(LinkedList)malloc(sizeof(LNode));    //建立一个头结点
    LNode *s,*r=L;                          //r为表尾指针
    x=getchar();                             //输入插入节点的data值,回车结束
    while(x!='\n'){
        s=(LNode *)malloc(sizeof(LNode));
        s->data=x;
        r->next=s;                          //把新节点接在尾结点后面
        r=s;                               //更新尾结点
        x=getchar();
    }
    r->next=NULL;                          //尾结点的next为NULL
    return L;
}

```

平均时间复杂度 $O(n)$

2、头插法建立

```

LinkedList list_HeadInsert(LinkedList &L){
    LNode *s;
    char x;
    L=(LinkedList)malloc(sizeof(LNode)); //创建一个头指针
    L->next=NULL;                       //初始化头结点为空链表（防止野指针）
    x=getchar();
    while(x!='\n')
    {
        s=(LNode*)malloc(sizeof(LNode));
        s->data=x;
        s->next=L->next;                //头插
        L->next=s;
        x=getchar();
    }
    return L;
}

```

平均时间复杂度 $O(n)$

头插法还能将一个节点插入到单链表的任意位置，还要存放指向后继节点的指针和指向前驱节点的指针。

2、定义双链表的结构体：

```

typedef struct LNode{
    ElemType data;        //数据域
    struct LNode *next;    //后继指针域
    struct LNode *prior;   //前驱指针域
}DNode,*DLinkedList;

```

3、初始化双链表

```

bool InitDLinkedList(DLinkedList &L){

```

```

    L=(DNode*)malloc(sizeof(DNode));
    if(L==NULL) return false;
    L->prior = NULL;
    L->next=NULL;
    return true;
}

```

4、双链表的插入

//在p节点之后插入s节点

```

bool InsertNextDNode(DNode *p,DNode *s){
    if(p==NULL||s==NULL) return false;
    s->next=p->next;                //插入（共4句）
    if(p->next!=NULL)                //如果p节点有后继节点
        p->next->prior=s;
    s->prior=p;
    p->next=s;                        //如果是最后一个节点会出现bug，此时推荐循环双链表
    return true;
}

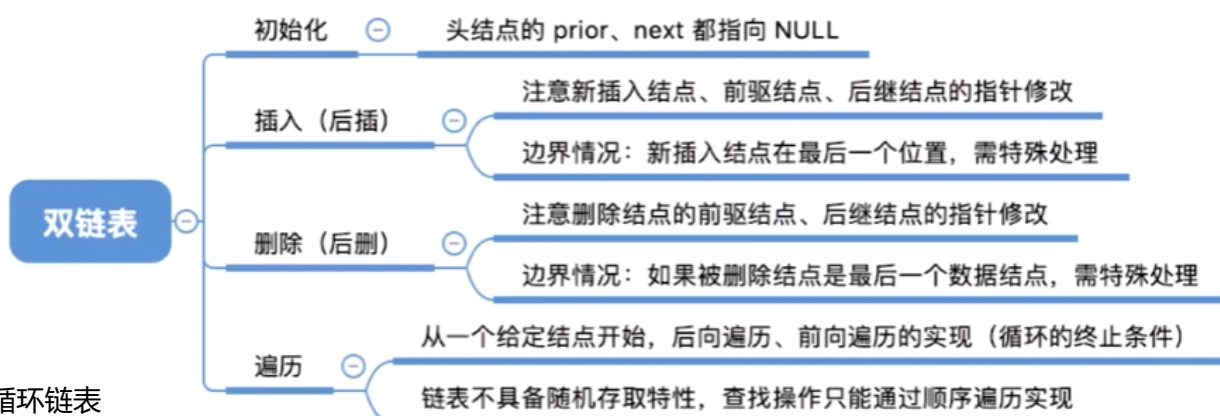
```

5、双链表的删除

```

bool DeleteNextDNode(DNode *p){
    if(p==NULL) return false;
    DNode *q=p->next;
    if(q==NULL) return false;
    p->next=q->next;                //删除（共2句）
    if(q->next!=NULL)
        q->next->prior=p;
    free(q);
    return true;
}

```



九、循环链表

1、定义：相比于非循环链表，循环链表的表尾节点的next域指向头结点，这样可以增加链表的灵活性。单链表从一个节点出发就只能找到后续的各个节点，而循环链表从一个节点出发可以找到任何一个节点。如果对链表的操作都是在头部和尾部，那么可以让指针指向表尾元素，方便修改。

2、循环单链表的结构体定义：

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode,*LinkList;
```

3、初始化一个循环单链表：

```
bool InitList(LinkList &L){
    L=(LNode*)malloc(sizeof(LNode));
    if(L==NULL) return false;
    L->next=L; //这里区别于普通单链表
    return true;
}
```

4、判断循环单链表是否为空：

```
bool Empty(LinkList L)
{
    if(L->next==L) return true;
    else return false;
}
```

5、判断节点p是否为循环单链表的表尾节点

```
bool isTail(LinkList L,LNode *p)
{
    if(p->next==L) return true;
    else return false;
}
```

6、定义循环双链表的结构体：

```
typedef struct LNode{
    ElemType data;    //数据域
    struct LNode *next; //后继指针域
    struct LNode *prior; //前驱指针域
}DNode,*DLinkList;
```

7、初始化空的循环双链表。

```
bool InitDLinkList(DLinkList &L){
    L=(DNode*)malloc(sizeof(DNode));
    if(L==NULL) return false;
    L->prior=L;
    L->next=L;
    return true;
}
```

8、判断循环双链表是否为空：


```

bool Empty(DLinkedList L){
    if(L->next==L) return true;
    else return false;
}

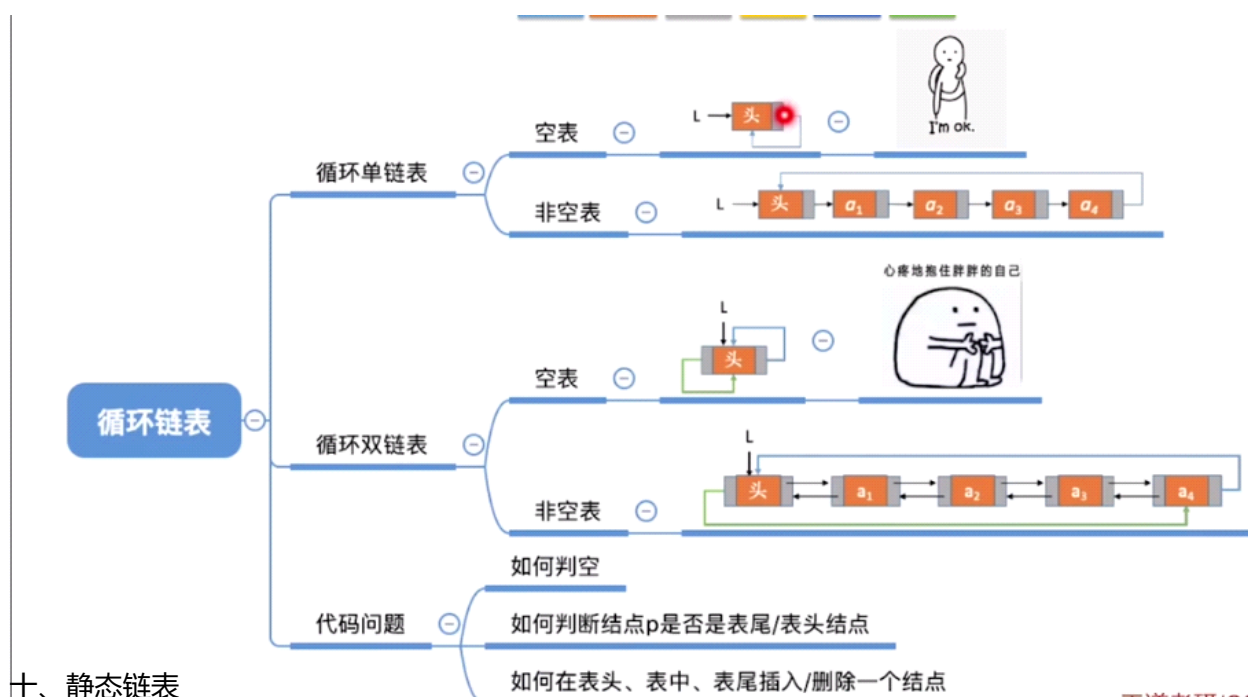
```

9、判断节点p是否为循环双链表的表尾节点：

```

bool isTail(DLinkedList L, DNode *p)
{
    if(p->next==L) return true;
    else return false;
}

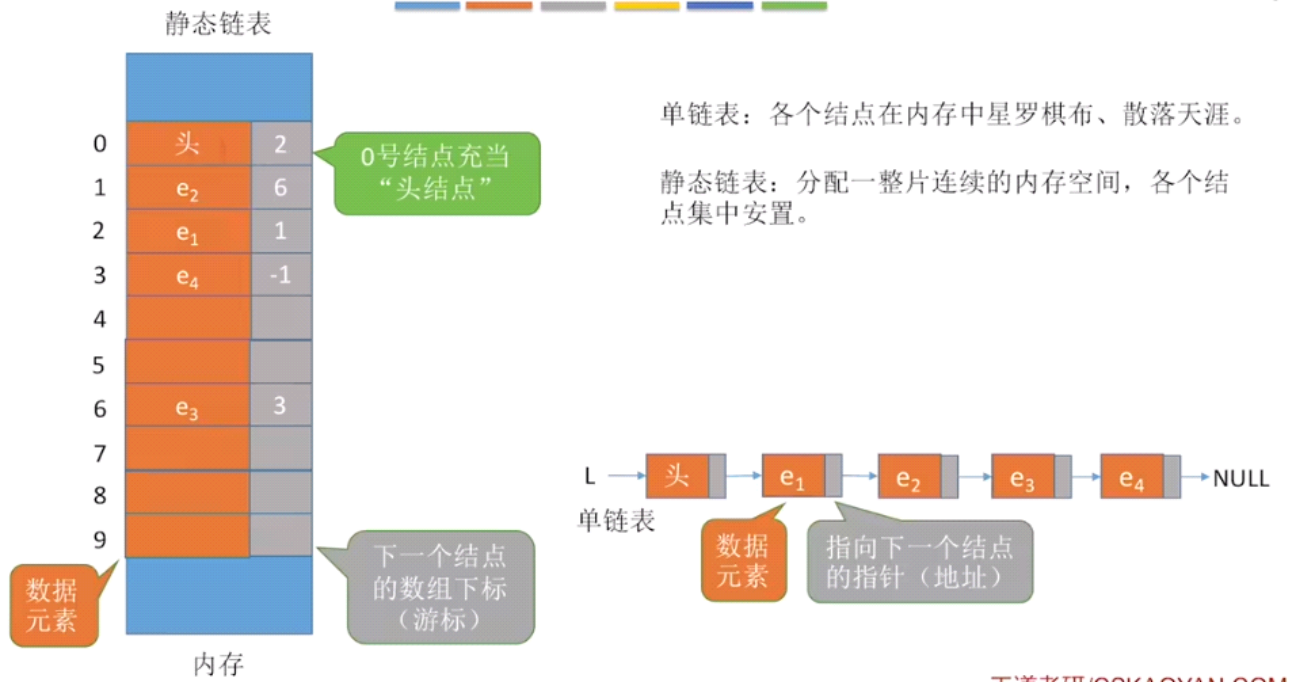
```



十、静态链表

1、定义：前面单链表的各个节点地址在内存中分散存放，而静态链表是分配一片连续的内存空间，各个节点集中安置，说白了就是用数组实现的一个链表。

王道考研/CSKAOYAN.CO



2、静态链表的结构体定义:

```
#define MaxSize 10
typedef struct{
    ElemType data;
    int next;
}SLinkList[MaxSize];    //结构体数组
```

3、静态链表的查找: 从头结点出发往后遍历

4、插入位序为i的节点:

先找到一个空闲节点, 存入数据, 从头节点出发找到位序为i-1的节点, 然后修改新节点的next, 然后修改i-1号节点的next。

5、删除某个节点:

从头结点出发找到前驱节点, 修改前驱节点的游标, 被删除节点的next设为空。

6、优缺点:

优点是增删改查不需要移动大量元素, 缺点是不能随机存取, 只能从头结点开始依次往后查找, 容量固定不可变, 适用于没有指针的语言写。

十一、顺序表和链表的对比

1、逻辑结构上: 都是线性结构, 线性表

2、存储结构上: 顺序表存储密度高, 支持随机存取, 但是大片连续空间分配不方便, 改变容量不方便。 链表的离散的小空间分配方便, 改变容量方便。但是不可以随机存取, 存储密度低

- 3、创建节点上：顺序表需要预留大片的连续空间，空间小了不够用，大了浪费内存
链表只需要分配一个头结点即可
- 4、增删节点上：顺序表插入删除需要频繁移动数据元素。 $O(n)$
链表找到目的位置直接插入删除即可，更快一点。 $O(n)$
- 5、查找节点上：顺序表按位查找可以直接找到($O(1)$)，按值查找 $O(n)$
链表按值查找和按位查找都为 $O(n)$

栈和队列

2024年2月3日 17:18

一、栈的基本概念

1、定义：栈是只允许在一端进行插入或者删除操作的线性表。

栈顶：允许插入和删除的一端

栈底：不允许插入和删除的一端

空栈：没有元素的栈

2、特点：后进先出，先进后出

3、基本操作：

创建（初始化栈）

销毁（销毁栈）

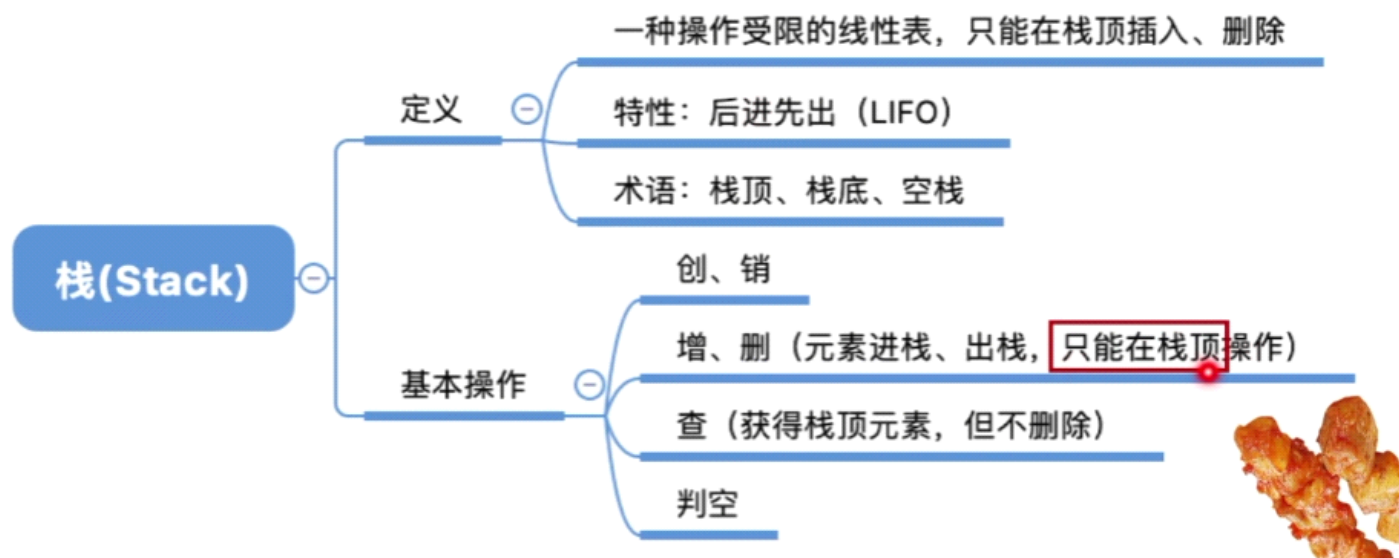
增（进栈）

删（出栈）

查（读取栈元素）

判断栈空

4、题型-判断出栈的合法性。



二、栈的顺序存储实现

1、栈的结构体表示：

```
#define MaxSize 10           //栈中元素最大个数
typedef struct{
    ElemType data[MaxSize];  //data域
    int top;                 //栈顶指针（位置，不是真指针）
}SqStack;
```

2、初始化顺序栈：

```
void InitStack(SqStack &S){
    S.top=-1;                //假设最上面元素为栈顶位置，则初始化栈顶指针为-1
```

```
}
```

3、判断栈空：

```
bool StackEmpty(SqStack S){  
    if(S.top==-1) return true;  
    else return false;  
}
```

4、新元素入栈：

```
bool Push(SqStack &S,ElemType x){  
    if(S.top==MaxSize-1) return false; //栈满就不入栈  
    S.top++; //修改栈顶指针  
    S.data[S.top]=x; //新元素入栈到栈顶指针处  
    return true;  
}
```

5、出栈：

```
bool Pop(SqStack &S,ElemType &x){ //x拿来备份数据的  
    if(S.top== -1) return false; //栈空还出什么栈  
    x=S.data[S.top]; //备份出栈数据  
    S.top--; //修改栈顶指针  
    return true;  
}
```

6、读取栈顶元素

```
bool GetTop(SqStack S,ElemType &x){  
    if(S.top== -1) return false; //空栈  
    x=S.data[S.top];  
    return true;  
}
```

7、共享栈：两个栈共享一片空间

```
#define MaxSize 10  
typedef struct{ //结构体定义  
    ElemType data[MaxSize]; //data域  
    int top0;  
    int top1;  
}ShStack;
```

```
void InitStack(ShStack &S) //初始化
```

```

{
    S.top0=-1;
    S.top1=MaxSize;
}

```

```

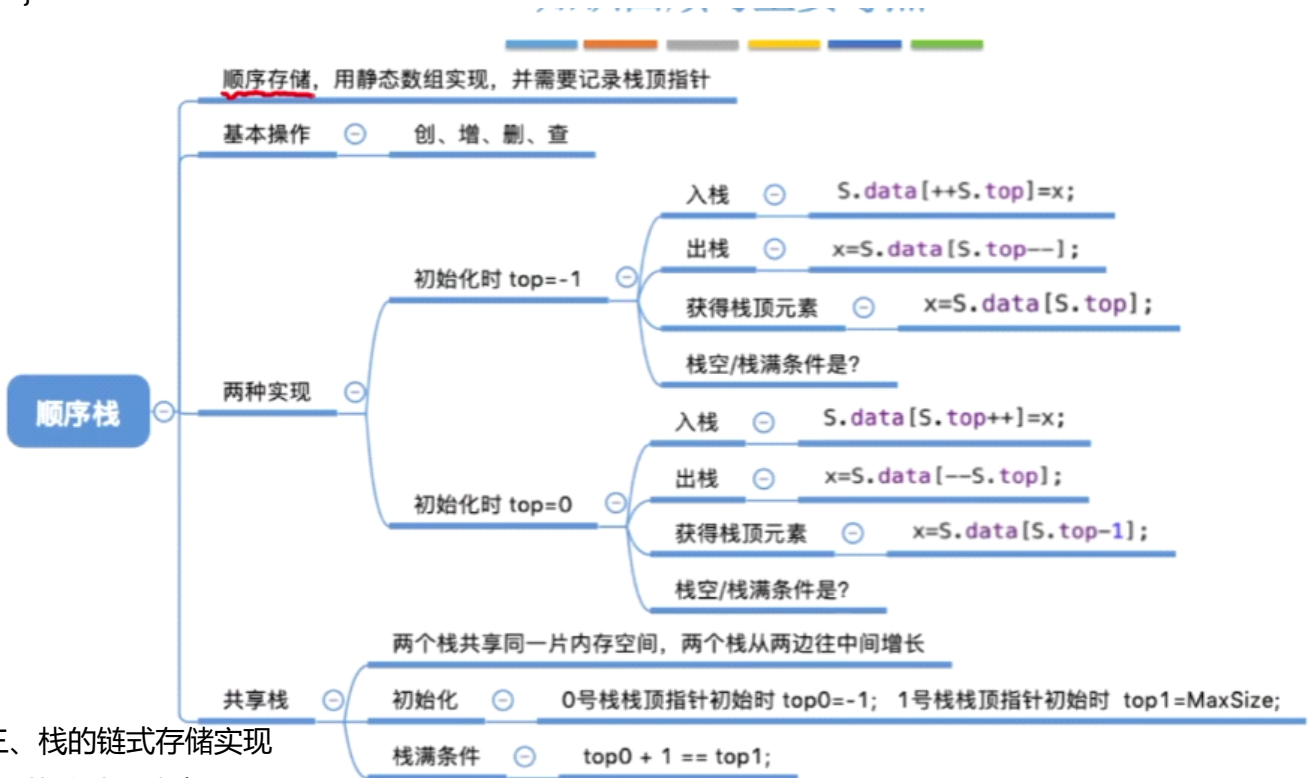
bool OccupyStack(ShStack &S) //判断栈满
{
    if(top0+1==top1) return true;
    else return false;
}

```

```

bool EmptyStack(ShStack &S) //判断栈空
{
    If(( S.top0=-1)&&(S.top1=MaxSize)) return true;
    else return false;
}

```



三、栈的链式存储实现

1、栈的结构体表示

```

typedef struct {
    ElemType data;           //数据域
    struct LNode *next;      //指针域
    struct LNode*top;        //栈顶指针
}LNode,*LinkList;

```

2、栈的初始化

```

bool InitStack(LinkList &L){ //带头节点
    L=(Lnode*)malloc(sizeof(LNode));
    if(L==NULL) return false;
    L->next=NULL;
}

```

```

L->top=L;
}

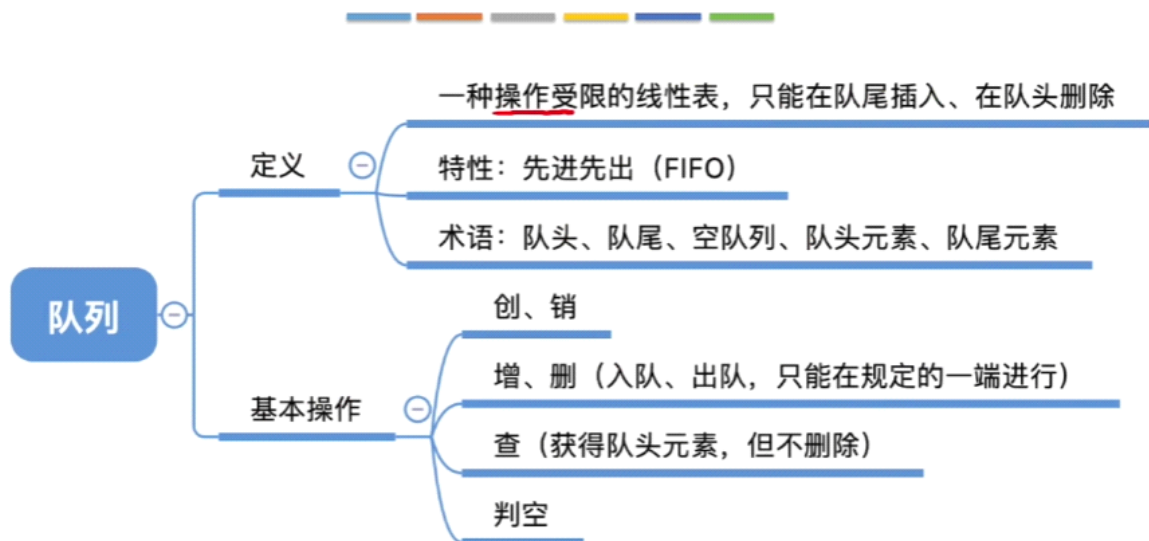
```

- 3、入栈：类似于单链表头插
- 4、出栈：类似于单链表后删
- 5、查找栈顶元素



四、队列的基本概念

- 1、队列的定义：只允许在一端进行插入，一端进行删除的线性表。
- 2、特点：先进先出、后进后出。
- 3、基本操作：
 - 创建：初始化队列
 - 销毁：销毁队列
 - 增：进队
 - 删：出队
 - 查：读取队列元素
 - 判断队空。



五、队列的顺序存储实现

1、结构体

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int front,rear;
}SqQueue;
```

2、入队

```
bool EnQueue(SqQueue &Q,ElemType x){
    if((Q.rear+1)%MaxSize==Q.front) return false; //队满
    Q.data[Q.rear]=x;                               //逻辑上入队
    Q.rear=(Q.rear+1)%MaxSize;                       //队尾逻辑上+1
    return true;
}
```

3、出队

```
bool DeQueue(SqQueue &Q,ElemType &x){
    if(Q.rear==Q.front) return false; //队空
    x=Q.data[Q.front+1];               //逻辑上出队
    Q.front=(Q.front+1)%MaxSize;       //队头逻辑上+1
    return true;
}
```

4、获得队头元素的值，用x返回

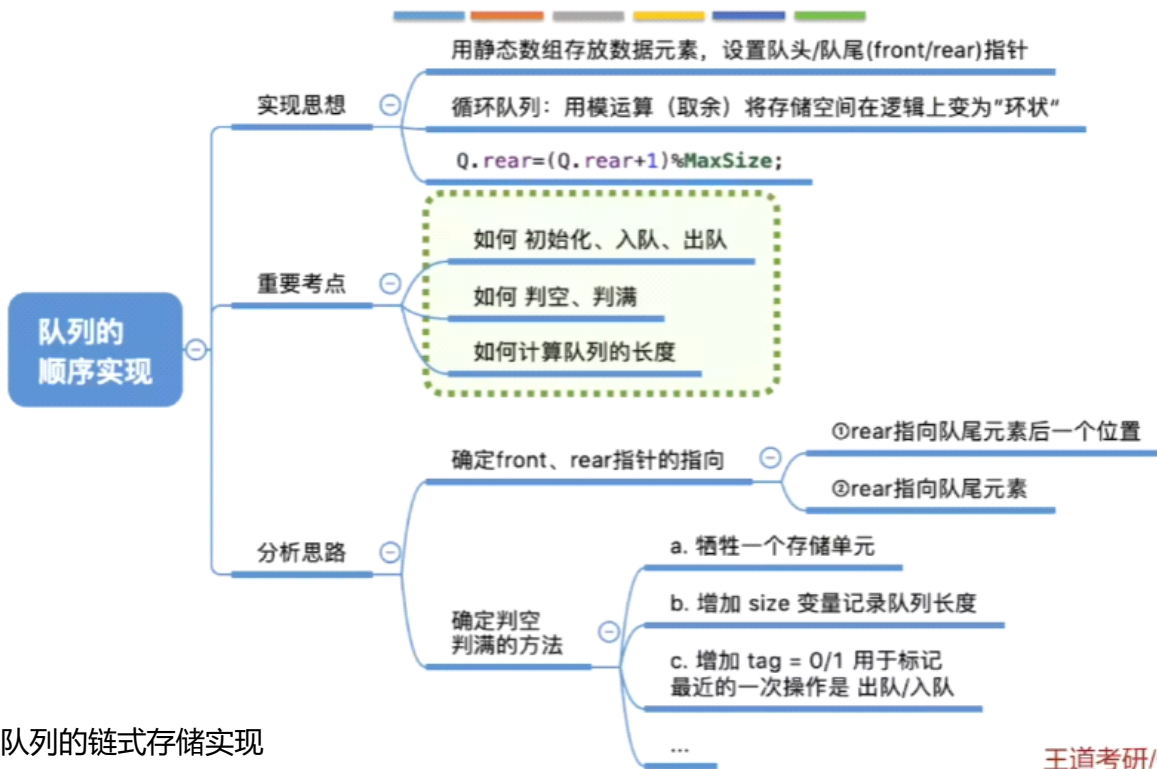
```
bool GetHead(SqQueue Q,ElemType &x){
    if(Q.rear==Q.front) return false;
    x=Q.data[Q.front];
    return true;
}
```

5、判断队空： $Q.rear == Q.front$

6、判断队满： $(Q.rear + 1) \% MaxSize == Q.front$

7、判断元素个数： $(rear + MaxSize - front) \% MaxSize$

8、以上区分队满和队空的做法会牺牲一个存储空间，为了利用那最后一点空间，可以在结构体内设立一个tag，插入时tag=1，删除时候tag=0，当front==rear&&tag==1时队满，当front==rear&&tag==0时队空。



王道考研/CSK

六、队列的链式存储实现

1、结构体

```
typedef struct LinkNode{
    ElemType data;
    struct LinkNode *next;
}LinkNode;
```

```
typedef struct{
    LinkNode *front,*rear;
}LinkQueue;
//队头指针和队尾指针
```

2、初始化:

```
void InitQueue(LinkQueue &Q){
    //带头节点
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode));
    Q.front->next=NULL;
    //队头用于出队, 因此初始值是NULL
}
```

```
void InitQueue(LinkQueue &Q){
    //不带头结点
    Q.front=NULL;
    Q.rear=NULL;
}
```

3、判断队列是否为空:

```
bool IsEmpty(LinkQueue Q){
    //带头节点
    if(Q.front==Q.rear) return true;
    else return false;
}
```

```

bool IsEmpty(LinkQueue Q){           //不带头结点
    if(Q.front==NULL) return true;
    else return false;
}

```

4、入队

```

void EnQueue(LinkQueue &Q,ElemType x){    //带头节点
    LinkNode *s=(LinkNode*)malloc(sizeof(LinkNode));
    s->data=x;
    s->next=NULL;
    Q.rear->next=s;
    Q.rear=s;
}

void EnQueue(LinkQueue &Q,ElemType x){    //不带头结点
    LinkNode *s=(LinkNode*)malloc(sizeof(LinkNode));
    s->data=x;
    s->next=NULL;
    if(Q.front==NULL){                  //没有元素时rear和front初始值为NULL
        Q.front=s;                      //此刻s是第一个节点
        Q.rear=s;
    }
    else{
        Q.rear->next=s;
        Q.rear=s;
    }
}

```

5、出队

```

bool DeQueue(LinkQueue &Q,ElemType &x){    //带头结点
    if(Q.front==Q.rear) return false;      //队空
    LinkNode *p=Q.front->next;             //Q.front为头结点，p指向要删除元素
    x=p->data;
    Q.front->next=p->next;                  //删除
    if(Q.rear==p)                          //删除的是最后一个节点
        Q.rear=Q.front;                   //改队尾指针
    free(p);                              //释放
    return true;
}

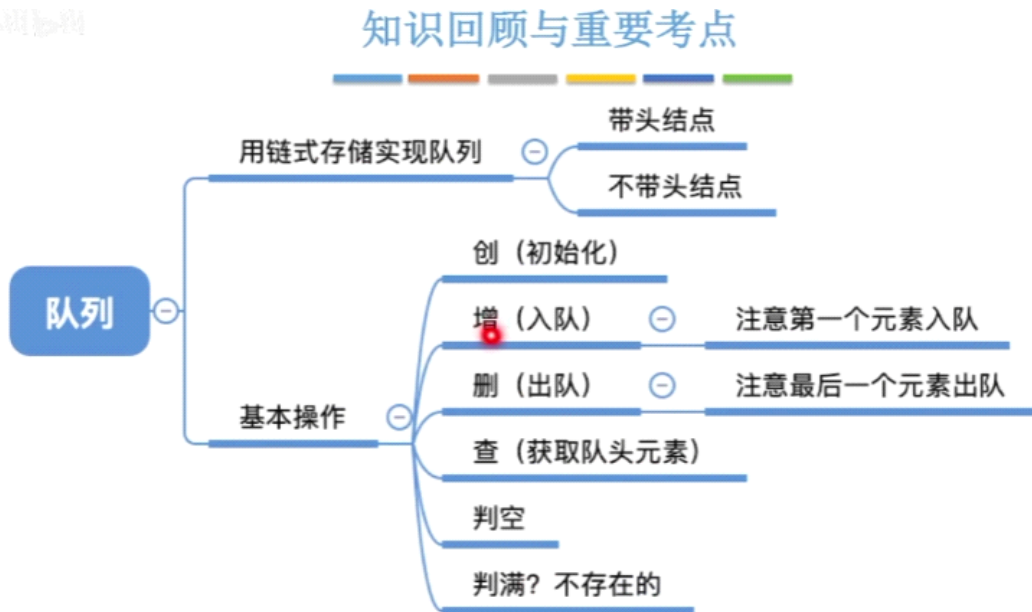
bool DeQueue(LinkQueue &Q,ElemType &x){    //带头结点
    if(Q.front==NULL) return false;
    LinkNode *p=Q.front;                   //Q.front为第一个节点，删的就是它
}

```

```

x=p->data;
Q.front=p->next;                                //删除
if(Q.rear==p) {                                  //如果删除的是最后一个元素
    Q.front=NULL;
    Q.rear=NULL;
}
free(p);
return true;
}

```



七、双端队列

1、定义：双端队列是两边都可以输入和删除的队列，除此之外还有输入受限的双端队列和输出受限的双端队列。

2、判断输出序列的合法性：

王道考研/CSKAC

考点：判断输出序列合法性

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$

1,2,3,4	2,1,3,4	3,1,2,4	4,1,2,3
1,2,4,3	2,1,4,3	3,1,4,2	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,3
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
1,4,2,3	2,4,1,3	3,4,1,2	4,3,1,2
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

卡特兰数：

14种合法出栈序列

$$\frac{1}{n+1}C_{2n}^n = \frac{1}{4+1}C_8^4 = 14$$

栈

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	<u>3,1,2,4</u>	<u>4,1,2,3</u>
1,2,4,3	2,1,4,3	<u>3,1,4,2</u>	<u>4,1,3,2</u>
1,3,2,4	2,3,1,4	3,2,1,4	4,2,1,4
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
<u>1,4,2,3</u>	<u>2,4,1,3</u>	<u>3,4,1,2</u>	<u>4,3,1,2</u>
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输入受限的双端队列

若数据元素输入序列为 1,2,3,4，则哪些输出序列是合法的，哪些是非法的？

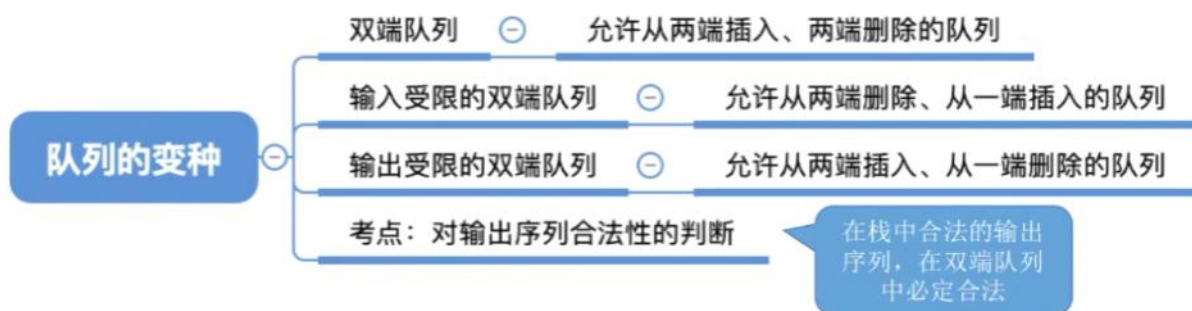
$$A_4^4 = 4! = 24$$



1,2,3,4	2,1,3,4	<u>3,1,2,4</u>	<u>4,1,2,3</u>
1,2,4,3	2,1,4,3	<u>3,1,4,2</u>	4,1,3,2
1,3,2,4	2,3,1,4	3,2,1,4	<u>4,2,1,3</u>
1,3,4,2	2,3,4,1	3,2,4,1	4,2,3,1
<u>1,4,2,3</u>	<u>2,4,1,3</u>	<u>3,4,1,2</u>	<u>4,3,1,2</u>
1,4,3,2	2,4,3,1	3,4,2,1	4,3,2,1

输出受限的双端队列

卡特兰数求共有多少个合法序列：

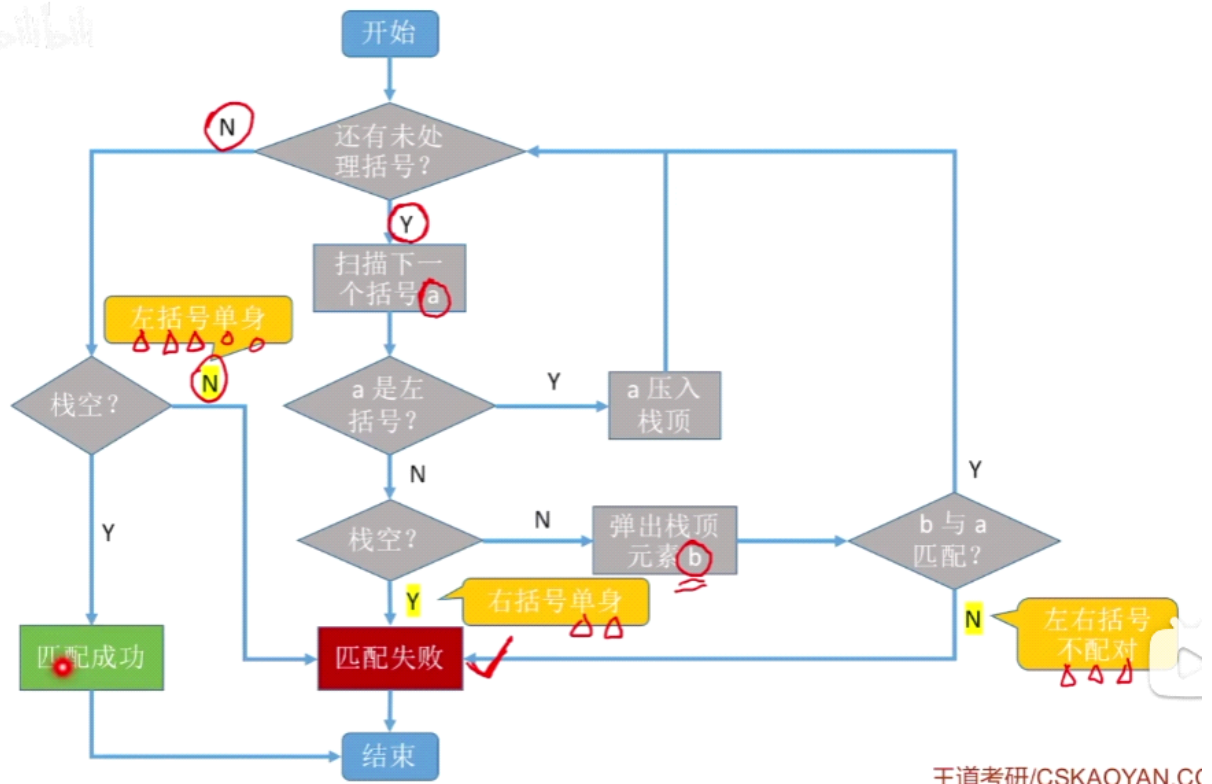


八、括号匹配问题：

1、用栈实现括号匹配，就依次扫描所有字符，遇到左括号就入栈，遇到右括号就弹出栈顶元素检查是否一样。当然也有匹配失败的情况：左括号匹配失败（匹配完栈里面还有左括号）、右括号匹配失败（有右括

号但是栈空没左括号了)、左右括号不匹配(左括号不等于右括号)

2、流程图:



3、算法实现

```
bool bracketCheck(char str[],int length){
    SqStack S;
    InitStack(S);
    for(int i=0;i<length;i++){
        if(str[i]=='('||str[i]=='['||str[i]=='{'){
            Push(S,str[i]);
        }
        else{
            if(StackEmpty(S)) return false;
            char topElem;
            Pop(S,topElem);
            if(str[i]==')'&&topElem!='(') return false;
            if(str[i]==']'&&topElem!='[') return false;
            if(str[i]=='}'&&topElem!='{') return false;
        }
    }
    return StackEmpty(S);
}
```

九、表达式求值:

1、三种算术表达式:

中缀表达式: 正常的算术表达式, 例如 $(3*(1+1)-8)$, 根据运算优先级计算

后缀表达式 (逆波兰表达式): 例如 $a+b$ 的后缀表达式为 $ab+$, $a+b-c$ 为 $ab+c-$, $a+b-c*d$ 为 $ab+cd*-$

前缀表达式 (波兰表达式): 例如 $a+b$ 的前缀表达式为 $+ab$, $a+b-c$ 为 $-+abc$, $a+b-c*d$ 为 $-+ab*cd$

注意: 后缀转换默认左优先原则-只要左边运算能先算, 那就先算左边

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

③
②
①
④
⑦
⑥
⑤

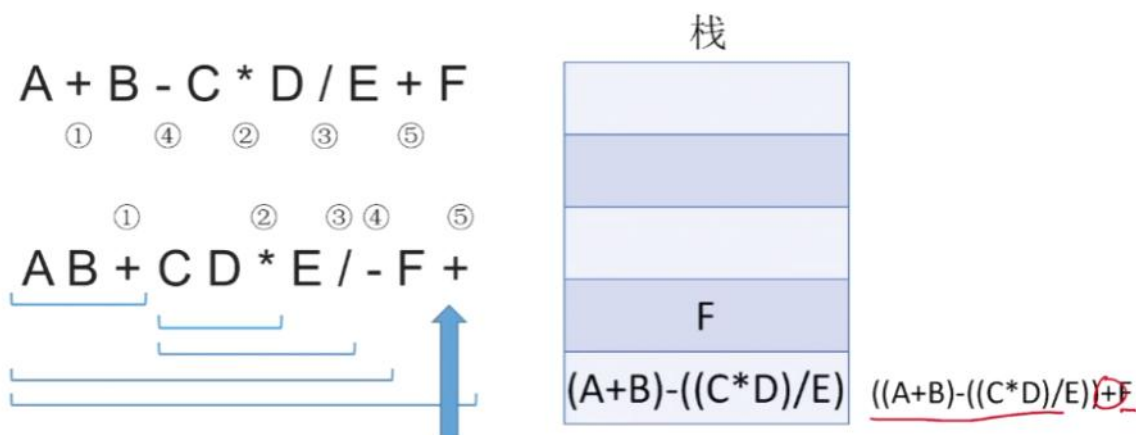
15 7 1 1 + - ÷ 3 × 2 1 1 + + -

中缀转前缀：
“右优先”

$$\begin{array}{ccccccccccc} - & \times & \div & 15 & - & 7 & + & 1 & 1 & 3 & + & 2 & + & 1 & 1 \\ \textcircled{7} & \textcircled{6} & \textcircled{5} & & \textcircled{4} & \textcircled{3} & & & & \textcircled{2} & & \textcircled{1} & & & \end{array}$$

2、后缀表达式的机器计算过程:

用栈实现后缀表达式的计算:



3、前缀表达式的机器计算过程:

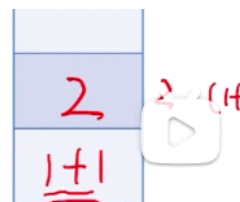
用栈实现前缀表达式的计算:

注意：先出栈的是“左操作数”

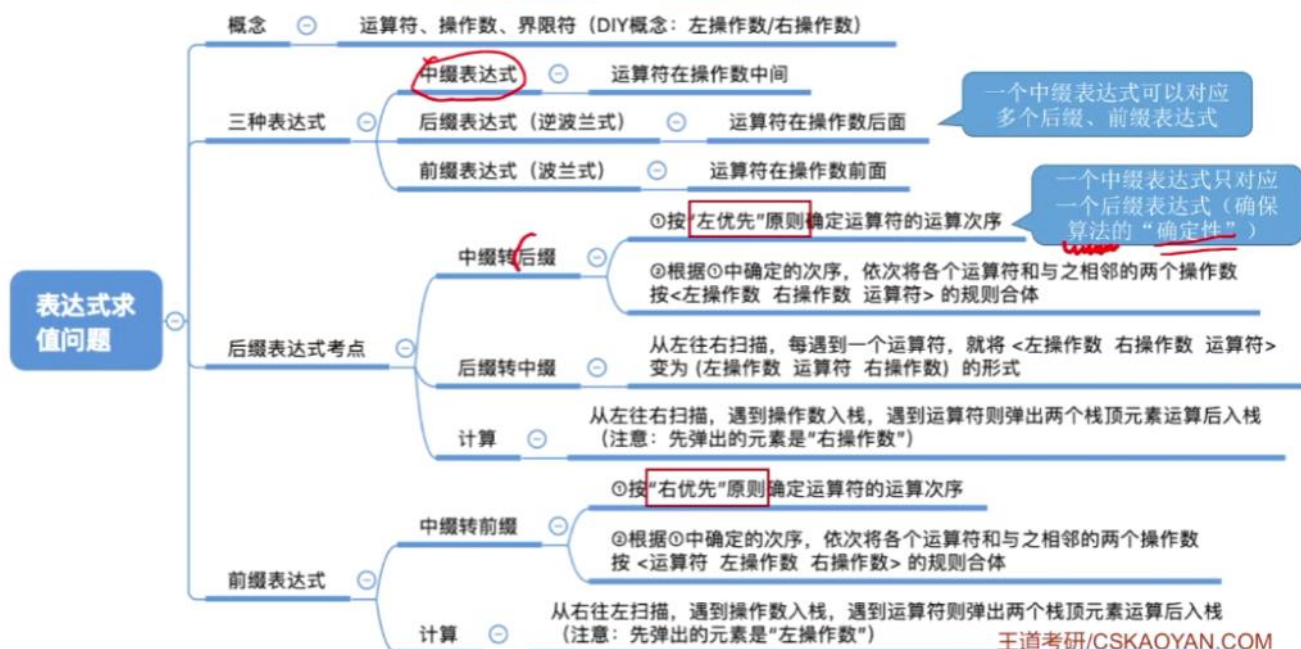
$$\underset{\textcircled{5}}{((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))}$$

$- \times \div 15 - 7 + 1 \quad 1 \quad 3 \quad \boxed{+} \quad 2 + 1 \quad 1$
 ⑦ ⑥ ⑤ ④ ③ ② ①

- × ÷ 15 - 7 + 1 1 3 (+) 2 + 1 1
 ⑦ ⑥ ⑤ ④ ③ ② ① ① ① ① ①



知识回顾与重要考点



4、中缀表达式转后缀表达式的机器计算过程:

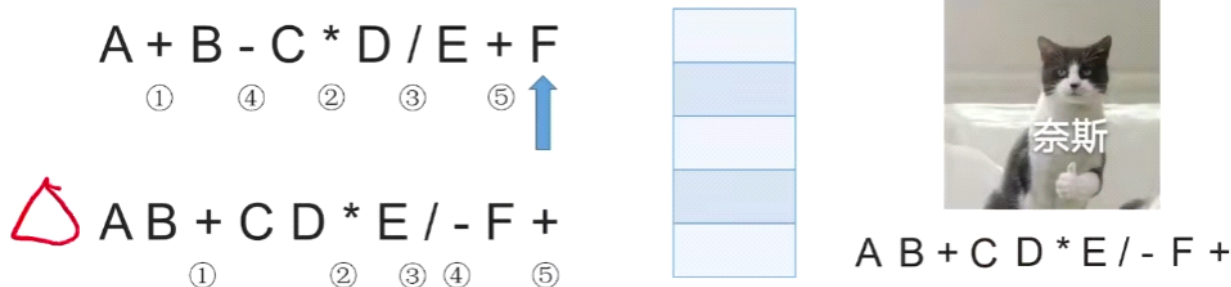
中缀表达式转后缀表达式 (机算)

初始化一个栈, 用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素, 直到末尾。可能遇到三种情况:

- 遇到操作数。直接加入后缀表达式。
- 遇到界限符。遇到“(”直接入栈; 遇到”)”则依次弹出栈内运算符并加入后缀表达式, 直到弹出“(”为止。注意:“(”不加入后缀表达式。
- 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符, 并加入后缀表达式, 若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后, 将栈中剩余运算符依次弹出, 并加入后缀表达式。



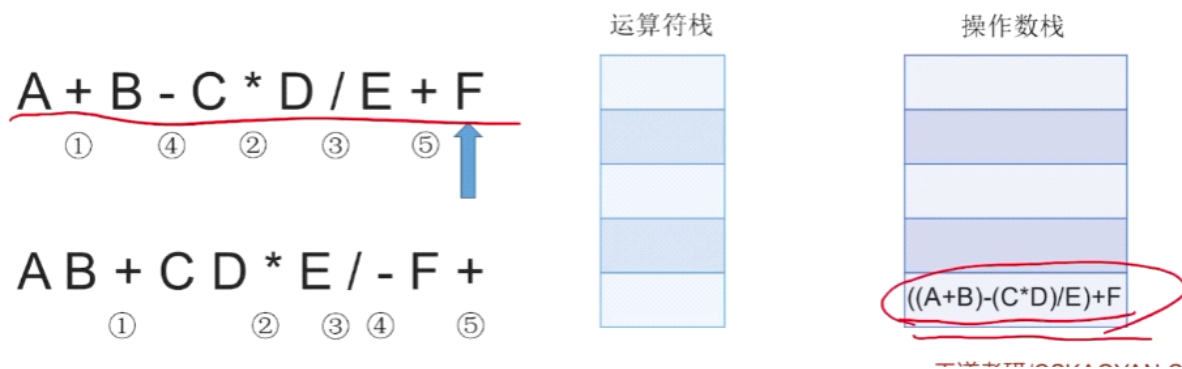
5、中缀表达式的计算:

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）



用栈实现中缀表达式转后缀表达式：

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到操作数。直接加入后缀表达式。
- ② 遇到界限符。遇到“(”直接入栈；遇到“)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

用栈实现后缀表达式的计算：

① 从左往右扫描下一个元素，直到处理完所有元素

② 若扫描到操作数则压入栈，并回到①；否则执行③

③ 若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

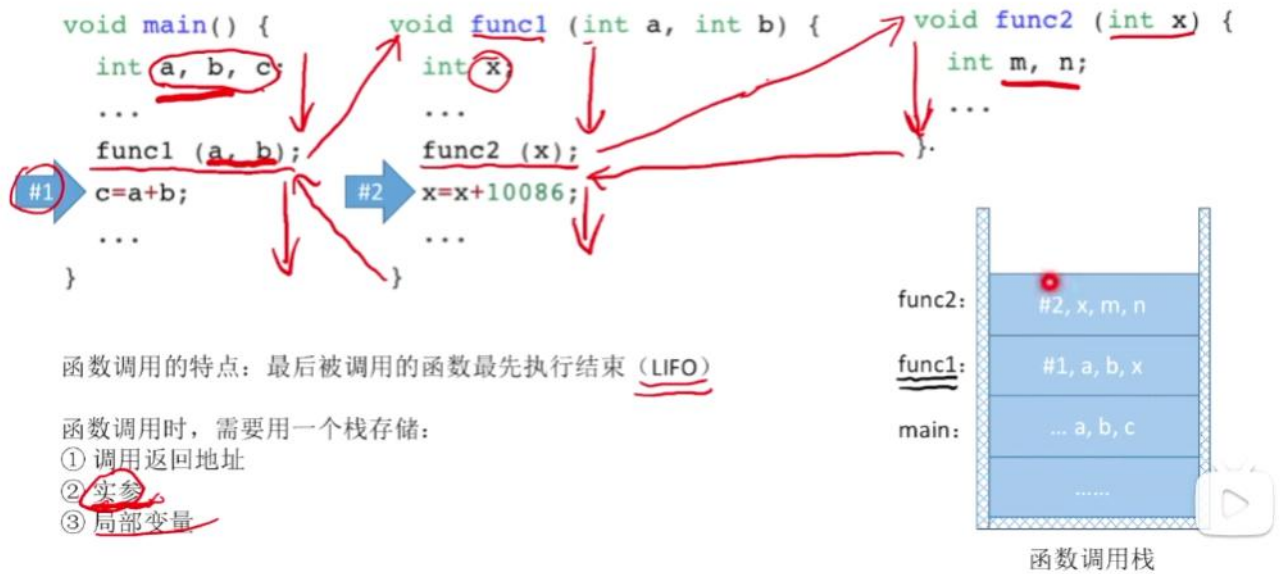
若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

十、栈在递归中的应用

1、函数调用栈：函数在调用时满足最先创建的最后释放，最后创建的最先释放，在调用函数的时候需要用到一个栈来存储这个函数所涉及到的返回地址、实参、局部变量等信息。

函数调用背后的过程



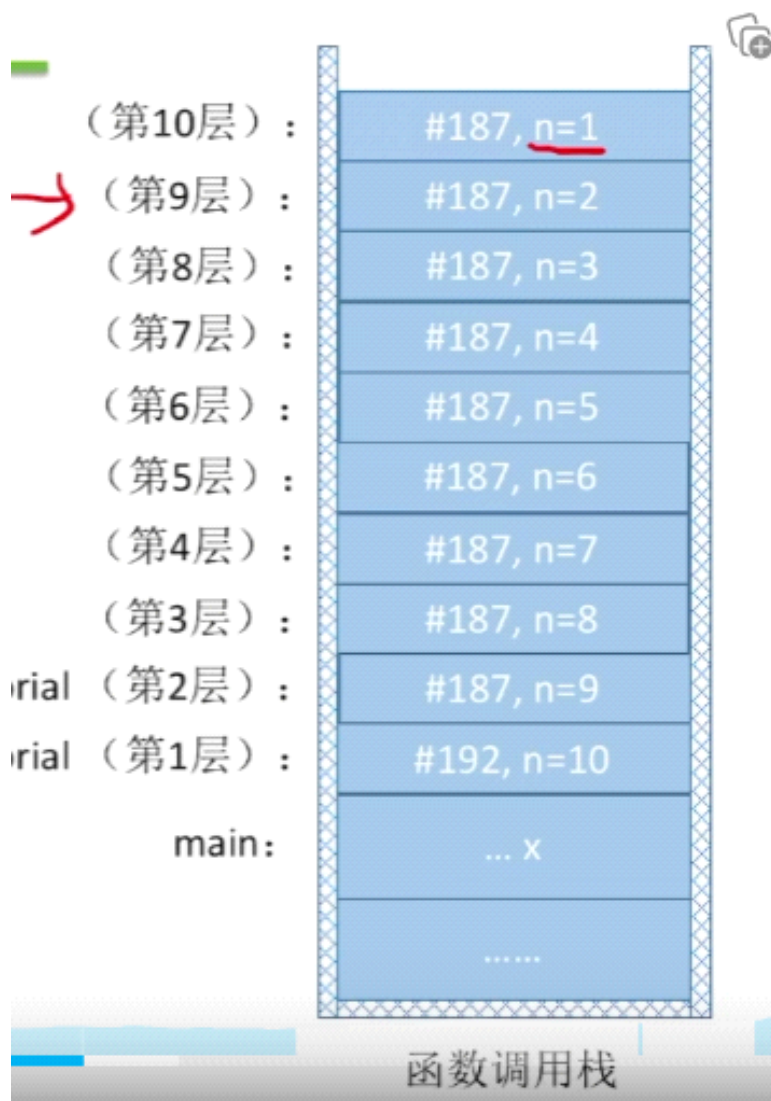
2、递归算法：可以把原始问题转换为属性相同，但是规模较小的问题。例如求阶乘和斐波那契数列。

3、递归算法求阶乘：

```

int factorial(int n){
    if(n==0||n==1) return 1;
    else return n*factorial(n-1);
}

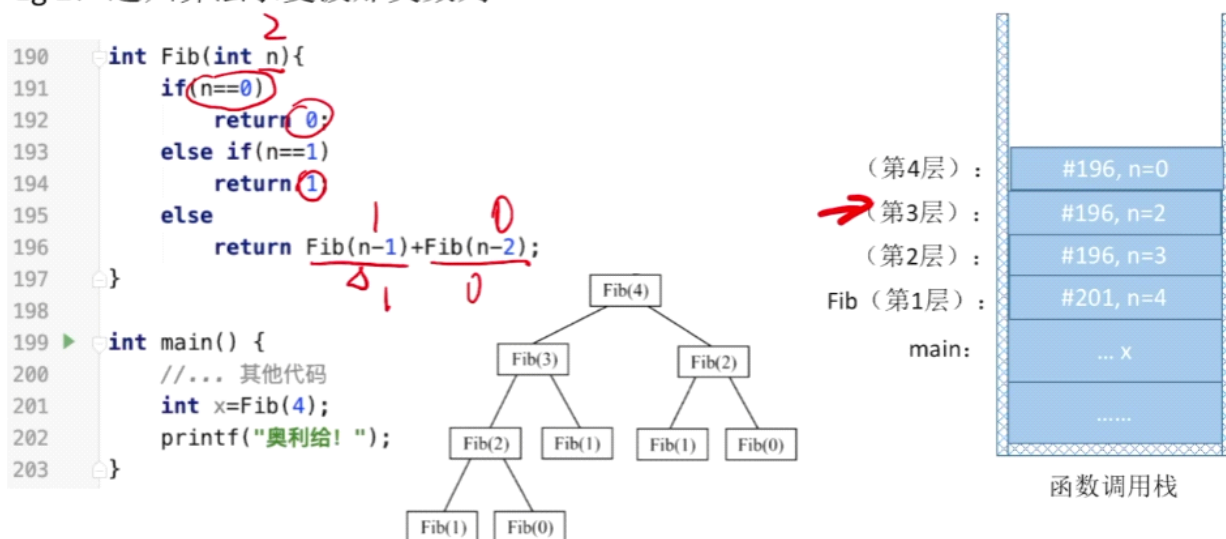
int main(){
    int x=factorial(10);
}
    
```



此代码在函数调用栈中创建了10层函数，递归层数太多可能会导致栈溢出。

4、递归算法求斐波那契数列：

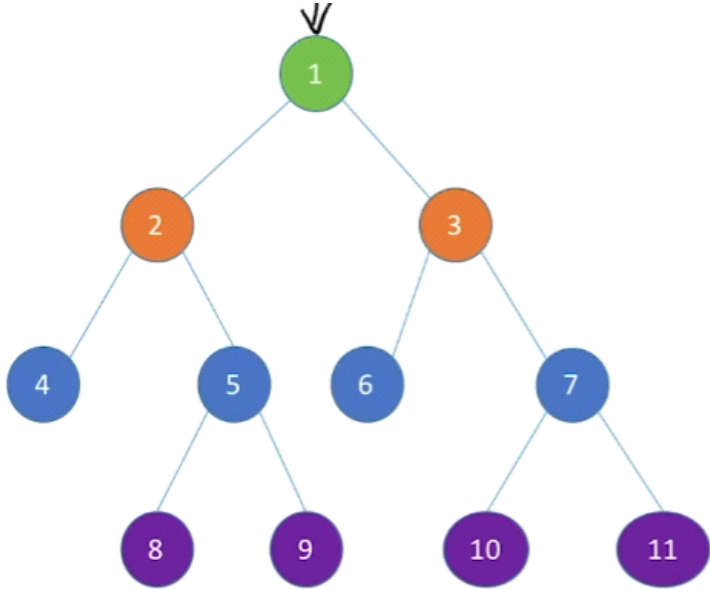
Eg 2: 递归算法求斐波那契数列



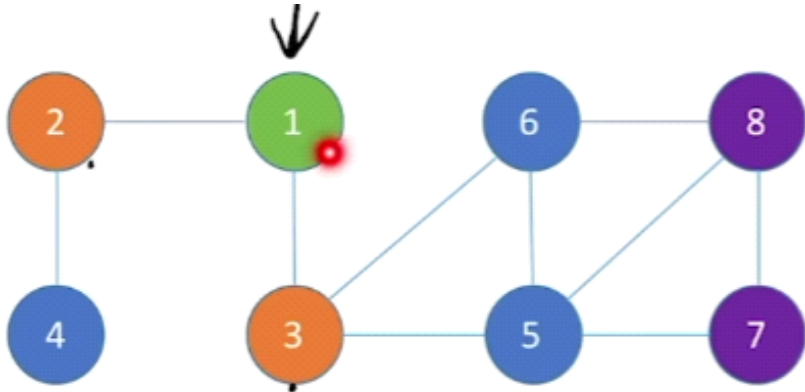
十一、队列的应用

1、树的层次遍历：如图，要对这图进行层次遍历，可以创建一个队列，1先入队，然后1的左右孩子入队，

入完队后1出队。然后2的左右孩子入队，入队完2出队。然后3的左右孩子入队，入队完3出队，以此直到所有元素出队为止。（没有左右孩子就直接出队）



2、图的广度优先遍历：如图，要遍历这个图，可以创建一个队列，从1入队开始，和1连接的节点2和3入队，然后1出队；然后和2连接的4节点入队，2出队；然后和3连接的5和6入队，3出队；然后4没有连接节点直接出队，由此直到队列没有元素为止。



3、队列在操作系统中的应用：当多个进程争着抢着有限的系统资源时，通常用队列的先来先服务策略（FCFS算法）。

Eg: CPU资源的分配



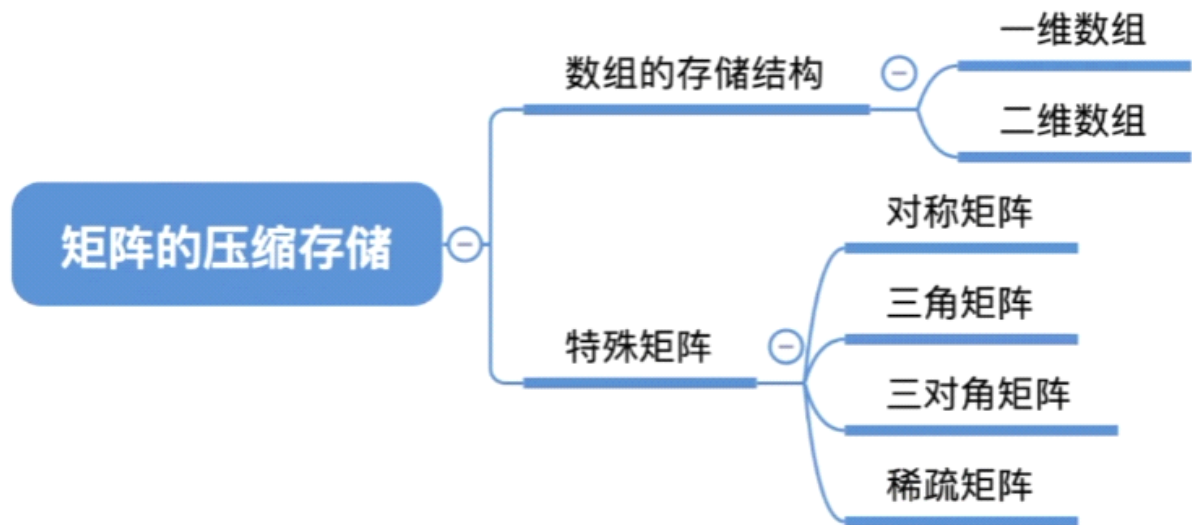
就绪进程队列



4、打印店当多台电脑连接一台打印机时候，系统会开辟一片缓冲区，按照先来先打印原则也能实现，能缓解打印机和主机的速度不匹配问题。

十二、特殊矩阵的压缩存储

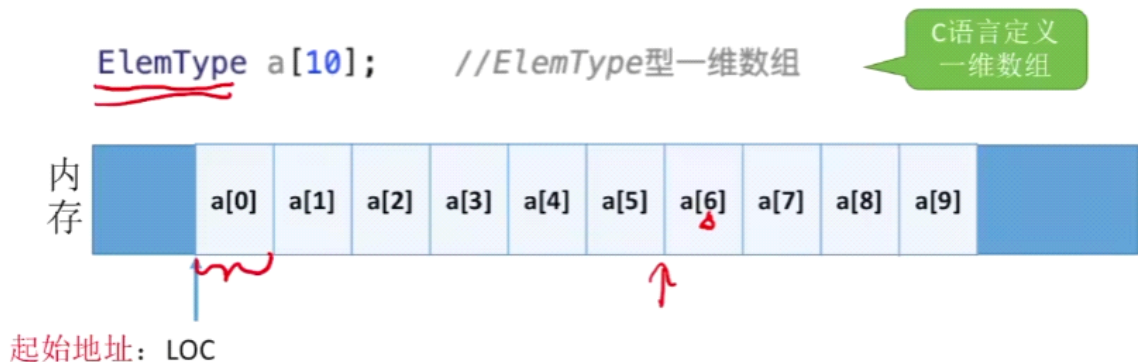
数组类型如下：



1、一维数组的存储结构：

± 哔哩哔哩

一维数组的存储结构



各数组元素大小相同，且物理上连续存放。

数组元素 $a[i]$ 的存放地址 = $\text{LOC} + i * \text{sizeof}(\text{ElemType})$ ($0 \leq i < 10$)

注：除非题目特别说明，否则数组下标默认从0开始

注意审题！
易错！

2、二维数组的存储结构：（图9行优先存储）（图10列优先存储）

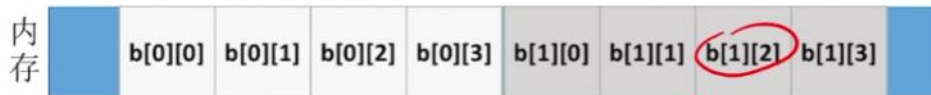
二维数组的存储结构

`ElemType b[2][4];` //2行4列的二维数组

C语言定义
二维数组

b[0][0]	b[0][1]	b[0][2]	b[0][3]
b[1][0]	b[1][1]	b[1][2]	b[1][3]

逻辑视角



行优先
存储

起始地址: LOC

M行N列的二维数组 $b[M][N]$ 中, 若按行优先存储, 则

$$b[i][j] \text{ 的存储地址} = \text{LOC} + (i * N + j) * \text{sizeof}(\text{ElemType})$$

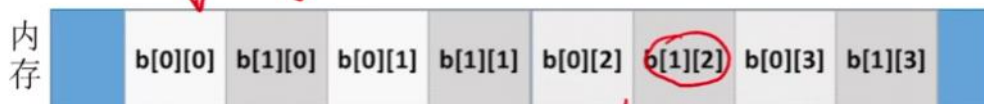
二维数组的存储结构

`ElemType b[2][4];` //2行4列的二维数组

C语言定义
二维数组

b[0][0]	b[0][1]	b[0][2]	b[0][3]
b[1][0]	b[1][1]	b[1][2]	b[1][3]

逻辑视角



列优先
存储

起始地址: LOC

M行N列的二维数组 $b[M][N]$ 中, 若按列优先存储, 则

$$b[i][j] \text{ 的存储地址} = \text{LOC} + (j * M + i) * \text{sizeof}(\text{ElemType})$$

3、普通矩阵的存储: 直接使用二维数组存储, 注意数组下标从0开始, 矩阵下标从1开始。

普通矩阵的存储

$a_{1,1}$	$a_{1,2}$	$a_{1,3}$		$a_{1,n-1}$	$a_{1,n}$
$a_{2,1}$	$a_{2,2}$	$a_{2,3}$		$a_{2,n-1}$	$a_{2,n}$
$a_{3,1}$	$a_{3,2}$	$a_{3,3}$		$a_{3,n-1}$	$a_{3,n}$
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
$a_{m,1}$	$a_{m,2}$	$a_{m,3}$		$a_{m,n-1}$	$a_{m,n}$

特殊矩阵

对称矩阵

三角矩阵

三对角矩阵

稀疏矩阵

都是方阵

可用二维数组存储

注意：描述矩阵元素时，行、列号通常从1开始；而描述数组时通常下标从0开始
(具体看题目给的条件，注意审题！)

某些特殊矩阵可以压缩存储空间

4、对称矩阵的存储：可以只存储主对角线+上三角或下三角区域（图11）以主对角线+下三角区域为例子，按照行优先原则将各个元素存入一维数组之中，一维数组的长度应该设置为 $(1+n)*n/2$ 。通过映射函数可以将已经存好的一维数组映射为矩阵。

$a_{1,1}$	$a_{1,2}$	$a_{1,3}$		$a_{1,n-1}$	$a_{1,n}$
$a_{2,1}$	$a_{2,2}$	$a_{2,3}$		$a_{2,n-1}$	$a_{2,n}$
$a_{3,1}$	$a_{3,2}$	$a_{3,3}$		$a_{3,n-1}$	$a_{3,n}$
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
$a_{n-1,1}$	$a_{n-1,2}$	$a_{n-1,3}$		$a_{n-1,n-1}$	$a_{n-1,n}$
$a_{n,1}$	$a_{n,2}$	$a_{n,3}$		$a_{n,n-1}$	$a_{n,n}$

若 n 阶方阵中任意一个元素 $a_{i,j}$ 都有 $a_{i,j} = a_{j,i}$ 则该矩阵为对称矩阵

普通存储： $n*n$ 二维数组

上三角区： $i < j$

下三角区： $i > j$

主对角线： $i = j$

对称矩阵的压缩存储

$a_{1,1}$	$a_{1,2}$	$a_{1,3}$		$a_{1,n-1}$	$a_{1,n}$
$a_{2,1}$	$a_{2,2}$	$a_{2,3}$		$a_{2,n-1}$	$a_{2,n}$
$a_{3,1}$	$a_{3,2}$	$a_{3,3}$		$a_{3,n-1}$	$a_{3,n}$
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
$a_{n-1,1}$	$a_{n-1,2}$	$a_{n-1,3}$		$a_{n-1,n-1}$	$a_{n-1,n}$
$a_{n,1}$	$a_{n,2}$	$a_{n,3}$		$a_{n,n-1}$	$a_{n,n}$

策略：只存储主对角线+下三角区

按行优先原则将各元素存入一维数组中。

$B[0]$ $B[1]$ $B[2]$ $B[3]$ $B[\frac{n(n+1)}{2}-1]$

$a_{1,1}$	$a_{2,1}$	$a_{2,2}$	$a_{3,1}$		$a_{n,n-1}$	$a_{n,n}$
-----------	-----------	-----------	-----------	-------	-------------	-----------

矩阵下标 $\rightarrow$ 一维数组下标

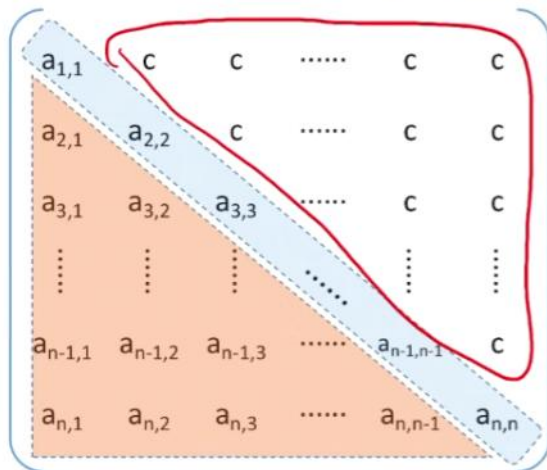
$a_{i,j} \rightarrow B[k]$

$a_{i,j} = a_{j,i}$ (对称矩阵性质)

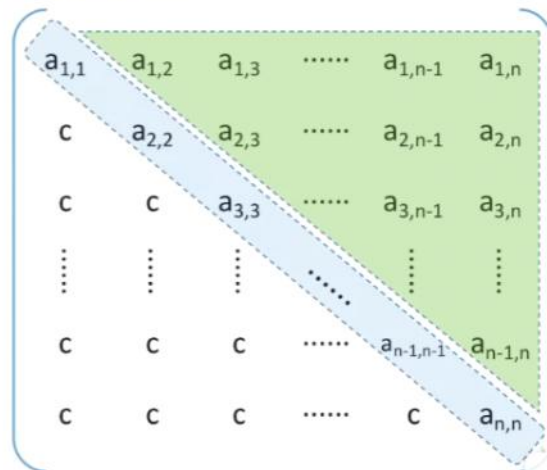
$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{j(j-1)}{2} + i - 1, & i < j \text{ (上三角区元素 } a_{ij} = a_{ji} \text{)} \end{cases}$$

5、三角矩阵的压缩：三角矩阵分为上三角矩阵和下三角矩阵

三角矩阵的压缩存储

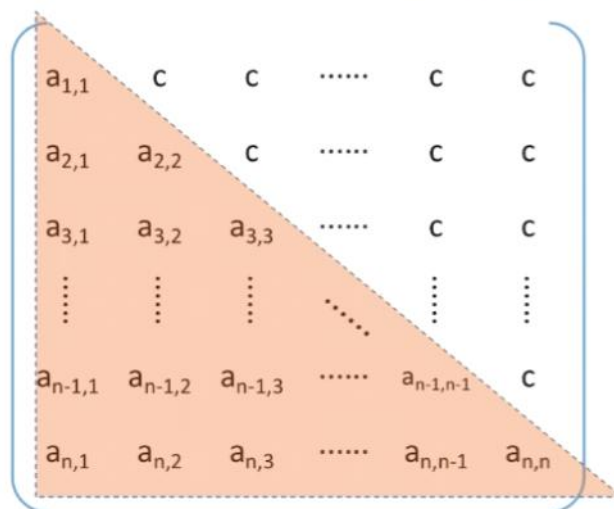


下三角矩阵：除了主对角线和下三角区，其余的元素都相同



上三角矩阵：除了主对角线和上三角区，其余的元素都相同

因此要进行压缩存储，可以按照行优先原则将橙色区域元素存入一维数组之中，并且在最后一个位置存储重复的一个值。



下三角矩阵：除了主对角线和下三角区，其余的元素都相同

压缩存储策略：按**行优先**原则将橙色区元素存入一维数组中。并在**最后一个位置存储常量c**

B[0] B[1] B[2] B[3] B[$\frac{n(n+1)}{2}-1$] B[$\frac{n(n+1)}{2}$]

a _{1,1}	a _{2,1}	a _{2,2}	a _{3,1}		a _{n,n}	c
------------------	------------------	------------------	------------------	-------	------------------	---



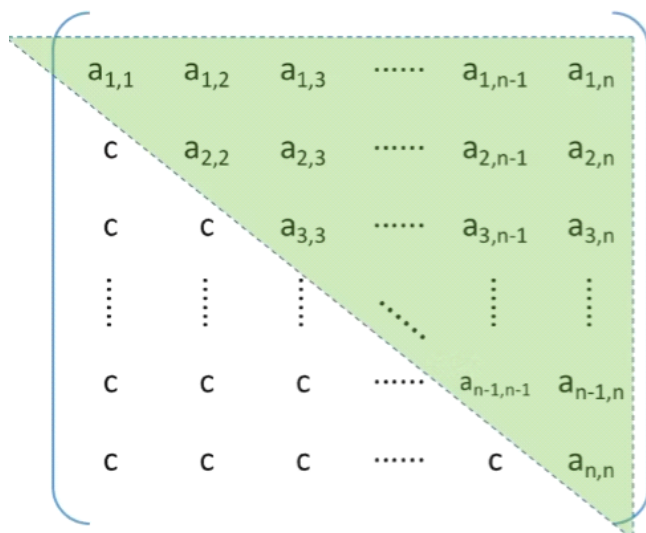
矩阵下标 → 一维数组下标

a_{i,j} (i ≥ j) → B[k]

Key: 按**行优先**的原则，a_{i,j}是第几个元素？

$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{ (下三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i < j \text{ (上三角区元素)} \end{cases}$$

三角矩阵的压缩存储



上三角矩阵：除了主对角线和上三角区，其余的元素都相同

压缩存储策略：按**行优先**原则将绿色区元素存入一维数组中。并在**最后一个位置**存储常量**c**

B[0] B[1] B[2] B[3] ... B[$\frac{n(n+1)}{2}-1$] B[$\frac{n(n+1)}{2}$]

a _{1,1}	a _{1,2}	a _{1,3}	a _{1,4}		a _{n,n}	c
------------------	------------------	------------------	------------------	-------	------------------	---



矩阵下标 → 一维数组下标

a_{i,j} (i ≤ j) → B[k]

Key: 按**行优先**的原则，a_{i,j}是第几个元素？

$$k = \begin{cases} \frac{(i-1)(2n-i+2)}{2} + (j-i), & i \leq j \text{ (上三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i > j \text{ (下三角区元素)} \end{cases}$$

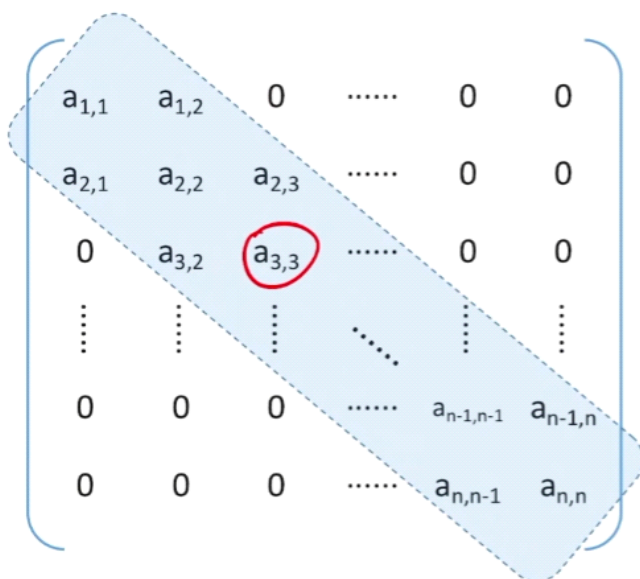


6、三对角矩阵的压缩存储：

三对角矩阵又称带状矩阵，当|i-j|>1时元素都为0。要压缩存储可以按照行优先的原则只存储带状部分。

三对角矩阵

三对角矩阵的压缩存储



三对角矩阵，又称**带状矩阵**：

当 |i-j|>1 时，有 a_{i,j} = 0 (1 ≤ i, j ≤ n)

压缩存储策略：

按**行优先**（或列优先）原则，只存储带状部分

B[0] B[1] B[2] B[3] ... B[3n-3]

a _{1,1}	a _{1,2}	a _{2,1}	a _{2,2}		a _{n,n-1}	a _{n,n}
------------------	------------------	------------------	------------------	-------	--------------------	------------------



矩阵下标 → 一维数组下标

a_{i,j} (|i-j| ≤ 1) → B[k]

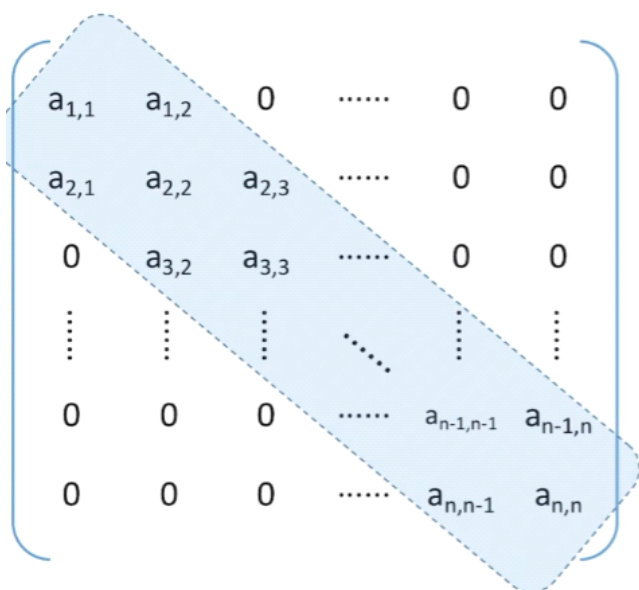
Key: 按**行优先**的原则，a_{i,j}是第几个元素？

前i-1行共 $3(i-1)-1$ 个元素

a_{i,j}是 i 行第 j-i+2 个元素

a_{i,j}是第 2i+j-2 个元素

三对角矩阵的压缩存储



若已知数组下标k, 如何得到 i, j ?

$B[k] \rightarrow a_{i,j}$

第 $k+1$ 个元素, 在第几行? 第几列?

前 $i-1$ 行共 $3(i-1)-1$ 个元素

前 i 行共 $3i-1$ 个元素

显然, $3(i-1)-1 < k+1 \leq 3i-1$

$$i \geq (k+2)/3$$

可以理解为“刚好”大于等于

$$i = \lceil (k+2)/3 \rceil$$

向上取整即可满足
“刚好”大于等于

王道书的计算逻辑: $3(i-1)-1 \leq k < 3i-1$

$$i \leq (k+1)/3+1$$

可以理解为“刚好”小于等于

$$i = \lfloor (k+1)/3+1 \rfloor$$

向下取整即可满足
“刚好”小于等于

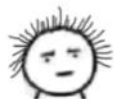
中国知网 CNKI KAOYAN.COM

7、稀疏矩阵的压缩存储:

稀疏矩阵是零元素占大部分的矩阵。压缩存储可以使用三元组 (行、列、值) 存储, 这样会失去随机存取的特性。除此之外也可以使用十字链表法存储。

稀疏矩阵的压缩存储

0	0	4	0	0	5
0	3	0	9	0	0
0	0	0	0	7	0
0	2	0	0	0	0
0	0	0	0	0	0



青少年

稀疏矩阵: 非零元素远远少于矩阵元素的个数

压缩存储策略:

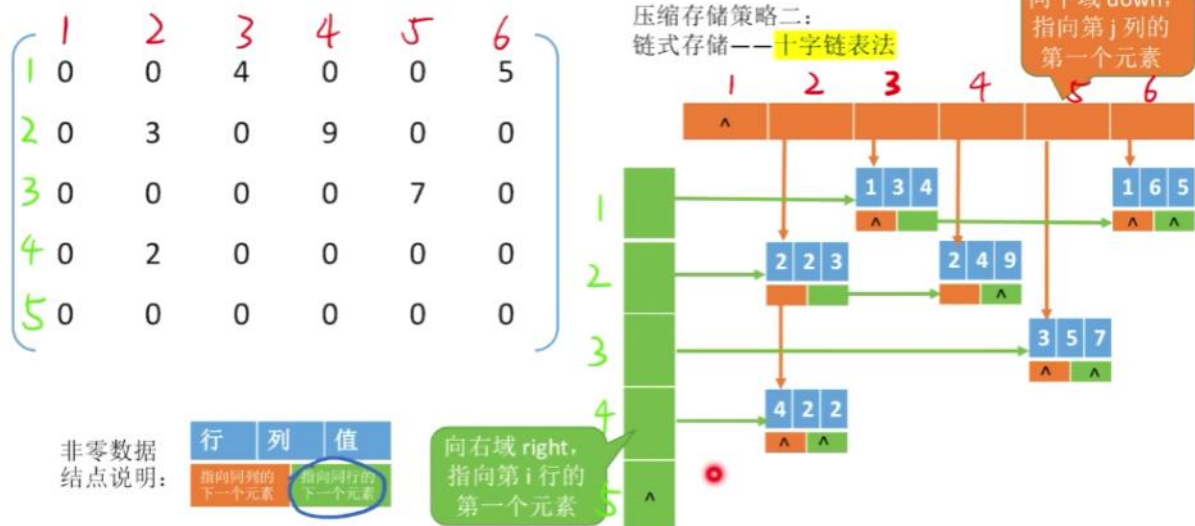
顺序存储——三元组 <行, 列, 值>

i (行)	j (列)	v (值)
1	3	4
1	6	5
2	2	3
2	4	9
3	5	7
4	2	2

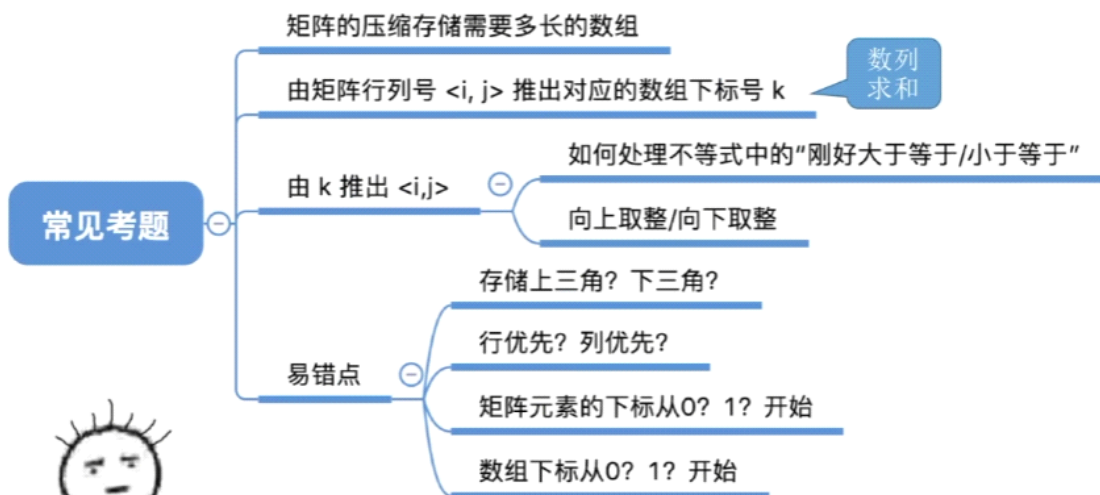
(注: 此处行、列标从1开始)

struct {
 i
 j
 v
}

稀疏矩阵的压缩存储



知识回顾与重要考点



问题

是否忘了，等差数列求和???

对于这类题型，学长不太建议大家死记公式，其实画画图找找对应关系就能解决。
方法如下：

- ① 画出二维数组/矩阵的**基本形状**。注意，下标从0/1开始？，行/列优先？
- ② **列出三列**基本信息，分别为数组元素，第几个，存储位置。
- ③ 根据数组元素下标====》第几个====》存储位置/对应数组下标

我们以本题示例讲解：

① 画出二维数组/矩阵的**基本形状**。

A=[
A[0][0] A[0][1] A[0][2] A[0][7] A[0][8] A[0][9]
A[1][0]
.....
.....
.....
.....
.....
.....
A[8][0] A[8][5] A[8][9]
]



② 列出**三列**基本信息

数组元素	第几个	存储位置/对应数组下标
A[0][0]	第1个	SA
A[0][1]	第2个	SA+3
A[0][2]	第3个	SA+3*2
⋮		
⋮		
⋮		
A[8][5]	第?个	

如果题目没有明确说明，一般是行优先，下标从0开始。

- ③ A[8][5]前面有A[0]~A[7]共八行元素，每行10个，总共 $8 \times 10 = 80$ 个。
A[8][0]~A[8][5]总共有6个元素，因此A[8][5]是第86个元素。

数组元素	第几个	
A[0][0]	第1个	SA
A[0][1]	第2个	SA+3
A[0][2]	第3个	SA+3*2
⋮		
⋮		
⋮		
A[8][5]	第86个	

而我们又知道，第1个元素对应SA，第2个元素对应SA+3，第3个元素对应SA+6，因此第86个元素对应 $SA + 3 \times 85 = SA + 255$ 。



串和KMP算法

2024年2月5日 14:35

一、串的定义：

串的定义

串，即**字符串**（String）是由零个或多个**字符**组成的有限序列。一般记为
 $S = 'a_1a_2\cdots a_n'$ ($n \geq 0$)

其中， S 是**串名**，单引号括起来的字符序列是串的值； a_i 可以是字母、数字或其他字符；串中字符的个数 n 称为**串的长度**。 $n = 0$ 时的串称为**空串**（用 $\emptyset$ 表示）。

例：

$S = \text{"HelloWorld!"}$

$T = \text{'iPhone 11 Pro Max'}$

注：有的地方用双引号（如Java、C）
有的地方用单引号（如Python）

子串：串中任意个**连续**的字符组成的子序列。

主串：包含子串的串。

字符在主串中的位置：字符在串中的序号。

子串在主串中的位置：子串的第一个字符在主串中的位置。

Eg: 'iPhone', 'Pro M' 是串T 的子串

Eg: T 是子串'iPhone'的主串

Eg: '1'在T中的位置是8(第一次出现)

Eg: '11 Pro'在T中的位置为8

空串 V.S 空格串：

$M = ''$

M是空串

$N = ' '$

N是由三个空格字符组成的空格串，每个空格字符占1B

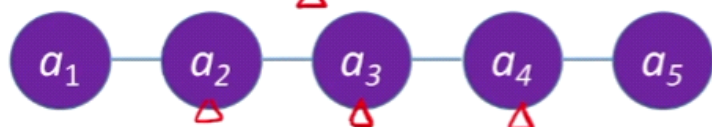
注意：位序从1开始
而不是从0开始

二、串和线性表的区别：

相比于线性表，只是里面的元素被限制为字符。操作对象不再是单个元素，而是整个字串作为一个整体操作。

串 V.S 线性表

串是一种特殊的**线性表**，数据元素之间呈**线性关系**



串的数据对象限定为**字符集**（如**中文字符**、**英文字符**、**数字字符**、**标点字符**等）

串的基本操作，如增删改查等**通常以子串为操作对象**

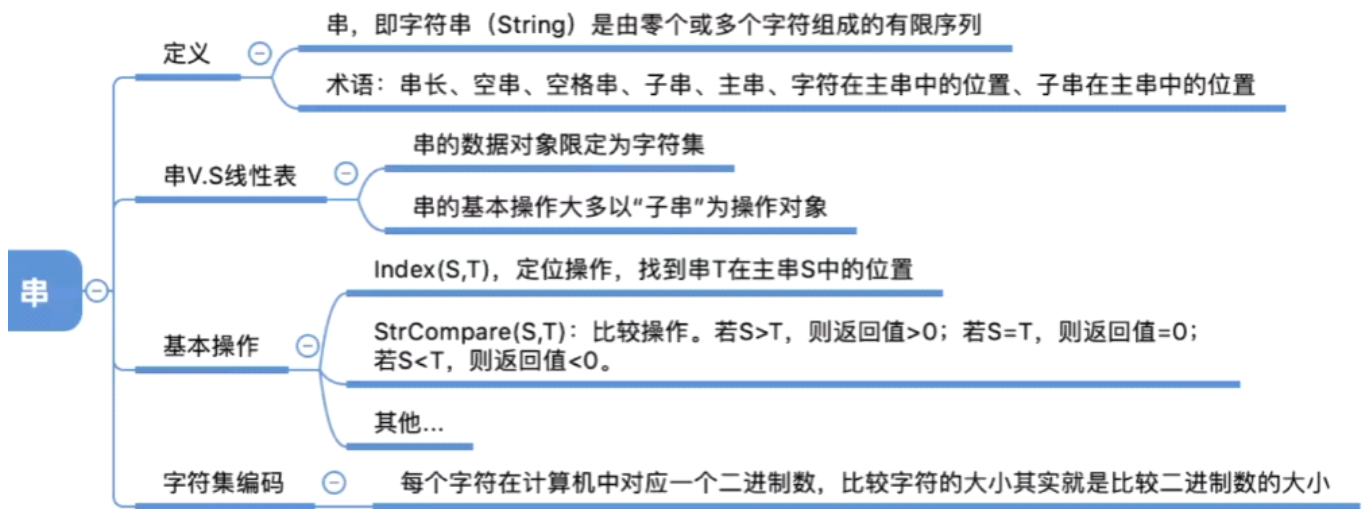


我们不一样

三、串的基本操作：

赋值（给串初始化）、复制（串S复制给串T）、判空（判断空串）、串长（返回串的元素个数）、清空（变成

空串)、销毁(空串+释放内存)、串连接(字符串的拼接)、求子串(返回第pos个字符开始长度为len的串)、定位操作(寻找子串在主串中的位置)、比较(比较字符串的大小, 英文用ASCII, 中文和其他语言用Unicode)



四: 串的存储结构

```
#define MAX 255
typedef struct{                //顺序存储结构体
    char ch[MAX];
    int length;
}SString;
```



```
typedef struct{                //顺序存储之动态数组存储
    char *ch;                  //指向分配内存的基地址
    int length;
```

```

}HString;
Hstring S;                //使用方法
S.ch=(char *)malloc(MAX*sizeof(char));
S.length=0;

typedef struct StringNode{
    char ch[4];            //可以每个节点存多个字符提升存储密度
    struct StringNode *next;
}StringNode , *String;

```

五、串的基本操作

1、求pos开始长度为len的子串：

```

bool SubString(SString &Sub,SString S,int pos,int len){
    if(pos+len-1>S.length) return false;    //子串范围越界了
    for(int i=pos;i<pos+len;i++)
        Sub.ch[i-pos+1]=S.ch[i];            //子串数据赋值给另一个串Sub
    Sub.length=len;                          //Sub串长为len
    return true;
}

```

2、比较S和T，S>T返回值>0，S<T返回值<0，S=T返回值=0：

```

int Strcompare(SString S,SString T){
    for(int i=1;i<=S.length&& i<=T.length;i++)
        if(S.ch[i]!=T.ch[i])                //有不不同的字符就根据ASCII码对比
            return S.ch[i]-T.ch[i];
    }
    return S.length-T.length;               //所有字符都相同就长度长的串更大
}

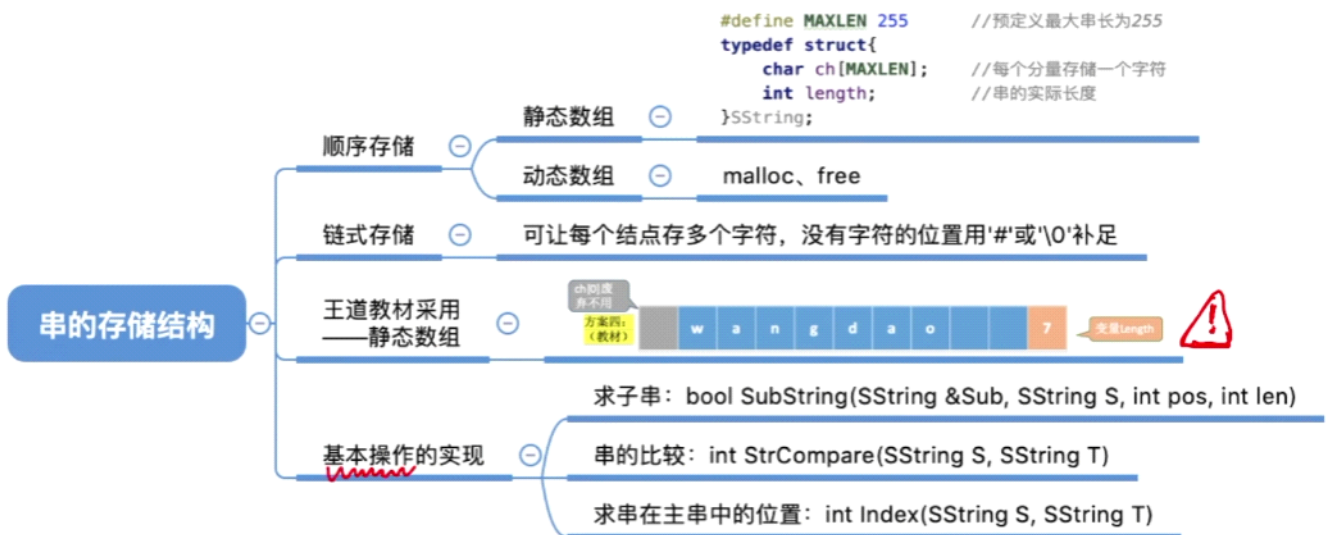
```

3、定位操作：在主串S中返回它的子串T第一次出现的位置，否则为0

```

int Index(SString S,SString T){
    Int i=1; n=StrLength(S),m=StrLength(T);
    SString temp;
    while(i<=n-m+1){                        //循环次数为S的长度减去T的长度+1
        SubString(temp,S,i,m);              //从S里面的第i个下标开始长度为T长度的子串，依次赋值到temp
        if(StrCompare(temp,T)!=0) i++;       //对比temp和T是否相等
        else return i;
    }
    return 0;                               //匹配失败
}

```

4、字符串的模式匹配算法——字符串里面找出某一段内容相同的子串

//朴素模式匹配算法

//(图6)主串中找出所有和它长度相等的子串——对比，最多对比 $n-m+1$ 次

```
int Index(SString S, SString T){
```

```
    int i=1, j=1;
```

```
    while(i<=S.length && j<=T.length){ //保证比较的是对应位置的元素
```

```
        if(S.ch[i]==T.ch[j]){ //相等的话比较下一对元素
```

```
            i++; j++;
```

```
        }
        else{
```

```
            i=i-j+2; //不相等的话i指向下一个子串的第一个位置
```

```
            j=1; //j回到匹配子串的第一个位置
```

```
        }
```

```
    }
```

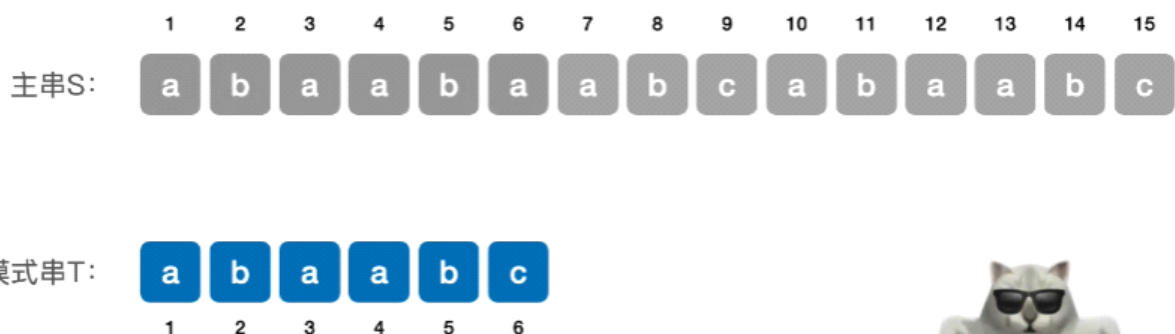
```
    if(j>T.length) return i-T.length;
```

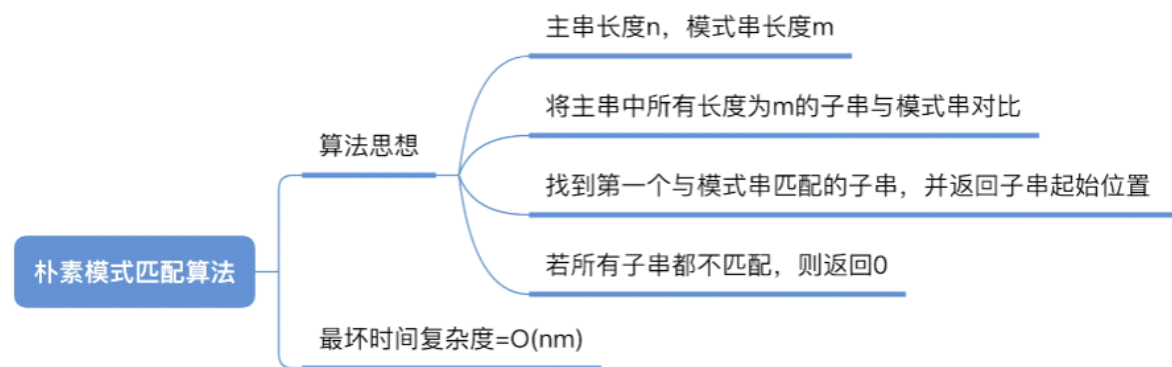
//匹配完j超过了T子串的长度说明所有元素都匹配，返回S子串当前的第一个位置

```
    else return 0;
```

```
}
```

最坏时间复杂度 $O(n*m)$





//KMP算法

```
int Index_KMP(SString S,SString T,int next[]){
    int i=1,j=1;
    while(i<=S.length&& j<=T.length){
        if(j==0||S.ch[i]==T.ch[j]){
            i++; j++;
        }
        else j=next[j];
    }
    if(j>T.length)
        return i-T.length;
    else
        return 0;
}
```

最坏时间复杂度 $O(m+n)$

5、求模式串next数组:

??

树

2025年9月9日星期二 下午2:09

一、树

1、树的常考性质

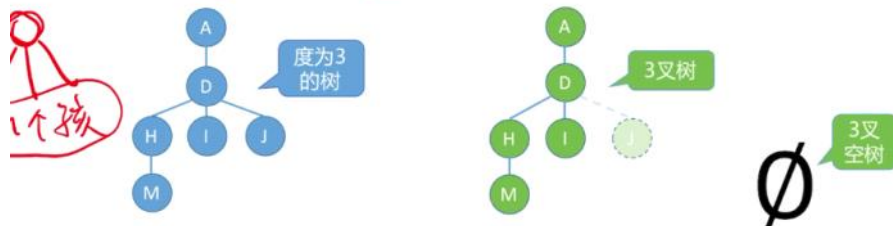
节点数=总度数+1，因为除了根节点外其他的节点像天线宝宝一样头上顶着个天线。

度为m和m叉树的区别：度为m的树至少有一个节点是m个孩子，m叉树没要求，可以是空树。

树的度——各节点的度的最大值

m叉树——每个结点最多只能有m个孩子的树

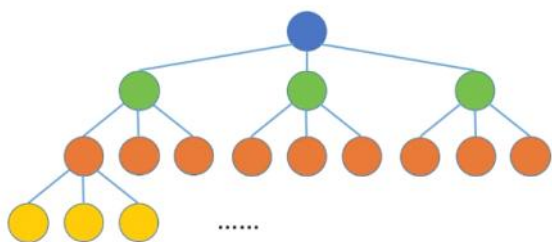
度为m的树	m叉树
任意结点的度 $\leq m$ (最多m个孩子)	任意结点的度 $\leq m$ (最多m个孩子)
至少有一个结点度 = m (有m个孩子)	允许所有结点的度都 $< m$
一定是非空树，至少有m+1个结点	可以是空树



度为m的树第i层最多有 m^{i-1} 个节点。

常见考点3：度为m的树第i层至多有 m^{i-1} 个结点 ($i \geq 1$)

m叉树第i层至多有 m^{i-1} 个结点 ($i \geq 1$)



第1层: m^0

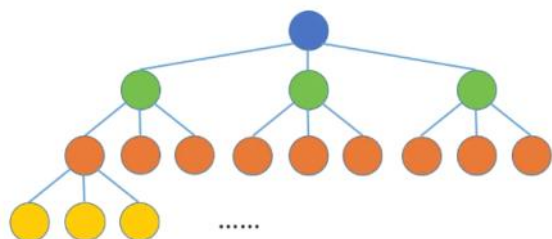
第2层: m^1 3^1

第3层: m^2 3^2

第4层: m^3

常见考点4：高度为h的m叉树至多有 $\frac{m^h-1}{m-1}$ 个结点。

等比数列求和公式: $a + aq + aq^2 + \dots + aq^{n-1} = \frac{a(1-qn)}{1-q}$



第1层: m^0

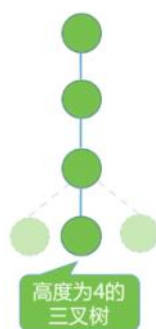
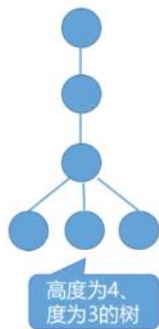
第2层: m^1

第3层: m^2

第4层: m^3

常见考点5：高度为h的m叉树至少有h个结点。

高度为h、度为m的树至少有 $h+m-1$ 个结点。



常见考点6：具有n个结点的m叉树的最小高度为 $\lceil \log_m(n(m-1)+1) \rceil$

高度最小的情况——所有结点都有m个孩子

前h-1层最多有几个结点 $\frac{m^{h-1}-1}{m-1} < n \leq \frac{m^h-1}{m-1}$ 前h层最多有几个结点

$$m^{h-1} < n(m-1) + 1 \leq m^h$$

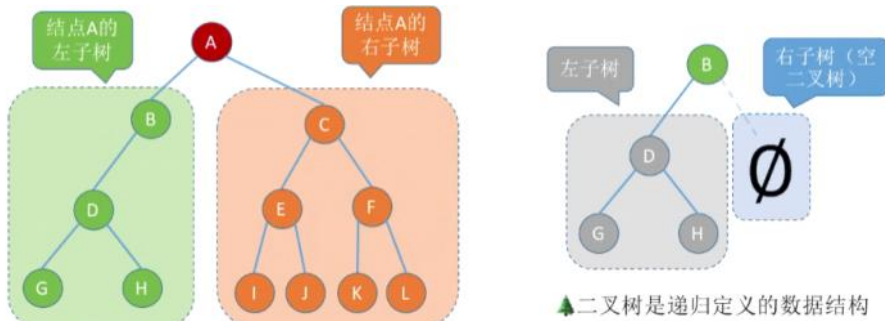
$$h-1 < \log_m(n(m-1)+1) \leq h$$

$$h_{min} = \lceil \log_m(n(m-1)+1) \rceil$$

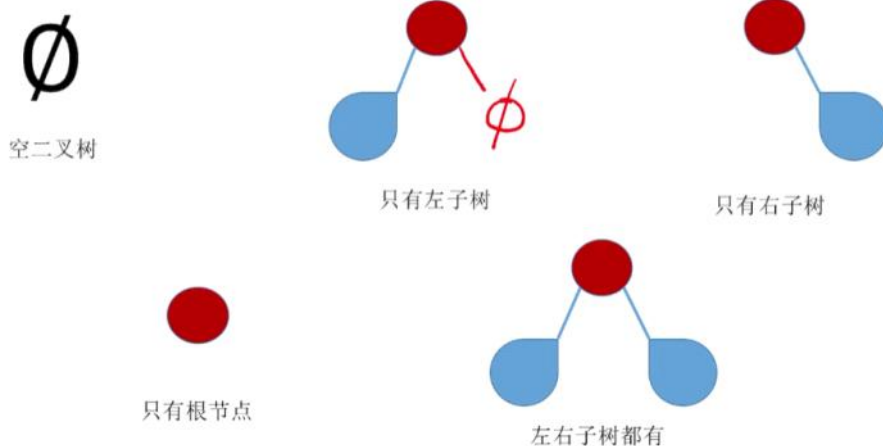
二、二叉树

1、基本概念

- (1) 二叉树是n个结点的有限集合，当n=0叫空二叉树，或者二叉树由一个根结点和左子树和右子树组成，左右子树又分别是一棵二叉树。
- (2) 二叉树的每个结点最多2个子树，左右子树不能颠倒。



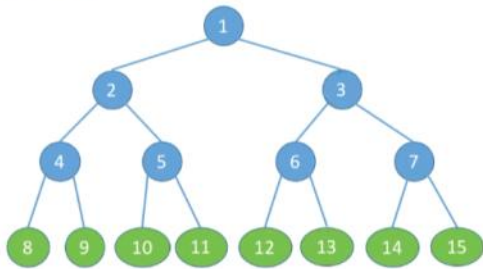
(3) 二叉树的5种状态



(4) 特殊的二叉树

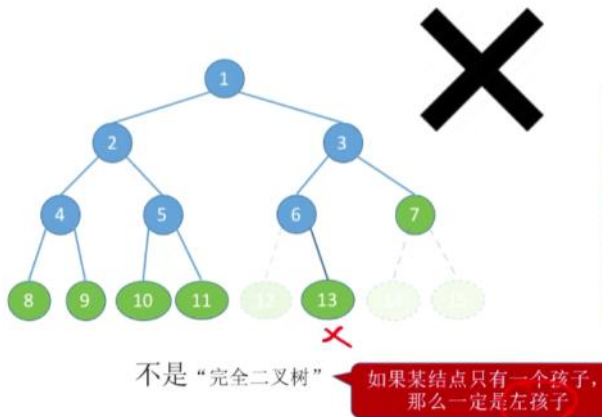
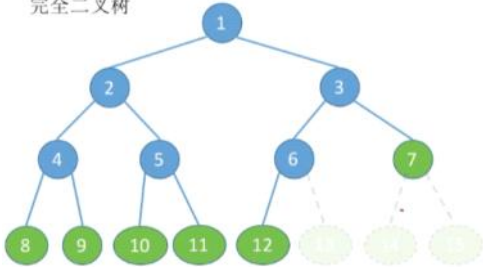
满二叉树：每个结点一定有2个孩子或者0个孩子，不能有1个孩子。注意每个节点的孩子都是自身节点编号的2倍或者2倍+1，双亲都是自身节点的1/2倍向下取整。

满二叉树。一棵高度为 h ，且含有 $2^h - 1$ 个结点的二叉树

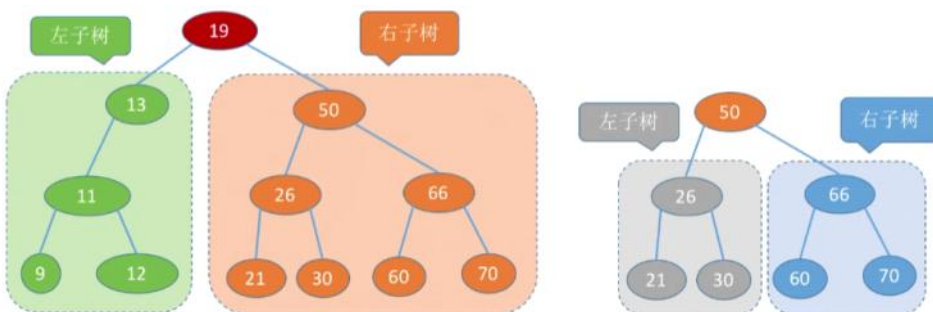


完全二叉树：满二叉树是特殊的完全二叉树，特点是**只有最后2层可能有叶子节点以及最多只有一个度为1的节点（必是左孩子）**，其孩子和父亲的顺序对应关系和满二叉树一样。

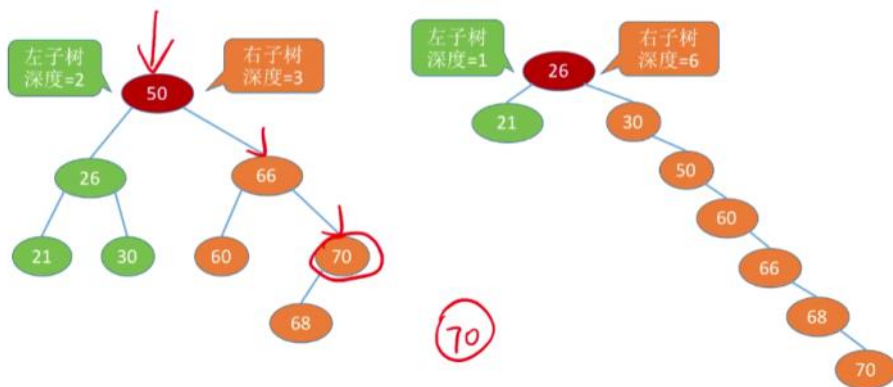
完全二叉树。当且仅当其每个结点都与高度为 h 的满二叉树中编号为 $1 \sim n$ 的结点一一对应时，称为完全二叉树



二叉排序树：二叉排序树的左子树上所有节点关键字都小于根节点，右子树上所有节点的关键字大于根节点，并且左子树上的所有关键字都小于右子树的关键字。二叉排序树可以用于元素的排序和搜索。



平衡二叉树：树上任意一个节点的**左子树和右子树的深度之差不超过1**。左右子树追求平衡。高度尽可能的低能提高查找速度。



2、二叉树的常考性质

常见考点1：设非空二叉树中度为0、1和2的结点个数分别为 n_0 、 n_1 和 n_2 ，则 $n_0 = n_2 + 1$
(叶子结点比二分支结点多一个)

假设树中结点总数为 n ，则

$$\textcircled{1} n = n_0 + n_1 + n_2$$

$$\textcircled{2} n = n_1 + 2n_2 + 1$$

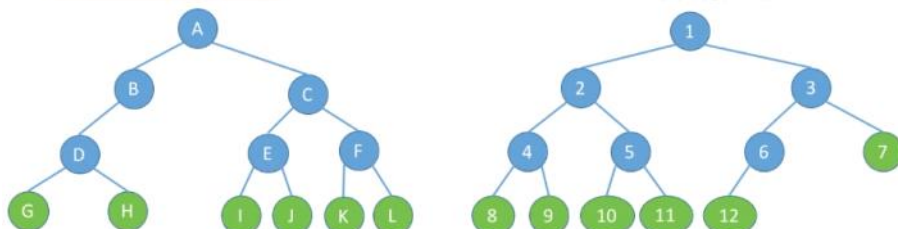


$$n_0 = n_2 + 1$$



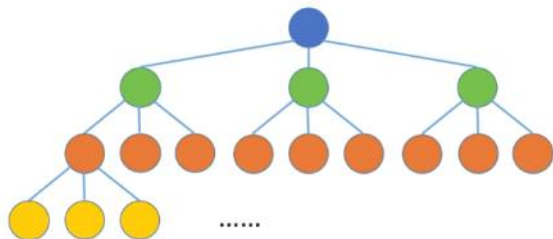
上面可都是重点

树的结点数=总度数+1



常见考点2：二叉树第 i 层至多有 2^{i-1} 个结点 ($i \geq 1$)

m 叉树第 i 层至多有 m^{i-1} 个结点 ($i \geq 1$)



第1层: m^0

第2层: m^1

第3层: m^2

第4层: m^3

常见考点1：具有 n 个 ($n > 0$) 结点的完全二叉树的高度 h 为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

高为 $h-1$ 的满二叉树共有 $2^{h-1} - 1$ 个结点

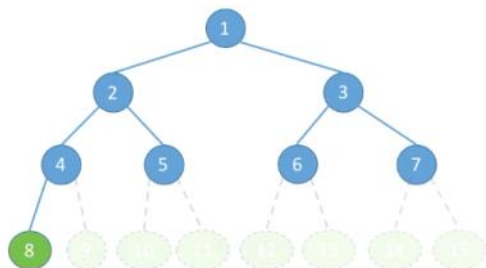
高为 h 的完全二叉树至少 2^{h-1} 个结点
至多 $2^h - 1$ 个结点



$$2^{h-1} \leq n < 2^h$$

$$h - 1 \leq \log_2 n < h$$

$$h = \lfloor \log_2 n \rfloor + 1$$



常见考点2: 对于完全二叉树, 可以由的结点数 n 推出度为0、1和2的结点个数为 n_0 、 n_1 和 n_2

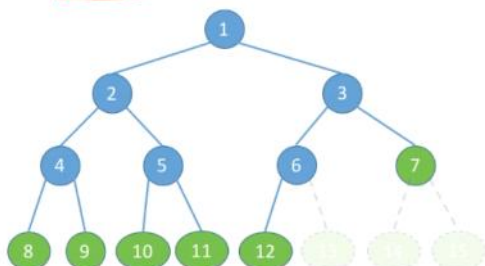
完全二叉树最多只有一个度为1的结点, 即

$n_1=0$ 或 1

$n_0 = n_2 + 1 \rightarrow n_0 + n_2$ 一定是奇数

若完全二叉树有 $2k$ 个 (偶数) 个结点, 则必有 $n_1=1$, $n_0=k$, $n_2=k-1$

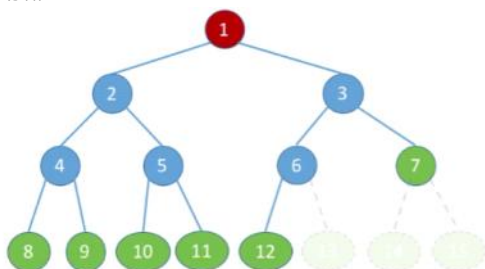
若完全二叉树有 $2k-1$ 个 (奇数) 个结点, 则必有 $n_1=0$, $n_0=k$, $n_2=k-1$



这个知道 $n_0=n_2+1$ 就行了, n_2 用 n_0 换掉。 n_1 要么0要么1, 都猜一个, 能整除的那个就是对的。

3、二叉树的存储结构

(1) 顺序存储



```
#define MaxSize 100
struct TreeNode {
    ElemType value; // 结点中的数据元素
    bool isEmpty; // 结点是否为空
};
```

TreeNode t[MaxSize];

定义一个长度为 MaxSize 的数组 t, 按照从上至下、从左至右的顺序依次存储完全二叉树中的各个结点

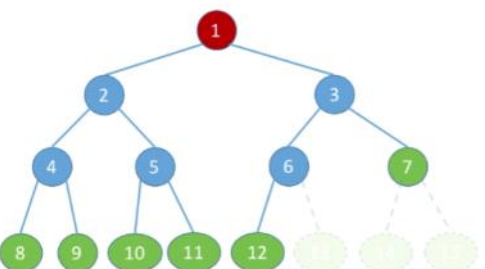


t[0] t[1] t[2]

可以让第一个位置空缺, 保证数组下标和结点编号一致

初始化时所有结点标记为空

```
for (int i=0; i<MaxSize; i++){
    t[i].isEmpty=true;
}
```



几个重要常考的基本操作:

- i 的左孩子 $\rightarrow 2i$
- i 的右孩子 $\rightarrow 2i+1$
- i 的父节点 $\rightarrow \lfloor i/2 \rfloor$
- i 所在的层次 $\rightarrow \lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

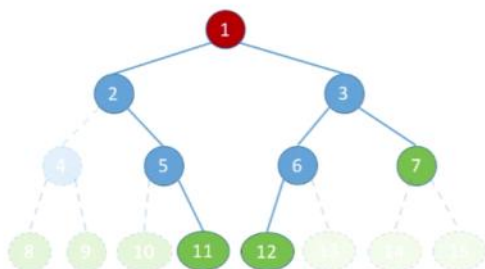
若完全二叉树中共有 n 个结点, 则

- 判断 i 是否有左孩子? $\rightarrow 2i \leq n$?
- 判断 i 是否有右孩子? $\rightarrow 2i+1 \leq n$?
- 判断 i 是否是叶子/分支结点? $\rightarrow i > \lfloor n/2 \rfloor$?



t[0] t[1] t[2]

注意顺序存储只适合完全二叉树, 如果是普通的二叉树就不能按照编号来反映其关系, 只能在存储的时候把二叉树的节点编号和完全二叉树对应起来, 但是这样会浪费大量的空间。



★ 二叉树的顺序存储中, 一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 $\rightarrow 2i$
- i 的右孩子 $\rightarrow 2i+1$
- i 的父节点 $\rightarrow \lfloor i/2 \rfloor$



(2) 链式存储

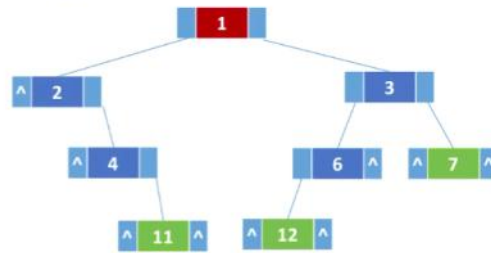
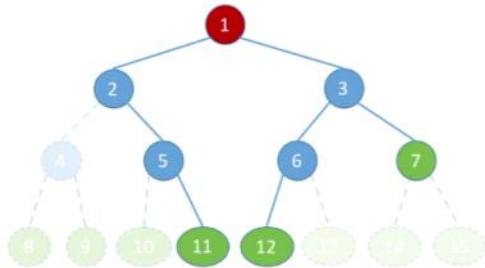
//二叉树的结点 (链式存储)

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

//数据域
//左、右孩子指针

可以用于构造
线索二叉树

★ n个结点的二叉链表共有 n+1 个空链域



```
struct ElemType{
    int value;
};
```

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

//定义一棵空树
BiTree root = NULL;

//插入根结点

```
root = (BiTree) malloc(sizeof(BiTNode));
root->data = {1};
root->lchild = NULL;
root->rchild = NULL;
```

//插入新结点

```
BiTNode * p = (BiTNode *) malloc(sizeof(BiTNode));
p->data = {2};
p->lchild = NULL;
p->rchild = NULL;
root->lchild = p; //作为根节点的左孩子
```



注意上面这种定义方式只包含了左右孩子的指针，如果要查找该节点的双亲节点，就需要从根节点开始遍历非常麻烦，因此在需要查找父节点的场景下应该多定义一个父节点指针。这个叫**三叉链表**。

//二叉树的结点 (链式存储)

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
    struct BiTNode *parent;
}BiTNode,*BiTree;
```

//数据域

//左、右孩子指针

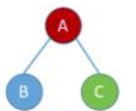
//父节点指针

三叉链表——方便
找父结点

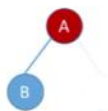
王道考研 WANGDAOYAN

三、二叉树的遍历

1、先序中序后序



先序遍历: ABC
中序遍历: BAC
后序遍历: BCA



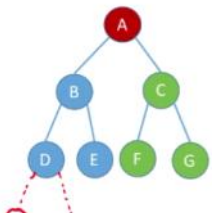
先序遍历: AB
中序遍历: BA
后序遍历: BA

右子树
为空树



先序遍历: AC
中序遍历: AC
后序遍历: CA

左子树
为空树

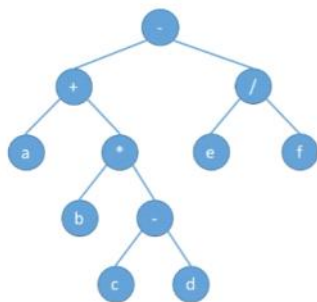


先序遍历: A B D E C F G
中序遍历: D B E A F C G
后序遍历: D E B F G C A

先序遍历: 根左右 (NLR)

中序遍历: 左根右 (LNR)

后序遍历: 左右根 (LRN)



先序遍历: $-+a*b-cd/ef$
 中序遍历: $a+b*c-d-e/f$
 后序遍历: $abcd-*+ef/-$

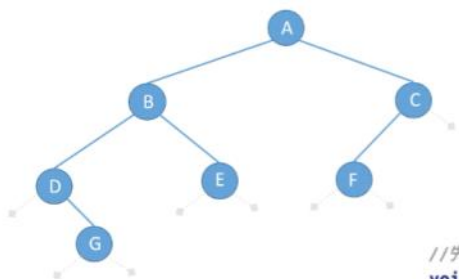
先序遍历 $\rightarrow$ 前缀表达式
 中序遍历 $\rightarrow$ 中缀表达式 (需要加界限符)
 后序遍历 $\rightarrow$ 后缀表达式

算数表达式的“分析树”

$a + b * (c - d) - e / f$

代码演示:

二叉树的遍历是通过递归来实现的, 代码只需调整调用的是根还是左还是右即可。

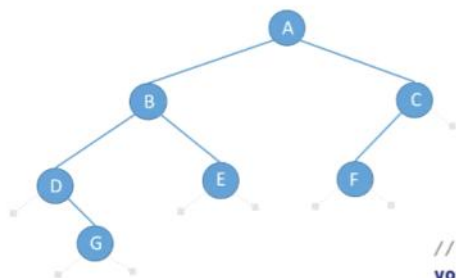


```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

```
//先序遍历
void PreOrder(BiTree T){
    if(T!=NULL){
        visit(T);           //访问根结点
        PreOrder(T->lchild); //递归遍历左子树
        PreOrder(T->rchild); //递归遍历右子树
    }
}
```

先序遍历 (PreOrder) 的操作过程如

1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ①访问根结点;
 - ②先序遍历左子树;
 - ③先序遍历右子树。

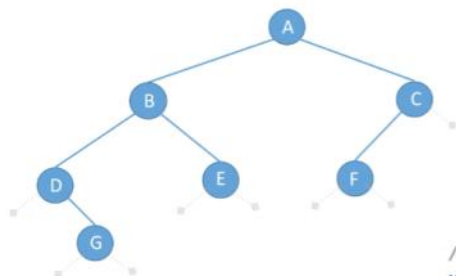


```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

```
//中序遍历
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild); //递归遍历左子树
        visit(T);           //访问根结点
        InOrder(T->rchild); //递归遍历右子树
    }
}
```

中序遍历 (InOrder) 的操作过程如下:

1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ①先序遍历左子树;
 - ②访问根结点;
 - ③先序遍历右子树。



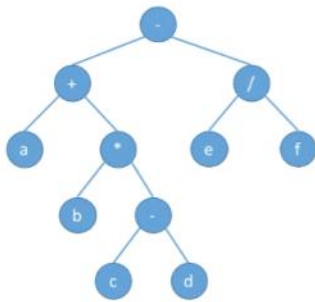
```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

```
//后序遍历
void PostOrder(BiTree T){
    if(T!=NULL){
        PostOrder(T->lchild); //递归遍历左子树
        PostOrder(T->rchild); //递归遍历右子树
        visit(T);           //访问根结点
    }
}
```

后序遍历 (InOrder) 的操作过程如下:

1. 若二叉树为空, 则什么也不做;
2. 若二叉树非空:
 - ①先序遍历左子树;
 - ②先序遍历右子树;
 - ③访问根结点。

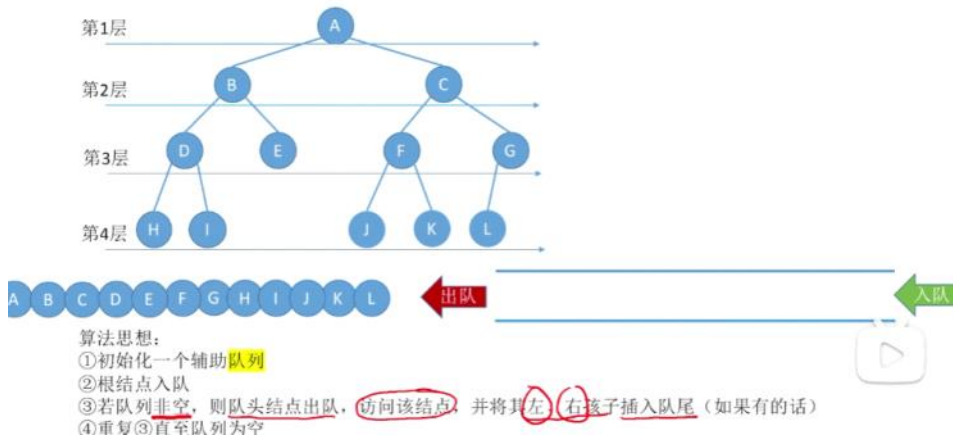
应用: 左右根顺序求树的深度



```
int treeDepth(BiTree T){
    if (T == NULL) {
        return 0;
    }
    else {
        int l = treeDepth(T->lchild);
        int r = treeDepth(T->rchild);
        //树的深度=Max(左子树深度, 右子树深度)+1
        return l>r ? l+1 : r+1;
    }
}
```

2、层次遍历 (BFS)

二叉树的层序遍历



代码实现

算法思想:

- ①初始化一个辅助队列
- ②根结点入队
- ③若队列非空, 则队头结点出队, 访问该结点, 并将其左、右孩子插入队尾 (如果有的话)
- ④重复③直至队列为空

```
//层序遍历
void LevelOrder(BiTree T){
    LinkQueue Q;
    InitQueue(Q);
    BiTree p;
    EnQueue(Q, T);
    while(!IsEmpty(Q)){
        DeQueue(Q, p);
        visit(p);
        if(p->lchild!=NULL)
            EnQueue(Q, p->lchild);
        if(p->rchild!=NULL)
            EnQueue(Q, p->rchild);
    }
}

//二叉树的结点 (链式存储)
typedef struct BiTNode{
    char data;
    struct BiTNode *lchild, *rchild;
}BiTNode, *BiTree;

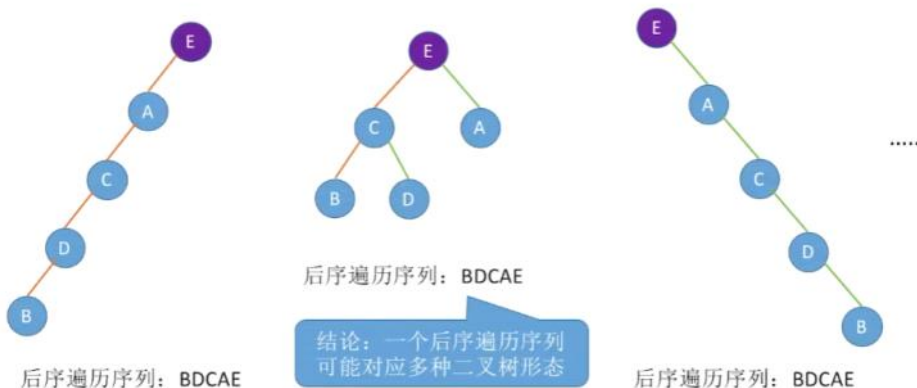
//链式队列结点
typedef struct LinkNode{
    BiTNode *data;
    struct LinkNode *next;
}LinkNode;

typedef struct{
    LinkNode *front, *rear; //队头队尾
}LinkQueue;
```

3、由遍历序列构造二叉树

如果只给出一棵二叉树的前中后序的遍历序列, 并不能确定二叉树的形态, 不能唯一确定一棵二叉树。

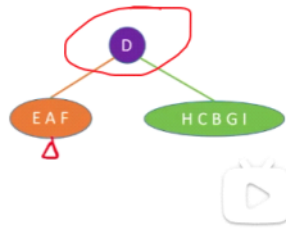
后序遍历: 前序遍历左子树、前序遍历右子树、根结点



但是如果能得到中序序列+其他序列, 就可以确定二叉树形态了:

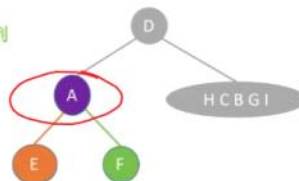
因为一个中序遍历序列中任何一个位置的符号都有可能是根节点, 并且先序的根节点在最前面, 后序的根节点在最后面, 因此就可以确定出根节点在中序序列中的位置:

例2：前序 + 中序遍历序列

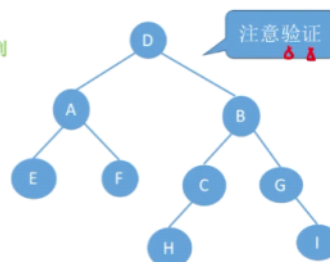


王道考研/CSKAQYAN.COM

例2：前序 + 中序遍历序列



例2：前序 + 中序遍历序列



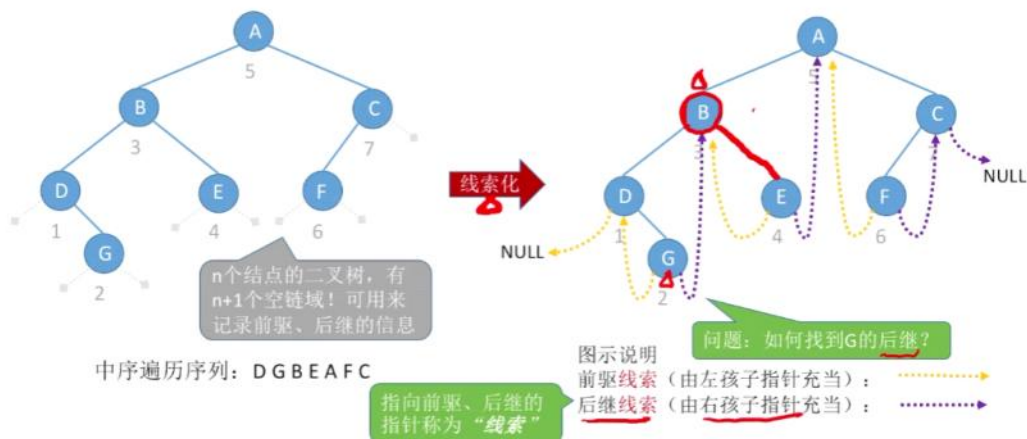
注意验证

因此这样就能确定一个二叉树了，后序序列和根序序列的逻辑也是如此，注意一定要有一个中序，没有中序的两两组合不能确定二叉树。

四、线索二叉树

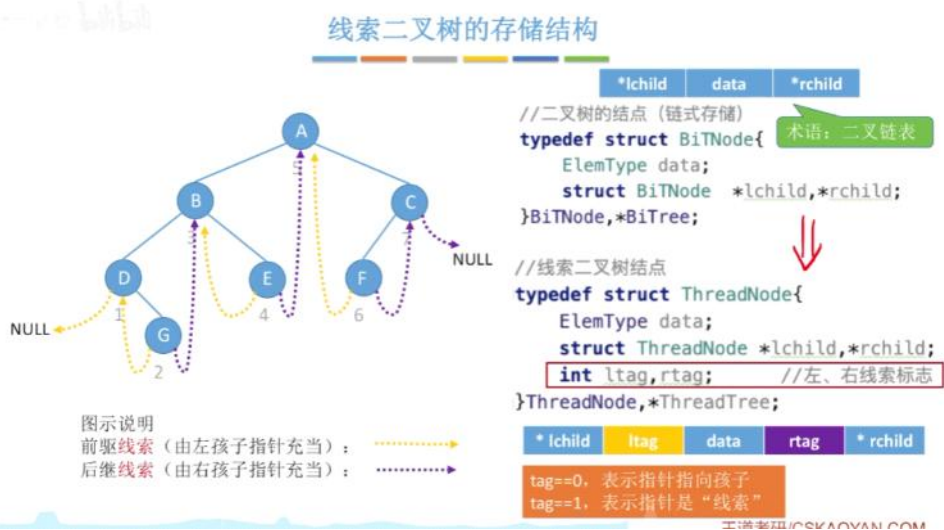
1、概念

普通二叉树如果不带双亲指针的话，如果要找一个节点的双亲节点，就需要从根节点开始出发，否则是无法完成中序遍历的；如果要找某个节点遍历序列的前驱，也是无法找到的，因为只有2个孩子指针只能向下。这样的话如果设计的二叉树会有大量的找前驱和后继的操作，普通二叉树是不好用的，可以用线索二叉树。

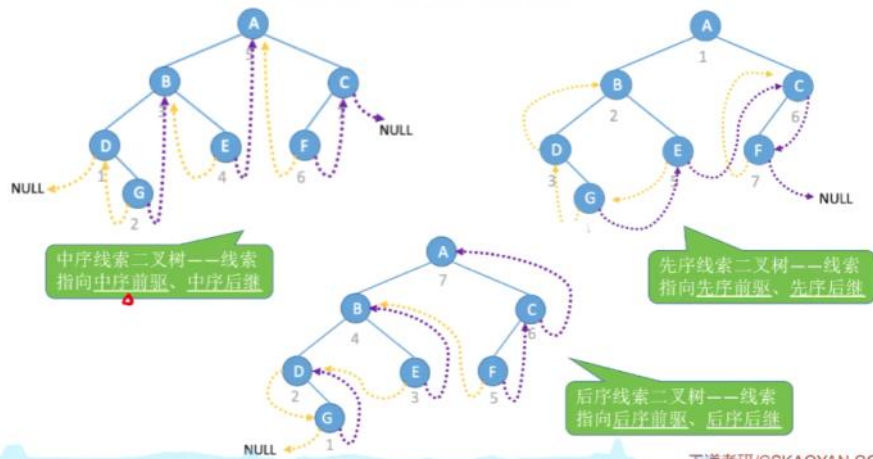


如上图的中序线索二叉树，GEF是叶子节点，左右孩子指针是空的，中序序列中G的前驱是D，让它的左孩子指针指向D，后继是B，让右孩子指针指向B。此时指向前驱、后继的指针叫做线索。这样找前驱和后继就简单多了。

2、存储结构



就多了个tag位表示指向的是孩子还是线索。



3、二叉树的线索化

线索化就是在代码中将普通的二叉树变成线索二叉树，具体的实现就是遍历一遍二叉树（分为中序先序后序层次），在遍历的过程中如果发现这个节点的孩子指针是空的，就让这个孩子指针指向前驱或者后继。再设置一个pre指针用来记录遍历时的前驱节点，跟着遍历指针动的，具体代码如下：

由二叉树建成的树，
ltag、rtag=0

中序线索化

```
typedef struct ThreadNode{
    ElemType data;
    struct ThreadNode *lchild,*rchild;
    int ltag,rtag; //左、右线索标志
}ThreadNode,* ThreadTree;

//中序遍历二叉树，一边遍历一边线索化
void InThread(ThreadTree T){
    if(T!=NULL){
        InThread(T->lchild); //中序遍历左子树
        visit(T); //访问根节点
        InThread(T->rchild); //中序遍历右子树
    }
}

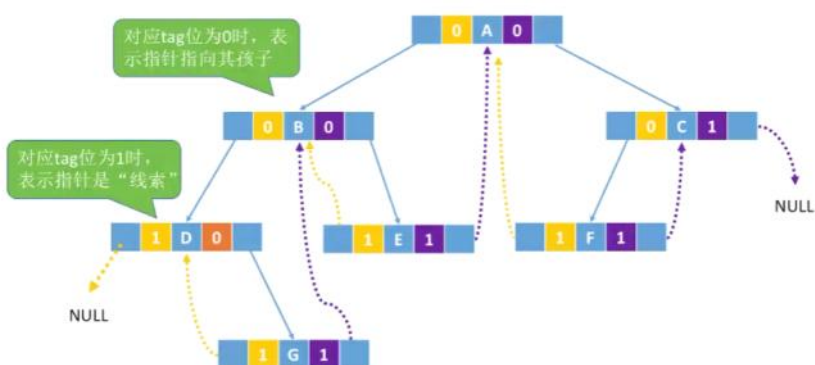
void visit(ThreadNode *q){
    if(q->lchild==NULL){ //左子树为空，建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){
        pre->rchild=q; //建立前驱节点的后继线索
        pre->rtag=1;
    }
    pre=q;
}

//中序线索化二叉树T
void CreateInThread(ThreadTree T){
    pre=NULL; //pre初始为NULL
    if(T!=NULL){ //非空二叉树才能线索化
        InThread(T); //中序遍历二叉树
        if(pre->rchild==NULL)
            pre->rtag=1; //处理遍历的最后一个结点
    }
}

//全局变量 pre，指向当前访问结点的前驱
ThreadNode *pre=NULL;
```

线索化之后变成这样：

中序线索二叉树



先序线索化（王道教材Style）

```
//先序线索化
void PreThread(ThreadTree p,ThreadTree &pre){
    if(p!=NULL){
        if(p->lchild==NULL){ //左子树为空，建立前驱线索
            p->lchild=pre;
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){
            pre->rchild=p; //建立前驱节点的后继线索
            pre->rtag=1;
        }
        pre=p;
        if(p->ltag==0) //标记当前节点为左孩子
            PreThread(p->lchild,pre); //递归
        PreThread(p->rchild,pre); //递归
    }
}

//先序线索化二叉树T
void CreatePreThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){ //非空二叉树，线索化
        PreThread(T,pre); //线索化二叉树
        if(pre->rchild==NULL) //处理遍历的最后一个结点
            pre->rtag=1;
    }
}
```

```

//后序线索化
void PostThread(ThreadTree p, ThreadTree &pre){
    if(p!=NULL){
        PostThread(p->lchild, pre); //递归，线索化左子树
        PostThread(p->rchild, pre); //递归，线索化右子树
        if(p->lchild==NULL){ //左子树为空，建立前驱线索
            p->lchild=pre;
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){
            pre->rchild=p;
            pre->rtag=1;
        }
        pre=p;
    } //if(p!=NULL)
}

```



```

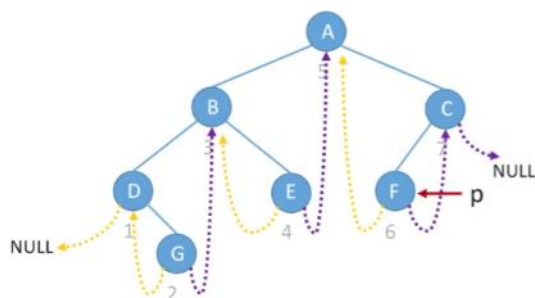
//后序线索化二叉树T
void CreatePostThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){ //非空二叉树，线索化
        PostThread(T, pre); //线索化二叉树
        if(pre->rchild==NULL) //处理遍历的最后一个结点
            pre->rtag=1;
    }
}

```

注意最后别忘了处理最后一个节点的右孩子和rtag。先序也有转圈问题，由于遍历顺序造成的。

4、在线索二叉树中查找前驱和后继

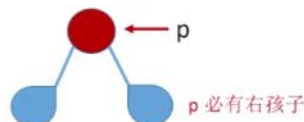
首先是**中序找后继**，如果这个节点已经线索化即rtag==1，那么就访问右孩子指针即可；如果没有线索化，说明是有孩子节点的，那么后继节点就是根据“左根右”得到右边的那个叶子节点，如果不是叶子节点就继续根据“左根右”的顺序访问左边的节点，一直往左直到**最左下角**。



中序遍历序列：D G B E A F C

在中序线索二叉树中找到指定结点*p
的中序后继 next

- ①若 $p \rightarrow rtag == 1$ ，则 $next = p \rightarrow rchild$
- ②若 $p \rightarrow rtag == 0$



中序遍历——左 根 右

左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

next = p的右子树中最左下结点

//找到以P为根的子树中，第一个被中序遍历的结点

```

ThreadNode *Firstnode(ThreadNode *p){
    //循环找到最左下结点(不一定是叶结点)
    while(p->ltag==0) p=p->lchild;
    return p;
}

```

在
的

- ①
- ②

//在中序线索二叉树中找到结点p的后继结点

```

ThreadNode *Nextnode(ThreadNode *p){
    //右子树中最左下结点
    if(p->rtag==0) return Firstnode(p->rchild);
    else return p->rchild; //rtag==1直接返回后继线索
}

```

//对中序线索二叉树进行中序遍历（利用线索实现的非递归算法）

```

void Inorder(ThreadNode *T){
    for(ThreadNode *p=Firstnode(T);p!=NULL; p=Nextnode(p))
        visit(p);
}

```

空间复杂度O(1)

然后是**中序找前驱**，根据“左根右”的顺序可知这个节点的前驱是左边的那个节点，如果已经线索化就直接返回左孩子指针，没有线索话就访问左孩子节点后**往右下角走到底**。

```

//找到以P为根的子树中，最后一个被中序遍历的结点
ThreadNode *Lastnode(ThreadNode *p){
    //循环找到最右下结点(不一定是叶结点)
    while(p->rtag==0) p=p->rchild;
    return p;
}

//在中序线索二叉树中找到结点p的前驱结点
ThreadNode *Prenode(ThreadNode *p){
    //左子树中最右下结点
    if(p->ltag==0) return Lastnode(p->lchild);
    else return p->lchild; //ltag==1直接返回前驱线索
}

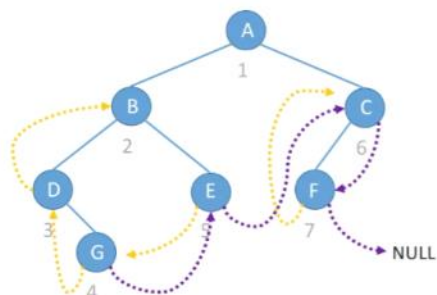
//对中序线索二叉树进行逆向中序遍历
void RevInorder(ThreadNode *T){
    for(ThreadNode *p=Lastnode(T);p!=NULL; p=Prenode(p))
        visit(p);
}

```

在r
的r
①;
②;

p 必有
中

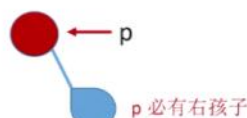
然后是**先序找后继**，根据“根左右”的顺序，线索化了的话直接返回即可，如果没有线索化且节点有左孩子，则先序后继为左孩子；如果没有左孩子说明一定有右孩子，那右孩子就是后继了。



先序遍历序列: ABDGECF

在先序线索二叉树中找到指定结点*p
的先序后继 next

- ①若 $p \rightarrow rtag == 1$ ，则 $next = p \rightarrow rchild$
- ②若 $p \rightarrow rtag == 0$



若p有左孩子，则先序后继为左孩子
先序遍历——根 左 右
根 (根 左 右) 右

若p没有左孩子，则先序后继为右孩子
先序遍历——根 右
根 (根 左 右)

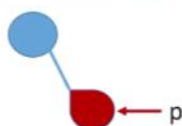
然后是**找先序前驱**， $ltag == 1$ 就直接访问左孩子指针；如果 $ltag == 0$ 那么前驱就不在左右孩子节点这边，这种情况下是找不到前驱的，只能从头开始找或者把二叉链表改成三叉链表。加入三叉链表后，得看看这个节点在什么位置，分三种情况如下（当然根节点的话是没有前驱的）：

- ①如果能找到p的父节点，且p是左孩子



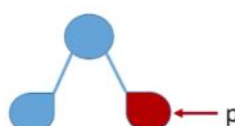
先序遍历——根 左 右
根 (根 左 右) 右
p的父节点即为其前驱

- ②如果能找到p的父节点，且p是右孩子，其左兄弟为空



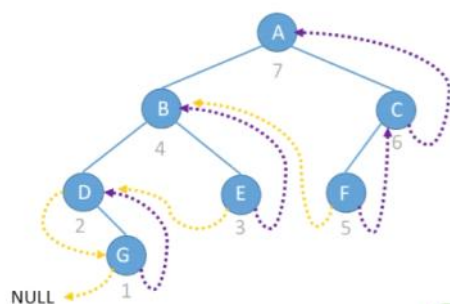
先序遍历——根 右
根 (根 左 右)
p的父节点即为其前驱

- ③如果能找到p的父节点，且p是右孩子，其左兄弟非空



根 左 右
p的前驱为左兄弟子树中最后一个被先序遍历的结点

然后是**后序找前驱**， $ltag == 1$ 时有右孩子就是右节点，没右孩子就是左节点。



后序遍历序列: GDEBFCA

在后序线索二叉树中找到指定结点*p
的后序前驱 pre

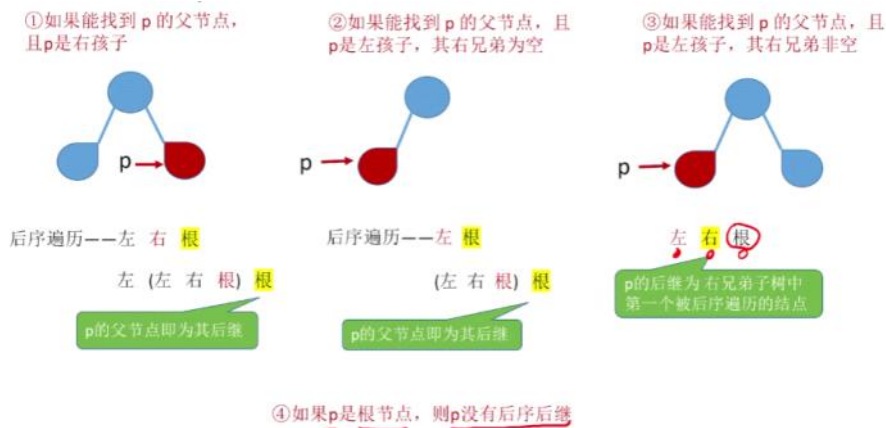
- ①若 $p \rightarrow ltag == 1$ ，则 $pre = p \rightarrow lchild$
- ②若 $p \rightarrow ltag == 0$



若p有右孩子，则后序前驱为右孩子
后序遍历——左 右 根
左 (左 右 根) 根

若p没有右孩子，则后序前驱为左孩子
后序遍历——左 根
(左 右 根) 根

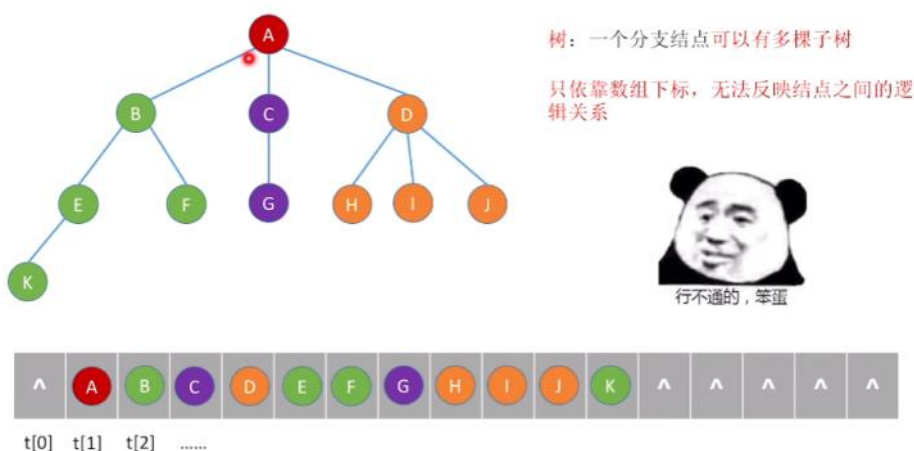
然后是后续找后继，ltag==1时左右子树找不到的，加入三叉链表，分三种情况如下：



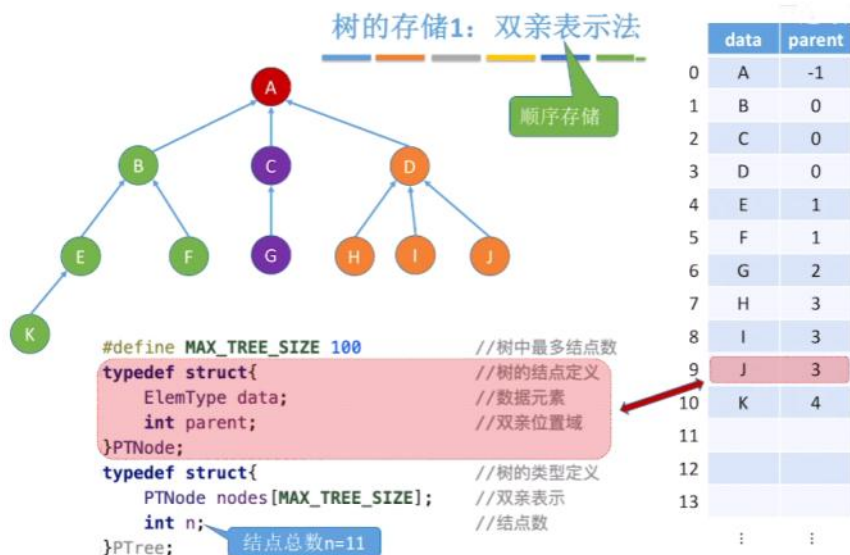
总而言之，先序找前驱，后序找后继需要三叉链表，没的话只能用土办法了。

五、树的存储结构

之前学过二叉树的存储结构，完全二叉树可以实现顺序存储，普通二叉树也可以牺牲一些空间实现顺序存储，当然也可以实现链式存储。现在要对数进行存储的话，如果用顺序存储，这么存进去就无法靠数组下标来反映节点之间的逻辑关系。



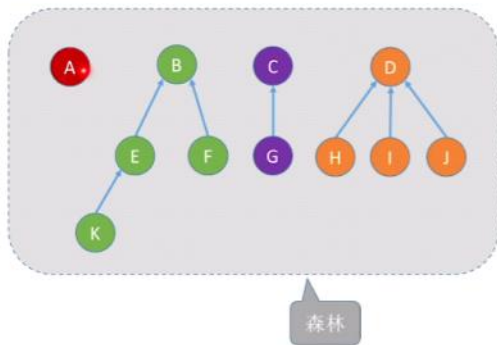
1、要用数组存储的话可以多记录下每个节点的双亲节点的数组下标，这种叫双亲表示法：



双亲表示法还可以用来存储森林，如下图：

拓展：双亲表示法存储“森林”

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合



	data	parent
0	A	-1
1	B	-1
2	C	-1
3	D	-1
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11		
12		
13		

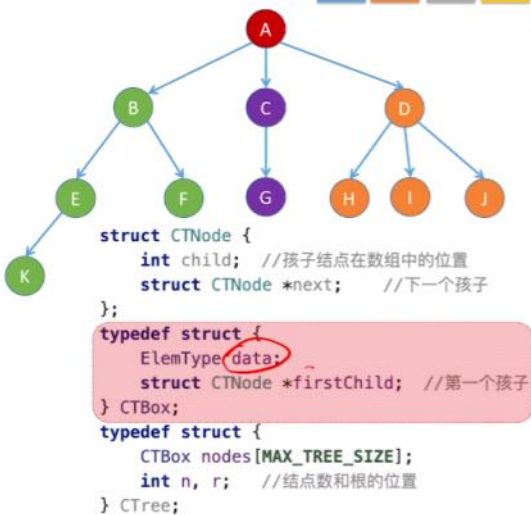
双亲表示法找父母是很容易，但是找孩子不简单，需要将整个数组遍历一下，因此适用于找父亲多找孩子少的情况。

2、孩子表示法

该方法是顺序存储+链式存储结合，数组中只存放数据和第一个孩子的指针，其余的孩子指针通过next指针连在一起。

树的存储2：孩子表示法

王道计算机教育
顺序存储+链式存储结合

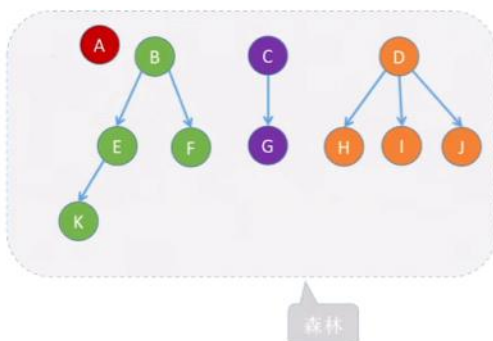


	data	*firstChild
0	A	1 → 2 → 3 ^
1	B	4 → 5 ^
2	C	6 ^
3	D	7 → 8 → 9 ^
4	E	10 ^
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^

王道考研/CSKAOYAN.COM

这种方法也可以拿来存储森林：

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合



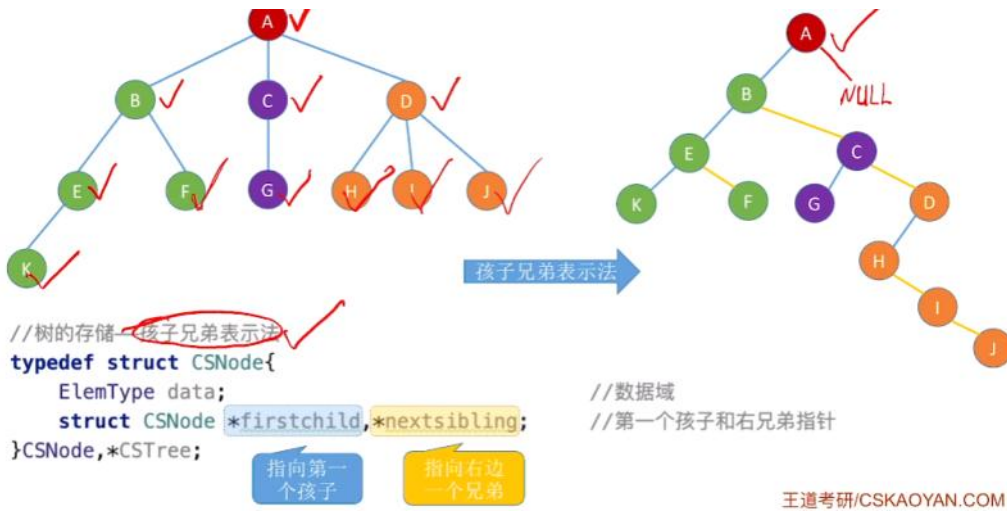
	data	*firstChild
0	A	^
1	B	4 → 5 ^
2	C	6 ^
3	D	7 → 8 → 9 ^
4	E	10 ^
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^

注：用孩子表示法存储森林，需要记录多个根的位置

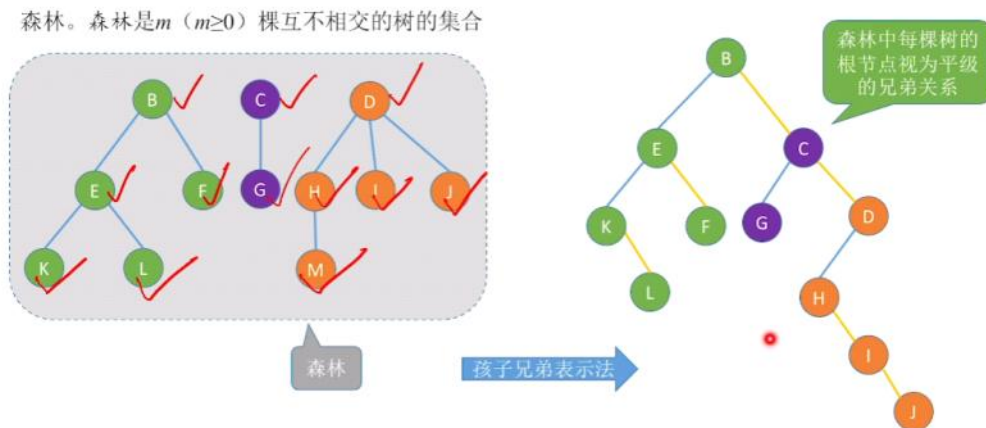
孩子表示法找孩子方便但是找父母节点不方便，适合找孩子多找父亲少的场景。1

3、孩子兄弟表示法

该表示法和二叉链表相似，但是指针的作用不同，一个指针指向第一个孩子，另一个指针指向左边的兄弟，把所有兄弟串起来。



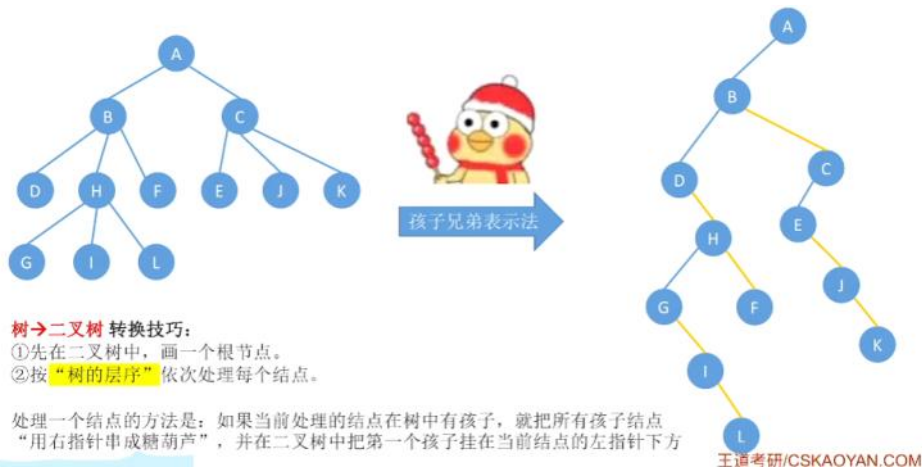
该表示法也可以存储森林（将每棵树的根节点看成平级的兄弟关系）：



六、树和二叉树和森林之间的转换

1、树转二叉树

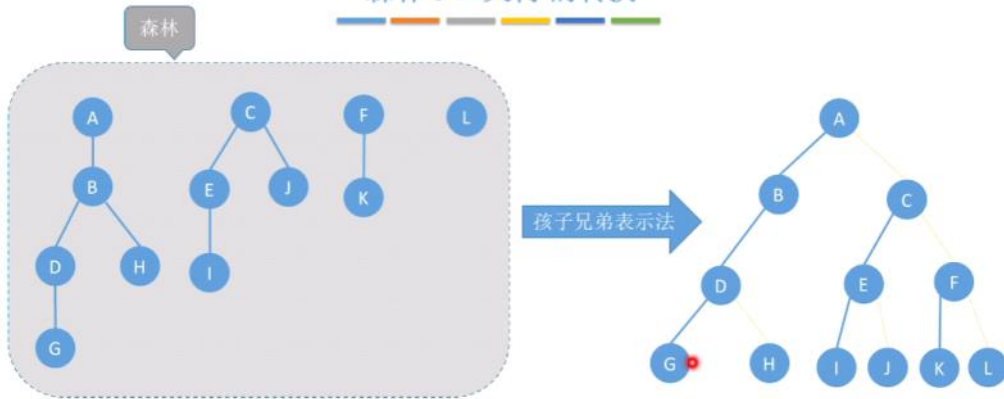
本质就是用孩子兄弟表示法存储树，一层一层从左往右的顺序处理节点



2、森林转二叉树

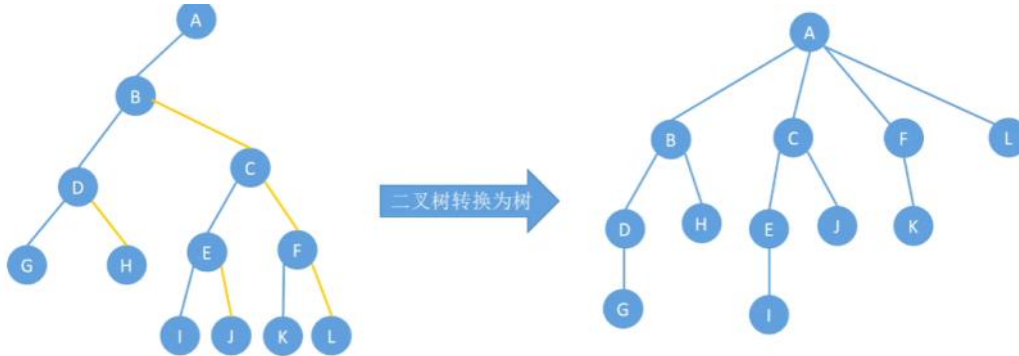
和上面一样用孩子兄弟表示法，就是刚开始的时候需要注意所有的根节点是平级顺序，先把所有的根节点串起来：

森林→二叉树的转换



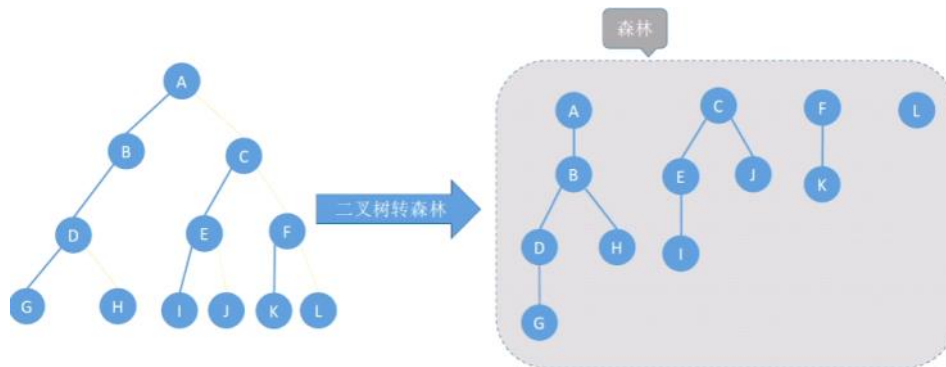
3、二叉树转树

看每个节点的左指针，断开，然后左指针指向的节点的右指针的那一串全部断开，按顺序挂到该节点下面：



4、二叉树转森林

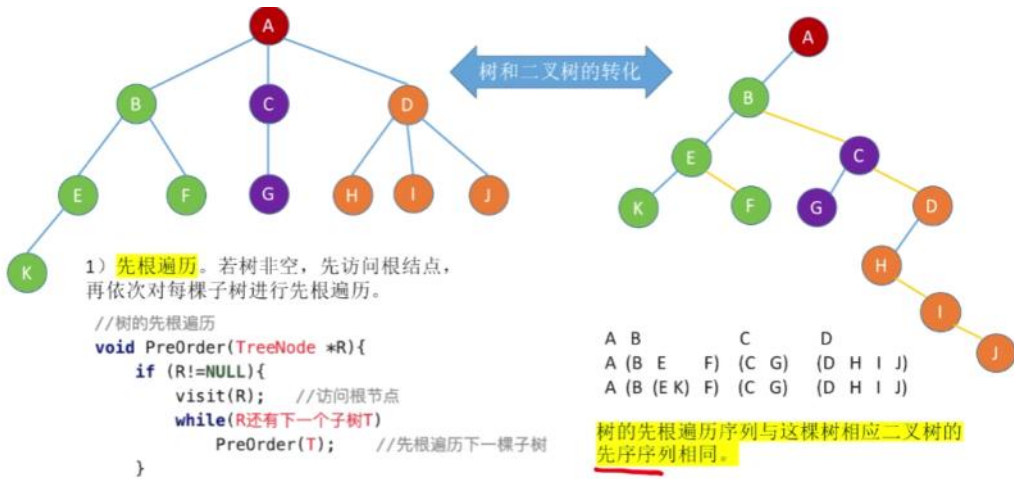
和二叉树转树一样的，只是刚开始的时候要把根节点右边的一串全部断开作为每个树的根节点：



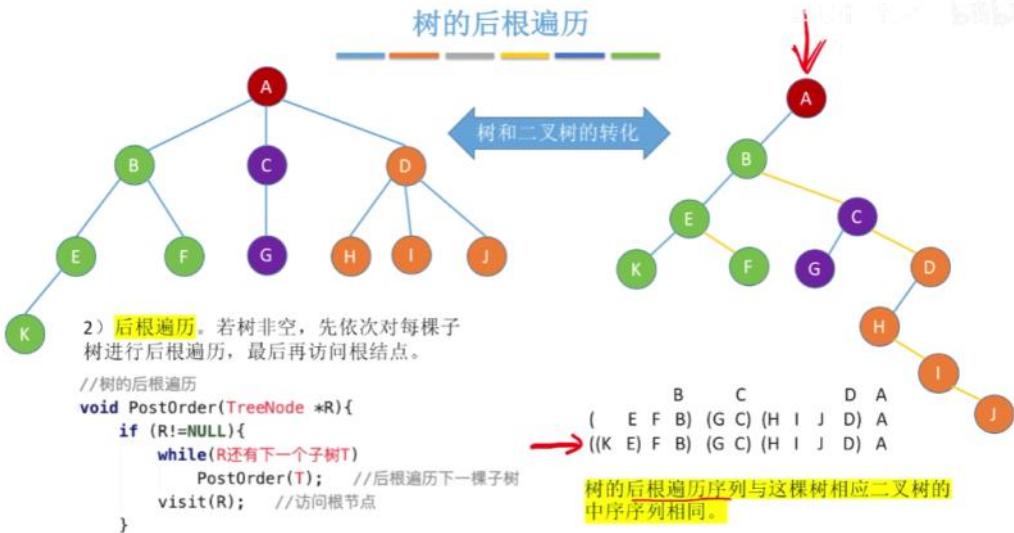
六、树的遍历

树也有递归特性，因此也可以对树进行递归遍历

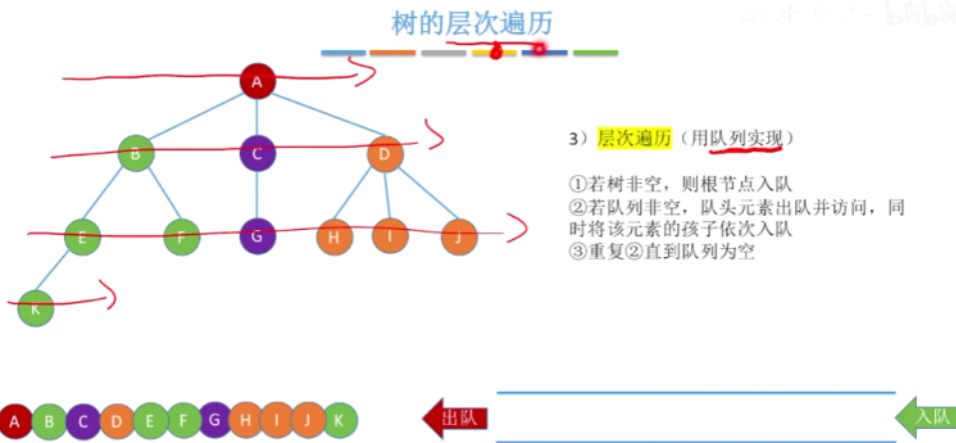
1、先跟遍历（根左一堆右一堆）



2、后根遍历（左一堆右一堆根）

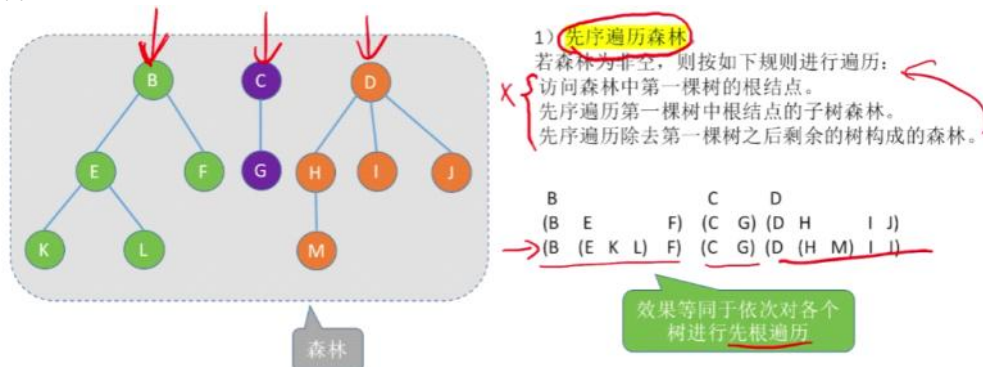


3、层次遍历（BFS实现）

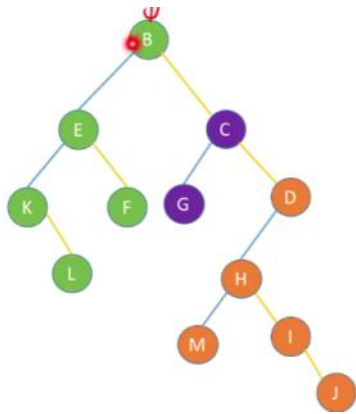


4、森林的遍历

(1) 先序



也可以先将森林转成二叉树后遍历。



1) 先序遍历森林。

若森林为非空，则按如下规则进行遍历：

访问森林中第一棵树的根结点。

先序遍历第一棵树中根结点的子树森林。

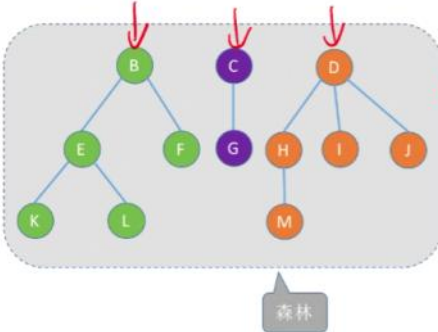
先序遍历除去第一棵树之后剩余的树构成的森林。

$$\begin{array}{ccccccc} & B & & C & & D & \\ (B & E & & F) & (C & G) & (D & H & & I & J) \\ \rightarrow (B & (E & K & L) & F) & (C & G) & (D & (H & M) & I & J) \end{array}$$

效果等同于依次对二叉树的先序遍历

(2) 中序 (假想出一个根节点成树来遍历)

注意这里的中序指的是森林转成二叉树后的中序遍历。



2) 中序遍历森林。

若森林为非空，则按如下规则进行遍历：

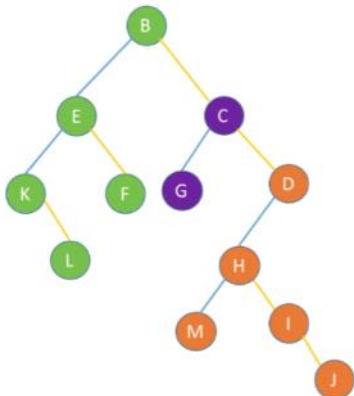
中序遍历森林中第一棵树的根结点的子树森林。

访问第一棵树的根结点。

中序遍历除去第一棵树之后剩余的树构成的森林。

$$\begin{array}{ccccccc} & B & & C & & D & \\ (& E & F & B) & (G & C) & (& H & I & J & D) \\ \rightarrow ((K & L & E) & F & B) & (G & C) & ((M & H) & I & J & D) \end{array}$$

效果等同于依次对各个树进行中序遍历



2) 中序遍历森林。

若森林为非空，则按如下规则进行遍历：

中序遍历森林中第一棵树的根结点的子树森林。

访问第一棵树的根结点。

中序遍历除去第一棵树之后剩余的树构成的森林。

$$\begin{array}{ccccccc} & B & & C & & D & \\ (& E & F & B) & (G & C) & (& H & I & J & D) \\ ((K & L & E) & F & B) & (G & C) & ((M & H) & I & J & D) \end{array}$$

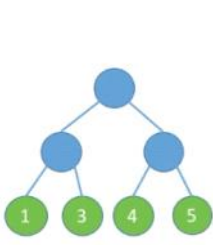
七、哈夫曼树

1、带权路径长度

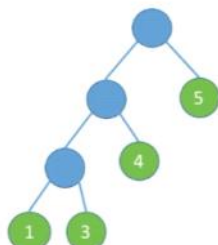
节点的权：每个节点里面塞的数值

节点的带权路径长度：从根节点到该节点的路径长度 (经过的边数) 和该节点上权值的乘积。

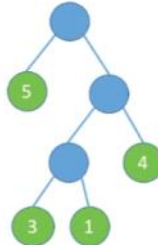
树的带权路径长度WPL：树种所有叶子节点的带权路径长度之和。



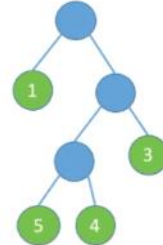
$$WPL = 2 \times 1 + 2 \times 3 + 2 \times 4 + 2 \times 5 = 26$$



$$WPL = 1 \times 5 + 2 \times 4 + 3 \times 1 + 3 \times 3 = 25$$



$$WPL = 1 \times 5 + 2 \times 4 + 3 \times 1 + 3 \times 3 = 25$$



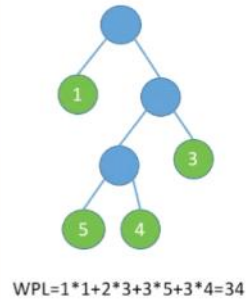
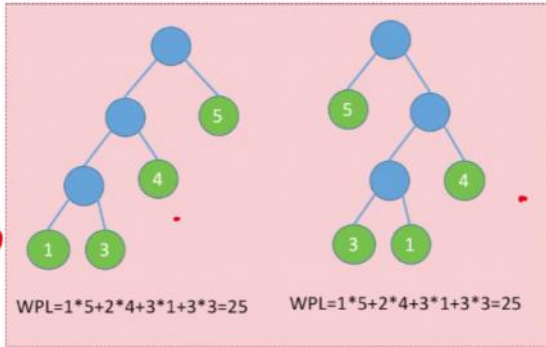
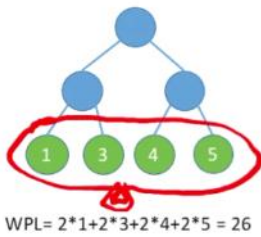
$$WPL = 1 \times 1 + 2 \times 3 + 3 \times 5 + 3 \times 4 = 34$$

2、哈夫曼树的定义

在含有n个带权叶子节点的二叉树中，其中带权路径长度WPL最小的二叉树叫哈夫曼树，也叫最优二叉树。

哈夫曼树的定义

都是哈夫曼树



在含有 n 个带权叶结点的二叉树中，其中带权路径长度（WPL）最小的二叉树称为哈夫曼树，也称最优二叉树

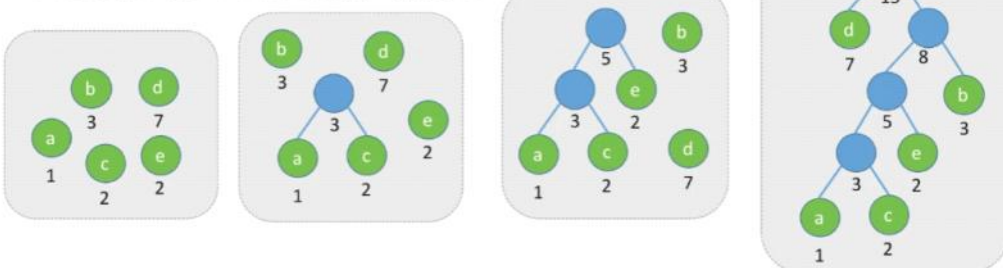
3、哈夫曼树的构造

节点中选择2个最小的作为兄弟，然后两个节点的权相加得到的值建立一个新节点，这个新节点是他们的双亲，然后双亲放回那堆节点中再次选2个最小的，一直反复。

哈夫曼树的构造

给定 n 个权值分别为 $w_1, w_2, \dots, w_n$ 的结点，构造哈夫曼树的算法描述如下：

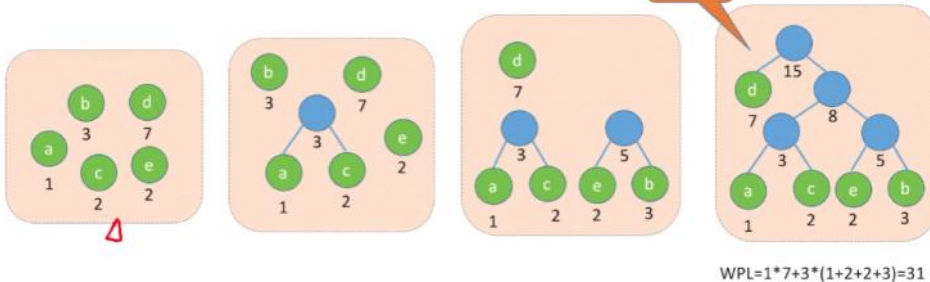
- 1) 将这 n 个结点分别作为 n 棵仅含一个结点的二叉树，构成森林 F 。
- 2) 构造一个新结点，从 F 中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树上根结点的权值之和。
- 3) 从 F 中删除刚才选出的两棵树，同时将新得到的树加入 F 中。
- 4) 重复步骤2) 和3)，直至 F 中只剩下一棵树为止。



$$WPL_{min} = 1 \times 7 + 2 \times 3 + 3 \times 2 + 4 \times 1 + 4 \times 2 = 31$$

哈夫曼中每个初始节点最终都会变成叶节点，权值越小离根节点越远，总节点个数为 $2n-1$ ，不存在度为1的节点，哈夫曼树也不唯一，但是他们的WPL必定相同。

左右顺序任意



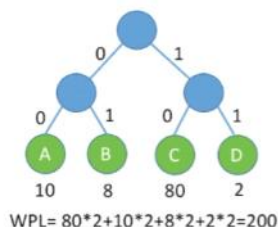
4、哈夫曼编码

哈夫曼编码的应用之一就是固定长度编码变成可变长度编码，就是允许对不同的字符用不等长的二进制位表示

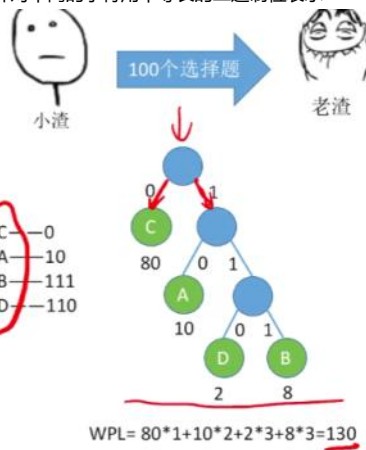
A—00
B—01
C—10
D—11

每个字符用长度为2的二进制表示

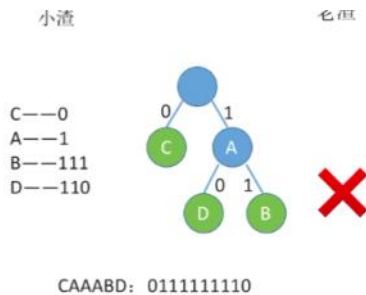
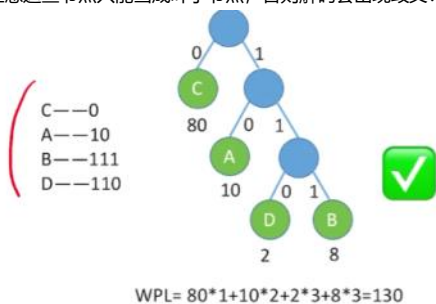
假设，100题中有80题选C，10题选A，8题选B，2题选D
所有答案的二进制长度 $= 80 \times 2 + 10 \times 2 + 8 \times 2 + 2 \times 2 = 200$ bit



C—0
A—10
B—111
D—110



注意这些节点只能当成叶子节点，否则解码会出现歧义：



如果没有一个编码是另一个编码的前缀，那么这个编码叫前缀编码。

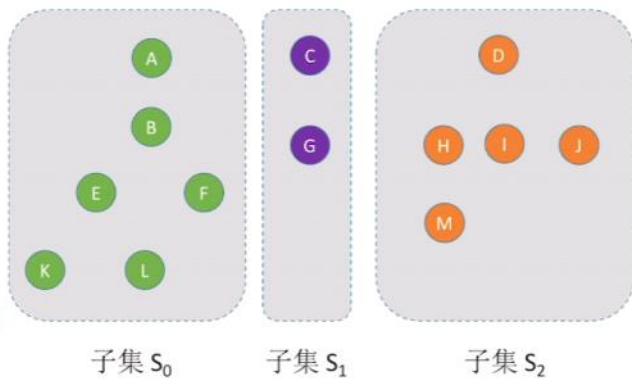
八、并查集

1、概念

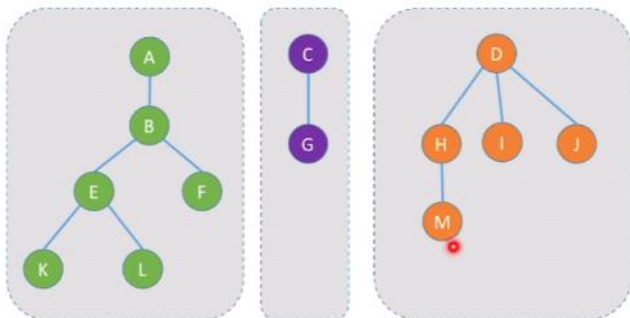
数据之间的逻辑关系之一有集合关系，那么怎么表示集合呢？



将各个元素划分为若干个互不相交的子集

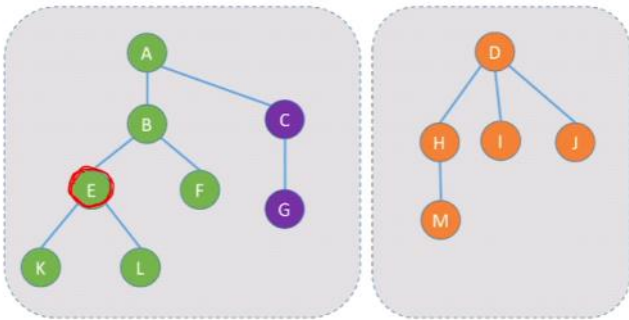


上面又划分了3个集合关系，元素之间的关系只有“属于同一个集合”和“不属于同一个集合”两种。上面这张图有点像森林的表示，因此可以组成森林的结构：

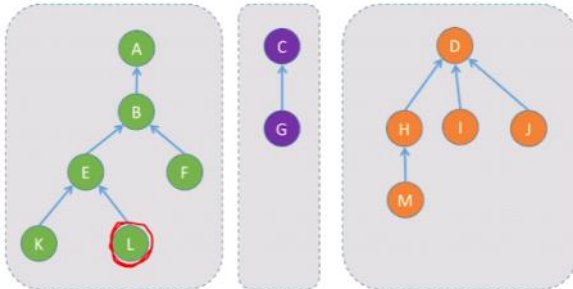


如果要查某一个元素从属于哪一个集合，那么让这个元素的节点一路向上查直到根节点，就可以直到自己属于哪棵树了，要判断2个元素是否属于同一集合也是这样。

如果要把2个集合合并为一个集合，只需要让一棵树成为另一棵树的孩子即可，如下图：



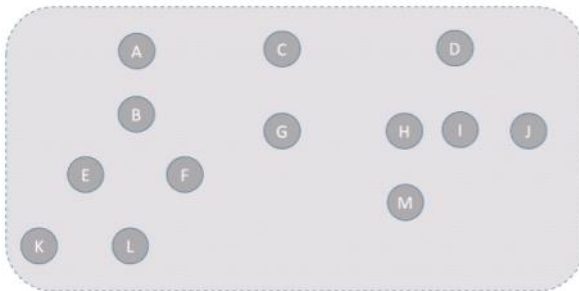
因此这就是并查集的由来，主要实现的是“并”和“查”。根据这个需求可以发现用双亲表示法来表示最为方便，方便向上查以及合并。



数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	0	-1	-1	1	1	2	3	3	3	4	4	7

2、代码实现

初始化：只需把数组值全部设置为-1即可



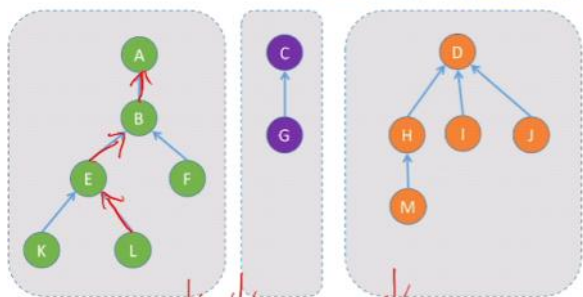
```
#define SIZE 13
int UFsets[SIZE]; // 集合元素数组

// 初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }
}
```

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

用一个数组 S[] 即可表示“集合”关系

“开查集”的代码实现——开、查

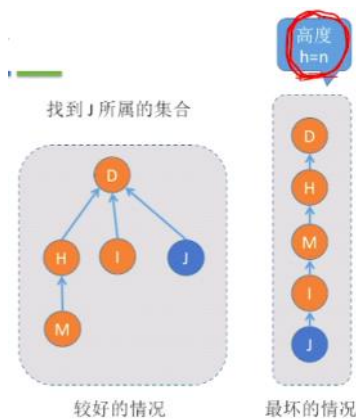


```
// Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0){ // 循环寻找x的根
        x=S[x];
    }
    return x; // 根的S[]小于0

// Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    // 要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    // 将Root2连接到另一根Root1下面
    S[Root2]=Root1;
}
```

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	0	-1	-1	1	1	2	3	3	3	4	4	7

时间复杂度上并union只需要O(1)，然而find需要O(n)的复杂度，和高度相关。



若结点数为 n ，Find 最坏时间复杂度为 $O(n)$

3、优化 Union

如果要优化查的时间复杂度，可以在 Union 建树的时候尽量让树长胖而不是长高，用根节点的绝对值表示树的节点总数（把根节点的-1换成其他负数就行）。

Union 操作的优化

```

//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点数
        S[Root1]=Root2; //小树合并到大树
    }
}

```

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-6	0	-2	-5	1	1	2	3	3	3	4	4	7

①用根节点的绝对值表示树的结点数
②Union操作，让小树合并到大树

王道考研/CSKAQYAN.COM

Union 操作的优化

```

#define SIZE 13
int UFSet[SIZE]; //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }
}

//Find “查”操作，找x所属集合（返回x所属根节点）
int Find(int S[],int x){
    while(S[x]>=0) //循环寻找x的根
        x=S[x];
    return x; //根的S[]小于0
}

//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    //将根Root2连接到另一根Root1下面
    S[Root2]=Root1;
}

```

该方法构造的树高不超过 $\lceil \log_2 n \rceil + 1$

优化

```

//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点数
        S[Root1]=Root2; //小树合并到大树
    }
}

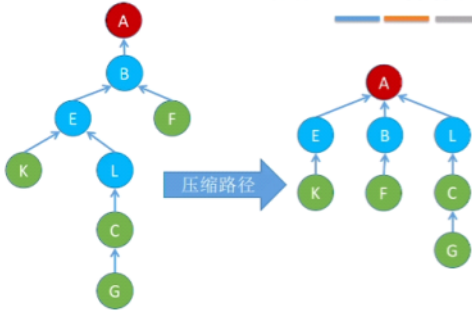
```

王道考研/CSKAQYAN.COM

4、优化 Find（压缩路径）

如果在查找根节点的途中顺便把路上所有的根节点挂到根节点的下面，就可以极大的压缩路径。

拓展：Find 操作的优化（压缩路径）



```
//Find “查”操作优化，先找到根节点，再进行“压缩路径”
int Find(int S[],int x){
    int root = x;
    while(S[root]>=0) root=S[root]; //循环找到根
    while(x!=root){ //压缩路径
        int t=S[x]; //t指向x的父节点
        S[x]=root; //x直接挂到根节点下
        x=t;
    }
    return root; //返回根节点编号
}
```

压缩路径——Find 操作，先找到根节点，再将查找路径上所有结点都挂到根结点下

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	11	-5	0	1	2	3	3	3	4	0	7

Eg: Find(S[], 11) //查找结点L所属集合

此时的时间复杂度压缩到O(a(n))

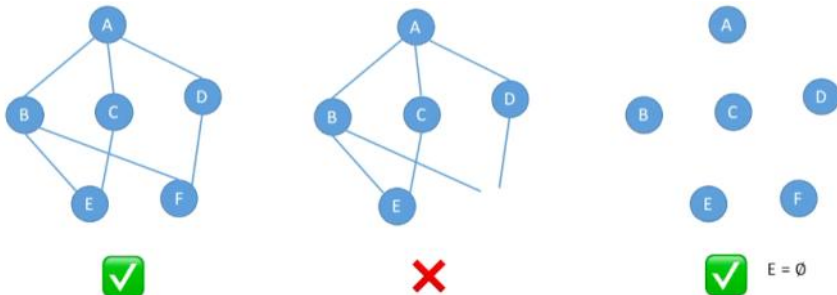


2025年9月11日星期四 下午6:30

一、基本概念

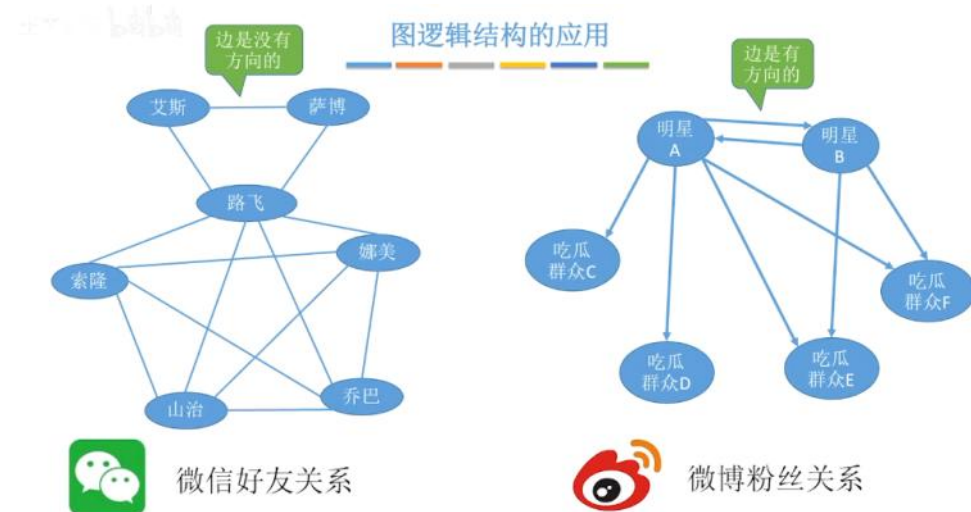
1、定义

一张图由多个顶点 v 和边 e 组成，每个边必须连接上2个顶点，顶点不能为空集但是边可以为空集。



2、有向图和无向图

有些图的边是带方向的，可以指单向的联系和双向的联系：



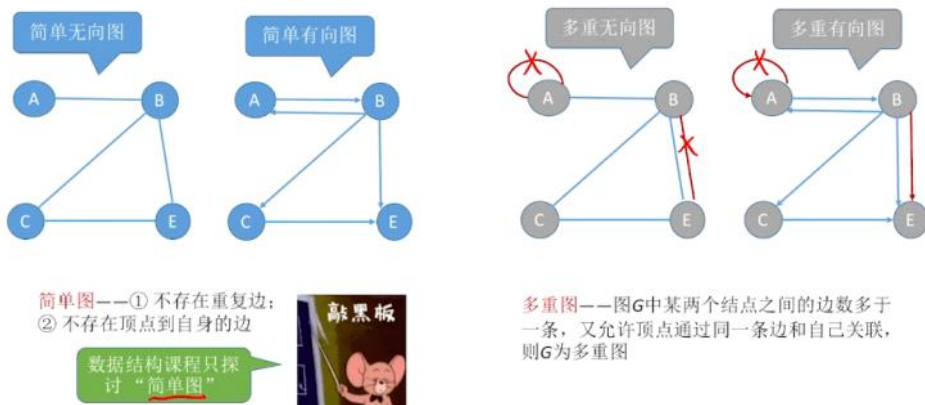
这种边带方向的叫有向图，不带方向的叫无向图。

有向图的边叫有向边（弧），用尖括号表示 $\langle v, w \rangle$ ，其中这里的 v 是弧尾， w 是弧头， $\langle v, w \rangle$ 叫做从 v 到 w 的弧， $\langle v, w \rangle$ 和 $\langle w, v \rangle$ 表示不一样的弧，他们的方向相反。

无向图的边叫无向边（边），用圆括号表示 (v, w) ，这里不分弧头弧尾，单纯表明顶点 w 和顶点 v 互为邻接点，或者说边 (v, w) 依附于两个顶点或者说顶点 v 和顶点 w 相关联。

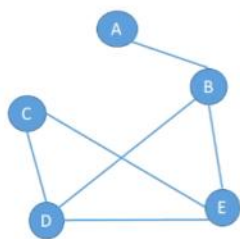
3、简单图和多重图

简单图不允许存在顶点到自己的边 $\langle v, v \rangle$ 和 (v, v) ，多重图反之。



4、顶点的度

顶点的度、入度、出度



对于无向图：顶点 v 的度是指依附于该顶点的边的条数，记为 $TD(v)$ 。

在具有 n 个顶点、 e 条边的无向图中， $\sum_{i=1}^n TD(v_i) = 2e$

即无向图的全部顶点的度的和等于边数的2倍

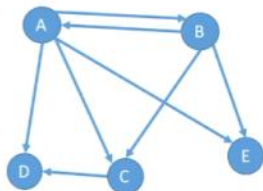
对于有向图：

入度是以顶点 v 为终点的有向边的数目，记为 $ID(v)$ ；

出度是以顶点 v 为起点的有向边的数目，记为 $OD(v)$ 。

顶点 v 的度等于其入度和出度之和，即 $TD(v) = ID(v) + OD(v)$ 。

在具有 n 个顶点、 e 条边的有向图中， $\sum_{i=1}^n ID(v_i) = \sum_{i=1}^n OD(v_i) = e$



5、顶和顶点的关系

路径——两点之间经过的路（顶点可以重复经过）

回路——两点之间的路径能构成一个环

简单路径——路径中所有的顶点不重复出现

简单回路——路径形成环，路径不重复

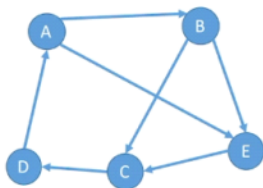
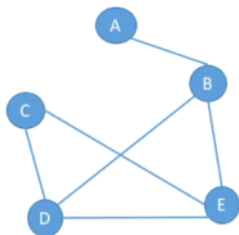
路径长度——两点之间经过多少边

点到点距离——两顶点之间最短路径，如果路不通，距离为无穷

连通——无向图两点之间有路径存在

强连通——有向图顶点之间有正向路径也有逆向路径

连通图、强连通图



若图 G 中任意两个顶点都是连通的，则称图 G 为连通图，否则称为非连通图。

若图中任何一对顶点都是强连通的，则称此图为强连通图。

常见考点：

对于 n 个顶点的无向图 G ，

若 G 是连通图，则最少有 $n-1$ 条边

若 G 是非连通图，则最多可能有 C_{n-1}^2 条边



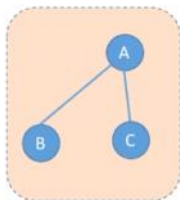
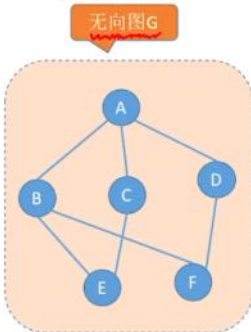
C_4^2

王道考研/CSKAOYAN.COM

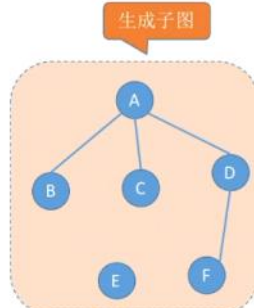
6、子图

原图上选一个子图，子图要满足图的定义

研究图的局部——子图



子图



生成子图

设有两个图 $G = (V, E)$ 和 $G' = (V', E')$ ，若 V' 是 V 的子集，且 E' 是 E 的子集，则称 G' 是 G 的子图。

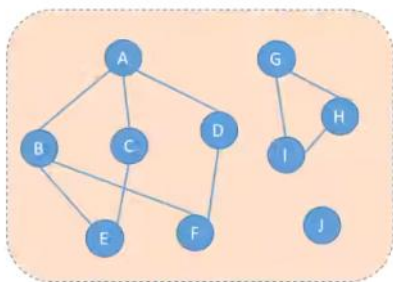
若有满足 $V(G') = V(G)$ 的子图 G' ，则称其为 G 的生成子图



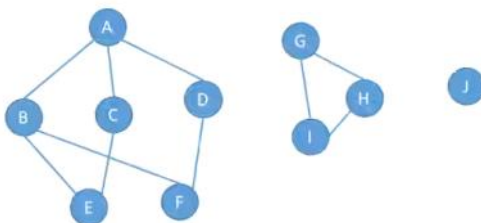
并非任意挑几个点、几条边都能构成子图

7、连通分量

无向图的极大连通子图称为连通分量，意思就是尽可能的包含更多的节点



无向图G

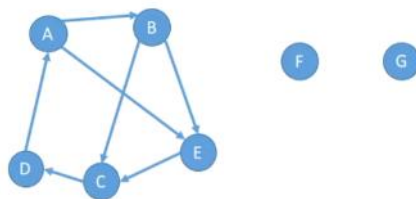
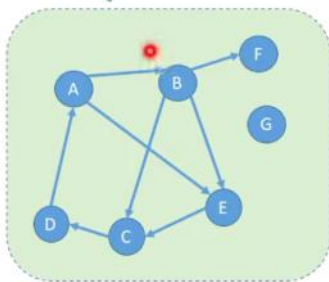


G的三个连通分量

8、强连通分量

有向图的极大强连通子图称为强连通分量，注意这个必须是双向能通到。

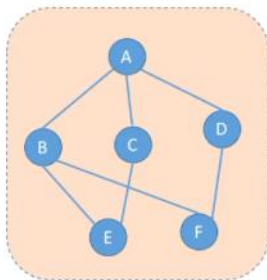
有向图G



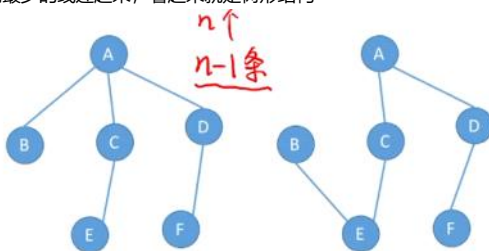
G的三个强连通分量

9、生成树

又叫做极小连通子图，就是无向图几个节点用最少的线连起来，看起来就是树形结构



无向图G



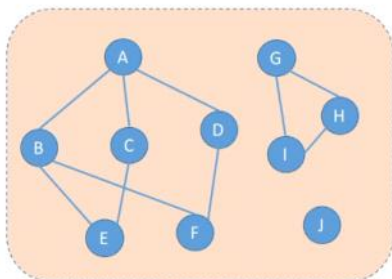
G的生成树1

G的生成树2

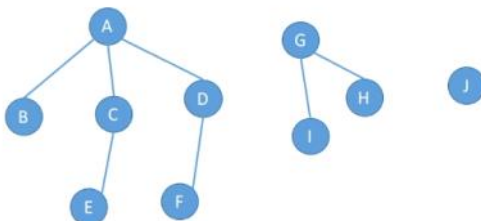
.....

10、生成森林

非连通图中连通分量的生成树构成生成森林。

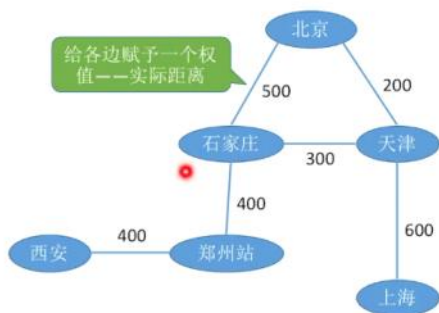


非连通无向图G



G的连通分量 → G的生成森林

11、边的权、带权图/网

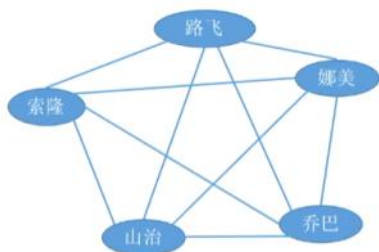


边的权——在一个图中，每条边都可以标上具有某种含义的数值，该数值称为该边的权值。
带权图/网——边上带有权值的图称为带权图，也称网。

带权路径长度——当图是带权图时，一条路径上所有边的权值之和，称为该路径的带权路径长度

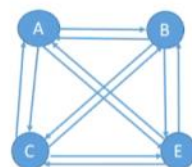
12、有向完全图和无向完全图

任意两点之间都加满边。



无向完全图——无向图中任意两个顶点之间都存在边

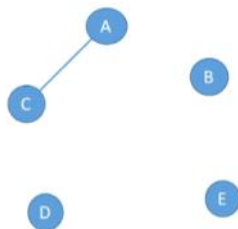
若无向图的顶点数 $|V|=n$ ，则
 $|E| \in [0, C_n^2] = [0, n(n-1)/2]$



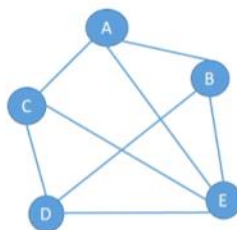
有向完全图——有向图中任意两个顶点之间都存在方向相反的两条弧

$n \uparrow$. C_n^2

13、稀疏图和稠密图



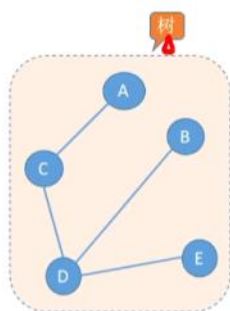
边数很少的图称为稀疏图



反之称为稠密图

14、其他特殊图

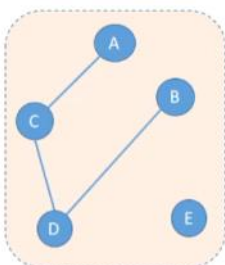
几种特殊形态的图



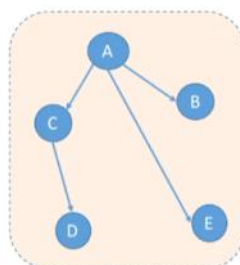
树——不存在回路，且连通的无向图

n 个顶点的树，必有 $n-1$ 条边。

常见考点： n 个顶点的图，若 $|E| > n-1$ ，则一定有回路



森林



有向树

有向树——一个顶点的入度为0，其余顶点的入度均为1的有向图，称为有向树。

15、概念总结自查



常见考点:

对于n个顶点的无向图G,

- 所有顶点的度之和= $2|E|$
- 若G是连通图, 则最少有n-1条边(树); 若 $|E|>n-1$, 则一定有回路
- 若G是非连通图, 则最多可能有 C_{n-1}^2 条边
- 无向完全图共有 C_n^2 条边

对于n个顶点的有向图G,

- 所有顶点的出度之和=入度之和= $|E|$
- 所有顶点的度之和= $2|E|$
- 若G是强连通图, 则最少有n条边(形成回路)
- 有向完全图共有 $2C_n^2$ 条边

王道考研/CSKAOYAN.COM

二、图的存储结构

1、邻接矩阵法

行和列分别是两个节点, 矩阵中的0表示两节点之间不存在边, 1表示存在。无向图的矩阵是对称的, 有向不一定对称。



```

#define MaxVertexNum 100 //顶点数目的最大值
typedef struct {
    char Vex[MaxVertexNum]; //顶点表
    int Edge[MaxVertexNum][MaxVertexNum]; //邻接矩阵, 边表
    int vexnum, arcnum; //图的当前顶点数和边数/弧数
} MGraph;

```

王道考研/CSKAOYAN.COM

结点数n的图 $G=(V, E)$ 的邻接矩阵A是 $n \times n$ 的。将G的顶点编号为 $v_1, v_2, \dots, v_n$, 则

$$A[i][j] = \begin{cases} 1, & \text{若}(v_i, v_j) \text{或}\langle v_i, v_j \rangle \text{是} E(G) \text{中的边} \\ 0, & \text{若}(v_i, v_j) \text{或}\langle v_i, v_j \rangle \text{不是} E(G) \text{中的边} \end{cases}$$

第i个结点的度 = 第i行(或第i列)的非零元素个数

第i个结点的出度 = 第i行的非零元素个数

第i个结点的入度 = 第i列的非零元素个数

第i个结点的度 = 第i行、第i列的非零元素个数之和

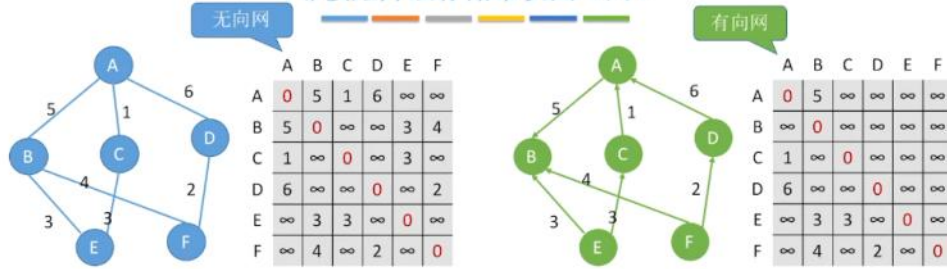


思考: 如何求顶点的度、入度、出度?

邻接矩阵法求顶点的度/出度/入度的时间复杂度为 $O(|V|)$

王道考研/CSKAOYAN.COM

邻接矩阵法存储带权图（网）



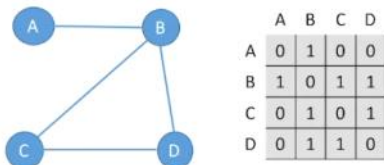
可用int的上限值表示“无穷”

```
#define MaxVertexNum 100
#define INFINITY 最大的int值
typedef char VertexType;
typedef int EdgeType;
typedef struct {
    VertexType Vex[MaxVertexNum];
    EdgeType Edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
} MGraph;
```

```
// 顶点数目的最大值
// 宏定义常量“无穷”
// 顶点的数据类型
// 带权图中边上权值的数据类型
// 顶点
// 边的权
// 图的当前顶点数和弧数
```

由此可见，邻接矩阵的空间复杂度是顶点数的平方，和边数无关，这个图适合存储稠密图，稀疏图会浪费空间，也可以用压缩矩阵的方法解决。

乘法性质：



设图G的邻接矩阵为A（矩阵元素为0/1），则Aⁿ的元素Aⁿ[i][j]等于由顶点i到顶点j的长度为n的路径的数目

	A	B	C	D
A	0	1	0	0
B	1	0	1	1
C	0	1	0	1
D	0	1	1	0

$$A^2[1][4] = a_{1,1}a_{1,4} + a_{1,2}a_{2,4} + a_{1,3}a_{3,4} + a_{1,4}a_{4,4} = 1$$

$$A^2[2][2] = a_{2,1}a_{1,2} + a_{2,2}a_{2,2} + a_{2,3}a_{3,2} + a_{2,4}a_{4,2} = 3$$

$$A^2[3][3] = a_{3,1}a_{1,3} + a_{3,2}a_{2,3} + a_{3,3}a_{3,3} + a_{3,4}a_{4,3} = 1$$

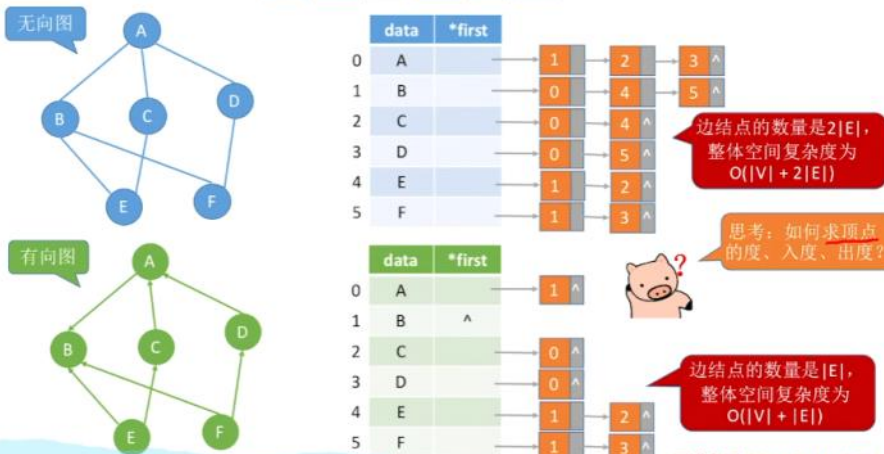
$$A^2[1][2] = a_{1,1}a_{1,2} + a_{1,2}a_{2,2} + a_{1,3}a_{3,2} + a_{1,4}a_{4,2} = 0$$

	A	B	C	D
A	1	0	1	1
B	0	3	1	1
C	1	1	2	1
D	1	1	1	2

王道考研/CSKAQYAN.COM

2、邻接表法

邻接表法

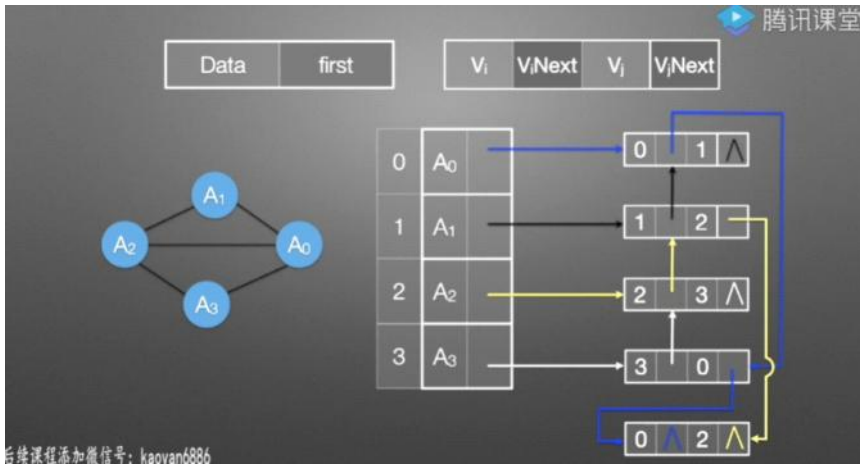


邻接表的指针表示的是出度的边，要查入度的话需要遍历整个邻接表。

注意：邻接表的指针没有先后之分，因此邻接表表示的会有很多种，邻接矩阵就只有一种。

	邻接表	邻接矩阵
空间复杂度	无向图O(V + 2 E)；有向图O(V + E)	O(V ²)
适用于	存储稀疏图	存储稠密图
表示方式	不唯一	唯一
计算度/出度/入度	计算有向图的度、入度不方便，其余很方便	必须遍历对应行或列
找相邻的边	找有向图的入边不方便，其余很方便	必须遍历对应行或列

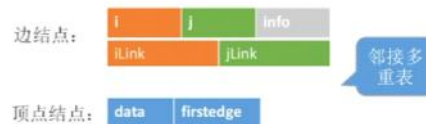
3、十字链表（针对有向图）



左边代表节点A0A1A2A3右边代表边(0,1)(1,2)(2,3)(3,0)(0,2), A0控制了(0,1)(0,2)(0,3)这三条边, 因此引出一条线将3个线连接起来; A1控制(1,2)(1,0)这2条边, 引出即可, 无向图中(0,1)和(1,0)表示的是同一个, 因此直接往上连接(0,1)节点即可。其他的节点也是这样。

4、邻接多重表

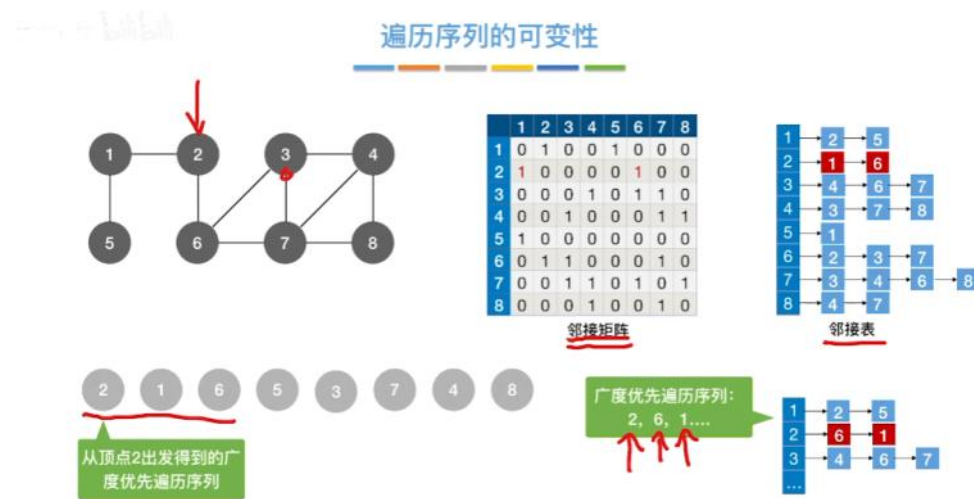
	邻接矩阵	邻接表	十字链表	邻接多重表
空间复杂度	$O(V ^2)$	无向图 $O(V + 2 E)$ 有向图 $O(V + E)$	$O(V + E)$	$O(V + E)$
找相邻边	遍历对应行或列 时间复杂度为 $O(V)$	找有向图的入边必须遍历整个邻接表	很方便	很方便
删除边或顶点	删除边很方便, 删除顶点需要大量移动数据	无向图中删除边或顶点都不方便	很方便	很方便
适用于	稠密图	稀疏图和其他	只能存无向图	只能存无向图
表示方式	唯一	不唯一	不唯一	不唯一



三、图的遍历

1、广度优先遍历BFS

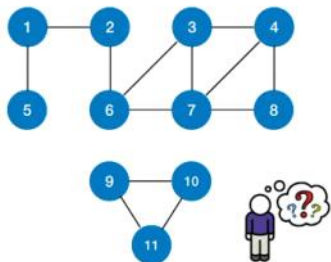
老朋友了, 用队列实现, 每次访问到一个节点就把该节点附近未访问的节点入队, 访问完出队, 依次类推。



如上图, 如果采用邻接矩阵存储, 则遍历序列是唯一的, 但是邻接表的话不唯一。

广度优先算法是不能遍历到没有连接到的节点的, 因此出现了final版的BFS算法: 就是扫描一下visited的bool数组, 发现了没访问过的节点就从这里出发调入BFS即可。

BFS算法 (Final版)



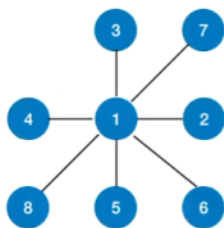
如果是非连通图，则无法遍历完所有结点

```
bool visited[MAX_VERTEX_NUM]; // 访问标记数组
void BFSTraverse(Graph G){ // 对图G进行广度优先遍历
    for(i=0;i<G.vexnum;++i)
        visited[i]=FALSE; // 访问标记数组初始化
    InitQueue(Q); // 初始化辅助队列Q
    for(i=0;i<G.vexnum;++i) // 从0号顶点开始遍历
        if(!visited[i]) // 对每个连通分量调用一次BFS
            BFS(G,i); // v1未访问过，从v1开始BFS
}

// 广度优先遍历
void BFS(Graph G,int v){ // 从顶点v出发，广度优先遍历图G
    visit(v); // 访问初始顶点v
    visited[v]=TRUE; // 对v做已访问标记
    Enqueue(Q,v); // 顶点v入队列Q
    while(!IsEmpty(Q)){ // 顶点v出队列
        Dequeue(Q,v); // 顶点v出队列
        for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w)){ // 检测v所有邻接点
            if(!visited[w]){ // w为v的尚未访问的邻接点
                visit(w); // 访问顶点w
                visited[w]=TRUE; // 对w做已访问标记
                Enqueue(Q,w); // 顶点w入队列
            }
        }
    }
}
```

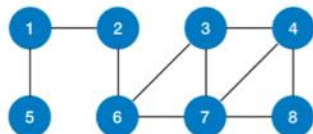
	1	2	3	4	5	6	7	8	9	10	11
visited	false	false	false	false	false	false	false	false	false	false	false

调用BFS的次数由连通分量数决定。



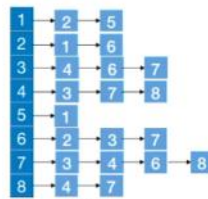
空间复杂度：最坏情况，辅助队列大小为 $O(|V|)$

复杂度分析



	1	2	3	4	5	6	7	8
1	0	1	0	0	1	0	0	0
2	1	0	0	0	0	1	0	0
3	0	0	0	1	0	1	1	0
4	0	0	1	0	0	0	1	1
5	1	0	0	0	0	0	0	0
6	0	1	1	0	0	0	1	0
7	0	0	1	1	0	1	0	1
8	0	0	1	0	0	1	0	0

邻接矩阵



邻接表

邻接矩阵存储的图：

访问 $|V|$ 个顶点需要 $O(|V|)$ 的时间

查找每个顶点的邻接点都需要 $O(|V|)$ 的时间，而总共有 $|V|$ 个顶点

时间复杂度 = $O(|V|^2)$

邻接表存储的图：

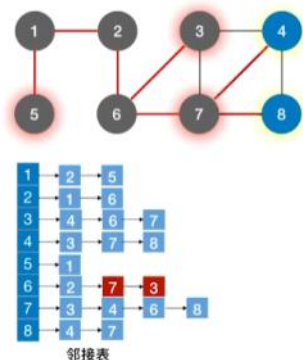
访问 $|V|$ 个顶点需要 $O(|V|)$ 的时间

查找各个顶点的邻接点共需要 $O(|E|)$ 的时间，

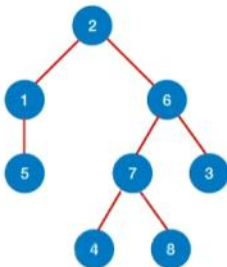
时间复杂度 = $O(|V|+|E|)$

BFS的访问可以出现生成树的，如下图从2节点开始的情况：

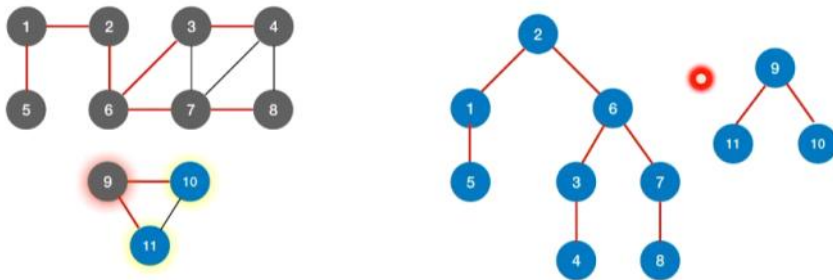
广度优先生成树



邻接表



对于非连通图来说，BFS访问会出现生成森林：



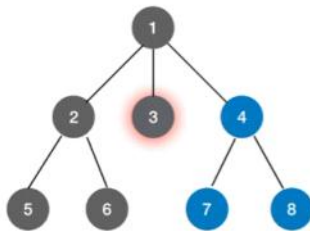
对非连通图的广度优先遍历，可得到广度优先生成森林

2、深度优先搜索DFS

也是老朋友了，是通过栈的递归实现的，以下是树的DFS算法（先根遍历）

==> bilibili

树的深度优先遍历

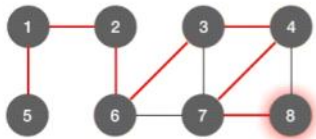


```
//树的先根遍历
void PreOrder(TreeNode *R){
    if (R!=NULL){
        visit(R); //访问根节点
        while(R还有下一个子树T)
            PreOrder(T); //先根遍历下一棵子树
    }
}
```

和广度优先一样，也会访问到已经访问过的节点，需要设置一个visited表示有没有访问过

==> bilibili

图的深度优先遍历



```
bool visited[MAX_VERTEX_NUM]; //访问标记数组

void DFS(Graph G,int v){ //从顶点v出发，深度优先遍历图G
    visit(v); //访问顶点v
    visited[v]=TRUE; //设已访问标记
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))
        if(!visited[w]){ //w为u的尚未访问的邻接顶点
            DFS(G,w);
        } //if
}
```



函数调用栈

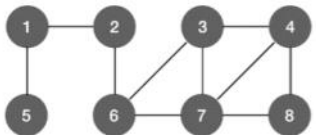
	1	2	3	4	5	6	7	8
visited	true	true	true	true	true	true	true	true

初始都为false

王道考研/CSKAOYAN.COM

==> bilibili

DFS算法（Final版）



如果是非连通图，则无法遍历完所有结点

```
bool visited[MAX_VERTEX_NUM]; //访问标记数组

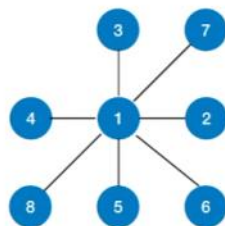
void DFSTraverse(Graph G){ //对图G进行深度优先遍历
    for(v=0;v<G.vexnum;++v)
        visited[v]=FALSE; //初始化已访问标记数据
    for(v=0;v<G.vexnum;++v)
        if(!visited[v])
            DFS(G,v);
}

void DFS(Graph G,int v){ //从顶点v出发，深度优先遍历图G
    visit(v); //访问顶点v
    visited[v]=TRUE; //设已访问标记
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))
        if(!visited[w]){ //w为u的尚未访问的邻接顶点
            DFS(G,w);
        } //if
}
```

	1	2	3	4	5	6	7	8	9	10	11
visited	true	true	true	true	true	true	true	true	false	false	false

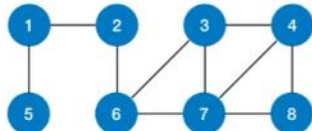
王道考研/CSKAOYAN.COM

复杂度分析



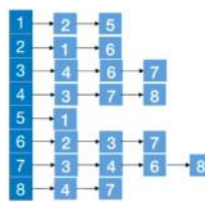
王道考研/CSKAQYAN.COM

复杂度分析



	1	2	3	4	5	6	7	8
1	0	1	0	0	1	0	0	0
2	1	0	0	0	0	1	0	0
3	0	0	0	1	0	1	1	0
4	0	0	1	0	0	0	1	1
5	1	0	0	0	0	0	0	0
6	0	1	1	0	0	0	1	0
7	0	0	1	1	0	1	0	1
8	0	0	0	1	0	0	1	0

邻接矩阵



邻接表

时间复杂度=访问各结点所需时间+探索各条边所需时间

邻接矩阵存储的图：

访问 $|V|$ 个顶点需要 $O(|V|)$ 的时间

查找每个顶点的邻接点都需要 $O(|V|)$ 的时间，而总共有 $|V|$ 个顶点

时间复杂度= $O(|V|^2)$

邻接表存储的图：

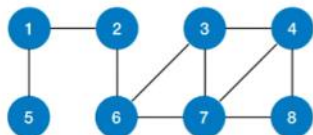
访问 $|V|$ 个顶点需要 $O(|V|)$ 的时间

查找各个顶点的邻接点共需要 $O(|E|)$ 的时间，

时间复杂度= $O(|V|+|E|)$

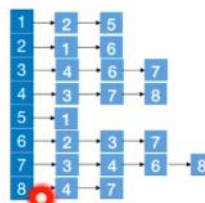
和BFS一样，用邻接表存储的话访问序列可能不唯一

深度优先遍历序列

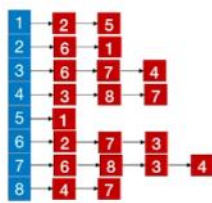


	1	2	3	4	5	6	7	8
1	0	1	0	0	1	0	0	0
2	1	0	0	0	0	1	0	0
3	0	0	0	1	0	1	1	0
4	0	0	1	0	0	0	1	1
5	1	0	0	0	0	0	0	0
6	0	1	1	0	0	0	1	0
7	0	0	1	1	0	1	0	1
8	0	0	0	1	0	0	1	0

邻接矩阵



邻接表



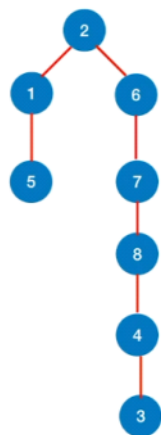
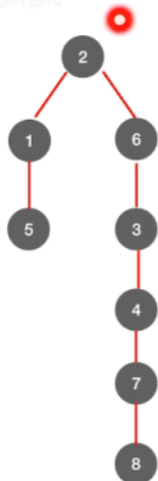
邻接表

同一个图的邻接矩阵表示方式唯一，因此深度优先遍历序列唯一

同一个图邻接表表示方式不唯一，因此深度优先遍历序列不唯一

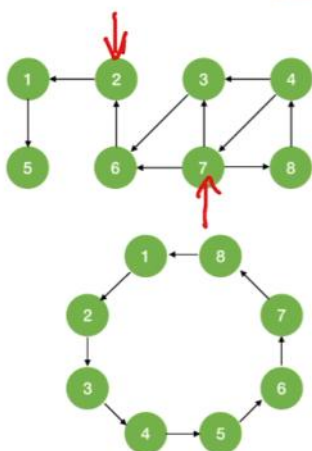
王道考研/CSKAQYAN.COM

深度优先生成树



王道考研/CSKAOYAN.COM

图的遍历与图的连通性



对有向图进行BFS/DFS遍历
调用BFS/DFS函数的次数要具体问题具体分析

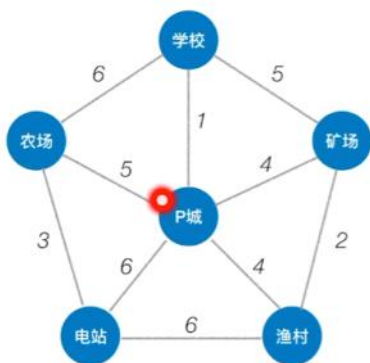
若起始顶点到其他各顶点都有路径，则只需调用1次
BFS/DFS 函数

对于强连通图，从任一结点出发都只需调用1次 BFS/DFS

王道考研/CSKAOYAN.COM

四、最小生成树

最小生成树（最小代价树）

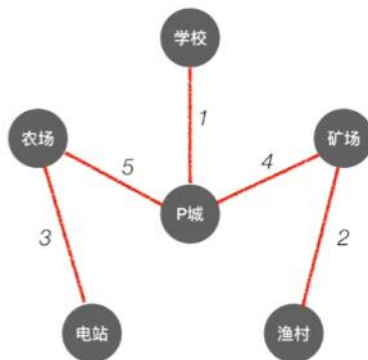
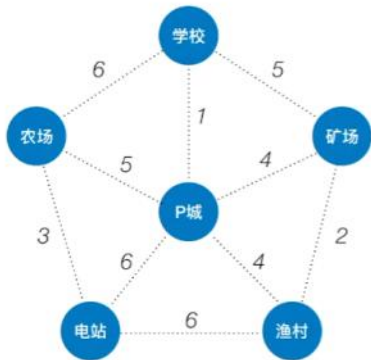


道路规划要求：所有地方都
连通，且成本尽可能的低

最小生成树研究的是一个带权的连通无向图，每棵树的权值不一样，设 R 为 G 的生成树集合，如果 T 为 R 的边的权值之和最小的生成树，那就是最小生成树MST。只有连通图才是生成树，非连通图的叫生成森林。

1、Prim算法

从某一顶点开始构建生成树，每次将代价最小的新顶点放入生成树，直到所有顶点都放入了。

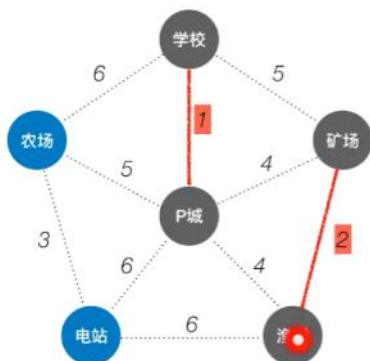


最小代价: 15

如上图, 从P城开始扫描已有的边, 发现去学校的最短, 连接上, 然后扫描已有的边发现矿场的最短, 连上在扫描, 然后就是渔村、农场、电站。

2、Kruskal算法

从边入手, 每次选择权值最小的边, 使得边的两头连通, 如果两端已经连通的就不选。



Kruskal 算法 (克鲁斯卡尔):

每次选择一条权值最小的边, 使这条边的两头连通 (原本已经连通的就不选)

直到所有结点都连通

Prim 算法 (普里姆):

从某一个顶点开始构建成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

时间复杂度: $O(|V|^2)$
适用于边稠密图

$|E|$ 大 ✓

Kruskal 算法 (克鲁斯卡尔):

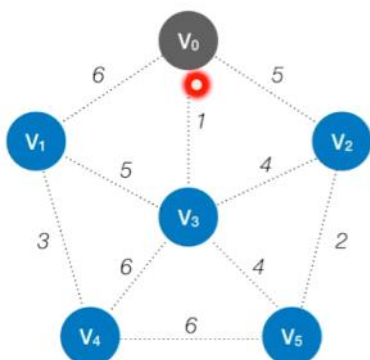
每次选择一条权值最小的边, 使这条边的两头连通 (原本已经连通的就不选)

直到所有结点都连通

时间复杂度: $O(|E|\log_2|E|)$
适用于边稀疏图

$|E|$ 小 ✓

Prim 算法的实现思想



初始: 从V0开始

标记各节点是否已加入树

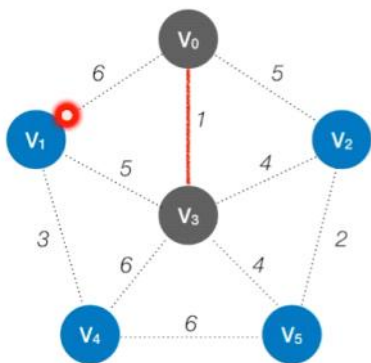
isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✗	✗	✗	✗

lowCost[6]

0	6	5	1	∞	∞
---	---	---	---	---	---

各节点加入树的最低代价



第1轮：循环遍历所有个结点，找到lowCast最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

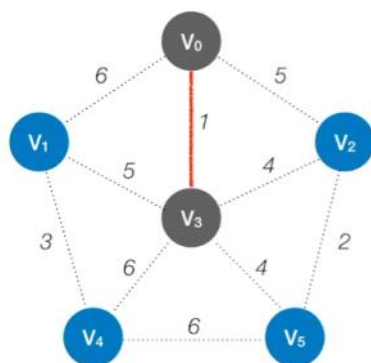
V0	V1	V2	V3	V4	V5
✓	✗	✗	✓	✗	✗

lowCost[6]

0	6	5	1	∞	∞
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCast值



第1轮：循环遍历所有个结点，找到lowCast最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

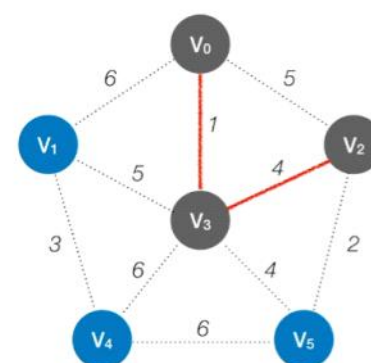
V0	V1	V2	V3	V4	V5
✓	✗	✗	✓	✗	✗

lowCost[6]

0	5	4	1	6	4
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCast值



第2轮：循环遍历所有个结点，找到lowCast最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

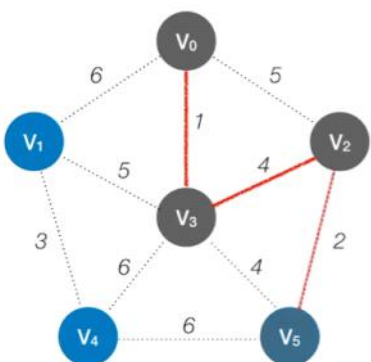
V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✗

lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCast值



第3轮：循环遍历所有个结点，找到lowCast最低的，且还没加入树的顶点

标记各节点是否已加入树

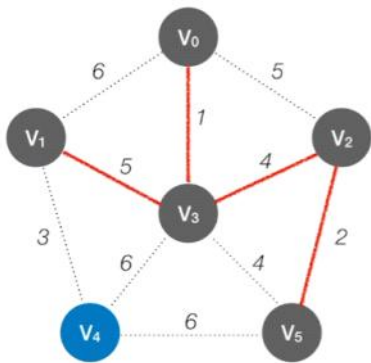
isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✗

lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价



第4轮：循环遍历所有个结点，找到lowCast最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✓	✓	✓	✗	✓

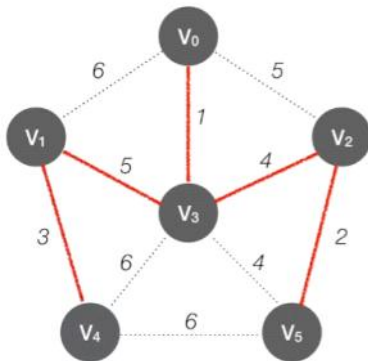


lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCast值



第5轮：循环遍历所有个结点，找到lowCast最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✓	✓	✓	✓	✓



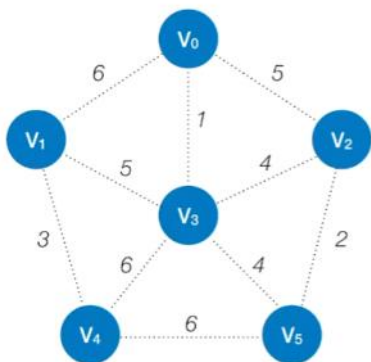
lowCost[6]

0	5	4	1	3	2
---	---	---	---	---	---

各节点加入树的最低代价

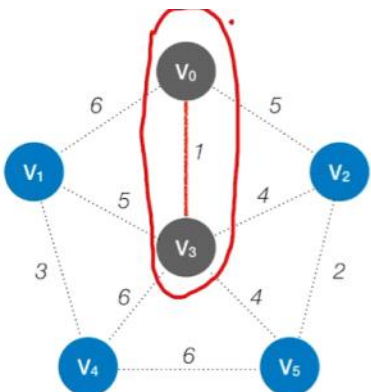
再次循环遍历，更新还没加入的各个顶点的lowCast值

Kruskal 算法的实现思想



初始：将各条边按权值排序

weight	Vertex1	Vertex2
1	V0	V3
2	V2	V5
3	V1	V4
4	V2	V3
4	V3	V5
5	V0	V2
5	V1	V3
6	V0	V1
6	V3	V4
6	V4	V5

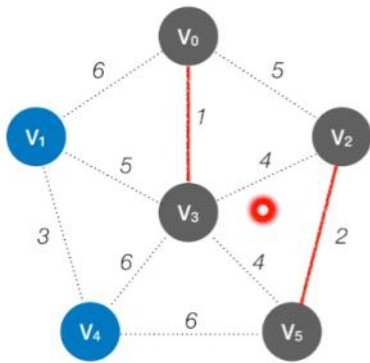


第1轮：检查第1条边的两个顶点是否连通（是否属于同一个集合）



weight	Vertex1	Vertex2
1	V0	V3
2	V2	V5
3	V1	V4
4	V2	V3
4	V3	V5
5	V0	V2
5	V1	V3
6	V0	V1
6	V3	V4
6	V4	V5

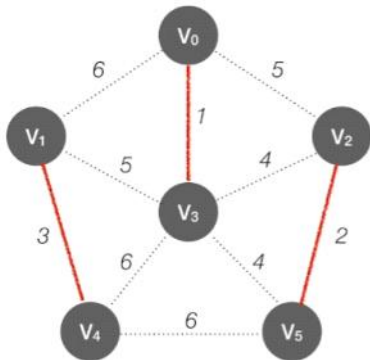
不连通，连起来！



第2轮：检查第2条边的两个顶点是否连通（是否属于同一个集合）

weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

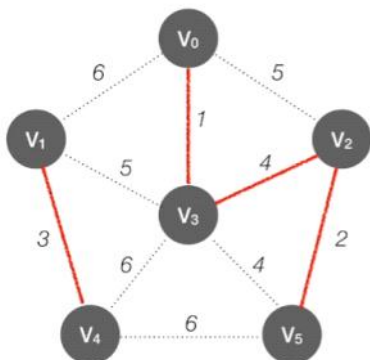
不连通，连起来！



第3轮：检查第3条边的两个顶点是否连通（是否属于同一个集合）

weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

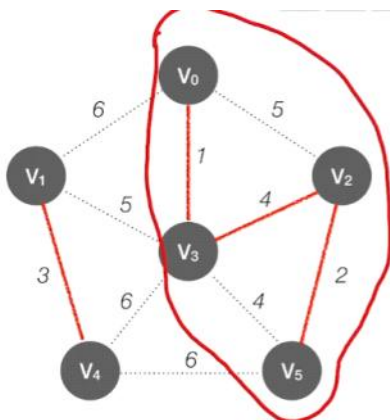
不连通，连起来！



第4轮：检查第4条边的两个顶点是否连通（是否属于同一个集合）

weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

不连通，连起来！



第5轮：检查第5条边的两个顶点是否连通（是否属于同一个集合）

weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

已连通，跳过！

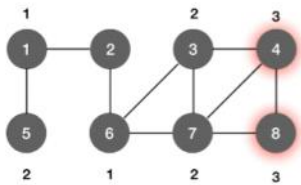
五、最短路径问题

最短路径问题可以分为单源最短路径（求两点之间最短路径）和各顶点之间的最短路径（点与点之间互相往来怎么走最近），前者用BFS和Dijkstra算法，后者用Floyd算法。

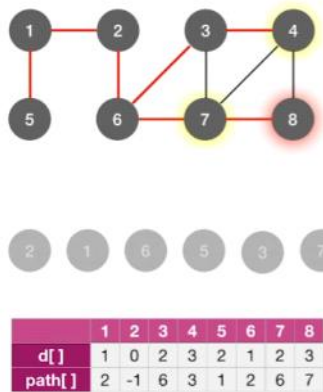


1、BFS求无权图单源最短路径

这个就是在BFS遍历的过程中，每遍历到一个节点，就将step步骤值存储到这个节点中。如下如从2开始：



代码实现



```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){ //BFS算法主过程
        DeQueue(Q,u); //队头元素u出队
        for(w=FirstNeighbor(G,u);w!=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){ //w为u的尚未访问的邻接顶点
                d[w]=d[u]+1; //路径长度加1
                path[w]=u; //最短路径应是从u到w
                visited[w]=TRUE; //设已访问标记
                EnQueue(Q,w); //顶点w入队
            }
        }
    }
}
```

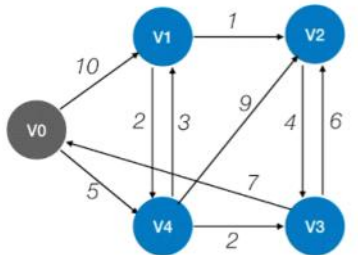
	1	2	3	4	5	6	7	8
d[]	1	0	2	3	2	1	2	3
path[]	2	-1	6	3	1	2	6	7

	1	2	3	4	5	6	7	8
visited	true	true	true	true	true	true	true	true

如上图d[]表示遍历到每个节点时处于第几步，path[]表示这个节点是从哪个节点连过来的（来的路），往回溯源即可得到路径。

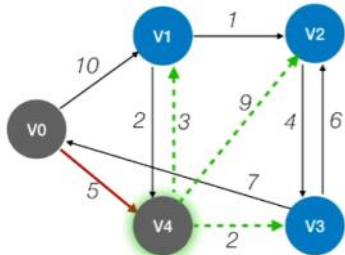
2、dijkstra算法解决有向带权图最短路径（其实无向图也可以用）

初始：从V0开始，初始化三个数组信息如下



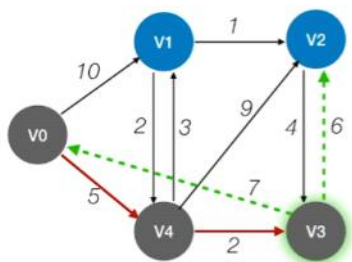
标记各顶点是否已找到最短路径	V0	V1	V2	V3	V4
final[5]	✓	✗	✗	✗	✗
最短路径长度	0	10	∞	∞	5
dist[5]	0	10	∞	∞	5
路径上的前驱	-1	0	-1	-1	0
path[5]	-1	0	-1	-1	0

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点V1，令final[i]=ture。



标记各顶点是否已找到最短路径	V0	V1	V2	V3	V4
final[5]	✓	✗	✗	✗	✗
最短路径长度	0	8	14	7	5
dist[5]	0	8	14	7	5
路径上的前驱	-1	4	4	4	0
path[5]	-1	4	4	4	0

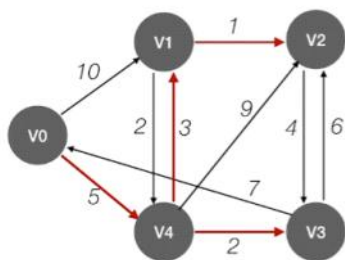
检查所有邻接自 V1 的顶点，若其 final 值为false，则更新 dist 和 path 信息



第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点V_i，令final[i]=ture。

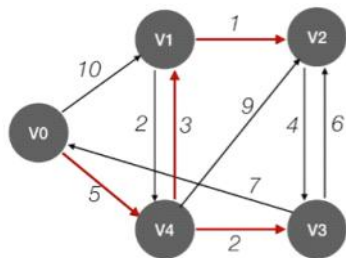
标记各顶点是否已找到最短路径	V0	V1	V2	V3	V4
final[5]	✓	✗	✗	✓	✓
最短路径长度	0	8	13	7	5
路径上的前驱	-1	4	3	4	0

检查所有邻接自 V_i 的顶点，若其 final 值为false，则更新 dist 和 path 信息



标记各顶点是否已找到最短路径	V0	V1	V2	V3	V4
final[5]	✓	✓	✓	✓	✓
最短路径长度	0	8	9	7	5
路径上的前驱	-1	4	1	4	0

检查所有邻接自 V_i 的顶点，若其 final 值为false，则更新 dist 和 path 信息

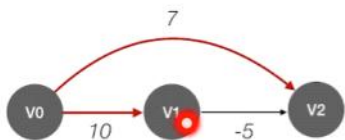


标记各顶点是否已找到最短路径	V0	V1	V2	V3	V4
final[5]	✓	✓	✓	✓	✓
最短路径长度	0	8	9	7	5
路径上的前驱	-1	4	1	4	0

V0到V2 的最短(带权)路径长度为：dist[2] = 9

通过 path[] 可知，V0到V2 的最短(带权)路径：V2 ← V1 ← V4 ← V0

注意如果有路径的权值为负数，那么迪杰斯科拉会失效。



标记各顶点是否已找到最短路径	V0	V1	V2
final[3]	✓	✓	✓
最短路径长度	0	10	7
路径上的前驱	-1	0	0

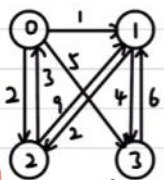
事实上V0到V2 的最短带权路径长度为 5

结论：Dijkstra 算法不适用于有负权值的带权图

3、Floyd算法

只讲解题方法：

每轮将K行K列以及主对角线固定住，第一轮固定1行1列，然后2行2列，以此类推。固定住后将行列对角线对应的值相加与原来的空对比即可，如果比原来的大就写原来的，比原来的小就更新最小值。

Floyd 回路 W_{ij} $O(V^3)$ 

- 1° 每轮定第k行k列与主对角线。
- 2° 对左列值相加与现有比较,取小。

	0	1	2	3
$A^{(1)}$	0	1	2	5
$A^{(2)}$	0	0	2	4
$A^{(3)}$	0	0	2	4
$A^{(4)}$	0	0	2	4

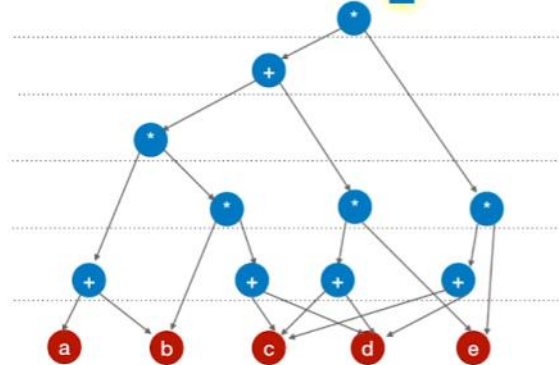
六、有向无环图

1、最简有向无环图表达式的解题方法

解题方法

$$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



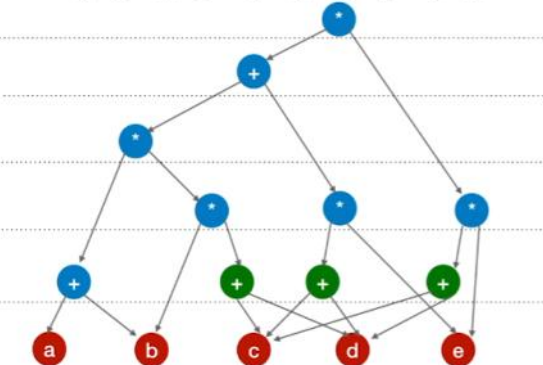
Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序 (先后顺序有点出入无所谓)

Step 3: 按顺序加入运算符, 注意“分层”

$$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

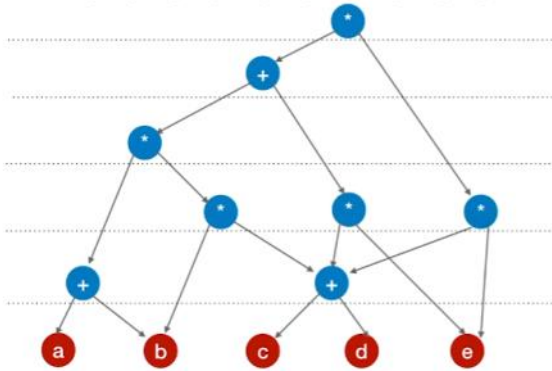
Step 2: 标出各个运算符的生效顺序 (先后顺序有点出入无所谓)

Step 3: 按顺序加入运算符, 注意“分层”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

$$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

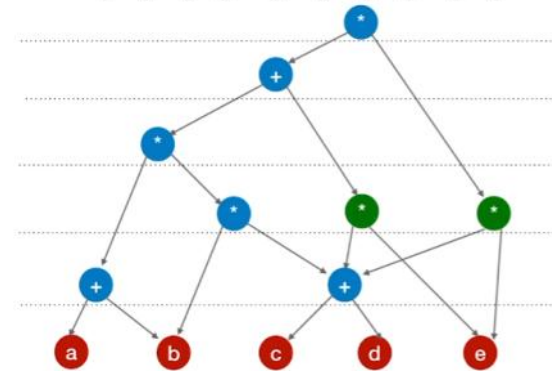
Step 2: 标出各个运算符的生效顺序 (先后顺序有点出入无所谓)

Step 3: 按顺序加入运算符, 注意“分层”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

$$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

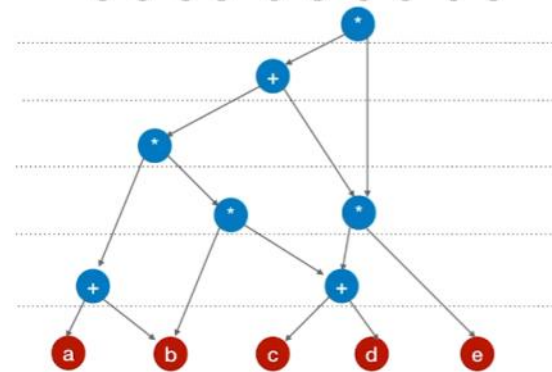
Step 2: 标出各个运算符的生效顺序 (先后顺序有点出入无所谓)

Step 3: 按顺序加入运算符, 注意“分层”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

$$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

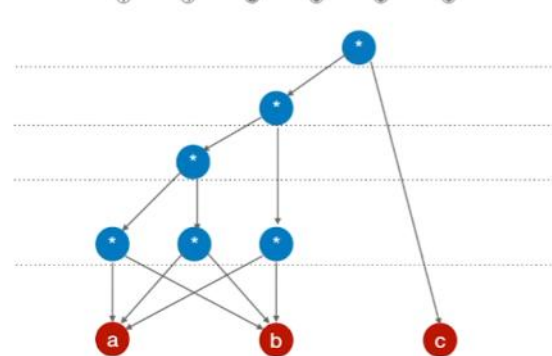
Step 2: 标出各个运算符的生效顺序 (先后顺序有点出入无所谓)

Step 3: 按顺序加入运算符, 注意“分层”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

$$(a*b)*(a*b)*(a*b)*c$$

① ④ ② ⑤ ③ ⑥

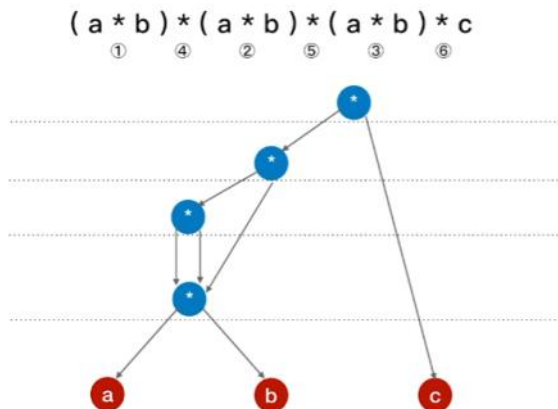


Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序 (先后顺序有点出入无所谓)

Step 3: 按顺序加入运算符, 注意“分层”

Step 4: 从底向上逐层检查同层的运算符是否可以合体



Step 1: 把各个操作数不重复地排成一排

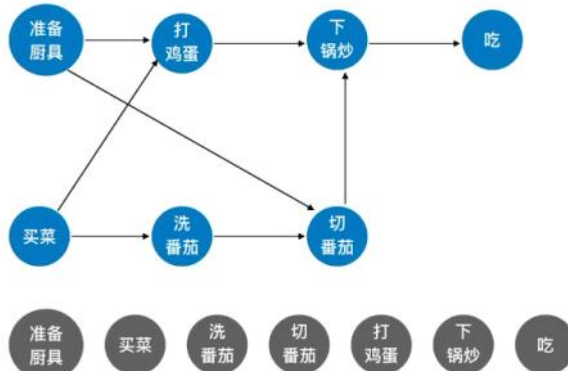
Step 2: 标出各个运算符的生效顺序 (先后顺序有点出入无所谓)

Step 3: 按顺序加入运算符, 注意“分层”

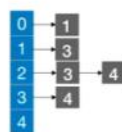
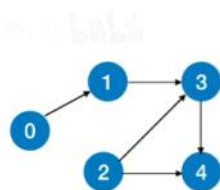
Step 4: 从底向上逐层检查同层的运算符是否可以合体

2、拓扑排序

就是找到做事的先后顺序, 在图中从AOV网中选择一个入度为0的顶点输出, 然后删除该顶点和他为起点的有向边, 一直重复直到全部删完。



每个AOV网可能有多个排序序列, 只能在有向无环图 (没有回路) 中使用拓扑排序。



每个顶点都需要处理一次

每条边都需要处理一次

```
bool TopologicalSort(Graph G){
    InitStack(S); //初始化栈, 存储入度为0的顶点
    for(int i=0; i<G.vexnum; i++){
        if(indegree[i]==0)
            Push(S, i); //将所有入度为0的顶点进栈
    }
    int count=0; //计数, 记录当前已经输出的顶点数
    while(!IsEmpty(S)){ //栈不为空, 则存在入度为0的顶点
        Pop(S, i); //栈顶元素出栈
        print[count++]=i; //输出顶点i
        for(p=G.vertices[i].firstarc; p=p->nextarc){
            //将所有i指向的顶点的入度减1, 并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(--indegree[v]==0)
                Push(S, v); //入度为0, 则入栈
        }
    }
    if(count<G.vexnum)
        return false; //排序失败, 有向图中有回路
    else
        return true; //拓扑排序成功
}
```

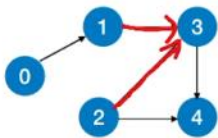
时间复杂度: $O(|V|+|E|)$

若采用邻接矩阵, 则需 $O(|V|^2)$

3、逆拓扑排序

逆拓扑排序就是将结果反推得到逆序列。

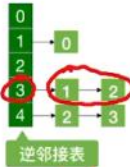
逆拓扑排序的实现



练习：模仿拓扑排序的思想实现**逆拓扑排序**
思考：使用不同的存储结构来对时间复杂度的影响

	0	1	2	3	4
0	0	1	0	0	0
1	0	0	0	1	0
2	0	0	0	1	1
3	0	0	0	0	1
4	0	0	0	0	0

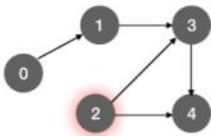
邻接矩阵



逆邻接表

```
bool TopologicalSort(Graph G){
    InitStack(S); //初始化栈，存储入度为0的顶点
    for(int i=0; i<G.vexnum; i++)
        if(indegree[i]==0)
            Push(S,i); //将所有入度为0的顶点进栈
    int count=0; //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){ //栈不为空，则存在入度为0的顶点
        Pop(S,i); //栈顶元素出栈
        print(count++); //输出顶点i
        for(p=G.vertices[i].firstarc; p!=NULL; p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈S
            v=p->adjvex;
            if(--indegree[v]==0)
                Push(S,v); //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false; //排序失败，有向图中有回路
    else
        return true; //拓扑排序成功
}
```

逆拓扑排序的实现 (DFS算法)



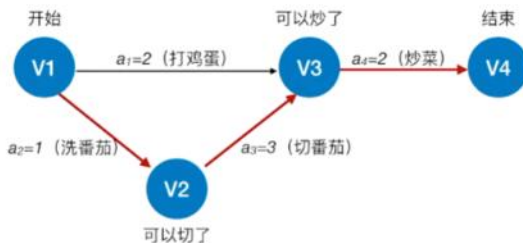
递归栈

```
void DFSTraverse(Graph G){ //对图G进行深度优先遍历
    for(v=0; v<G.vexnum; ++v)
        visited[v]=FALSE; //初始化已访问标记数据
    for(v=0; v<G.vexnum; ++v)
        if(!visited[v])
            DFS(G,v);
}

void DFS(Graph G,int v){ //从顶点v出发，深度优先遍历图G
    visited[v]=TRUE; //设已访问标记
    for(w=FirstNeighbor(G,v); w>=0; w=NextNeighbor(G,v,w)){
        if(!visited[w]){ //w为u的尚未访问的邻接顶点
            DFS(G,w);
        } //if
    }
    print(v); //输出顶点
}
```

逆拓扑排序序列：4 3 1 0 2

4、关键路径

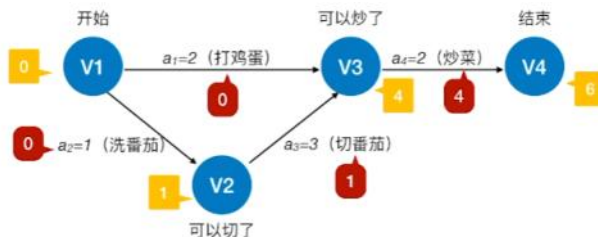


从源点到汇点的有向路径可能有多条，所有路径中，具有最大路径长度的路径称为**关键路径**，而把关键路径上的活动称为**关键活动**

完成整个工程的最短时间就是关键路径的长度，若关键活动不能按时完成，则整个工程的完成时间就会延长

事件最早开始时间：某个顶点要开始最早要多久

活动最早开始时间：某条边的起点节点的事件最早开始时间



我好饿

最快多久可以开始做？
最快多久可以开切？
最快多久可以开炒？
最快多久可以吃？

现在，马上！
1分钟
4分钟
6分钟

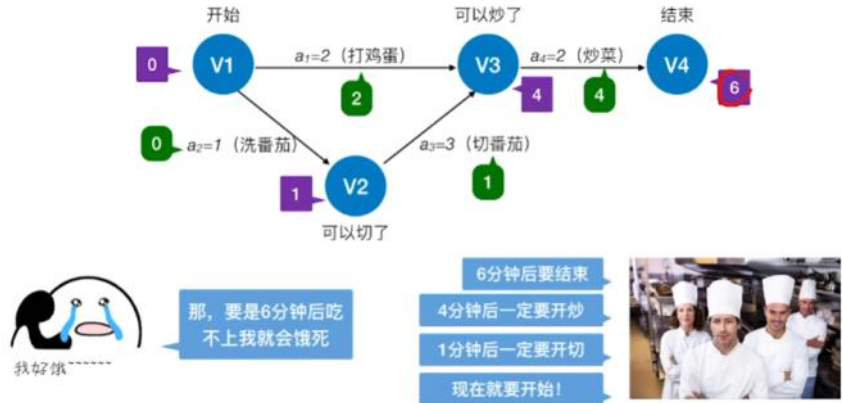


事件 v_i 的最早发生时间 e_i ——决定了所有从 v_i 开始的活动的最早开工时间

活动 a_i 的最早开始时间 $e(i)$ ——指该活动弧的起点所表示的事件的最早发生时间

事件的最迟发生时间：不推迟整个工程完成的前提下，该事件最迟必须发生的时间

活动的最迟开始时间：某条边活动终点的最迟发生时间减去该活动所需的时间



事件 v_k 的最迟发生时间 $vl(k)$ ——它是指在不推迟整个工程完成的前提下，该事件最迟必须发生的时间。

活动 a_i 的最迟开始时间 $li(i)$ ——它是指该活动弧的终点所表示事件的最迟发生时间与该活动所需时间之差。

时间余量：就是在不增加工期的情况下，该活动可以拖延的时间，时间余量=活动最迟开始时间-活动最早开始时间

如果一个活动的**时间余量为0**，那么说明该活动必须如期完成，是**关键活动**。

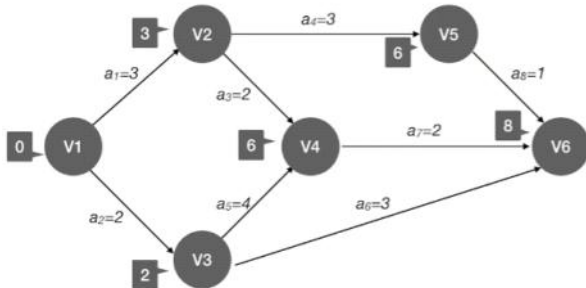
求关键路径的步骤：

① 求所有事件的最早发生时间 $ve()$

按**拓扑排序**序列，依次求各个顶点的 $ve(k)$ ：

$ve(\text{源点}) = 0$

$ve(k) = \max\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱



拓扑序列：V1、V3、V2、V5、V4、V6

$ve(1) = 0$

$ve(3) = 2$

$ve(2) = 3$

$ve(5) = 6$

$ve(4) = \max\{ve(2) + 2, ve(3) + 4\} = 6$

$ve(6) = \max\{ve(5) + 1, ve(4) + 2, ve(3) + 3\} = 8$

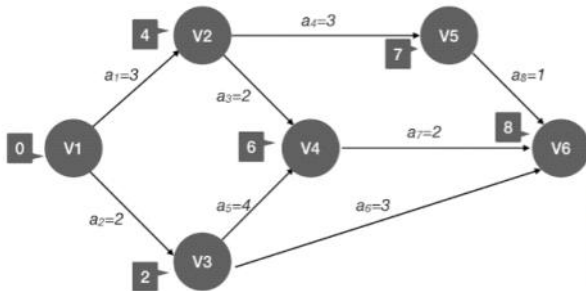
	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8

② 求所有事件的最迟发生时间 $vl()$

按**逆拓扑排序**序列，依次求各个顶点的 $vl(k)$ ：

$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \min\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继



逆拓扑序列：V6、V5、V4、V2、V3、V1

$vl(6) = 8$

$vl(5) = 7$

$vl(4) = 6$

$vl(2) = \min\{vl(5) - 1, vl(4) - 2\} = 4$

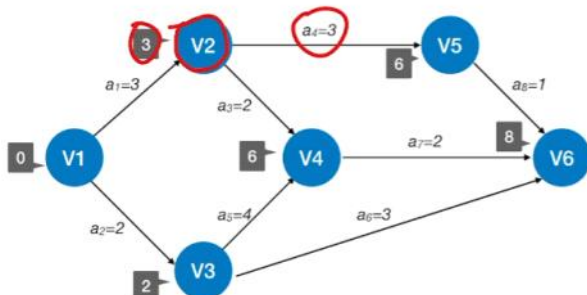
$vl(3) = \min\{vl(4) - 4, vl(6) - 3\} = 2$

$vl(1) = 0$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

③ 求所有活动的最早发生时间 $e()$

若边 $\langle v_k, v_j \rangle$ 表示活动 a_i ，则有 $e(i) = ve(k)$



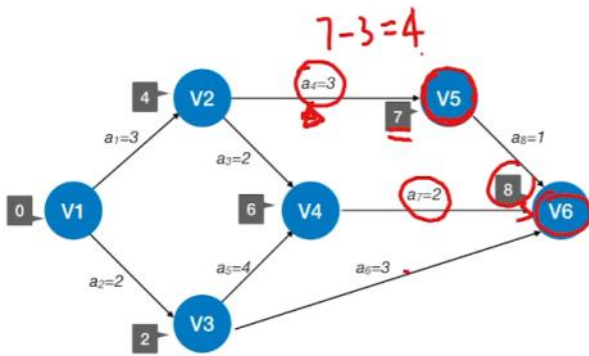
	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

	a1	a2	a3	a4	a5	a6	a7	a8
$e(k)$	0	0	3	3	2	2	6	6

④ 求所有活动的最迟发生时间 $l(i)$

若边 $\langle v_i, v_j \rangle$ 表示活动 a_i , 则有 $l(i) = vl(j) - \text{Weight}(v_i, v_j)$

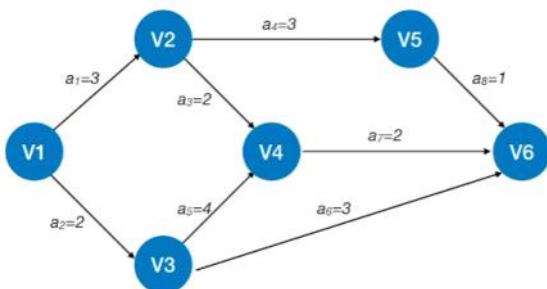
	V1	V2	V3	V4	V5	V6
ve(k)	0	3	2	6	6	8
vl(k)	0	4	2	6	7	8



⑤ 求所有活动的时间余量 $d(i)$

$d(i) = l(i) - e(i)$

	V1	V2	V3	V4	V5	V6
ve(k)	0	3	2	6	6	8
vl(k)	0	4	2	6	7	8



	a1	a2	a3	a4	a5	a6	a7	a8
e(k)	0	0	3	3	2	2	6	6
l(k)	1	0	4	4	2	5	6	7
d(k)	1	0	1	1	0	3	0	1

① 求所有事件的最早发生时间 $ve()$

② 求所有事件的最迟发生时间 $vl()$

③ 求所有活动的最早发生时间 $e()$

④ 求所有活动的最迟发生时间 $l()$

⑤ 求所有活动的时间余量 $d()$

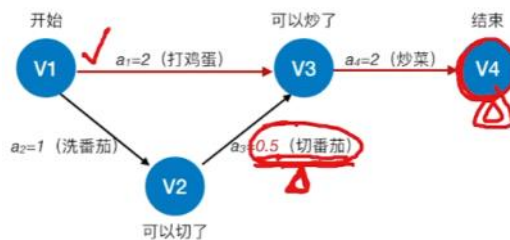
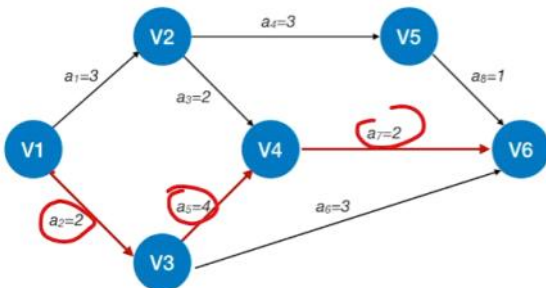
$d(k)=0$ 的活动就是关键活动, 由关键活动可得关键路径

	V1	V2	V3	V4	V5	V6
ve(k)	0	3	2	6	6	8
vl(k)	0	4	2	6	7	8

	a1	a2	a3	a4	a5	a6	a7	a8
e(k)	0	0	3	3	2	2	6	6
l(k)	1	0	4	4	2	5	6	7
d(k)	1	0	1	1	0	3	0	1

关键活动: a_2, a_5, a_7

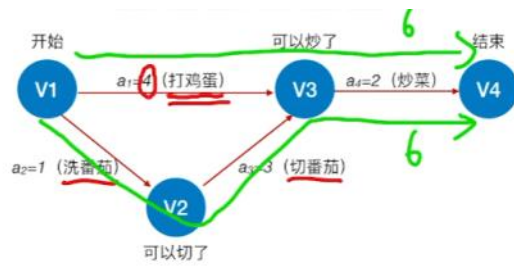
关键路径: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V6$



若关键活动耗时增加, 则整个工程的工期将增长

缩短关键活动的时间, 可以缩短整个工程的工期

当缩短到一定程度时, 关键活动可能会变成非关键活动



可能有多条关键路径，只提高一条关键路径上的关键活动速度并不能缩短整个工程的工期，只有加快那些包括在所有关键路径上的关键活动才能达到缩短工期的目的。

查找

2025年9月16日星期二 下午3:01

一、基本概念

1、查找相关

查找：找想要的元素

查找表：在查找表里查元素

关键字：查找的主键，关键字查找的唯一标识

2、对查找表常见操作

①查找符合条件的数据元素

②插入、删除某个数据元素

学号	姓名	语文	数学	英语
202001	铁柱	99	97	98
202002	狗剩	86	23	90
202003	旺财	43	66	76
202004	愣子	12	16	11
202005	铁柱	9	17	25

只需进行操作①——静态查找表

仅关注查找速度即可

订单号	菜品	下单时间	已完成
1	黄焖Jimmy饭	30min	是
2	鱼香Rose	20min	否
3	皮蛋solo粥	15min	是
4	梅菜Crow	10min	否
5	过桥Michelle	2min	否

也要进行操作②——动态查找表

除了查找速度，也要关注
插/删操作是否方便实现

3、查找算法的评价指标

查找长度——在查找运算中，需要对比关键字的次数称为查找长度

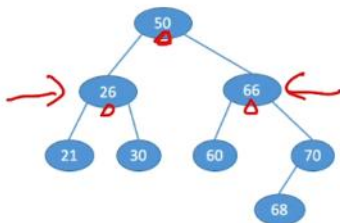
平均查找长度 (ASL, Average Search Length) —— 所有查找过程中进行关键字的比较次数的平均值



$$ASL = \sum_{i=1}^n P_i C_i$$

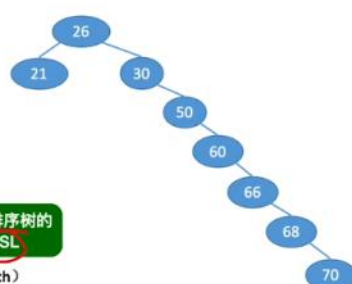
数据元素个数 n
查找第 i 个元素的查找长度 C_i
查找第 i 个元素的概率 P_i

类似于二叉排序树，平均查找长度的计算方法都是相似的

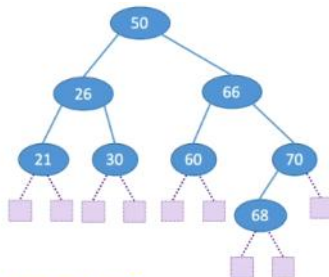


查找成功的平均查找长度 ASL (Average Search Length)

$$ASL = (1*1 + 2*2 + 3*3 + 4*4)/8 = 2.625$$



$$ASL = (1*1 + 2*2 + 3*3 + 4*4 + 5*5 + 6*6 + 7*7)/8 = 3.75$$



查找失败的平均查找长度 ASL (Average Search Length)

$$ASL = (3*7 + 4*2)/9 = 3.22$$

评价一个查找算法的效率时，
通常考虑查找成功/查找失败
两种情况的 ASL



$$ASL = (2*3 + 3+4+5+6+7*2)/9 = 4.22$$

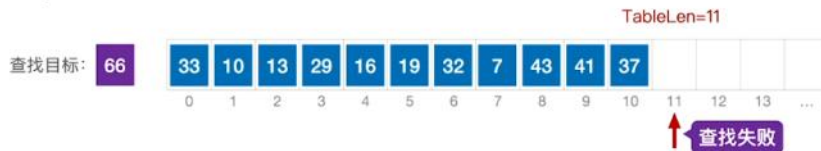
二、顺序查找

从头到尾挨个查，通常用于线性表

顺序查找的实现

```
typedef struct{           //查找表的数据结构 (顺序表)
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

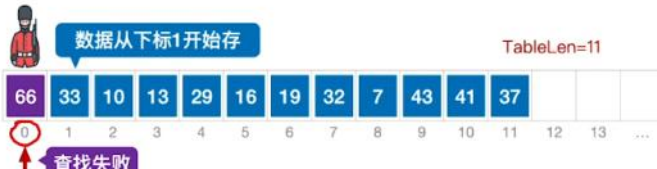
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen;&& ST.elem[i]!=key; ++i);
    //查找成功,则返回元素下标; 查找失败,则返回-1
    return i==ST.TableLen? -1 : i;
}
```



还有一种形式就是将数组下标0的位置空出来存放关键字, 这个关键字称为哨兵, 然后从后往前进行查找, 方便判断值是否越界

```
typedef struct{           //查找表的数据结构 (顺序表)
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;       //哨兵
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key; --i); //从后往前找
    return i;             //查找成功,则返回元素下标; 查找失败,则返回0
}
```



在分析查找效率的情况下, 每个元素被查找到的概率都是 $1/n$, 第一个被查找的需要查1步, 第二个被查找的元素需要查2步依次下去可以计算出查找成功的ASL, 另外查找失败的情况是遍历了 $n+1$ 个元素, 都是 $O(n)$ 的时间复杂度。



$$ASL_{成功} = \frac{1 + 2 + 3 + \dots + n}{n} = \frac{n+1}{2}$$

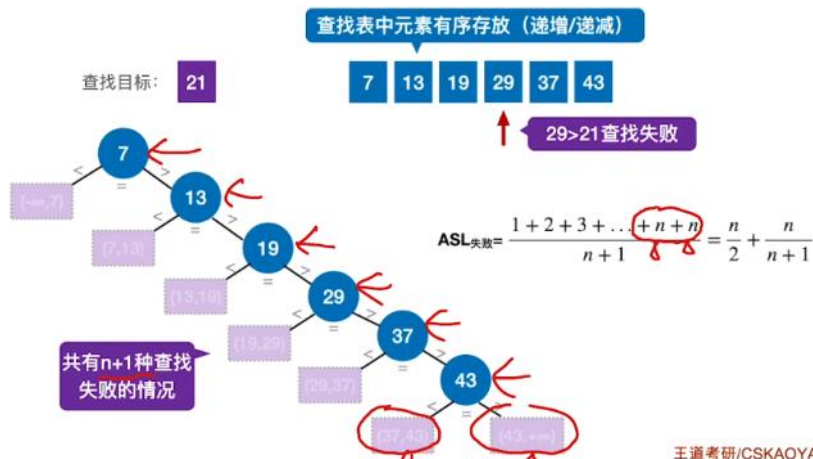
$$ASL_{失败} = n+1$$

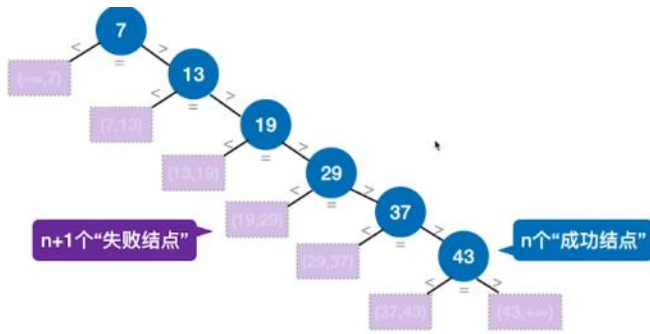
$O(n)$



2、顺序查找的优化 (有序表)

有序表的元素都是有序存放的, 一个个查找的时候如果要查21, 在经过29后就可以停下来了, 因为后面的更大更不可能是21, 此时可以画出ASL的判定树:





一个成功结点的查找长度 = 自身所在层数
 一个失败结点的查找长度 = 其父节点所在层数
 默认情况下, 各种失败情况或成功情况都等概率发生

3、顺序查找的优化 (被查概率不等)

如果将被查概率大的元素放在前面, 也能提高查找效率:

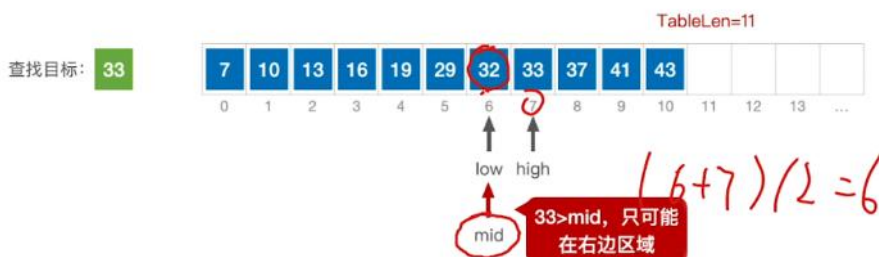
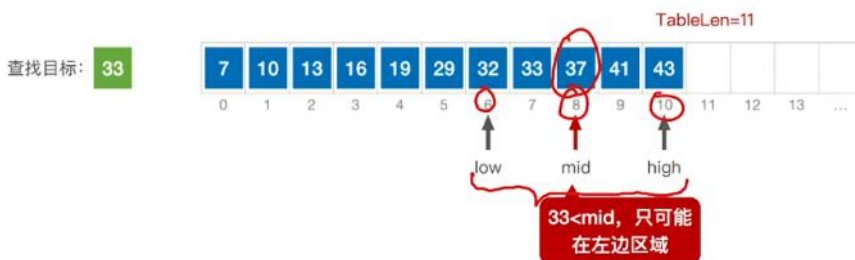
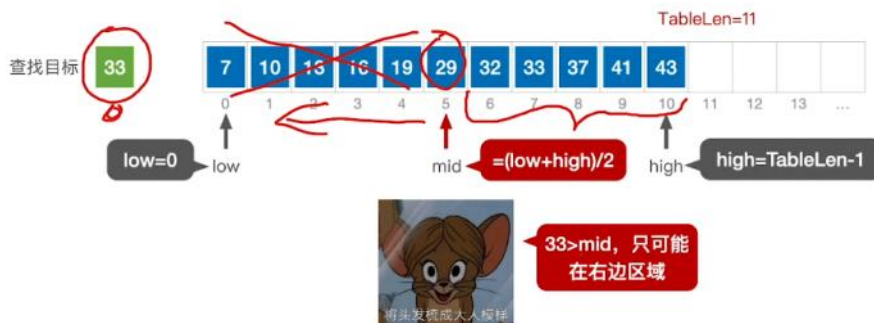
被查概率
 7: 15%
 13: 5%
 19: 10%
 29: 40%
 37: 28%
 43: 2%

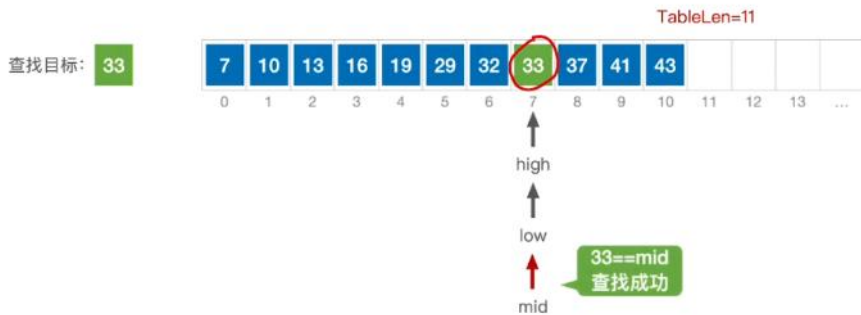
7 13 19 29 37 43
 $ASL_{成功} = 1 \cdot 0.15 + 2 \cdot 0.05 + 3 \cdot 0.1 + 4 \cdot 0.4 + 5 \cdot 0.28 + 6 \cdot 0.02 = 3.67$

被查概率大的放在靠前位置
 29 37 7 19 13 43
 $ASL_{成功} = 1 \cdot 0.4 + 2 \cdot 0.28 + 3 \cdot 0.15 + 4 \cdot 0.1 + 5 \cdot 0.05 + 6 \cdot 0.02 = 2.18$

三、折半查找

折半查找仅仅适用于有序的顺序表。每次相除将区间进行一分为二因此叫二分查找。





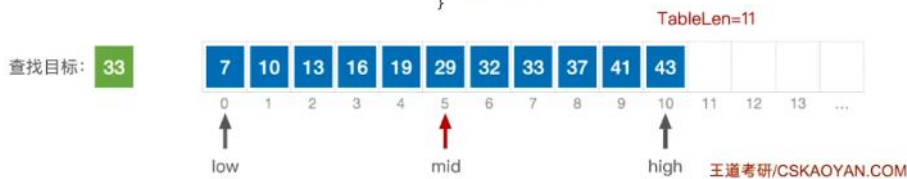
查找失败的例子:



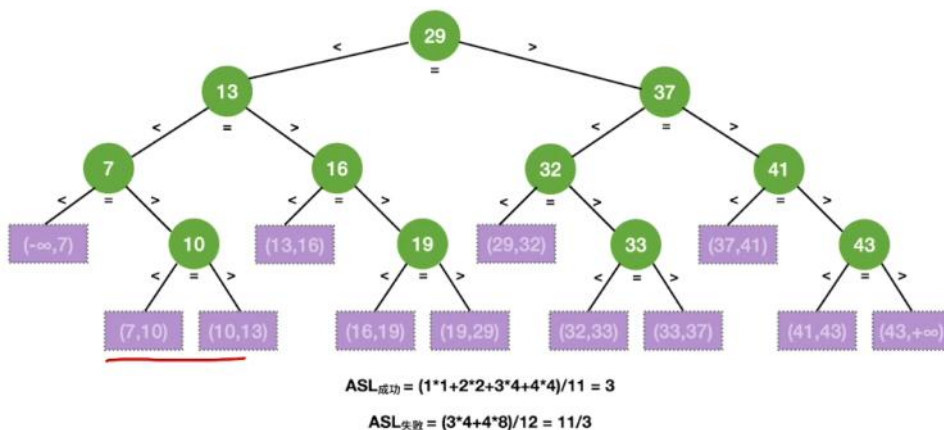
代码实现:

```
typedef struct{
    ElemType *elem;
    int TableLen;
}SSTable;

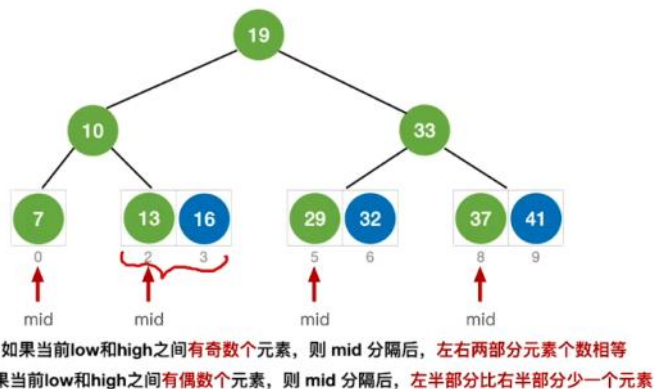
//折半查找
int Binary_Search(SSTable L, ElemType key){
    int low=0, high=L.TableLen-1, mid;
    while(low<=high){
        mid=(low+high)/2;
        if(L.elem[mid]==key)
            return mid;
        else if(L.elem[mid]>key)
            high=mid-1;
        else
            low=mid+1;
    }
    return -1;
}
```



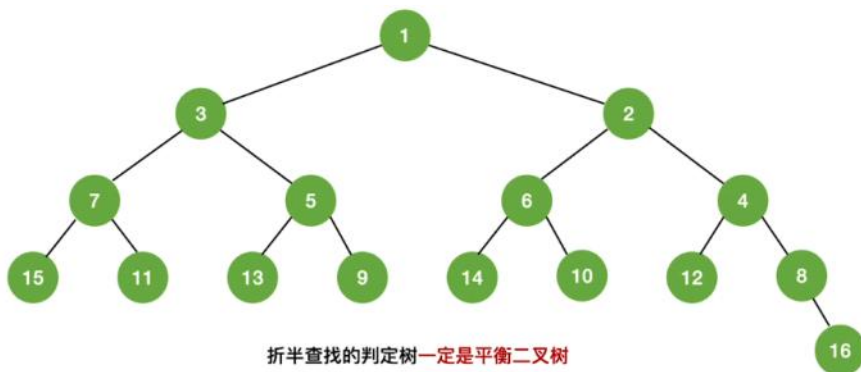
查找效率分析:



查找判定树的构造:



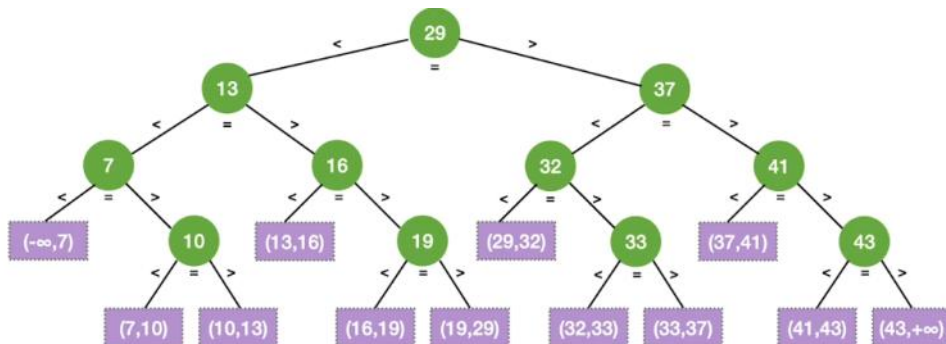
由此可见右子树比左子树最多多1个节点或者0个节点。



折半查找的判定树一定是平衡二叉树

折半查找的判定树中，只有最下面一层是不满的
因此，元素个数为 n 时树高 $h = \lceil \log_2(n+1) \rceil$

王道考研/CSKAOYAN.COM



判定树结点关键字：左<中<右，满足二叉排序树的定义
失败结点： $n+1$ 个（等于成功结点的空链域数量）

树高 $h = \lceil \log_2(n+1) \rceil$

查找成功的ASL $\leq h$

折半查找的时间复杂度 = $O(\log_2 n)$

注：该树高不包含失败结点

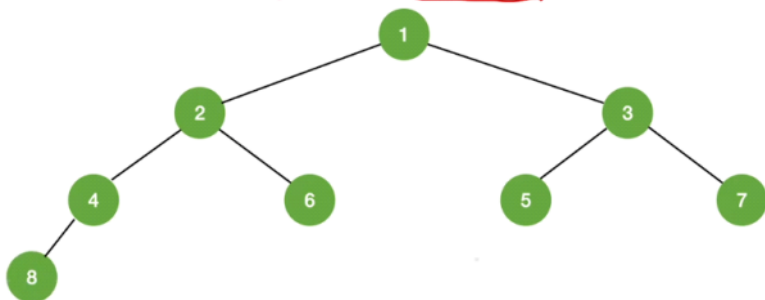
查找失败的ASL $\leq h$

王道考研/CSKAO

注意，上面的都是 $mid = (low + high) / 2$ 向下取整的情况，如果是向上取整，那么就是反过来即左子树比右子树的部分要多一个元素。

折半查找的判定树中，若 $mid = \lfloor (low + high) / 2 \rfloor$ ，则对于任何一个结点，必有：

左子树结点数-右子树结点数=0或1



四、分块查找

如果线性表里的元素可以分成多个块且每个块都有各自的特点，就可以向上建立一个索引表来进行分块查找。

“索引表”中保存每个分块的最大关键字和分块的存储区间

10	20	30	40	50
[0,1]	[2,5]	[6,8]	[9,10]	[11,13]

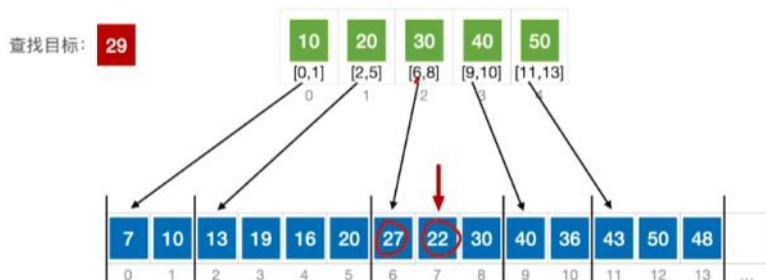
```
//索引表
typedef struct {
    ElemType maxValue;
    int low,high;
}Index;

//顺序表存储实际元素
ElemType List[100];
```

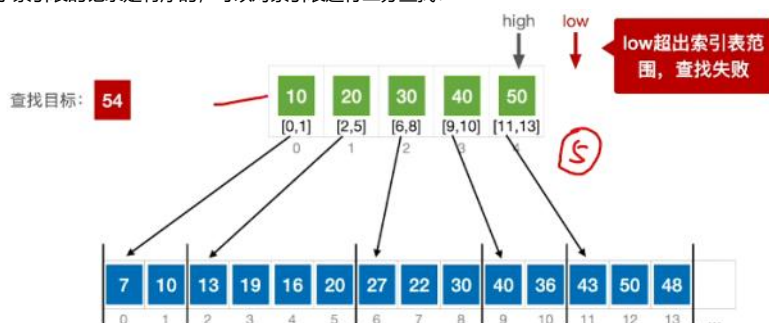


特点：块内无序、块间有序

如果要查就先查索引表找到对应的分块，然后在分块内顺序查找。如果查到了那就好，没查到的话就可以发现指针“越块”了，此时就说明查找失败。

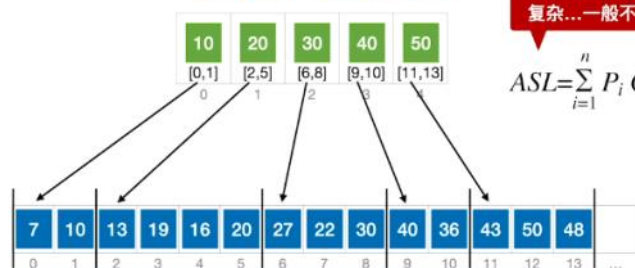


由于索引的记录是有序的，可以对索引表进行二分查找：



若索引表中不包含目标关键字，则折半查找索引表最终停在 $low > high$ ，要在 low 所指分块中查找

查找效率分析 (ASL)



查找失败的情况更复杂...一般不考

$$ASL = \sum_{i=1}^n P_i C_i$$

共有14个元素，各自被查概率为 $1/14$

若索引表采用顺序查找，则 7: 2次、10: 3次、13: 3次...
若索引表采用折半查找，则 30: 4次、27: 2次?



查找效率分析 (ASL)



假设，长度为 n 的查找表被均匀地分为 b 块，每块 s 个元素

设索引查找和块内查找的平均查找长度分别为 L_I 、 L_S ，则分块查找的平均查找长度为

$$ASL = L_I + L_S$$

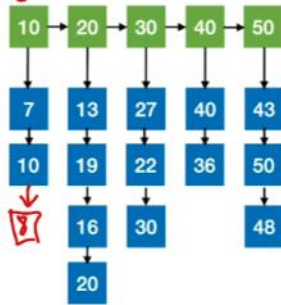
用顺序查找索引表，则 $L_I = \frac{(1+2+\dots+b)}{2} = \frac{b+1}{2}$, $L_S = \frac{(1+2+\dots+s)}{s} = \frac{s+1}{2}$

$$\text{则 } ASL = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2 + 2s + n}{2s}, \text{ 当 } s = \sqrt{n} \text{ 时, } ASL_{\min} = \sqrt{n} + 1$$

查找效率无非就是查找索引的时间+查找块内的时间。可以对ASL求导或者基本不等式得出最小的查找时间时 s 的值。

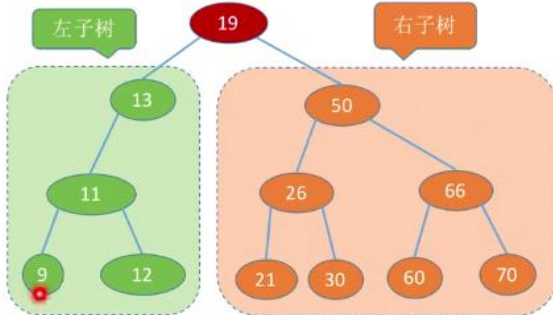


若查找表是“动态查找表”，有木有更好的实现方式？——链式存储



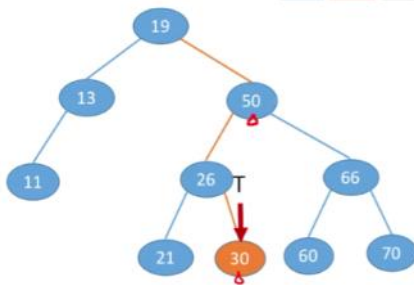
五、二叉排序树BST

二叉排序树就是左子树的值<根节点的值<右子树节点的值，使用中序遍历可以得到递增的有序序列。



1、查找过程

二叉排序树的查找



左子树结点值 < 根结点值 < 右子树结点值

若树非空，目标值与根结点的值比较：
若相等，则查找成功；
若小于根结点，则在左子树上查找，否则在右子树上查找。

查找成功，返回结点指针；查找失败返回NULL

例1：查找关键字为30的结点

```
//在二叉排序树中查找值为 key 的结点
BSTNode *BST_Search(BSTree T, int key){
    while(T != NULL && key != T->key){ //若树空或等于根结点值，则结束循环
        if(key < T->key) T = T->lchild; //小于，则在左子树上查找
        else T = T->rchild; //大于，则在右子树上查找
    }
    return T;
}
```

//二叉排序树结点
typedef struct BSTNode{
 int key;
 struct BSTNode *lchild, *rchild;
}BSTNode, *BSTree;

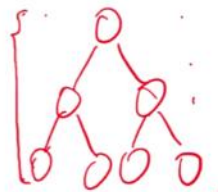
二叉排序树的查找

```
//在二叉排序树中查找值为 key 的结点
BSTNode *BST_Search(BSTree T, int key){
    while(T != NULL && key != T->key){ //若树空或等于根结点值，则结束循环
        if(key < T->key) T = T->lchild; //小于，则在左子树上查找
        else T = T->rchild; //大于，则在右子树上查找
    }
    return T;
}
```

最坏空间复杂度O(1)

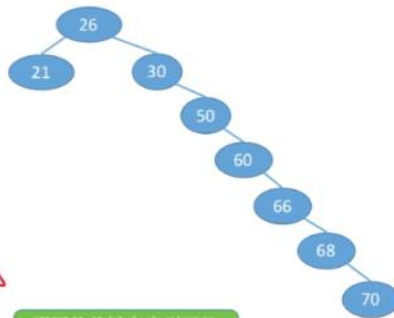
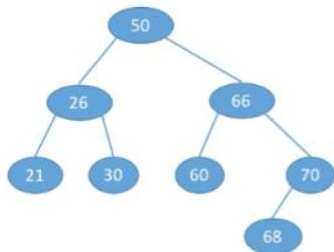
```
//在二叉排序树中查找值为 key 的结点（递归实现）
BSTNode *BSTSearch(BSTree T, int key){
    if (T == NULL)
        return NULL; //查找失败
    if (key == T->key)
        return T; //查找成功
    else if (key < T->key)
        return BSTSearch(T->lchild, key); //在左子树中找
    else
        return BSTSearch(T->rchild, key); //在右子树中找
}
```

最坏空间复杂度O(h)



2、二叉排序树的构造

从第二个节点开始，小的放左子树大的放右子树：



例1: 按照序列str={50, 66, 60, 26, 21, 30, 70, 68}建立BST

例2: 按照序列str={50, 26, 21, 30, 66, 60, 70, 68}建立BST

例3: 按照序列str={26, 21, 30, 50, 60, 66, 68, 70}建立BST

不同的关键字序列可能得到同款二叉排序树

也可能得到不同款二叉排序树

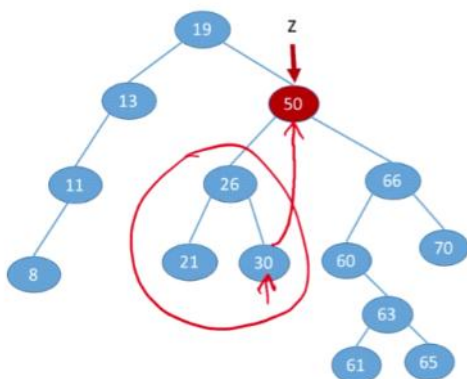
3、二叉树排序的删除

如果删的是叶子节点那直接删;

如果只有左子树或者右子树, 那么将这个子树跟节点替代它就行了;

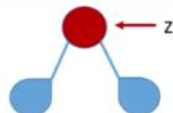
如果既有左子树又有右子树, 那么用这个节点的中序遍历的直接前驱或者直接后继替代这个节点即可, 替代完后删除前驱或者后继叶节点。

③ 若结点z有左、右两棵子树, 则令z的直接后继(或直接前驱)替代z, 然后从二叉排序树中删去这个直接后继(或直接前驱), 这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历, 可以得到一个递增的有序序列



中序遍历——左 根 右

(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

z的前驱: z的左子树中最右下结点(该节点一定没有右子树)

4、查找效率分析

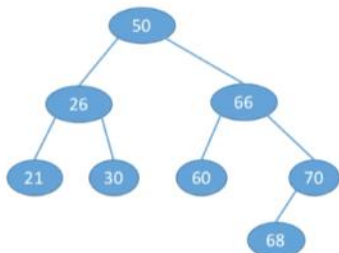
二叉排序树的查找效率很大程度上由树高决定, 因此如果树高过长的话, 会影响查找效率。

论坛 bbyl
在树中, 找到最下层的
一个结点需要对比h次

查找效率分析

最好情况: n个结点的二叉
树最小高度为 $\lfloor \log_2 n \rfloor + 1$ 。
平均查找长度 = $O(\log_2 n)$

查找长度——在查找运算中, 需要对比关键字的次数称为查找长度, 反映了查找操作时间复杂度

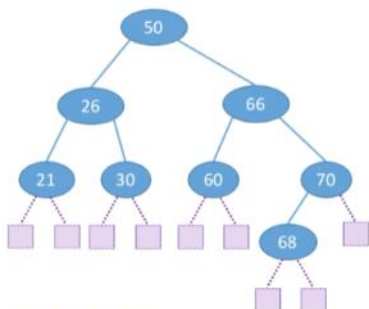


最坏情况: 每个结点只有
一个分支, 树高h=结点数
n。平均查找长度 = $O(n)$

查找成功的平均查找长度 ASL (Average Search Length)

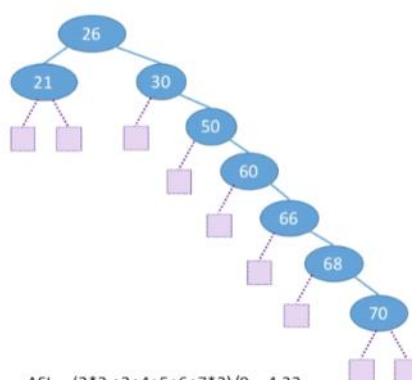
$$ASL = (1*1 + 2*2 + 3*4 + 4*1)/8 = 2.625$$

$$ASL = (1*1 + 2*2 + 3*1 + 4*1 + 5*1 + 6*1 + 7*1)/8 = 3.75$$



查找失败的平均查找长度 ASL (Average Search Length)

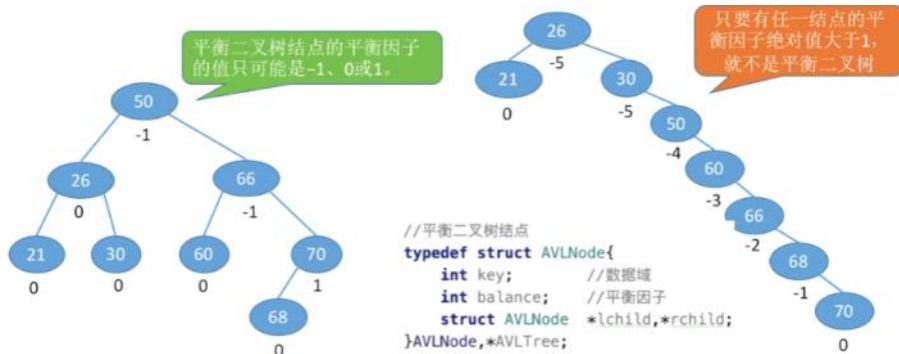
$$ASL = (3*7 + 4*2)/9 = 3.22$$



$$ASL = (2*3 + 3+4+5+6+7*2)/9 = 4.22$$

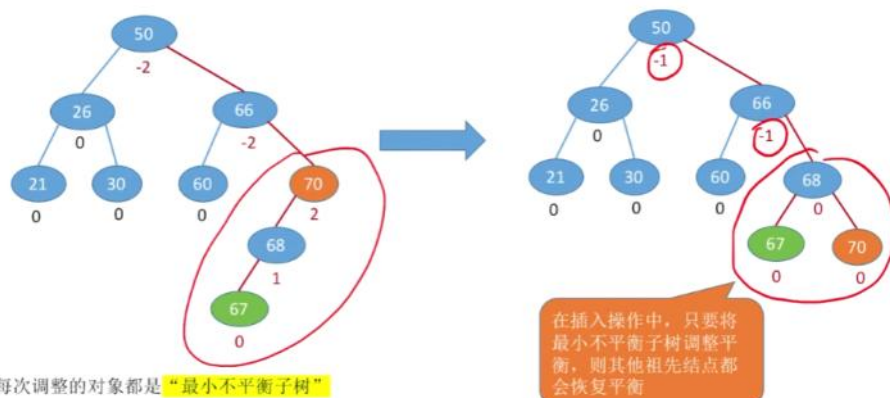
六、平衡二叉树

平衡二叉树简称平衡树AVL树，定义就是树上的任意一个节点的左子树和右子树的高度之差不超过1,因此引入了节点的平衡因子，平衡因子=左子树高-右子树高。



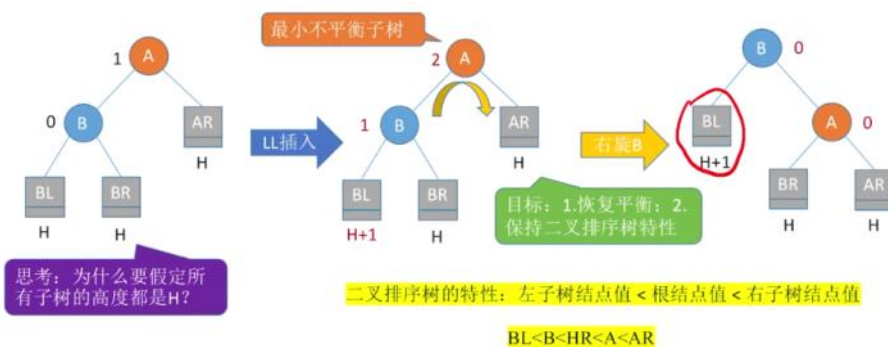
平衡二叉树的查找效率就是二叉排序树最好情况下的查找效率 $O(\log 2n)$ 。

1、平衡二叉树的插入



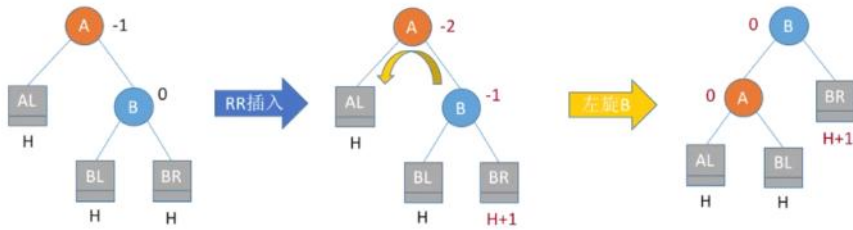
如上图，在插入的时候如果平衡因子过低，就需要将最小不平衡子树调整平衡即可，这个最小不平衡子树的调整规则如下。

2、调整最小不平衡子树



1) LL平衡旋转（右单旋转）。由于在结点A的左孩子（L）的左子树（L）上插入了新结点，A的平衡因子由1增至2，导致以A为根的子树失去平衡，需要一次向右的旋转操作。将A的左孩子B向右上旋转代替A成为根结点，将A结点向右下旋转成为B的右子树的根结点，而B的原右子树则作为A结点的左子树。

调整最小不平衡子树 (RR)

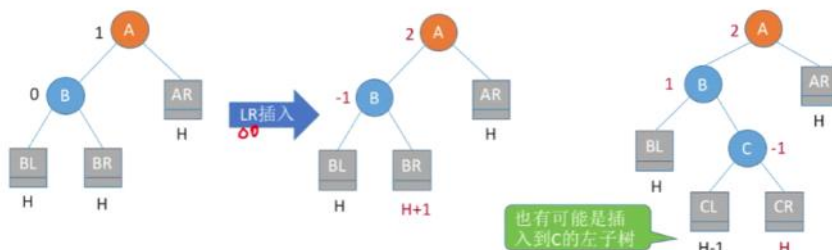


二叉排序树的特性：左子树结点值 < 根结点值 < 右子树结点值

$$AL < A < BL < B < BR$$

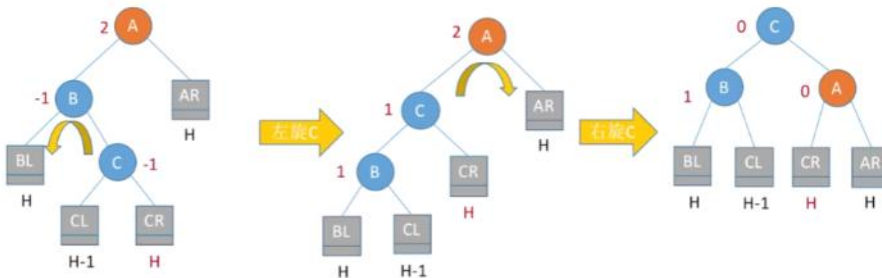
2) RR平衡旋转（左单旋转）。由于在结点A的右孩子（R）的右子树（R）上插入了新结点，A的平衡因子由-1减至-2，导致以A为根的子树失去平衡，需要一次向左的旋转操作。将A的右孩子B向左上旋转代替A成为根结点，将A结点向左下旋转成为B的左子树的根结点，而B的原左子树则作为A结点的右子树

调整最小不平衡子树 (LR)



3) LR平衡旋转（先左后右双旋转）。由于在A的左孩子（L）的右子树（R）上插入新结点，A的平衡因子由1增至2，导致以A为根的子树失去平衡，需要进行两次旋转操作，先左旋转后右旋转。先将A结点的左孩子B的右子树的根结点C向左上旋转提升到B结点的位置，然后再把该C结点向右上旋转提升到A结点的位置

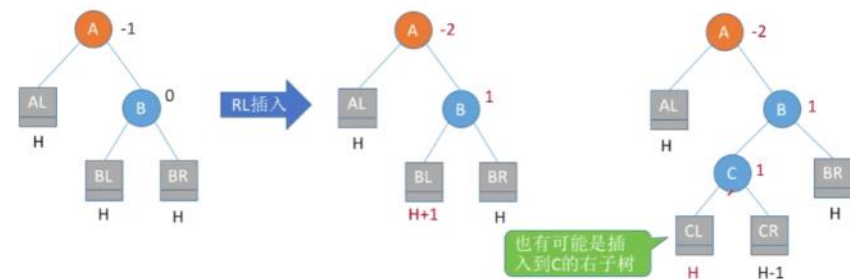
调整最小不平衡子树 (LR)



$$BL < B < CL < C < CR < A < AR$$

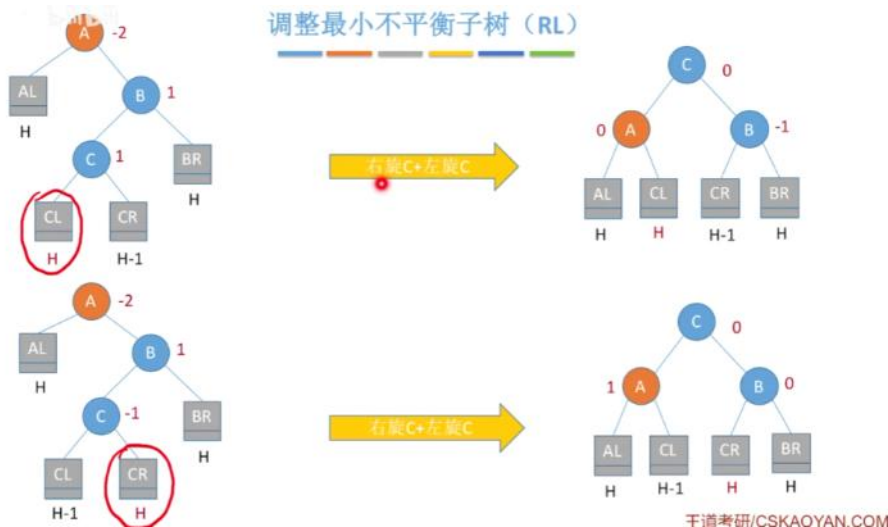
调整最小不平衡子树 (RL)

AL < A < CL < C < CR < B < BR



4) RL平衡旋转 (先右后左双旋转)。由于在A的右孩子(R)的左子树(L)上插入新结点, A的平衡因子由-1减至-2, 导致以A为根的子树失去平衡, 需要进行两次旋转操作, 先右旋转后左旋转。先将A结点的右孩子B的左子树的根结点C向右上旋转提升到B结点的位置, 然后再把该C结点向左上旋转提升到A结点的位置

调整最小不平衡子树 (RL)



王道考研/CSKAOYAN.COM

对了旋转完后建议用“左根右”来验证一下是不是有序的。

3、平衡二叉树的删除

平衡二叉树的删除操作具体步骤:

①删除结点 (方法同“二叉排序树”)

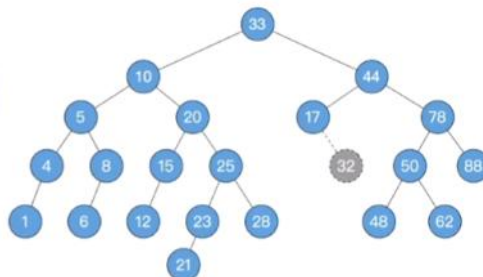
- 若删除的结点是叶子, 直接删。
- 若删除的结点只有一个子树, 用子树顶替删除位置。
- 若删除的结点有两棵子树, 用前驱 (或后继) 结点顶替, 并转换为对前驱 (或后继) 结点的删除。

②一路向北找到最小不平衡子树, 找不到就完结撒花

③找最小不平衡子树下, “个头”最高的儿子、孙子

④根据孙子的位置, 调整平衡 (LL/RR/LR/RL)

⑤如果不平衡向上传导, 继续②



平衡二叉树的删除操作具体步骤:

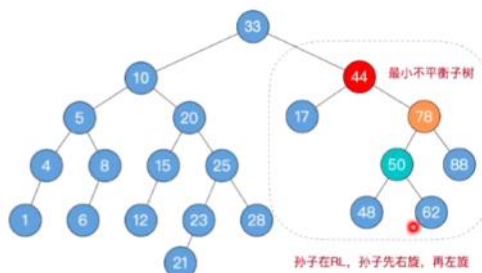
①删除结点 (方法同“二叉排序树”)

②一路向北找到最小不平衡子树, 找不到就完结撒花

③找最小不平衡子树下, “个头”最高的儿子、孙子

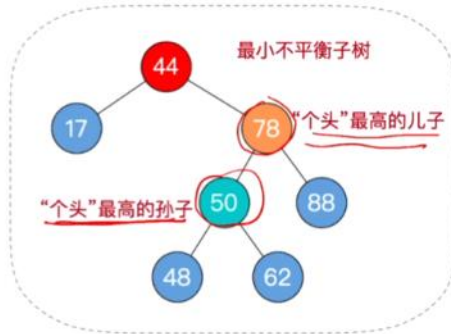
④根据孙子的位置, 调整平衡 (LL/RR/LR/RL)

⑤如果不平衡向上传导, 继续②



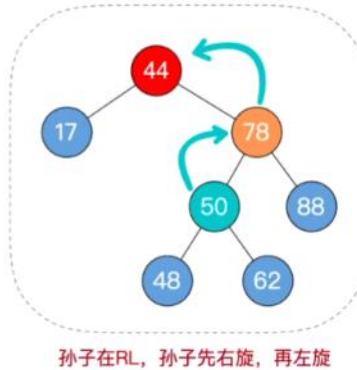
平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



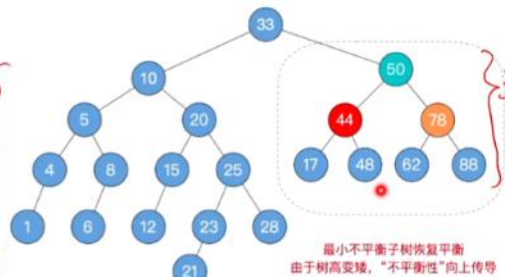
平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②



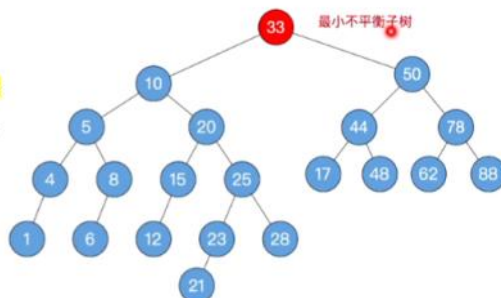
平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



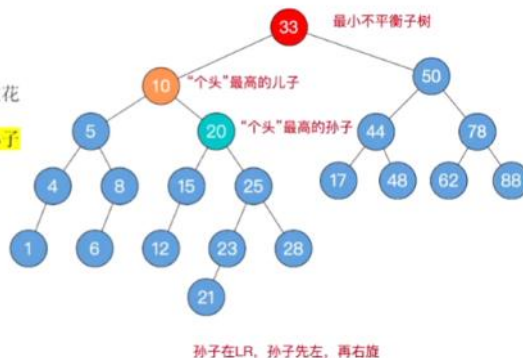
平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



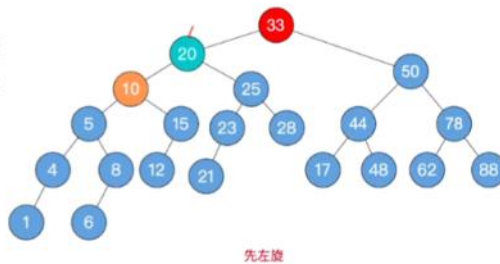
平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



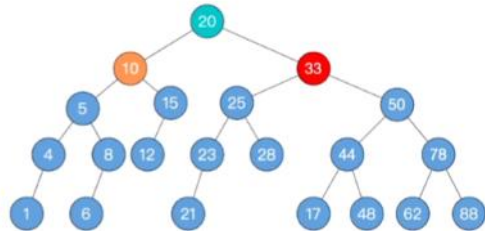
平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②



平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



删除的时间复杂度是 $O(\log 2n)$ 。

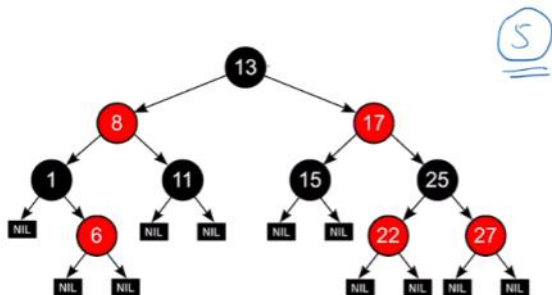
七、红黑树RBT

平衡二叉树AVL在插入删除的时候很容易破坏平衡，导致频繁调整树的形态，这个的时间开销是很大的。相比之下红黑树的插入删除什么的不会破坏平衡的特性，因此时间开销要比平衡二叉树低很多。因此在以查为主，很少插入/删除的场景使用平衡二叉树更合适；经常插入删除的场景使用红黑树比较合适。

1、定义

红黑树是一个二叉排序树，需要满足以下条件：

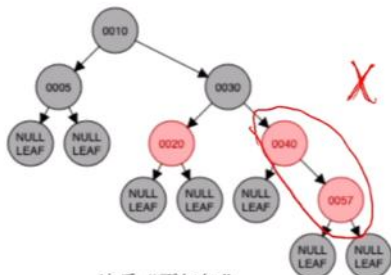
- (1) 每个节点要么红色要么黑色。根节点必黑色。叶子节点也是黑色。
- (2) 不存在两个相邻的红色节点（父子节点都是一个黑色一个红色）
- (3) 每个节点从该节点到任意节点的路径上所含黑节点数量相同，例如下图13节点到6节点经过1个黑节点，到27也是经过1个黑节点。



左根右，相对黑
不红红，黑路同



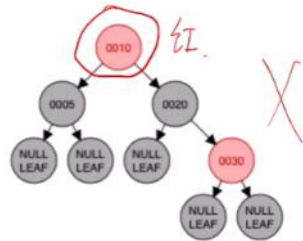
张口就是freestyle



违反“不红红”



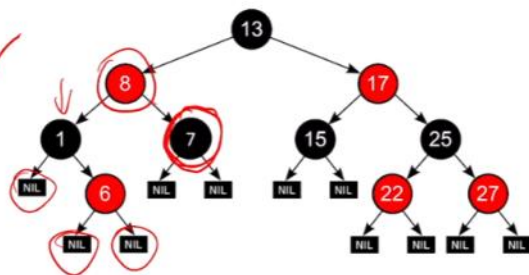
标准确认



左根右，根叶黑
不红红，黑路同



张口就是freestyle



违反“左根右”

2、性质

- 性质1: 从根节点到叶节点的最长路径不大于最短路径的2倍
- 性质2: 有n个内部节点的红黑树高度 $h \leq 2\log_2(n+1)$

→ 红黑树查找操作时间复杂度 = $O(\log_2 n)$

查找效率与AVL
树同等数量级

3、查找

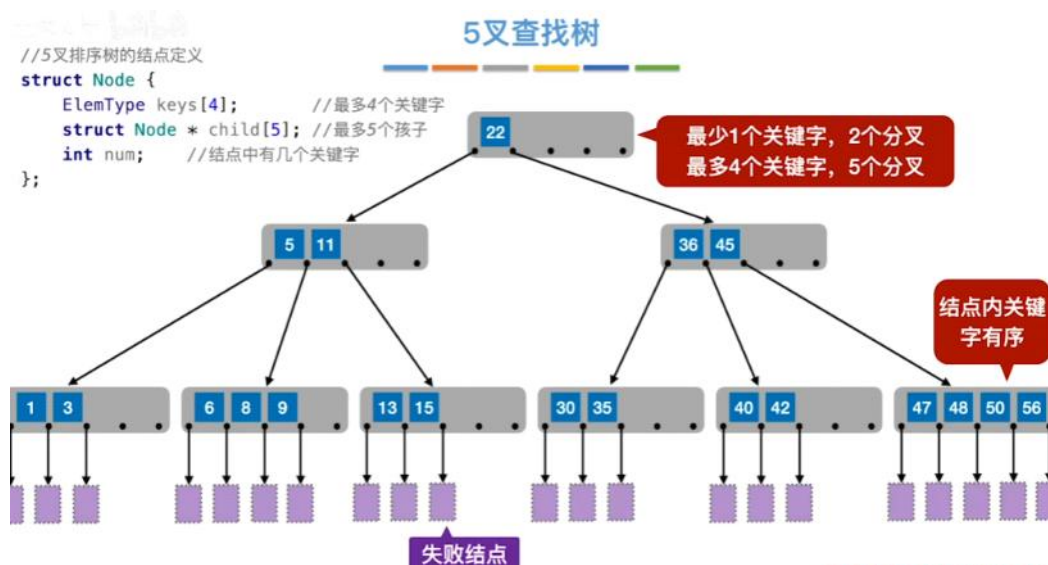
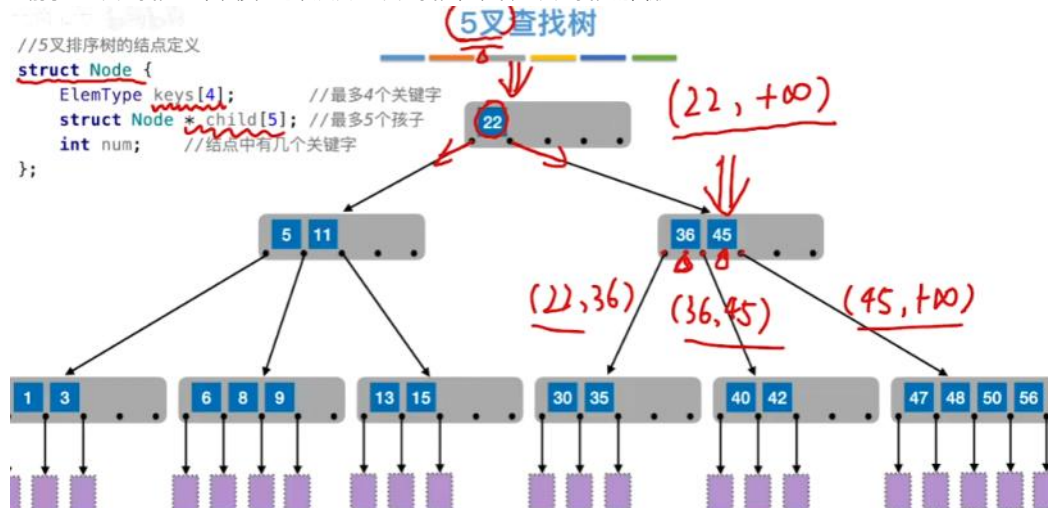
红黑树的查找和平衡二叉树什么的一样，从根节点开始，左孩子小右孩子大，如果最后找到一个空叶子节点，则查找失败。

4、插入

未完待续

八、B树

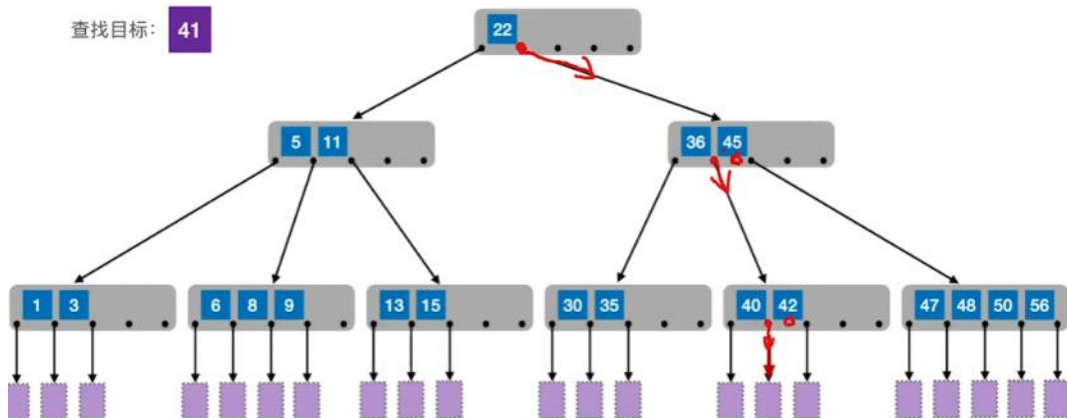
之前学过二叉查找树BST，其实也可以扩展成m叉查找树，以下是5叉查找树的图形：



1、B树的查找

和二叉排序树的查找是一样的，每层从左到右依次扫描，最后查到失败节点说明没找到。还可以用二分查找进行优化。

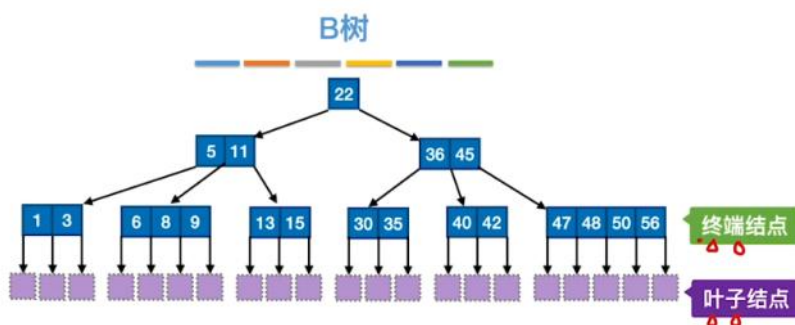
查找目标: 41



为了保证查找效率, 规定m叉查找树中除了根节点外, 任何节点至少有 $\lceil m/2 \rceil$ 个分叉, 即至少包含有 $\lceil m/2 \rceil - 1$ 个关键字; 此外, 还规定对于任何一个节点的所有子树的高度都要相同 (保证查找次数)。

上述条件都满足, 那就是B树了。

2. B树的插入



B树, 又称多路平衡查找树, B树中所有结点的孩子个数的最大值称为B树的阶, 通常用m表示。一棵m阶B树或为空树, 或为满足如下特性的m叉树:

- 1) 树中每个结点至多有m棵子树, 即至多含有m-1个关键字。
- 2) 若根结点不是终端结点, 则至少有两棵子树。
- 3) 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树, 即至少含有 $\lceil m/2 \rceil - 1$ 个关键字。
- 5) 所有的叶子结点都出现在同一层次上, 并且不带信息 (可以视为外部结点或类似于折半查找判定树的查找失败结点, 实际上这些结点不存在, 指向这些结点的指针为空)。

3. B树的高度

B树的高度

问题: 含n个关键字的m叉B树, 最小高度、最大高度是多少?

最大高度——让每个结点包含的关键字、分叉尽可能的少。记 $k = \lceil m/2 \rceil$

	最少结点数	最少关键字数
第一层	1	1
第二层	2	$2(k-1)$
第三层	$2k$	$2k(k-1)$
第四层	$2k^2$	$2k^2(k-1)$
...	...	...
第h层	$2k^{h-2}$	$2k^{h-2}(k-1)$

h层的m阶B树至少包含关键字总数 $1 + 2(k-1)(k^0 + k^1 + k^2 + \dots + k^{h-2}) = 1 + 2(k^{h-1} - 1)$

若关键字总数少于这个值, 则高度一定小于h, 因此 $n \geq 1 + 2(k^{h-1} - 1)$

$$\text{得, } h \leq \log_k \frac{n+1}{2} + 1 = \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$$

含n个关键字的m叉B树, $\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$

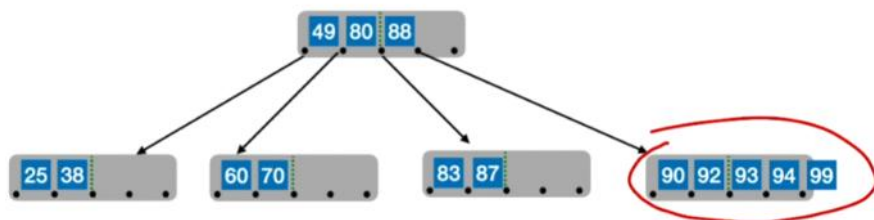
注意B树每个索引里1个主键可以划分2个子树, 因为这个主键将数值区间一分为二, 因此n个主键可以划分出n+1个子树。

2、B树的插入

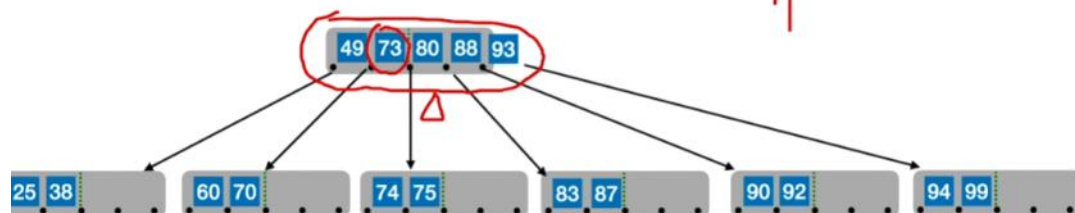
B树最初只有1个根节点, 先把数据插入到根节点:



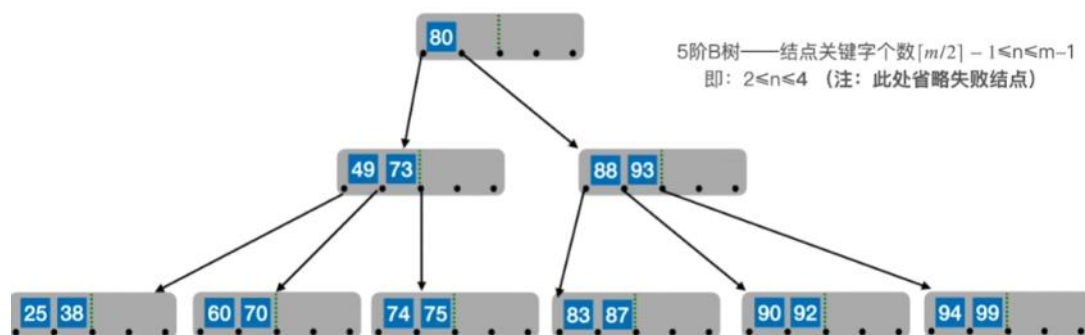
一直插直到关键字的个数超过了节点规定的数量:



好了现在根节点的个数已经超出规定个数了，此时需要将中间元素提取出来建立一个新的根节点，继续往上分裂：



在插入key后，若导致原结点关键字数超过上限，则从中间位置 ($\lceil m/2 \rceil$) 将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置 ($\lceil m/2 \rceil$) 的结点插入原结点的父结点

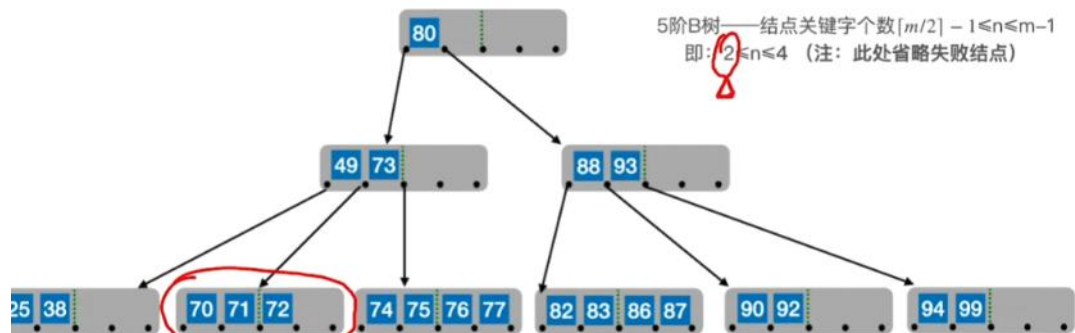


5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ (注：此处省略失败结点)

这样插入可以保证b数一直满足相关的定义。

3、B树的删除

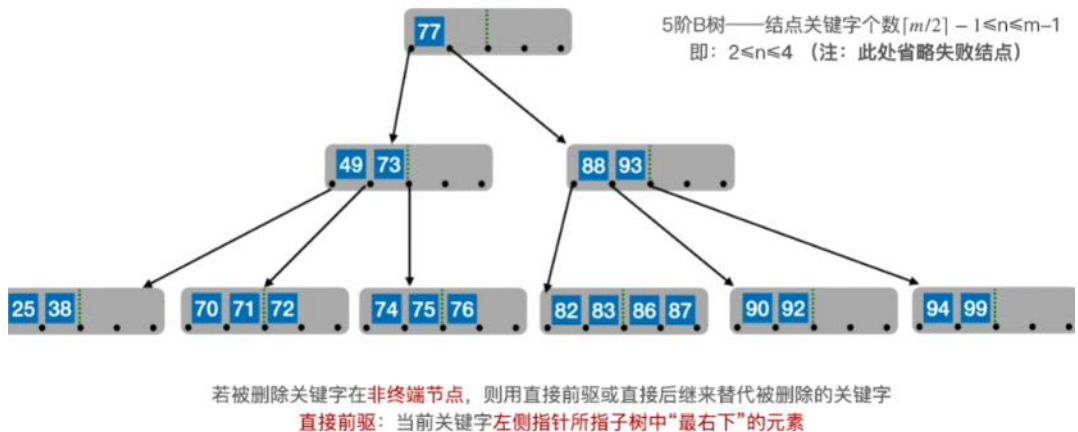
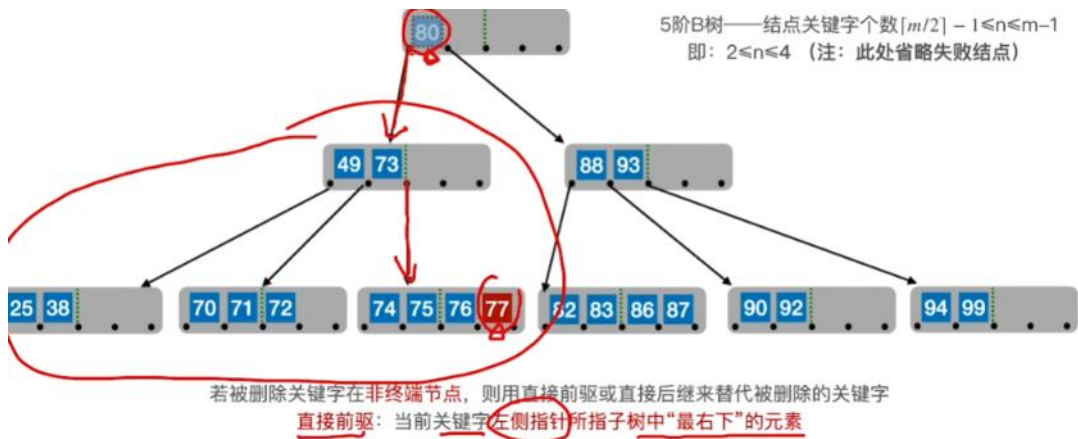
如果是终端节点就直接删除



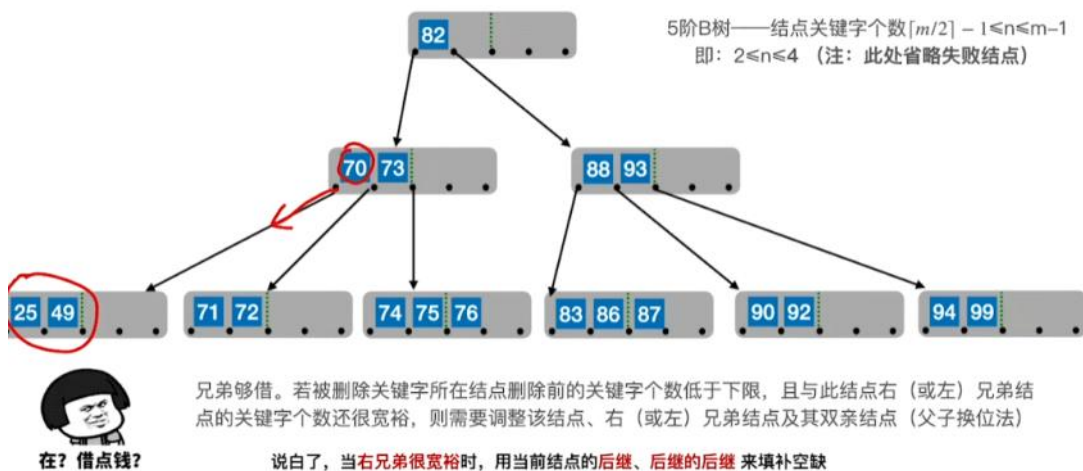
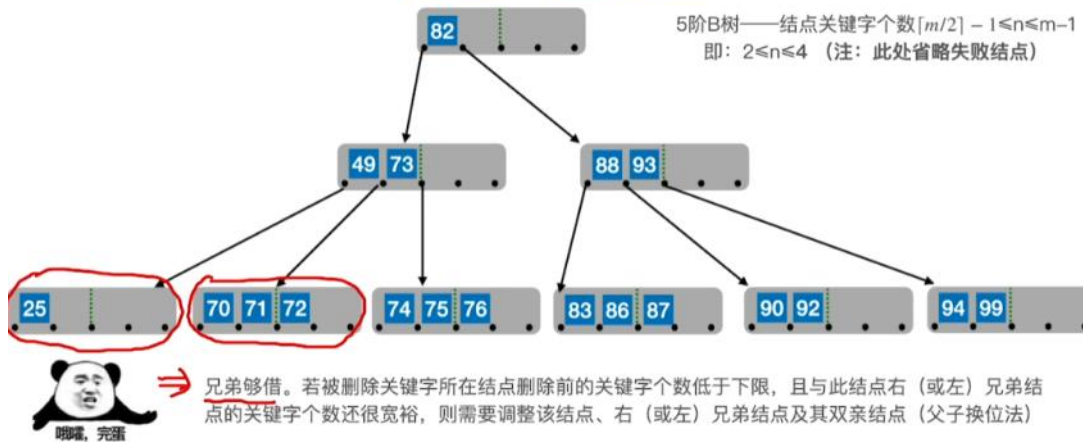
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ (注：此处省略失败结点)

若被删除关键字在终端节点，则直接删除该关键字 (要注意节点关键字个数是否低于下限 $\lceil m/2 \rceil - 1$)

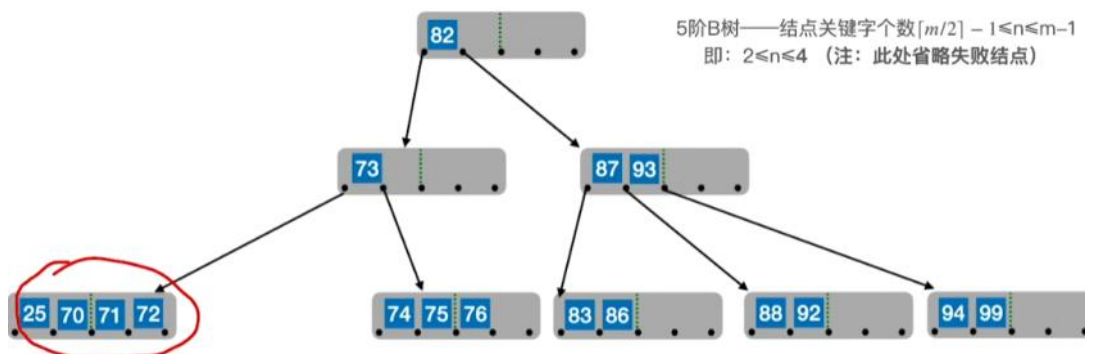
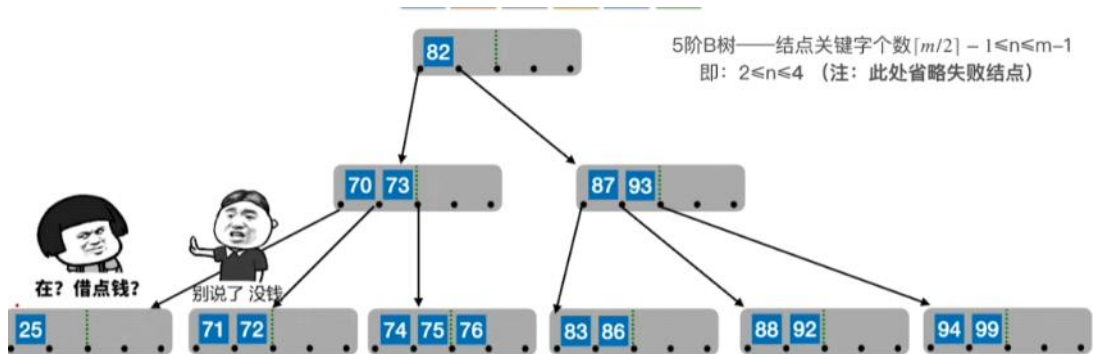
如果删的是根节点，删完后没根节点了，就找出它的前驱或者后继来顶替他的位置，这样就转换成对终端节点的删除问题了：



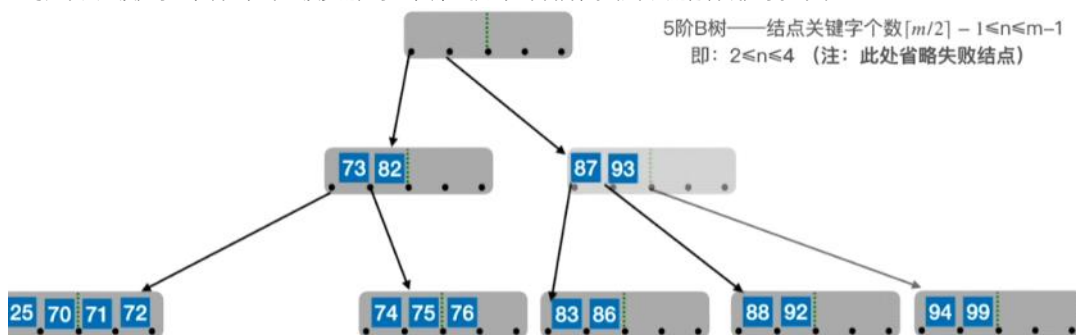
当某个节点删除后低于规定的个数, 那么就把旁边兄弟的元素拿过来, 为了保证有序性, 需要先将兄弟元素放到根节点顶替根节点的某个元素的位置, 将根节点的元素拉下来:



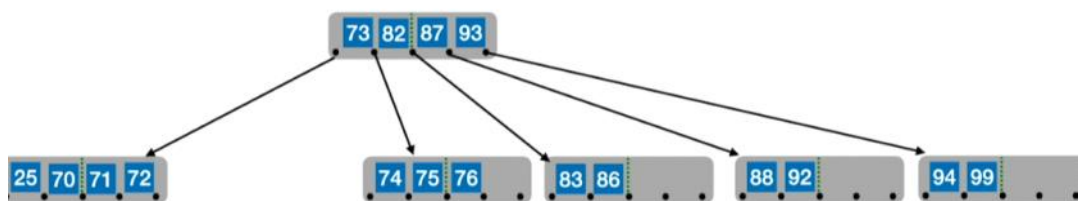
除了右兄弟可以借, 左兄弟也可以的, 拿元素的过程和上面一样。如果两边的兄弟手头都没元素了, 就将两兄弟合并, 父节点的那个分界线元素也一起拉下来:



此时父节点也没元素了, 并且兄弟也没多的元素了, 那么就两兄弟合并, 把根节点的分界线元素拉下来:

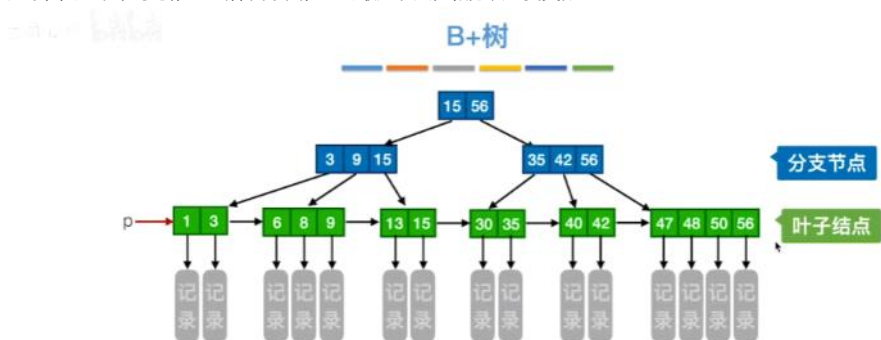


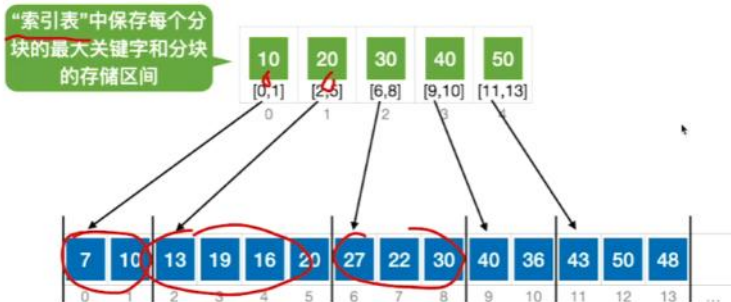
兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限, 且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$, 则将关键字删除后与左(或右)兄弟结点及双亲结点中的关键字进行合并



九、B+树

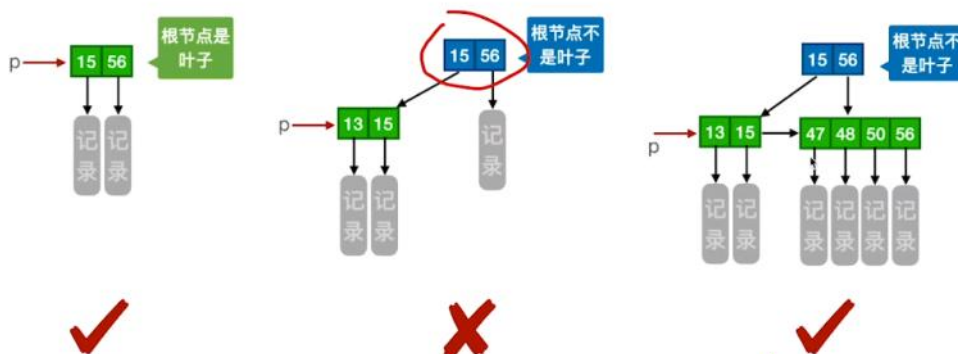
如下图的B+树, 索引位置会保留下面位置的最大节点, 和分块查找类似。





特点：块内无序、块间有序

B+树的非叶根节点至少有两棵子树，其他每个分支节点至少有 $\lceil m/2 \rceil$ 棵子树。



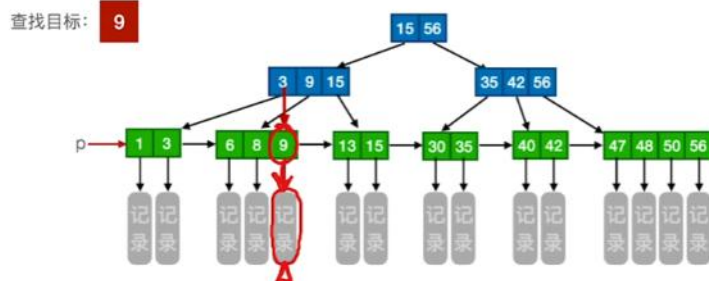
可以理解为：要追求“绝对平衡”，即所有子树高度要相同

2) 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。

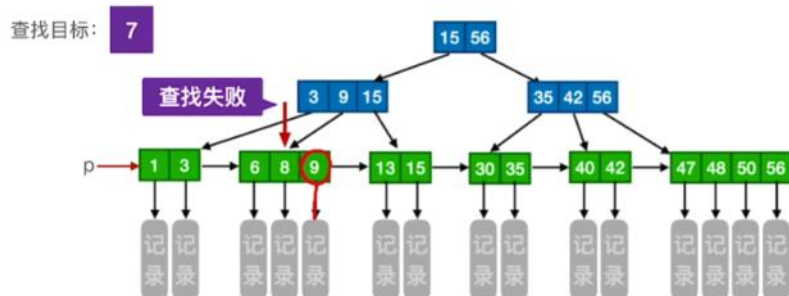
除了这些，B+树的节点子树个数和关键字的个数是相等的，所有叶子节点包含全部关键字以及指向记录的指针，叶子节点关键字按照大小顺序排列，并且相邻叶子节点按照大小顺序相互链接起来（支持顺序查找）。

1、B+树的查找

B+树的查找一般和分块查找一样，找到对应的索引节点，然后在叶子节点中找到数据，注意就算是索引节点中找到了，那也得去叶子节点中遍历（和B树不一样），因为数据都放在叶子节点里了。



多路查找



多路查找

B+树中，无论查找成功与否，最终一定都要走到最下面一层结点

除了索引的层层查找，还可以对叶子节点进行顺序查找，在上图中从p指针开始进行顺序遍历。

	m阶B树	m阶B+树
类比	二叉查找树的进化——>m叉查找树	分块查找的进化——>多级分块查找
关键字与分叉	n个关键字对应n+1个分叉（子树）	n个关键字对应n个分叉
结点包含的信息	所有结点中都包含记录的信息	只有最下层叶子结点才包含记录的信息（可使树更矮）
查找方式	不支持顺序查找。查找成功时，可能停在任何一层结点，查找速度“不稳定”	支持顺序查找。查找成功或失败都会到达最下一层结点，查找速度“稳定”
相同点	除根节点外，最少 $\lceil m/2 \rceil$ 个分叉（确保结点不要太“空”） 任何一个结点的子树都要一样高（确保“绝对平衡”）	

九、哈希表

1、定义

散列表是一种数据结构，特点是可以根据数据元素的关键字来计算出它在散列表中的存储地址；散列函数 $\text{Addr}=\text{H}(\text{key})$ 建立了“关键字->存储地址”的映射关系。类似于随机存取，如果可以计算出地址，时间复杂度直接干到 $O(1)$ 了。

例：某散列表的长度为13，散列函数 $\text{H}(\text{key})=\text{key}\%13$ 。依次将数据元素 19、14、23 插入散列表：



2、冲突

上面的求余找地址可能会发生数据的冲突，例如下图1号内存地址已经有数据14了，再放个1进去就冲突。此时1和14是同义词。

例：某散列表的长度为13，散列函数 $\text{H}(\text{key})=\text{key}\%13$ 。依次将数据元素 19、14、23 插入散列表：



继续插入元素 1：存储地址 $=1\%13=1$ ，该地址已经存储了其他元素——发生“冲突”



为了防止冲突，可以修改求余的取值，构造更合适的散列函数，减少冲突的发生。当然再精妙的散列函数也难免会冲突，此时需要对冲突进行处理，第一个就是拉链法，第二是开放地址法。

3、构造散列函数

设计散列函数时应该注意什么？

散列函数（哈希函数）： $\text{Addr}=\text{H}(\text{key})$ 建立了“关键字”→“存储地址”的映射关系。

设计散列函数时应该注意什么：

- 定义域必须涵盖所有可能出现的关键字。
反例： $\text{H}(\text{key}) = \sqrt{\text{key}} \% 13$ ，不支持关键字为负值
- 值域不能超出散列表的地址范围。
反例： $\text{H}(\text{key}) = \text{key} \% 15$ ，可能被映射到非法地址13、14
- 尽可能减少冲突。散列函数计算出来的地址应尽可能均匀分布在地址空间。
反例： $\text{H}(\text{key}) = (\text{key} \times 13) \% 13$ ，任何关键字都会被映射到地址0
- 散列函数应尽量简单，能够快速计算出任意一个关键字对应的散列地址。
反例： $\text{H}(\text{key}) = (\text{key}!) \% 13$ ，若关键字值较大，计算阶乘耗时高



方法1：除留余数法，尽量找质数作为余数

除留余数法

除留余数法 —— $H(key) = key \% p$

散列表表长为 m ，取一个不大于 m 但最接近或等于 m 的质数 p

注：质数又称素数。指除了1和此整数自身外，不能被其他自然数整除的数

适用场景：较为通用，只要关键字是整数即可

例：散列表表长 $m=13$ 。13是质数，因此可令散列函数 $H(key)=key\%13$

0	1	2	3	4	5	6	7	8	9	10	11	12

例：散列表表长 $m=15$ 。不大于15且最接近15的质数是13，因此可令散列函数 $H(key)=key\%13$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
													^	^

除留余数法 —— $H(key) = key \% p$

散列表表长为 m ，取一个不大于 m 但最接近或等于 m 的质数 p

注：质数又称素数。指除了1和此整数自身外，不能被其他自然数整除的数

原因：对质数取余，可以分布更均匀，从而减少冲突

设：可能出现的关键字= $\{2,4,6,8,10,12,14\}$

散列表表长8，散列函数 $H(key)=key\%8$

0	1	2	3	4	5	6	7

8 2 4 6
10 12 14

对合数取余，散列地址分布不均匀，易发生冲突

散列表表长8，散列函数 $H(key)=key\%7$

0	1	2	3	4	5	6	7
							^

14 8 2 10 4 12 6

对质数取余，散列地址分布均匀，不易发生冲突

方法2：直接定址法：

直接定址法 —— $H(key) = key$ 或 $H(key) = a*key + b$

其中， a 和 b 是常数。这种方法计算最简单，且不会产生冲突。若关键字分布不连续，空位较多，则会造成存储空间浪费。

适用场景：关键字分布基本连续

例：用散列表存储某班级的30个学生信息，班内学生学号为(1120112176~1120112205)

可令 $H(key) = key - 1120112176$

0	1	2	3	4	5	...	...	...	26	27	28	29
...176	...177	...178	...179	...180	...181	...	...	...	...202	...203	...204	...205

方法3：数字分析法：

数字分析法

数字分析法 —— 选取数码分布较为均匀的若干位作为散列地址

设关键字是 r 进制数（如十进制数），而 r 个数码在各位上出现的频率不一定相同，可能在某些位上分布均匀一些，每种数码出现的机会均等；而在某些位上分布不均匀，只有某几种数码经常出现，此时可选取数码分布较为均匀的若干位作为散列地址。

适用场景：关键字集合已知，且关键字的某几个数码位分布均匀

例：要求将手机用户的信息存入长度为10000的散列表。以“手机号码”作为关键字设计散列函数

138XXX2875
138XXX1682
138XXX9125
.....
199XXX1684
199XXX1236

以手机号后四位作为散列地址

0	1	2	3	...	...	9998	9999

方法4：平方取中法

平方取中法

平方取中法——取关键字的平方值的中间几位作为散列地址。

具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系，因此使得散列地址分布比较均匀。

适用场景：关键字的每位取值都不够均匀。

例：某汽车品牌，以（底盘号、发动机号、外观号）作为一款汽车型号的标识。该品牌有100种车型，欲将这些车型信息存入长度为100的散列表，如何设计散列函数？

车型号	平方	取中
底盘号有：28、34、39、60...	605030 →	366061300900
发动机号有：50、52、54...	345220 →	119176843400
外观号有：20、25、30、35...	395225 →	156202800625
	605435 →	366551539225

关键字的各个数码位都分布不均匀

中间几位的值受每个数码位影响

985
× 985

4925
7880
8865
97025

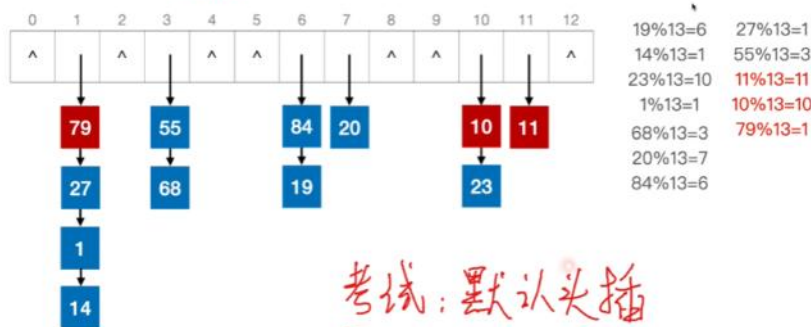
985²

4、拉链法处理冲突

拉链法就是把所有的同义词存储在一个链表之中

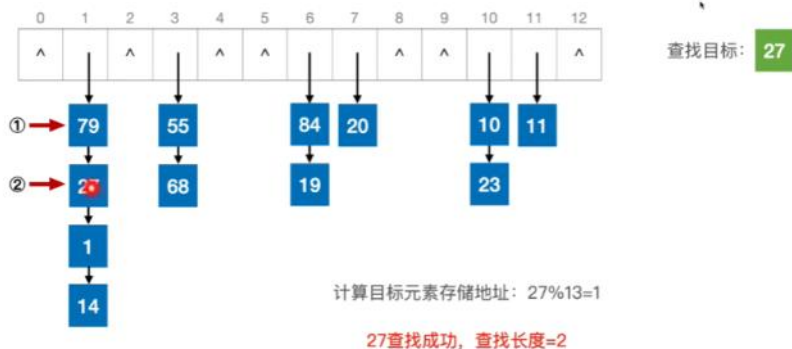
散列表的插入操作（拉链法解决冲突）

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。依次插入关键字 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}

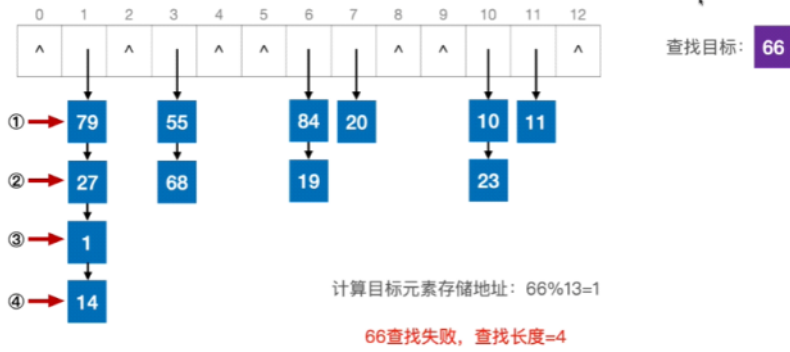


散列表的查找操作（拉链法解决冲突）

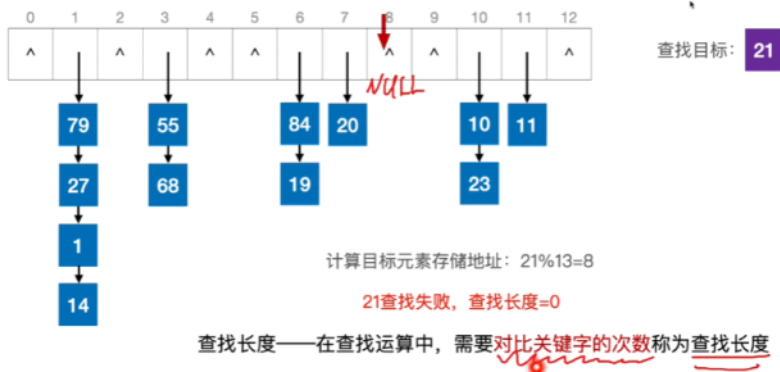
例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次查找元素 27, 20, 66, 21，分析每次查找操作的过程和查找长度。



例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次查找元素 27, 20, 66, 21，分析每次查找操作的过程和查找长度。

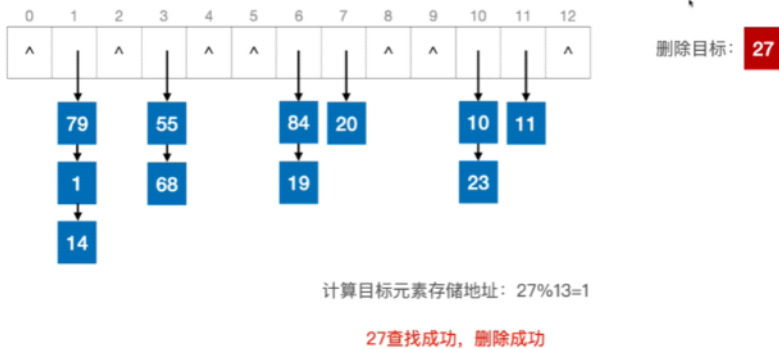


例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次查找元素 27, 20, 66, 21，分析每次查找操作的过程和查找长度。



散列表的删除操作 (拉链法解决冲突)

例：若散列表长度为13，散列函数 $H(key)=key\%13$ ，用拉链法解决冲突。在下述散列表中依次删除元素 27, 20, 66。

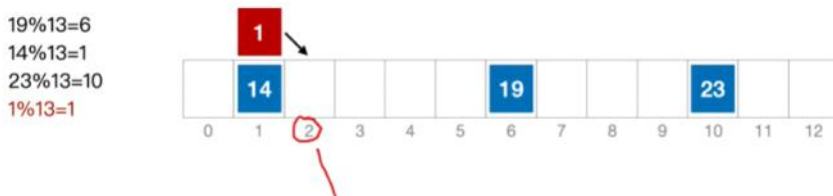


对散列表的增删查，又回到链表的操作了。如果想提升一点点查找效率，可以在插入的时候构造一个有序的链表，这样可以方便一点点查找。

5、开放定址法解决冲突

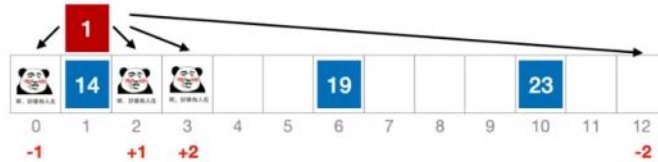
该方法就是如果发生冲突，就给这个元素找一个新的空位置，这样一个地址既对同义词开放又对非同义词开放，因此叫开放定址法。

例：某散列表的长度为13，散列函数 $H(key)=key\%13$ 。依次将数据元素 19、14、23、1 插入散列表：



一般新的位置就在原来位置的左边或者右边，如果两边都有元素了，就需要探测出一个新的位置：

$19\%13=6$
 $14\%13=1$
 $23\%13=10$
 $1\%13=1$



待解决的问题：用什么规则确定“另一个空闲位置”？

思路：需确定一个“探测的顺序”，从初始散列地址出发，去寻找下一个空闲位置。

eg: $d_0=0, d_1=1, d_2=-1, d_3=2, d_4=-2, \dots$

注: d_i 表示第 i 次发生冲突时，下一个探测地址与初始散列地址的相对偏移量。

开放定址法的基本原理

根据散列函数 $H(key)$ ，求得初始散列地址。若发生冲突，如何找到“另一个空闲位置”？



m 个地址

发生第 i 次冲突时的散列地址

$$H_i = (H(key) + d_i) \% m$$

初始散列地址

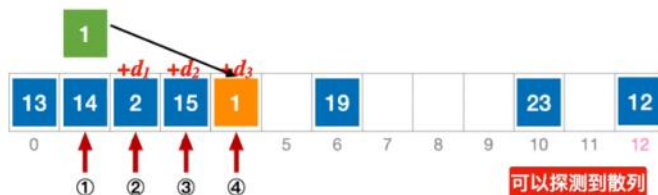
偏移量

四种常用方法构造探测序列 d_i

注: $0 \leq i \leq m-1$

- 线性探测法 $d_i = 0, 1, 2, 3, \dots, m-1$ (down d_{m-1})
- 平方探测法 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 。其中 $k \leq m/2$
- 双散列法 $d_i = i \times \text{hash}_2(key)$ 。其中 $\text{hash}_2(key)$ 是另一散列函数
- 伪随机序列法 d_i 是一个伪随机序列，如 $d_i = 0, 5, 3, 11, \dots$

四种探测法的规则如下：



可以探测到散列表的每个地址

发生第 i 次冲突时的散列地址

$$H_i = (H(key) + d_i) \% m$$

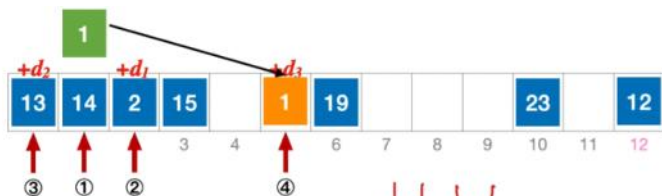
初始散列地址

偏移量

线性探测法, $d_i = 0, 1, 2, 3, \dots, m-1$

初始散列地址 $H_0 = 1\%13 = 1$ ，发生冲突(第1次)
 $H_1 = (1 + 1) \% 13 = 2$ ，发生冲突(第2次)
 $H_2 = (1 + 2) \% 13 = 3$ ，发生冲突(第3次)
 $H_3 = (1 + 3) \% 13 = 4$ ，未发生冲突，插入位置#4

注：查找操作原理类似，根据探测序列依次对比各存储单元内的关键字。
 若探测到目标关键字，则查找成功。
 若探测到空单元，则查找失败。



发生第 i 次冲突时的散列地址

$$H_i = (H(key) + d_i) \% m$$

初始散列地址

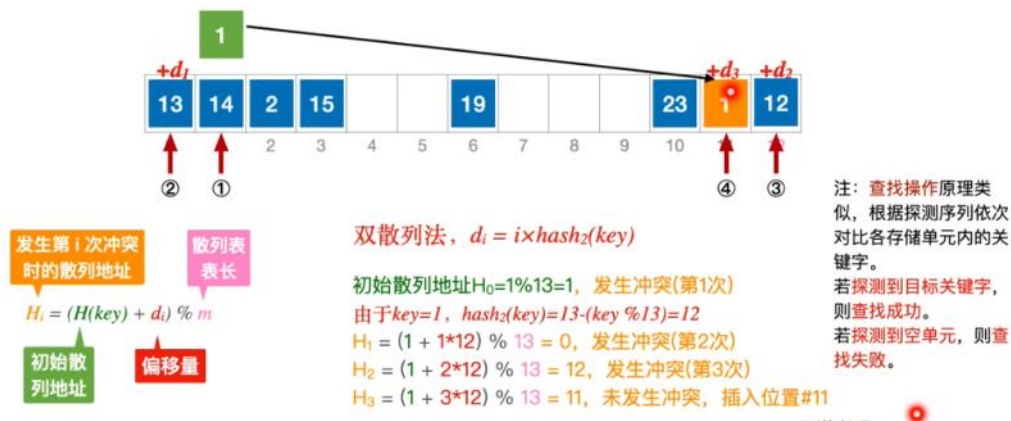
偏移量

平方探测法, $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 。其中 $k \leq m/2$

初始散列地址 $H_0 = 1\%13 = 1$ ，发生冲突(第1次)
 $H_1 = (1 + 1) \% 13 = 2$ ，发生冲突(第2次)
 $H_2 = (1 + -1) \% 13 = 0$ ，发生冲突(第3次)
 $H_3 = (1 + 4) \% 13 = 5$ ，未发生冲突，插入位置#5

注：查找操作原理类似，根据探测序列依次对比各存储单元内的关键字。
 若探测到目标关键字，则查找成功。
 若探测到空单元，则查找失败。

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用双散列法解决冲突，假设 $hash_2(key)=13-(key\%13)$ 。分析：插入元素1、查找元素1的过程



伪随机序列法 (插入、查找操作)

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用伪随机序列法解决冲突，假设伪随机序列 $d_i=0, 5, 3, 11, \dots$ ，其中 d_i 表示第 i 次发生冲突时的增量。分析：插入元素1、查找元素1的过程



特别注意：关于删除操作

例：长度为13的散列表状态如下图所示，散列函数 $H(key)=key\%13$ ，采用线性探测法解决冲突。



正确示范：删除元素15

- 计算元素15的初始散列地址 $= 15\%13 = 2$ 。对比位置#2，关键字不等于15；
- 根据线性探测法的探测序列，继续对比位置#3，关键字等于15；
- 逻辑删除元素15，将位置#3标记为“已删除”

注：无论线性探测法、平方探测法、双散列法、伪随机序列法原理都一样。删除元素时，只能逻辑删除

注意：采用“开放定址法”时，删除元素不能简单地将被删元素的空间置为空，否则将截断在它之后的探测路径，可以做一个“已删除”标记，进行逻辑删除。

特别注意：关于删除操作

注意：采用“开放定址法”时，删除元素不能简单地将被删元素的空间置为空，否则将截断在它之后的探测路径，可以做一个“已删除”标记，进行逻辑删除。



注：新元素也可以插入到已被“逻辑删除”的地址

带来的问题：查找效率低下，散列表看起来很满，实则很空。

Tips：可以不定期整理散列表内的数据。



6、散列查找的性能分析（用的线性探测法）

散列查找的性能分析

例：设散列表HT长度为16，初始为空。散列函数 $H(key) = key \% 13$ ，处理冲突采用线性探测法。依次插入关键字序列 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}

- (1) 请画出所构造的散列表
- (2) 请计算在等概率情况下，查找成功、查找失败的平均查找长度ASL。



计算每个关键字的初始散列地址，若发生冲突，采用线性探测法解决冲突：

$H(19) = 19 \% 13 = 6$
 $H(14) = 14 \% 13 = 1$
 $H(23) = 23 \% 13 = 10$
 $H(1) = 1 \% 13 = 1 \rightarrow 2$
 $H(68) = 68 \% 13 = 3$
 $H(20) = 20 \% 13 = 7$
 $H(84) = 84 \% 13 = 6 \rightarrow 7 \rightarrow 8$
 $H(27) = 27 \% 13 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
 $H(55) = 55 \% 13 = 3 \rightarrow 4 \rightarrow 5$
 $H(11) = 11 \% 13 = 11$
 $H(10) = 10 \% 13 = 10 \rightarrow 11 \rightarrow 12$
 $H(79) = 79 \% 13 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$

$$ASL_{成功} = \frac{1 + 1 + 1 + 2 + 1 + 1 + 3 + 4 + 3 + 1 + 3 + 9}{12} = 2.5$$

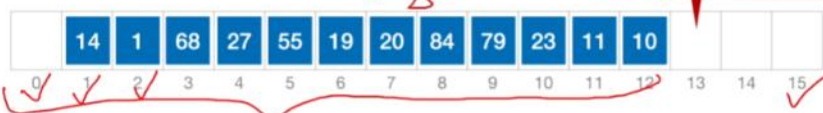
计算“查找成功”的ASL：假设查找每一个已存在关键字的概率相同

如上图，在插入的时候如果已经满了，每多偏移一步，查找时就要多一步，例如14求余后可以直接得到，查找次数是1，而1在插入的时候多了一步，查找的次数是2，其他的也是这样。

例：设散列表HT长度为16，初始为空。散列函数 $H(key) = key \% 13$ ，处理冲突采用线性探测法。依次插入关键字序列 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}

- (1) 请画出所构造的散列表
- (2) 请计算在等概率情况下，查找成功、查找失败的平均查找长度ASL。

Tips：任何一个关键字的初始散列地址都不可能落在13-15



$$ASL_{失败} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2}{13} = 7$$

计算“查找失败”的ASL：假设出现每一个初始散列地址的概率相同

$$ASL_{失败} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 1 + 1}{16} = 6.5$$

经典错误！注意：虽然散列表长度为16，但散列函数的取值范围仅有13种可能性

如上图，如果要查找的位置是空的，那么就查找失败，如果没空就会往后一直查直到空为止；注意查找的总次数（分母）必须等于散列函数的取值范围而不是散列表的总长度。

- (3) 若删除元素20，请再计算在等概率情况下，查找成功、查找失败的平均查找长度ASL。



$$ASL_{成功} = \frac{1 + 1 + 1 + 2 + 1 + 3 + 4 + 3 + 1 + 3 + 9}{11} = 2.63$$

$H(19) = 19 \% 13 = 6$
 $H(14) = 14 \% 13 = 1$
 $H(23) = 23 \% 13 = 10$
 $H(1) = 1 \% 13 = 1 \rightarrow 2$
 $H(68) = 68 \% 13 = 3$
 $H(84) = 84 \% 13 = 6 \rightarrow 7 \rightarrow 8$
 $H(27) = 27 \% 13 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
 $H(55) = 55 \% 13 = 3 \rightarrow 4 \rightarrow 5$
 $H(11) = 11 \% 13 = 11$
 $H(10) = 10 \% 13 = 10 \rightarrow 11 \rightarrow 12$
 $H(79) = 79 \% 13 = 1 \rightarrow 2 \rightarrow \dots \rightarrow 9$



注意易错点：采用开放定址法时，删除一个元素是逻辑删除，而非物理删除！这一点将影响查找长度的计算

注意查找位置为空和被删除的概念不一样，删除只是逻辑上的删除，数据仍然在那里，探测到这里时还会继续往后探测。

(3) 若删除元素20，请再计算在等概率情况下，查找成功、查找失败的平均查找长度ASL。

	14	1	68	27	55	19	逻辑删除	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

$H(19) = 19\%13 = 6$
 $H(14) = 14\%13 = 1$
 $H(23) = 23\%13 = 10$
 $H(1) = 1\%13 = 1 \rightarrow 2$
 $H(68) = 68\%13 = 3$
 $H(84) = 84\%13 = 6 \rightarrow 7 \rightarrow 8$
 $H(27) = 27\%13 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
 $H(55) = 55\%13 = 3 \rightarrow 4 \rightarrow 5$
 $H(11) = 11\%13 = 11$
 $H(10) = 10\%13 = 10 \rightarrow 11 \rightarrow 12$
 $H(79) = 79\%13 = 1 \rightarrow 2 \rightarrow \dots \rightarrow 9$



注意！注意！

$$ASL_{成功} = \frac{1 + 1 + 1 + 2 + 1 + 3 + 4 + 3 + 1 + 3 + 9}{11} = 2.63$$
$$ASL_{失败} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2}{13} = 7$$

注意易错点：采用开放定址法时，删除一个元素是逻辑删除，而非物理删除！这一点将影响查找长度的计算

7、装填因子

装填因子反映的就是散列表满的程度，计算的话就是表中记录数量/散列表的长度。

散列表的装填因子 $\alpha = \frac{\text{表中记录数 } n}{\text{散列表长度 } m}$

反映了一个散列表“满”的程度

显然，装填因子越大，越容易发生冲突。从而导致插入、查找操作效率降低，ASL增大



例：如上所示，散列表HT长度为16，散列函数 $H(\text{key}) = \text{key} \% 13$ ，其中已插入12个元素，则装填因子 $\alpha = 12/16 = 0.75$

8、聚集（堆积）现象

就是多个数据的初始地址被映射到了同一个地方。一堆元素堆积在一起

例：散列表长度为27，散列函数 $H(\text{key}) = \text{key} \% 23$ ，采用线性探测法处理冲突



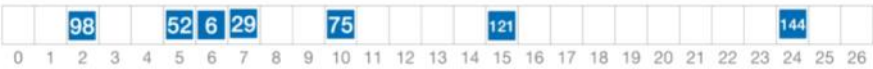
聚集（堆积）现象：在处理冲突的过程中，几个初始散列地址不同的元素争夺同一个后继散列地址的现象称作“聚集”（或称作“堆积”）

线性探测法在发生冲突时，总是往后探测相邻的后一个单元，很容易造成同义词、非同义词的“聚集（堆积）现象”，从而影响查找效率，导致ASL提升

线性探测法容易导致聚集现象，如果改成其他的探测法，可能会缓解这个现象，从而提升查找效率。

使用“平方探测法”减少聚集现象

例：散列表长度为27，散列函数 $H(\text{key}) = \text{key} \% 23$ ，采用平方探测法（二次探测法）处理冲突



平方探测法（又称二次探测法）： $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ ，其中 $k \leq m/2$

$d_0 = 0, d_1 = 1, d_2 = -1, d_3 = 4, d_4 = -4, d_5 = 9, d_6 = -9 \dots$

排序

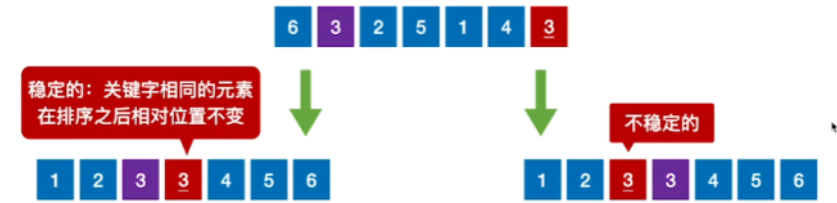
2025年9月20日星期六 下午1:29

一、基本概念

排序就是把一堆元素根据满足的关键字进行升序或者降序的排序。

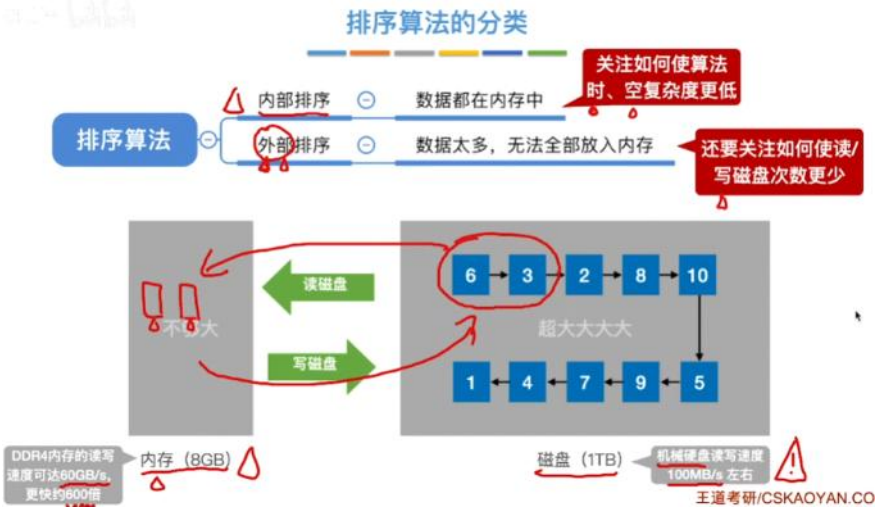
1、排序算法的性能指标

时间复杂度、空间复杂度、稳定性。前面的学过了，稳定性指的是2个相同的元素原本有先后顺序，在经过排序后这个先后顺序是否能保持。



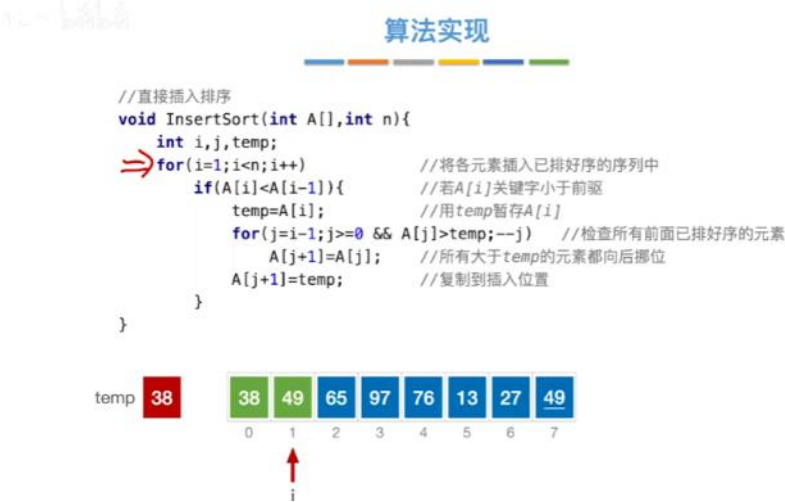
至于稳定是否比不稳定好，这个得看实际的需求。

2、排序的分类



二、插入排序

1、以升序为例，算法思想就是从前往后扫描所有待排序的节点，如果这个节点比前面的小，就放入temp中，依次比较前面节点的大小（边比较边将指针目前的节点后移），直到遇到比它更小或者等于的节点停下，将该节点插入到更小节点的后继即可。



当然也可以使用带哨兵的算法，区别就是本来该放入temp的节点放到A[0]即可。

算法实现（带哨兵）

```
//直接插入排序（带哨兵）
void InsertSort(int A[],int n){
    int i,j;
    for(i=2;i<=n;i++){
        if(A[i]<A[i-1]){
            A[0]=A[i]; //依次将A[2]~A[n]插入到前面已排序序列
            //若A[i]关键字小于其前驱，将A[i]插入有序表
            //复制为哨兵，A[0]不存放元素
            for(j=i-1;A[0]<A[j];--j){ //从后往前查找待插入位置
                A[j+1]=A[j]; //向后挪位
            }
            A[j+1]=A[0]; //复制到插入位置
        }
    }
}
```



优点：不用每轮循环都判断 $j \geq 0$



空间复杂度： $O(1)$

最好时间复杂度（全部有序）： $O(n)$

最坏时间复杂度（全部逆序）： $O(n^2)$

平均时间复杂度： $O(n^2)$

算法稳定性：稳定 老可稳

2、折半插入排序

原来的temp节点是顺序向后寻找的，如果采用折半查找的话能提升效率，先找到应该插入的位置然后移动元素。

思路：先用折半查找找到应该插入的位置，再移动元素



当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

优化——折半插入排序

```
//折半插入排序
void InsertSort(int A[],int n){
    int i,j,low,high,mid;
    for(i=2;i<=n;i++){
        A[0]=A[i]; //依次将A[2]~A[n]插入到前面的已排序序列
        //将A[i]暂存到A[0]
        low=1;high=i-1; //设置折半查找的范围
        while(low<=high){ //折半查找(默认递增有序)
            mid=(low+high)/2; //取中点
            if(A[mid]>A[0]) high=mid-1; //查找左半子表
            else low=mid+1; //查找右半子表
        }
        for(j=i-1;j>=high+1;--j)
            A[j+1]=A[j]; //统一后移元素，空出插入位置
        A[high+1]=A[0]; //插入操作
    }
}
```

这操作也就那样吧



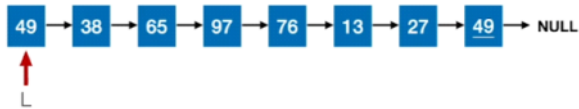
比起“直接插入排序”，比较关键字的次数减少了，但是移动元素的次数没变，整体来看时间复杂度依然是 $O(n^2)$

当 $low > high$ 时折半查找停止，应将 $[low, i-1]$ 内的元素全部右移，并将 $A[0]$ 复制到 low 所指位置

当 $A[mid] == A[0]$ 时，为了保证算法的“稳定性”，应继续在 mid 所指位置右边寻找插入位置



对链表进行插入排序



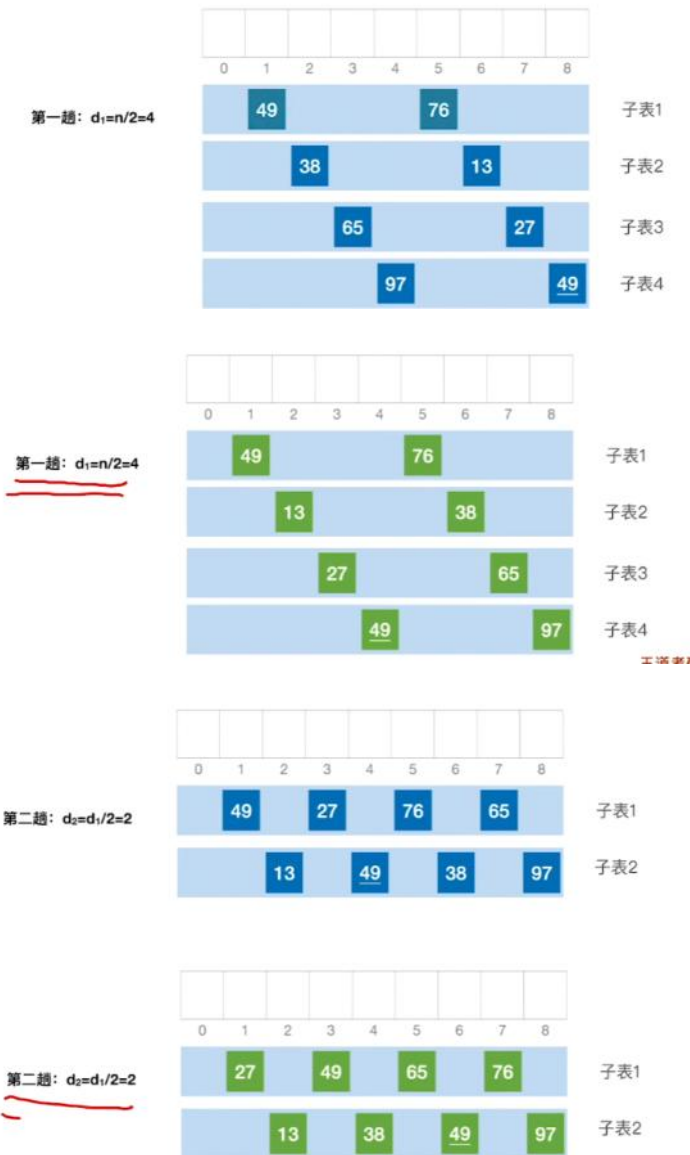
移动元素的次数变少了，但是关键字对比的次数依然是 $O(n^2)$ 数量级，
整体来看时间复杂度依然是 $O(n^2)$

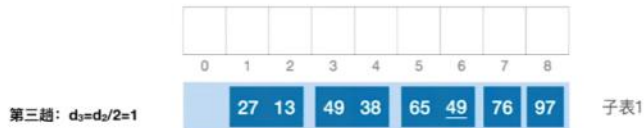
三、希尔排序

1、插入排序中如果表中元素原本就有序，那么需要的时间是最少的，这是最好的情况，希尔排序就是先追求表中元素的局部有序，再逐渐逼近全局有序。

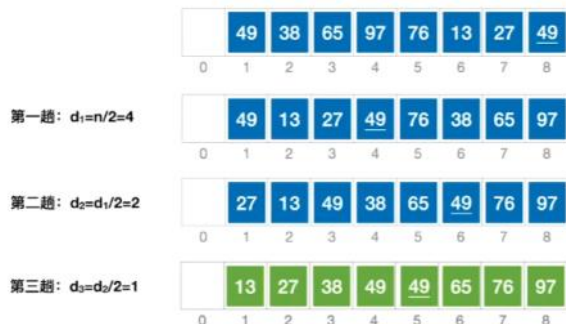
具体方法就是将排序表分割成若干个子表，每个子表都有对应的增量 d ，例如第一次的增量是4，那么这个子表的值就是1、5、9、14...，这个子表排序完后就缩小增量继续排序，直到增量为1排序完即可。

希尔排序：先将待排序表分割成若干形如 $L[i, i + d, i + 2d, \dots, i + kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量 d ，重复上述过程，直到 $d=1$ 为止。





整个表已呈现出“基本有序”，对整体再进行一次“直接插入排序”



2、代码实现

算法实现

```
//希尔排序
void ShellSort(int A[],int n){
    int d, i, j;
    //A[0]只是暂存单元，不是哨兵，当j<=0时，插入位置已到
    for(d= n/2; d>=1; d=d/2) //步长变化
        for(i=d+1; i<=n; ++i)
            if(A[i]<A[i-d]){ //需将A[i]插入有序增量子表
                A[0]=A[i]; //暂存在A[0]
                for(j= i-d; j>0 && A[0]<A[j]; j-=d)
                    A[j+d]=A[j]; //记录后移，查找插入的位置
                A[j+d]=A[0]; //插入
            }
}
```

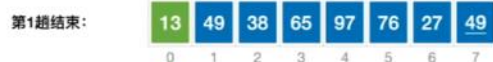


该排序方法的**时间复杂度**难以估计，**和增量的选择有关**，如果增量刚开始就是1那就会退化为插入排序算法，如果是最坏的情况下，那时间复杂度就是 $O(n^2)$ 。
该排序算法**只能基于顺序存储来实现**。

四、冒泡排序

该算法是**基于交换排序**来实现的排序，每趟扫描都将前后顺序不一的元素两两交换，扫描完一趟就能将最大的元素“沉底”，再扫描一趟就将第二大的元素“沉底”，直到所有的元素都有序。

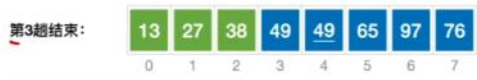
从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。



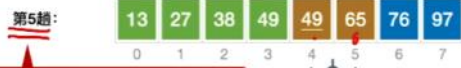
第一趟排序使关键字值最小的一个元素“冒”到最前面



第2趟结束后，最小的两个元素会“冒”到最前边



第3趟结束后，最小的3个元素会“冒”到最前边



若某一趟排序没有发生“交换”，说明此时已经整体有序。



△△△△

算法实现

```
//交换
void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}

//冒泡排序
void BubbleSort(int A[],int n){
    for(int i=0;i<n-1;i++){
        bool flag=false;
        for(int j=n-1;j>i;j--){
            if(A[j-1]>A[j]){
                swap(A[j-1],A[j]);
                flag=true;
            }
        }
        if(flag==false)
            return;
    }
}
```

算法性能分析

空间复杂度: O(1)

最好情况 (有序):

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

 比较次数=n-1; 交换次数=0
 最好时间复杂度=O(n)

最坏情况 (逆序):

8	7	6	5	4	3	2	1
---	---	---	---	---	---	---	---

 比较次数=(n-1)+(n-2)+...+1 = $\frac{n(n-1)}{2}$ = 交换次数
 最坏时间复杂度=O(n²)
 平均时间复杂度=O(n²)

冒泡排序是否适用于链表?

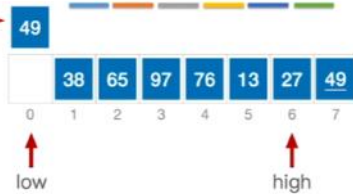


五、快速排序

1、该算法的思想就是每次在表中随意选取一个元素作为基准元素，将大于基准元素的值放在基准元素的右边，小于基准元素的元素放在基准元素的左边。这样就将表一分为二，确定好了该基准元素的位置。然后分别在左表和右表重复此操作直到排序完成。

快速排序

更小的元素都交换到左边
更大的元素都交换到右边



快速排序

枢轴（基准）元素



用第一个元素把待排序序列“划分”为两个部分。左边更小，右边更大。该元素的最终位置已确定

low high

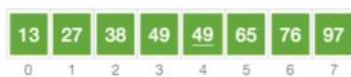
快速排序



快速排序

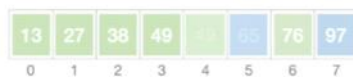


快速排序



2. 代码实现

快速排序



第四层
QuickSort

l=4, h=3
#97, l=4, h=5
p=4
#97, l=4, h=7
p=6
#98, l=0, h=7
p=3

递归工作栈

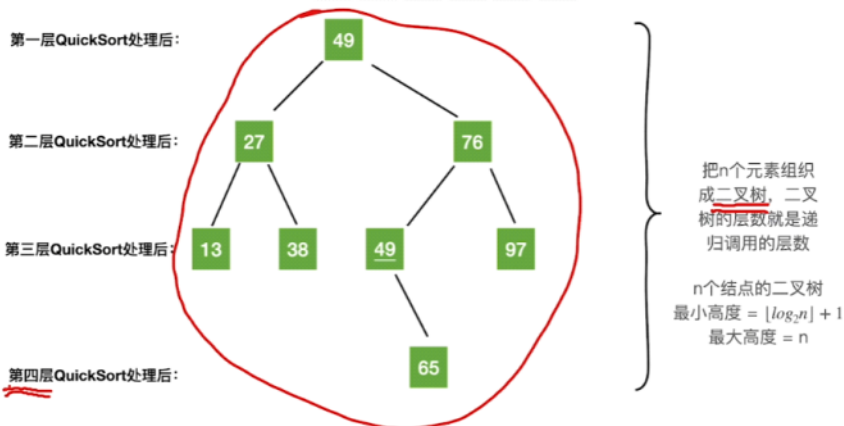
```
//用第一个元素将待排序序列划分成左右两个部分
int Partition(int A[],int low,int high){
    int pivot=A[low]; //第一个元素作为枢轴
    while(low<high){ //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high]; //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low]; //比枢轴大的元素移动到右端
    }
    A[low]=pivot; //枢轴元素存放到最后位置
    return low; //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

3. 算法效率

时间复杂度和递归的深度相关，空间复杂度也是。

算法效率分析



如果这个序列本身就是有序的, 那么用快速排序就很麻烦了, 因为会再次打乱重组, 这样时间复杂度就变成了 $O(n \log n)$ 最坏复杂度, 最好的复杂度就是 $O(n^2)$, 平均复杂度是 $O(n \log n)$, 对了这个快速排序不稳定。

六、简单选择排序

简单选择排序的每一趟待排序元素中选取关键字最小的元素加入有序序列, 如下图:





```
//简单选择排序
void SelectSort(int A[],int n){
    for(int i=0;i<n-1;i++){
        int min=i;
        for(int j=i+1;j<n;j++){
            if(A[j]<A[min]) min=j;
        }
        if(min!=i) swap(A[i],A[min]);
    }
}
```

//一共进行n-1趟
//记录最小元素位置
//在A[1...n-1]中选择最小的元素
//更新最小元素位置
//封装的swap()函数共移动元素3次



空间复杂度: $O(1)$



无论有序、逆序、还是乱序, 一定需要 $n-1$ 趟处理

总共需要对比关键字 $(n-1)+(n-2)+\dots+1 = \frac{n(n-1)}{2}$ 次
元素交换次数 $< n-1$



稳定性: 不稳定

适用性: 既可以用于顺序表, 也可用于链表

七、堆排序

1、大根堆和小根堆

大根堆就是根节点的值大于左右孩子的值, 小根堆就是根节点的值小于左右孩子的值。

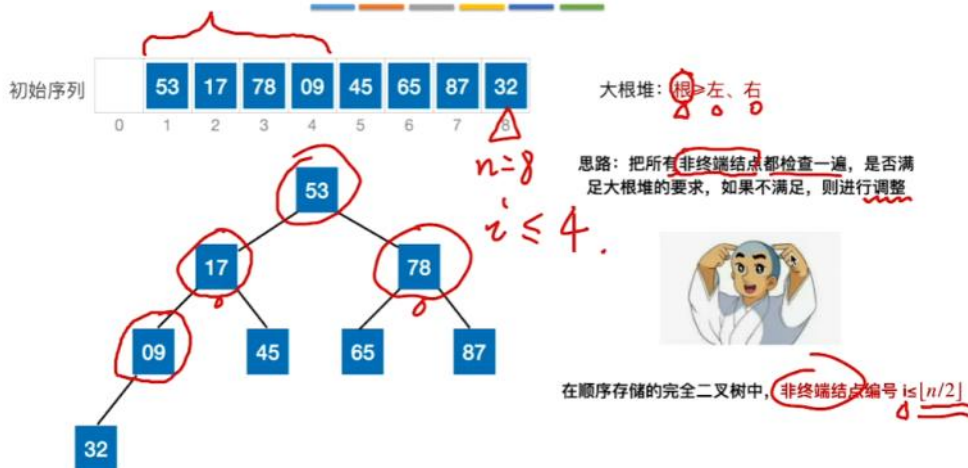
若 n 个关键字序列 $L[1..n]$ 满足下面某一条性质，则称为堆 (Heap)：

- ① 若满足： $L(i) \geq L(2i)$ 且 $L(i) \geq L(2i+1)$ ($1 \leq i \leq n/2$) —— 大根堆 (大顶堆)
- ② 若满足： $L(i) \leq L(2i)$ 且 $L(i) \leq L(2i+1)$ ($1 \leq i \leq n/2$) —— 小根堆 (小顶堆)

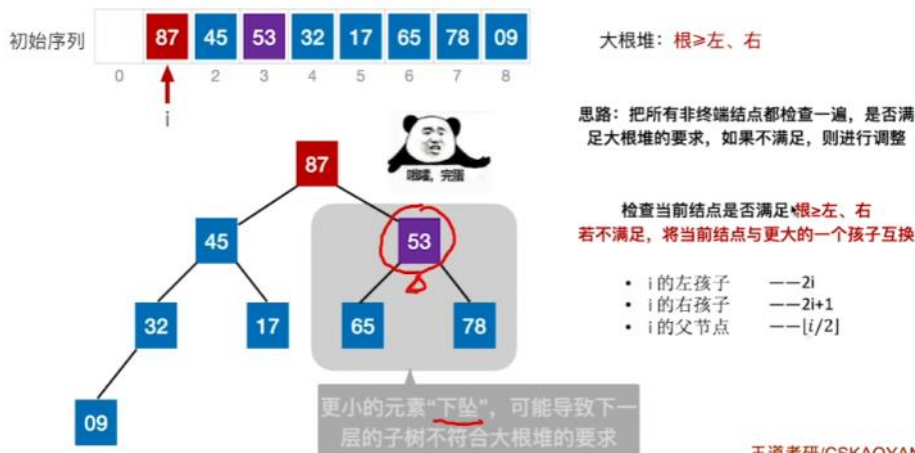


堆排序是基于选择排序生成的，选择排序就是在待排序中选取关键字最大或者最小的元素加入有序子序列。

建立大根堆



如上图，在 $i \leq \lfloor n/2 \rfloor$ 的范围内从右往左依次扫描，第一个09节点的左孩子比它大，就和它交换；然后是78节点，右孩子更大就交换，然后是17节点，此时左右孩子都比他大，就选取最大的那个进行交换（便于保持大根堆特性），最后是53节点和87互换，如下图

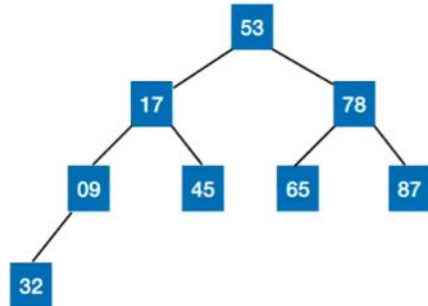


此时又不满足大根堆的要求，53的孩子更大，此时就一直往下换直到满足要求。

王道考研/CSKAQYAN.COI

建立大根堆（代码）

初始序列



```

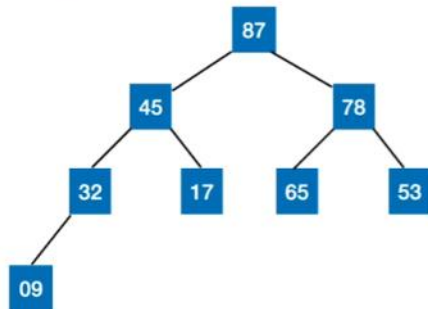
//建立大根堆
void BuildMaxHeap(int A[],int len){
    for(int i=len/2;i>0;i--) //从后往前调整所有非终端结点
        HeadAdjust(A,i,len);
}

//将以 k 为根的子树调整为大根堆
void HeadAdjust(int A[],int k,int len){
    A[0]=A[k]; //A[0]暂存子树的根结点
    for(int i=2*k;i<=len;i*=2){ //沿key较大的子结点向下筛选
        if(i<len&&A[i]<A[i+1])
            i++; //取key较大的子结点的下标
        if(A[0]>=A[i]) break; //筛选结束
        else{
            A[k]=A[i]; //将A[i]调整到双亲结点上
            k=i; //修改k值,以便继续向下筛选
        }
    }
    A[k]=A[0]; //被筛选结点的值放入最终位置
}
    
```

2. 堆排序

建立大根堆后,可以将待排序的节点放到根节点然后进行下坠操作来完成排序。

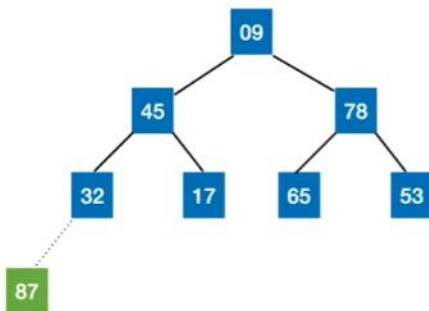
大根堆



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

大根堆



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

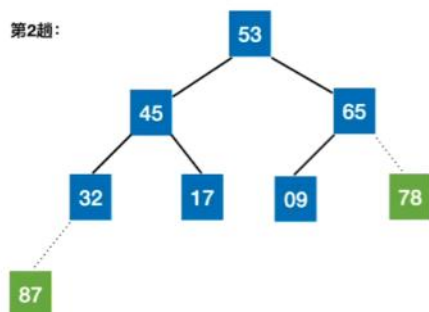
堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

大根堆



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

第2趟：



堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

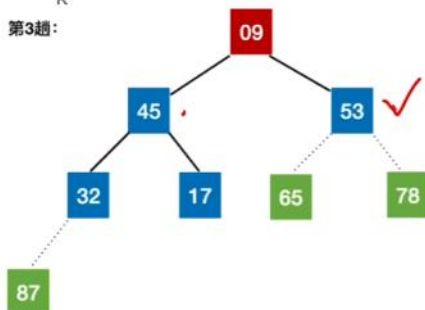
并将待排序元素序列再次调整为大根堆（小元素不断“下坠”）

大根堆



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

第3趟：



堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

并将待排序元素序列再次调整为大根堆（小元素不断“下坠”）

//将以 k 为根的子树调整为大根堆
void HeadAdjust(int A[],int k,int len)

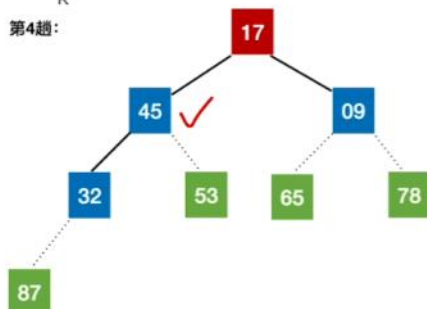
len=5

大根堆



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

第4趟：



堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

并将待排序元素序列再次调整为大根堆（小元素不断“下坠”）

//将以 k 为根的子树调整为大根堆
void HeadAdjust(int A[],int k,int len)

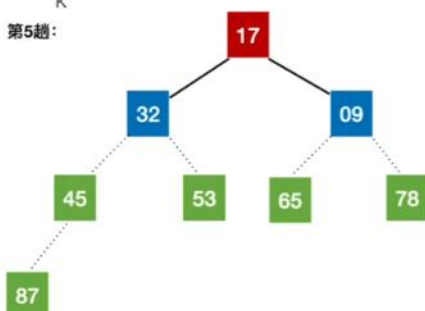
len=4

大根堆



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

第5趟：



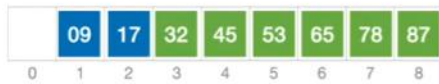
堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

并将待排序元素序列再次调整为大根堆（小元素不断“下坠”）

//将以 k 为根的子树调整为大根堆
void HeadAdjust(int A[],int k,int len)

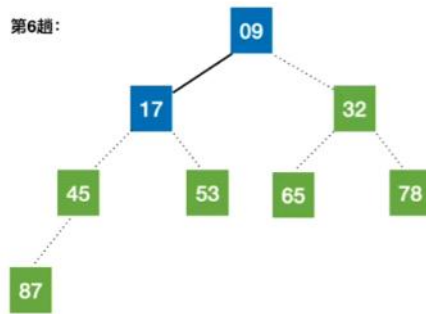
len=3

大根堆



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

第6趟：



堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

并将待排序元素序列再次调整为大根堆（小元素不断“下坠”）

基于大根堆进行排序

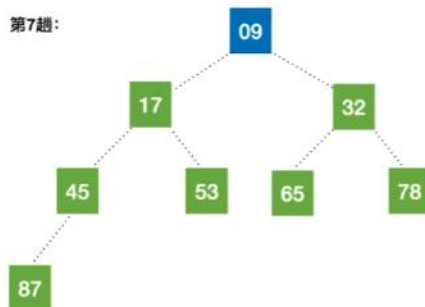
大根堆

只剩下最后一个待排序元素，不用再调整



选择排序：每一趟在待排序元素中选取关键字最大的元素加入有序子序列

第7趟：



堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

并将待排序元素序列再次调整为大根堆（小元素不断“下坠”）

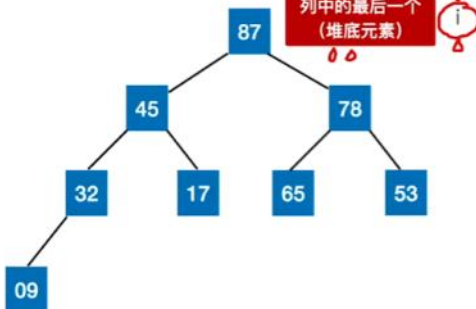
因此在经过n-1趟后完成了排序，得到了一个递增的序列，如果是基于小根堆来排序的话，得到的是递减的序列。

基于大根堆进行排序（代码）

大根堆



指向待排序元素序列中的最后一个（堆底元素）



//建立大根堆

void BuildMaxHeap(int A[],int len)

//将以 k 为根的子树调整为大根堆

void HeadAdjust(int A[],int k,int len)

//堆排序的完整逻辑

void HeapSort(int A[],int len){

BuildMaxHeap(A, len); //初始建堆

for(int i=len; i>1; i--){ //n-1趟的交换和建堆过程

swap(A[i], A[1]); //堆顶元素和堆底元素交换

HeadAdjust(A, 1, i-1); //把剩余的待排序元素整理成堆

}

堆排序：每一趟将堆顶元素加入有序子序列（与待排序序列中的最后一个元素交换）

并将待排序元素序列再次调整为大根堆（小元素不断“下坠”）

算法效率分析

```
//建立大根堆
void BuildMaxHeap(int A[],int len){
    for(int i=len/2;i>0;i--){
        HeadAdjust(A,i,len);
    }
}

//堆排序的完整逻辑
void HeapSort(int A[],int len){
    BuildMaxHeap(A,len);
    for(int i=len;i>1;i--){
        swap(A[i],A[1]);
        HeadAdjust(A,1,i-1);
    }
}

//将以 k 为根的子树调整为大根堆
void HeadAdjust(int A[],int k,int len){
    A[0]=A[k];
    for(int i=2*k;i<=len;i*=2){
        if(i<len&&A[i]<A[i+1])
            i++;
        if(A[0]>=A[i]) break;
        else{
            A[k]=A[i];
            k=i;
        }
    }
    A[k]=A[0];
}
```

堆排序的时间复杂度 = $O(n) + O(n \log_2 n) = O(n \log_2 n)$

还有，堆排序不稳定。

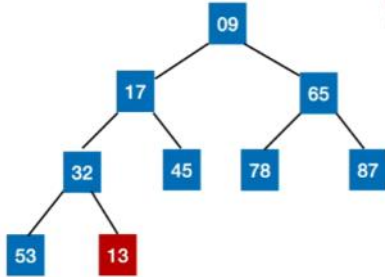
3、对堆的插入操作

在堆中插入新元素

小根堆



- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$

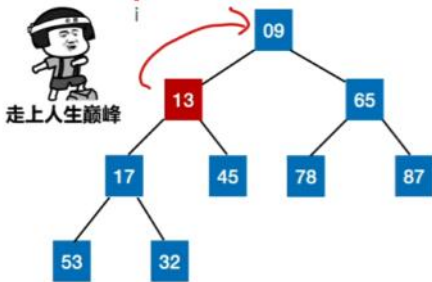


对于小根堆，新元素放到表尾，与父节点对比，若新元素比父节点更小，则将二者互换。新元素就这样一路“上升”，直到无法继续上升为止

小根堆



- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$



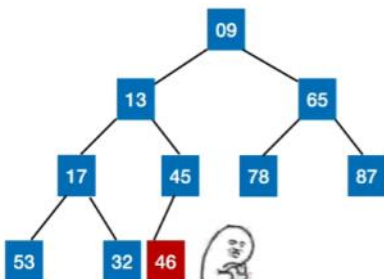
对于小根堆，新元素放到表尾，与父节点对比，若新元素比父节点更小，则将二者互换。新元素就这样一路“上升”，直到无法继续上升为止

对比关键字的次数 = 3次

小根堆



- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$



对于小根堆，新元素放到表尾，与父节点对比，若新元素比父节点更小，则将二者互换。新元素就这样一路“上升”，直到无法继续上升为止

对比关键字的次数 = 1次

4、对堆的删除操作

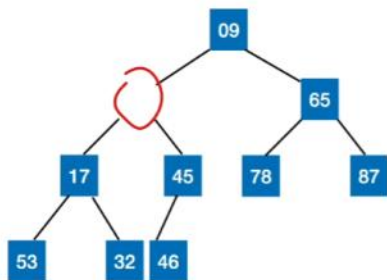
如果删除的是分支节点，就用堆底的元素来填补这个空缺。填补完后进行下坠操作即可。

在堆中删除元素

小根堆



- i 的左孩子 $\rightarrow 2i$
- i 的右孩子 $\rightarrow 2i+1$
- i 的父节点 $\rightarrow \lfloor i/2 \rfloor$

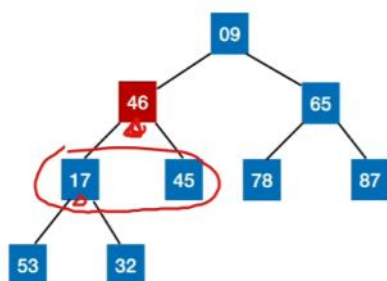


在堆中删除元素

小根堆



- i 的左孩子 $\rightarrow 2i$
- i 的右孩子 $\rightarrow 2i+1$
- i 的父节点 $\rightarrow \lfloor i/2 \rfloor$

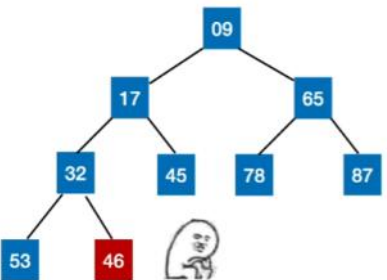


被删除的元素用堆底元素替代，然后让该元素不断“下坠”，直到无法下坠为止。

小根堆



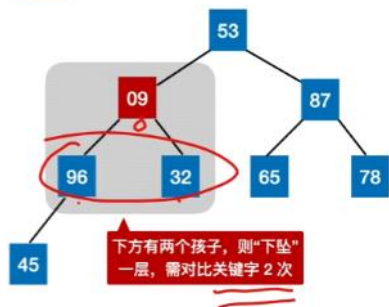
- i 的左孩子 $\rightarrow 2i$
- i 的右孩子 $\rightarrow 2i+1$
- i 的父节点 $\rightarrow \lfloor i/2 \rfloor$



被删除的元素用堆底元素替代，然后让该元素不断“下坠”，直到无法下坠为止。

对比关键字的次数 = 4次

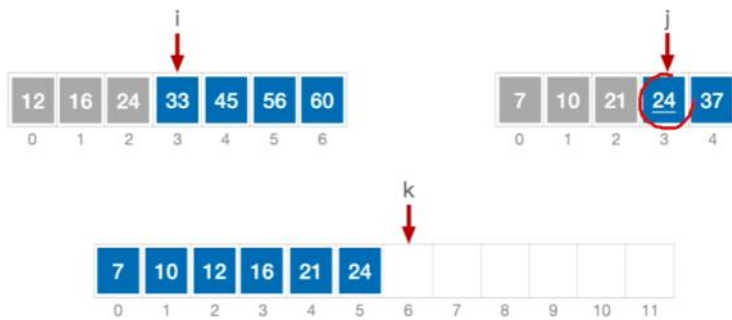
注意下坠的时候如果有左右孩子，那么需要对比2次选最大的那个：



八、归并排序

1、归并

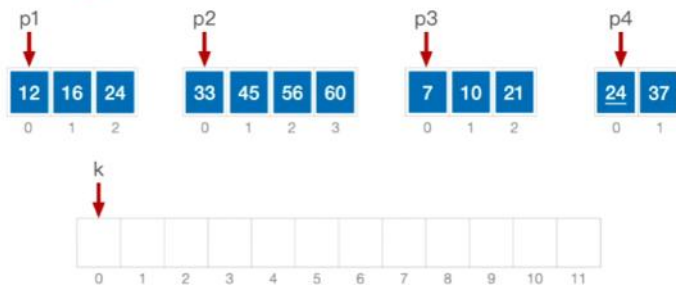
归并就是把2个或者多个已经有序的序列合并成1个。



对比 i、j 所指元素，选择更小的一个放入 k 所指位置

“4路”——四合一 “4路”归并

归并：把两个或多个已经有序的序列合并成一个

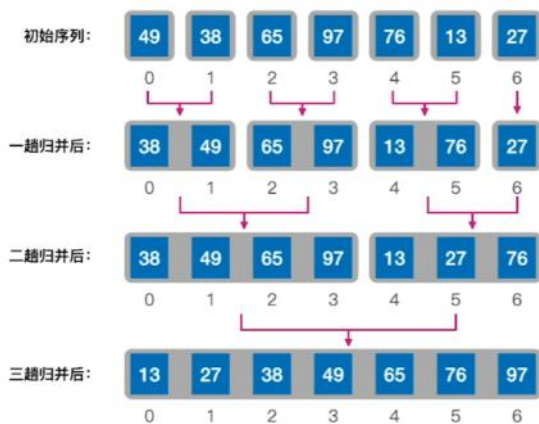


“4路”归并——每选出一个元素需对比关键字3次
结论：m路归并，每选出一个元素需要对比关键字 m-1 次

2、归并排序

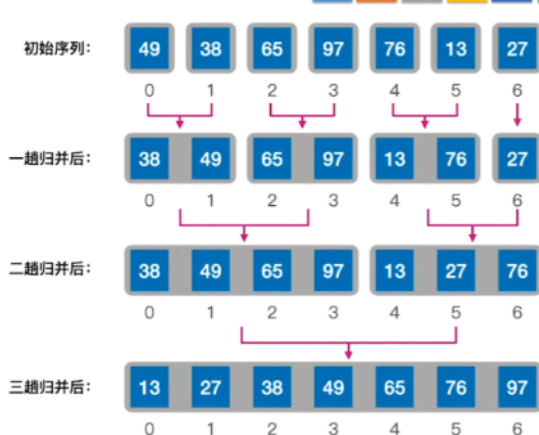
归并排序就是最开始把每个元素单独看成一个子序列，每轮都对相邻子序列两两归并，直到归并成一个序列。

在内部排序中一般采用2路归并 归并排序（手算模拟）



核心操作：把数组内的两个有序序列归并为一个

算法效率分析



2路归并的“归并树”——形态上就是一棵倒立的二叉树

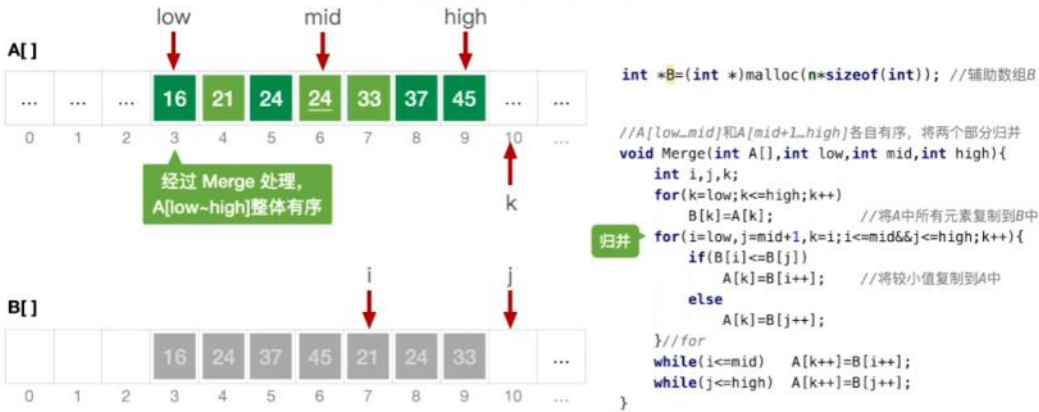


二叉树的第h层最多有 2^{h-1} 个结点
若树高为h，则应满足 $n \leq 2^{h-1}$
即 $h-1 = \lceil \log_2 n \rceil$

结论：n个元素进行2路归并排序，归并趟数 $= \lceil \log_2 n \rceil$

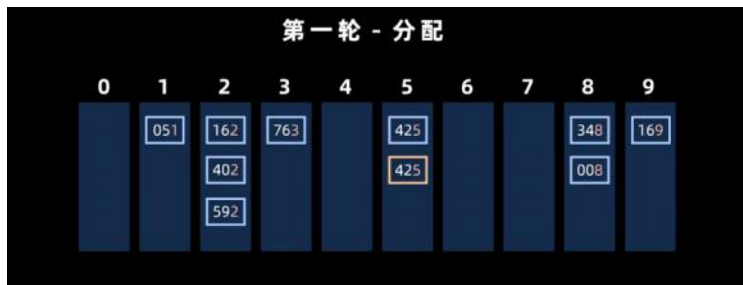
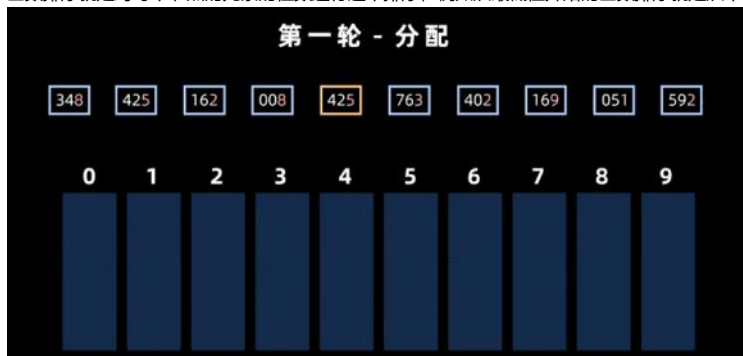
每趟归并时间复杂度为 $O(n)$ ，则算法时间复杂度为 $O(n \log_2 n)$

空间复杂度 $= O(n)$ ，来自于辅助数组B

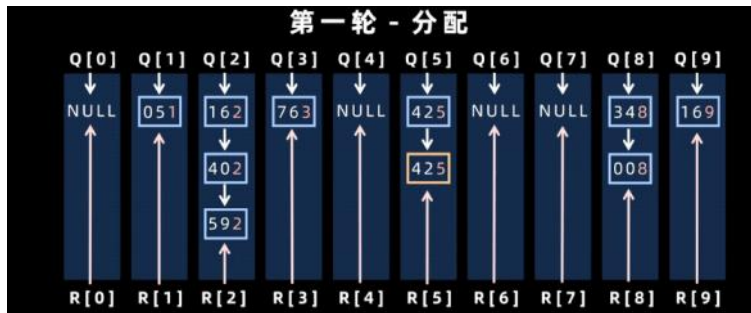


九、基数排序

基数排序就是对每个节点的元素的位数进行逐个排序，例如从最低位开始的基数排序就是从个位数开始排，按照每个个位位置进行分配，如下图：

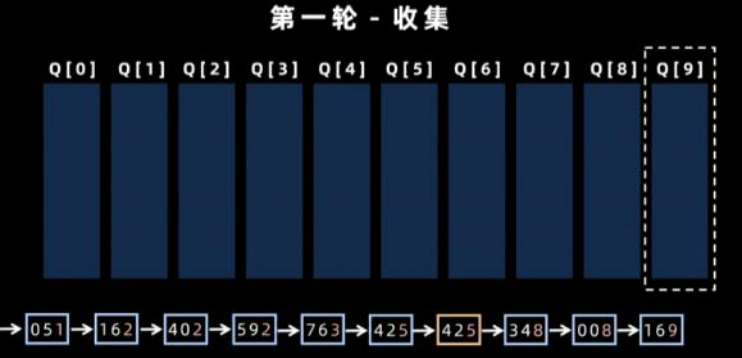
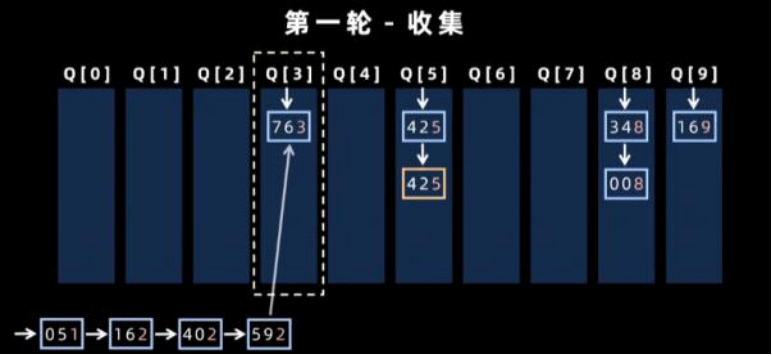


这个存放节点的桶非常适合用链表来表示，二维数组可能会浪费空间。

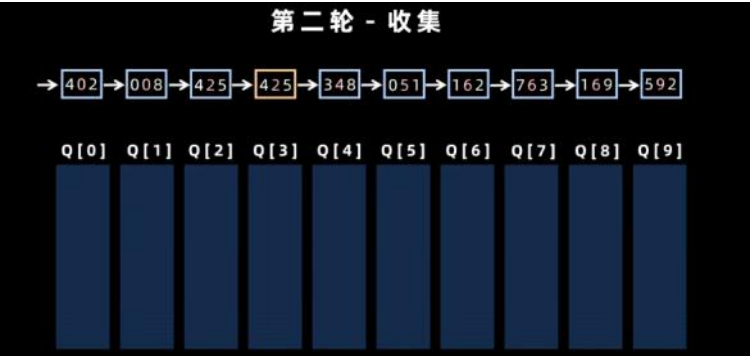
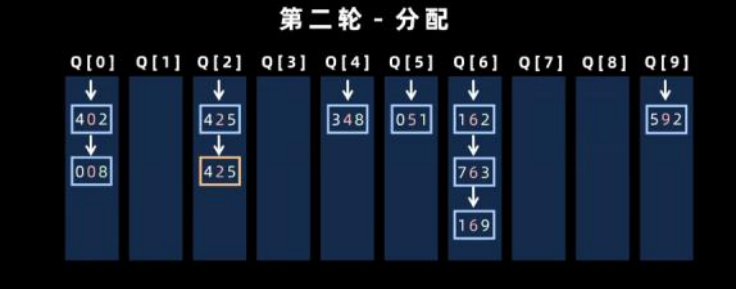


如上图，会有指向“队头”和“队尾”的指针，这样每个桶里的元素就类似于结构体了。

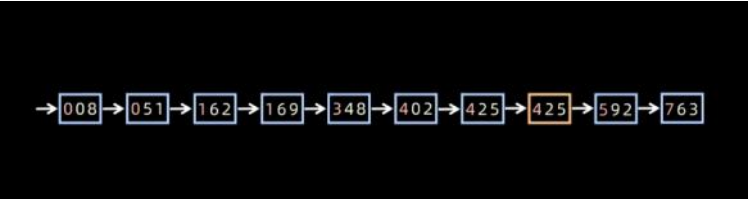
分配完后把桶里的元素再次拿出来：

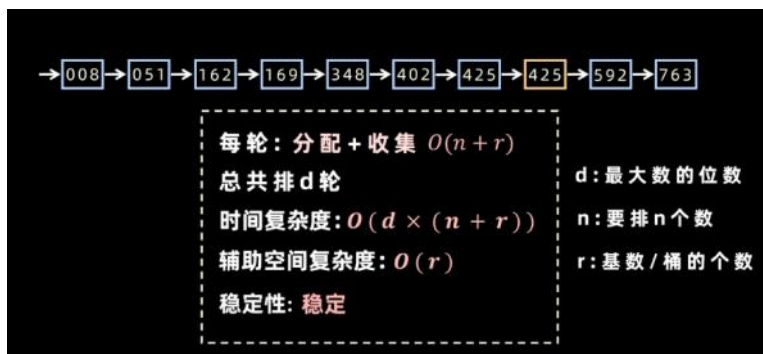


此时个位数有序，接下来就是进行十位数的分配，然后就是百位数。



十位分配完后会发现后两位已经从小到大有序了，最后分配完百位的，整体就有序了：

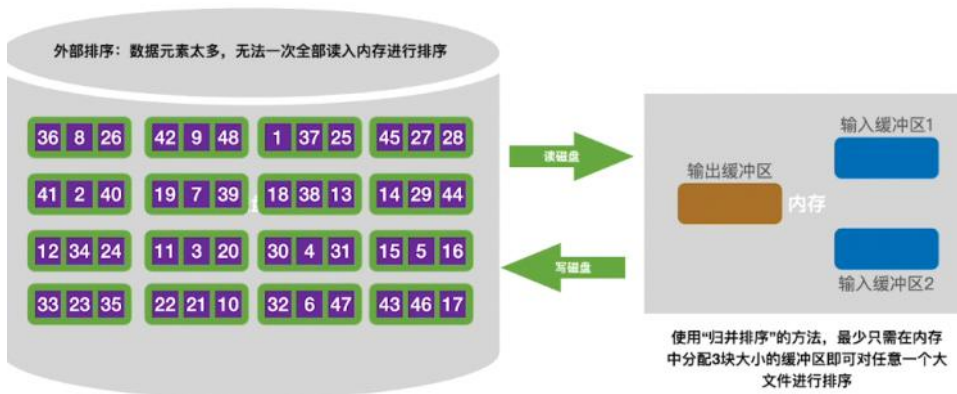




因此这种排序方式适合出生日期，身份证号码之类的进行排序。

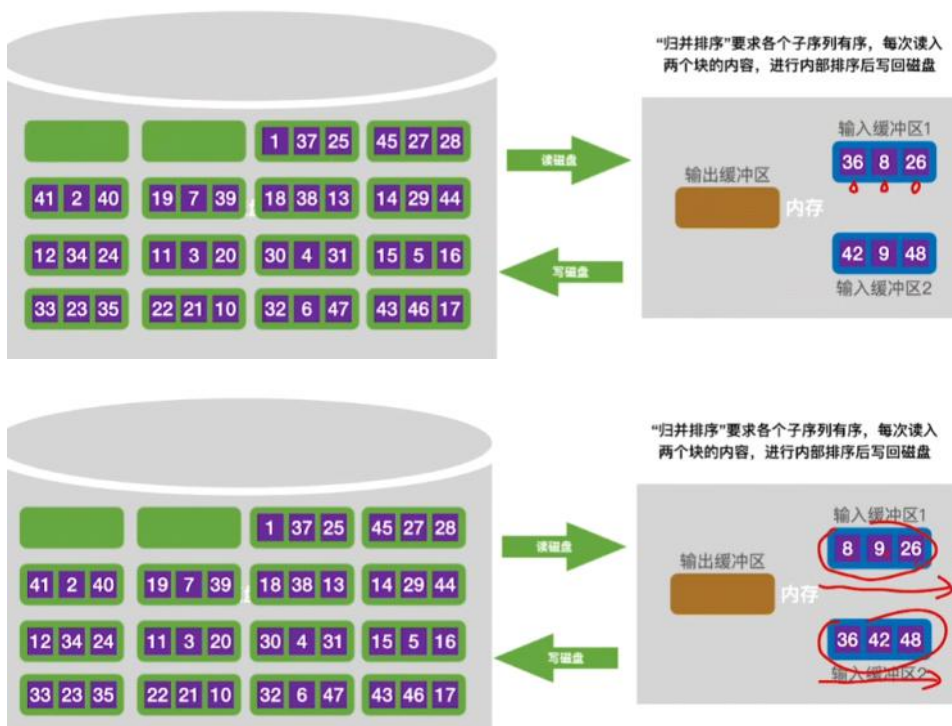
九、外部交换

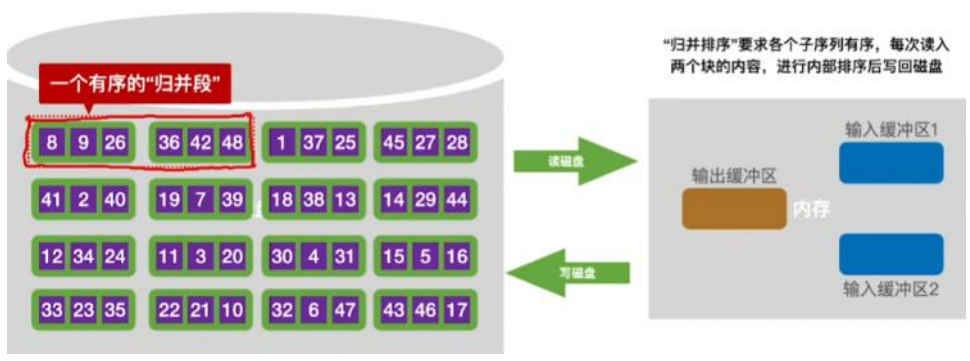
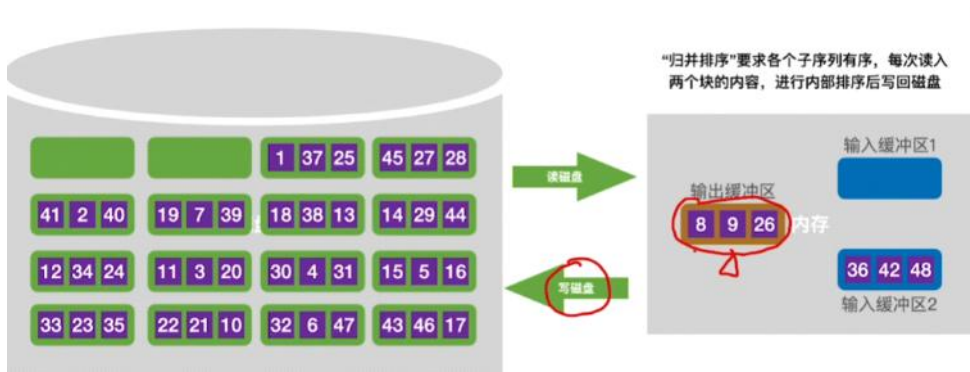
外部排序指的是排序外存内的元素，排序的时候需要进行外存和内存之间的数据交换，这个排序的过程是**基于归并排序**实现的。



1、归并过程

例如下面这样，先从外存中抽出2块数据放入内存，在内存中对这两组数据进行排序，排序完将数据块写回内存，形成一个有序的归并段。

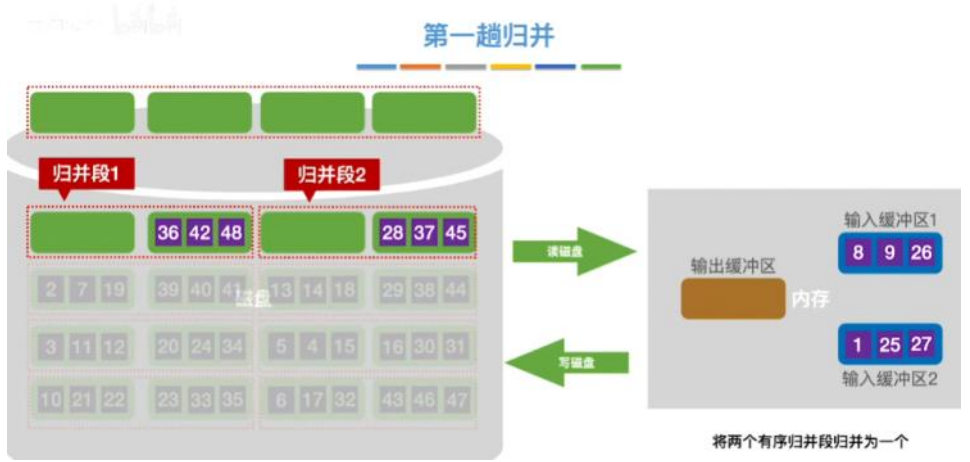


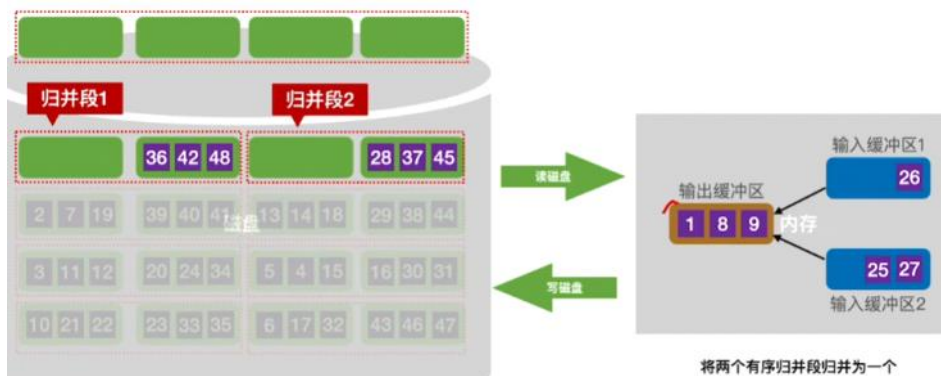


如下图，就这样所有的磁盘块都初步排序了下形成了一个初始的归并段，接下来对这些归并段继续进行第二趟归并排序。

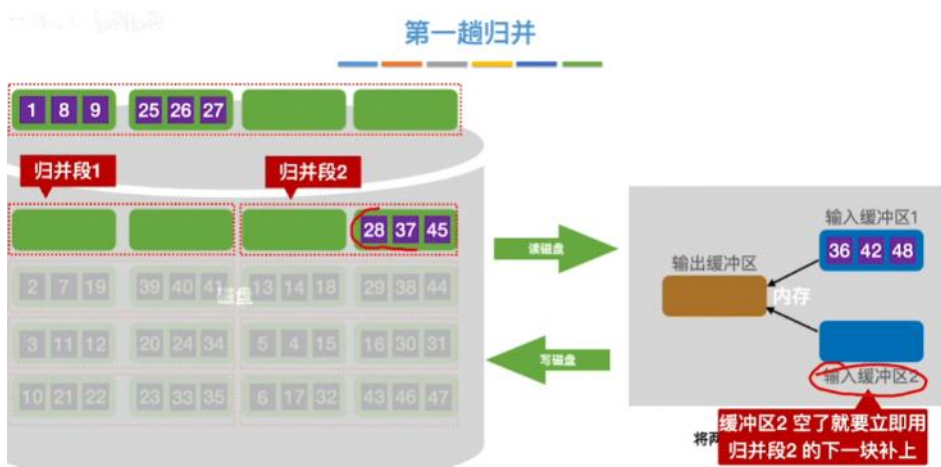
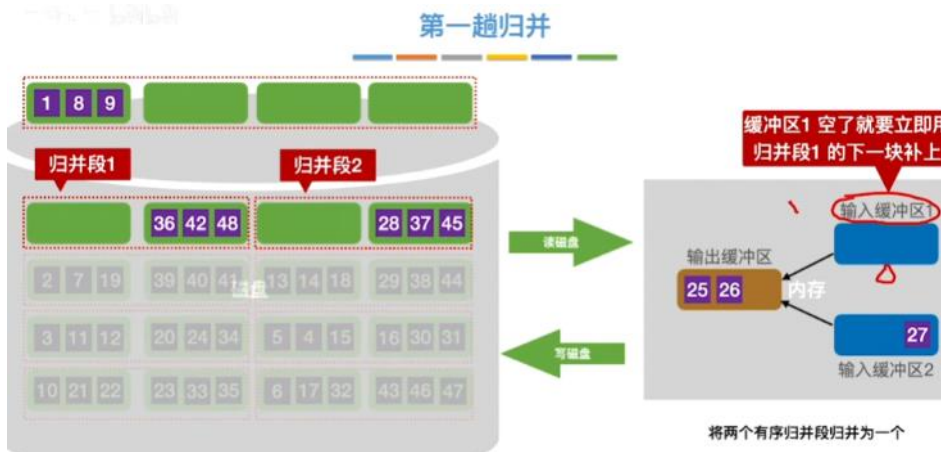


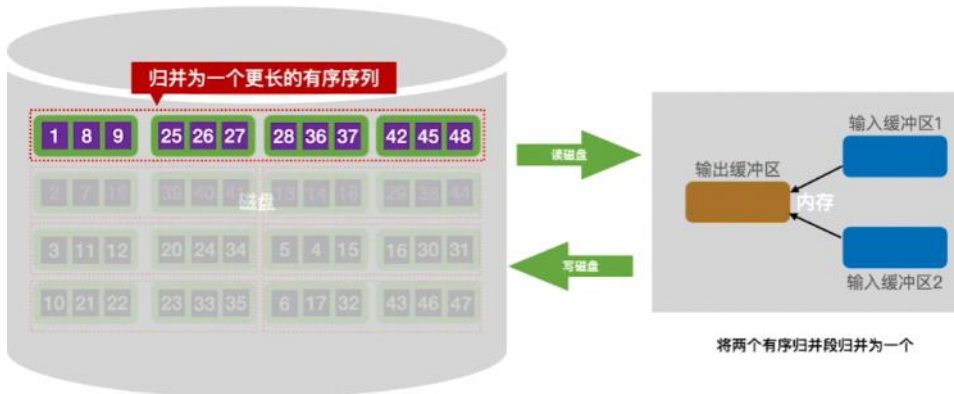
现在将归并段1和归并段2中最小的那块放入内存，将这部分最小的元素放入输出缓冲区，然后写回磁盘，此时能保证这次写回磁盘的是最小的那些数字。





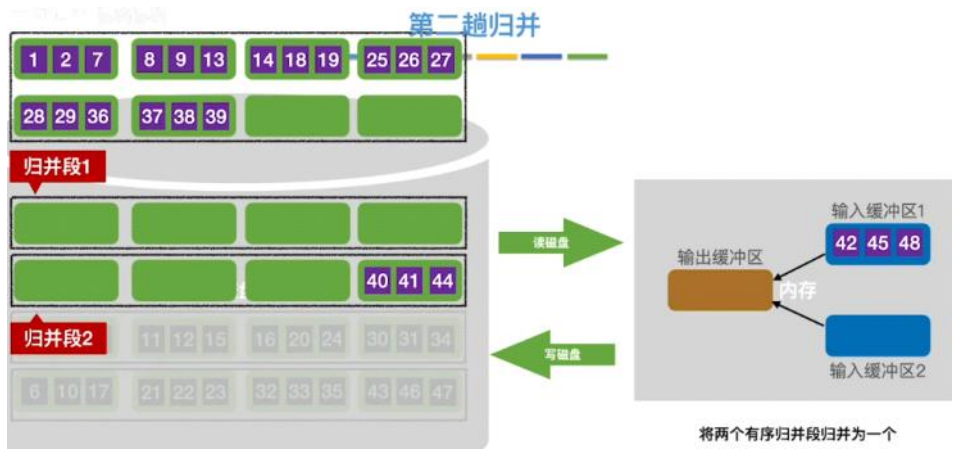
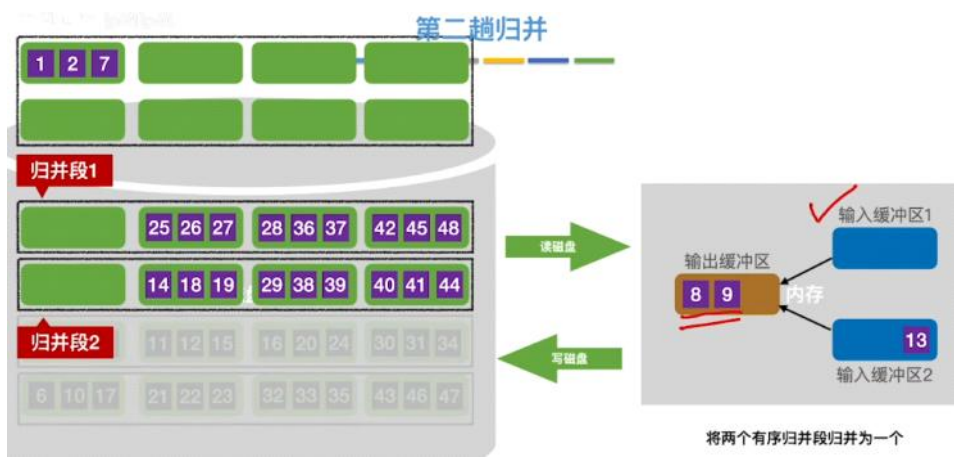
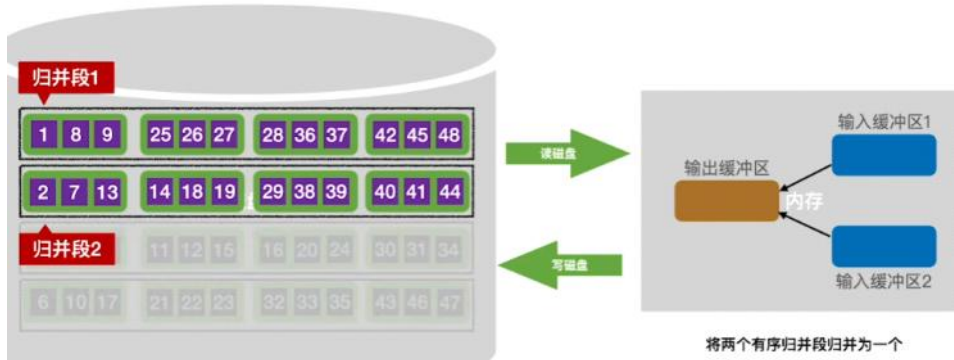
此时继续排序将最小的放入输出缓冲区，现在输入缓冲区有部分空了，那就用归并段1的下一块继续补上。



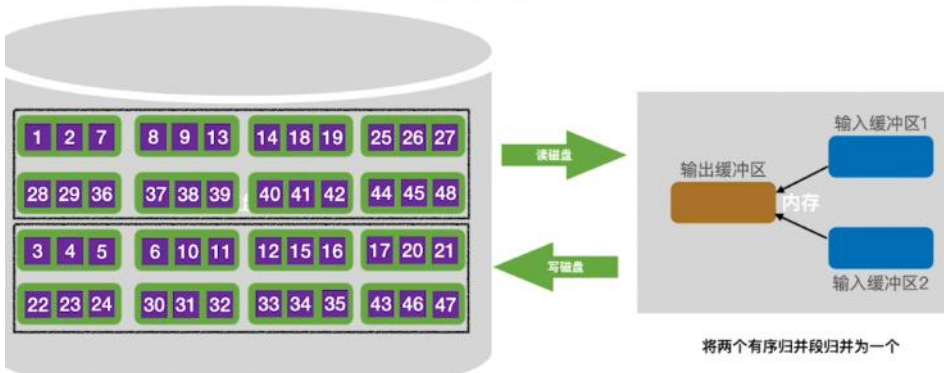


就这样将图中4行的序列归并好，现在开始第二趟归并：

第二趟归并



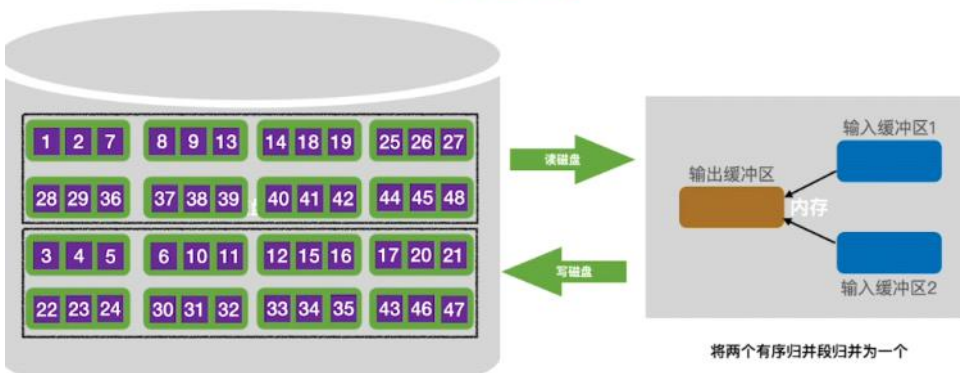
第二趟归并



现在进行第三趟归并，归并完后全部有序了。

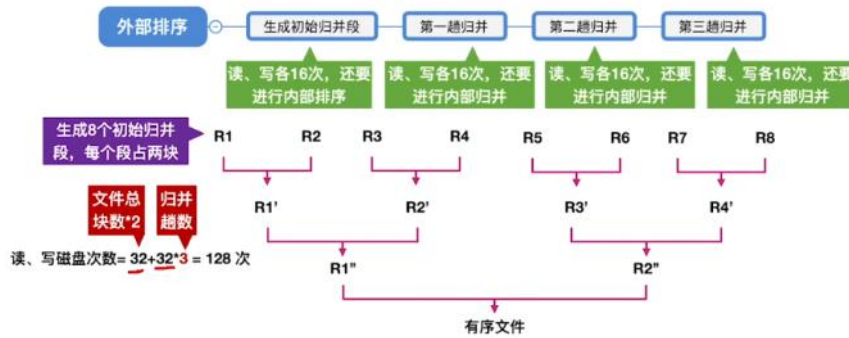
第三趟归并

把2个有序子序列
(归并段) 归并



2、时间开销

时间开销分析



外部排序时间开销=读写外存的时间+内部排序所需时间+内部归并所需时间

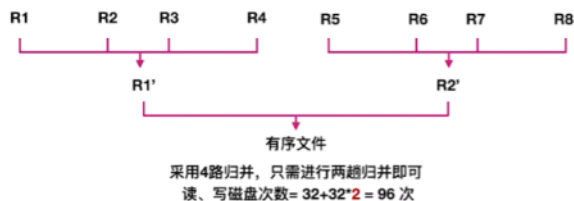
可以通过扩大输入缓冲区来提高归并效率。

优化：多路归并



优化：多路归并

外部排序时间开销=读写外存的时间+内部排序所需时间+内部归并所需时间



重要结论：采用多路归并可以减少归并趟数，从而减少磁盘I/O(读写)次数

对 r 个初始归并段，做 k 路归并，则归并树可用 k 叉树表示若树高为 h ，则归并趟数 $= h-1 = \lceil \log_k r \rceil$ 推导： k 叉树第 h 层最多有 k^{h-1} 个结点则 $r \leq k^{h-1}$, $(h-1)_{\min} = \lceil \log_k r \rceil$

k 越大， r 越小，归并趟数越少，读写磁盘次数越少

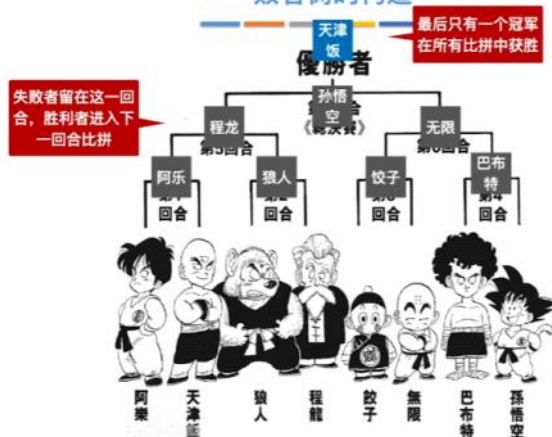
多路归并带来的负面影响：

① k 路归并时，需要开辟 k 个输入缓冲区，内存开销增加。② 每挑选一个关键字需要对比关键字 $(k-1)$ 次，内部归并所需时间增加

当然多路归并在减少排序趟数的同时，会增加内存区域排序的负担，降低效率，此时可以通过败者树之类的方法来进行优化。

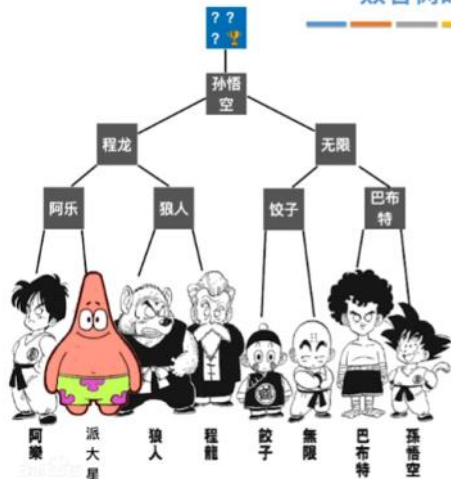
3、败者树

败者树的构造



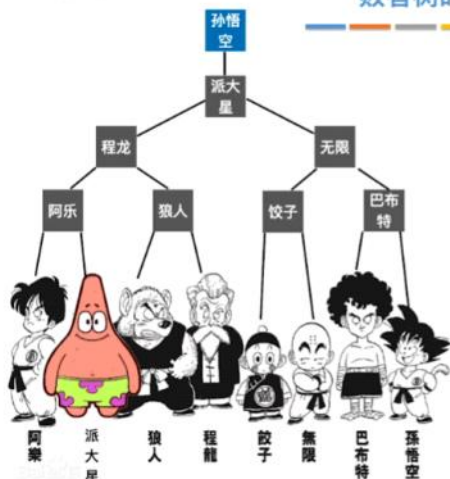
败者树类似于比武，两两组成一回合，成功的上升，失败的留在原地，最终选出最后的胜利者。这个过程过完就构造出了一个败者树，现在将冠军踢掉，在叶子结点处新增一个选手，就只需要在选手所在的子树上进行对决。

败者树的使用



王道考研/CSKAO

败者树的使用

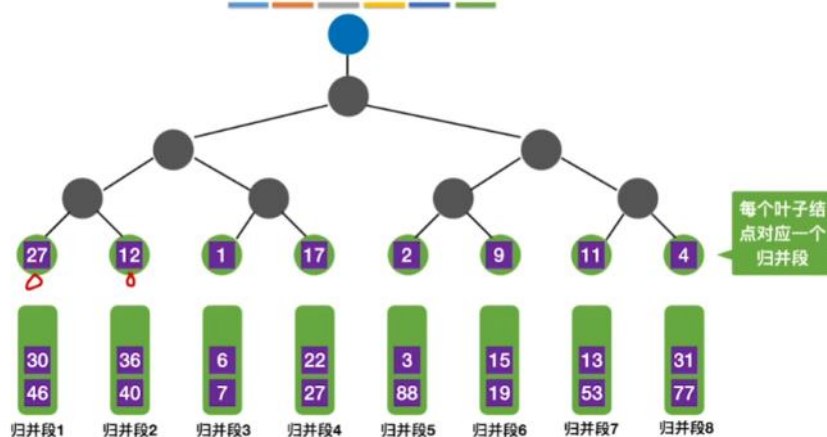


基于已经构建好的败者树，选出新的胜者只需进行 3 场比赛

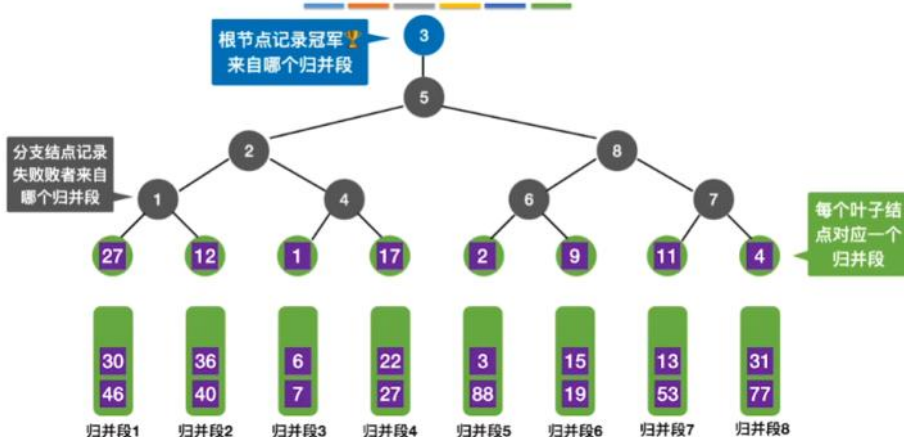
王道考研/CSKAQYAN.COM

在计算机上的应用也是如此，将8个归并段的内部进行排序一下，然后把最顶层的放入树中：

败者树在多路平衡归并中的应用

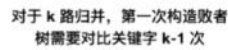


败者树在多路平衡归并中的应用

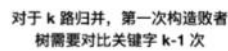


此时得到了一个初始的败者树，得到了一个最小的元素（根节点），将这个元素记录下来后踢出树中，在该元素所在的归并段中继续抽一个出来。

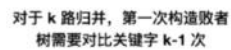
1



1



1



2

4、置换选择排序

